

```
const shopName = "우리편의점";  
  
let price = 1200;  
  
console.log(price);  
  
const cart = ["아메리카노", "케이크"];  
  
cart.forEach((item) => console.log(item));  
  
document.querySelector("#list").textContent = "완료";  
  
const res = await fetch(url);
```

2 권 · 화면과 서버

자바스크립트 기초

값을 화면에 찍는 것부터 시작해서, 브라우저에서 도는 할 일 목록을 직접 만드는 데까지 갑니다.

읽고, 따라 치고, 출력이 같은지 맞춰 보세요.

7

단원

33

개념

166

직접 해보기

146

문제

차례

09	구조분해와 스프레드	13
01	배열 구조분해	14
02	객체 구조분해	22
03	스프레드: 배열 펼치기	32
04	스프레드: 객체 펼치기	41
05	나머지 매개변수	49
10	DOM 조작	69
01	DOM 이란과 요소 선택	70
02	내용 바꾸기	78
03	스타일과 클래스	85
04	요소 만들고 붙이기	93
05	입력값 읽기	101
11	이벤트	119
01	click 이벤트	120
02	이벤트 객체	127
03	input 과 change	134
04	form submit	140
05	이벤트 위임	146
12	비동기 fetch	159
01	동기와 비동기, setTimeout	160
02	콜백 지옥과 Promise	163
03	fetch 로 데이터 받기	166
04	async 와 await	178
05	에러 처리	182
13	종합 연습	201

01	성적 관리 프로그램	202
02	장바구니	207
03	할 일 목록	212
04	사용자 검색	217
14	OpenAPI와 Postman	223
01	Open API가 무엇인가	224
02	주소 만들기 URL과 쿼리스트링	231
03	Postman으로 먼저 확인하기	238
04	키가 필요한 API	245
05	안 되는 경우들	252
15	React 가기 전에	267
01	파일 나눠 쓰기 (모듈)	268
02	localStorage	274
03	날짜와 무작위 (Date, <code>Math.random</code>)	283
04	class 맛보기	291
부록 가	연습문제·종합연습 정답	307
부록 나	심화문제 정답	373

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작 전 준비 (처음 한 번만)

1. Node.js 설치

01~09단원은 **Node.js**라는 프로그램으로 실행합니다. 없으면 아무것도 안 돌아갑니다.

1. <https://nodejs.org> 접속
2. 왼쪽의 **LTS** 버튼을 눌러 내려받기 (오른쪽 Current 말고 왼쪽입니다)
3. 받은 파일을 실행 → 계속 "다음"만 누르면 됩니다
4. 설치가 끝나면 **컴퓨터를 켜다 켜거나, 열려 있던 터미널을 모두 닫고 다시 여세요**

설치가 됐는지 확인하는 방법 — 터미널에 이렇게 칩니다.

```
node -v
```

v22.14.0 처럼 **v로 시작하는 숫자**가 나오면 성공입니다.

'node'은(는) 내부 또는 외부 명령... 이 아닙니다 가 나오면 아직 설치가 안 됐거나 터미널을 다시 열지 않은 것입니다. 터미널을 닫았다가 새로 열어 보세요.

2. VS Code 설치

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받아 설치하세요.

설치 후 VS Code 를 열고 **파일 → 폴더 열기**로 이 **JS자료** 폴더를 통째로 여세요. 파일 하나만 여는 것보다 폴더째 여는 편이 훨씬 편합니다.

3. 알아두면 좋은 것

- **HTML과 CSS는 몰라도 됩니다.** 10~12단원 파일 위쪽에 있는 `<style>`, `<body>` 같은 코드는 화면 모양을 만드는 부분입니다. 이 자료에서는 가르치지 않습니다. `<script>` 안부터 읽으세요.
- **12단원은 인터넷 연결이 필요합니다.** 인터넷에서 데이터를 받아오는 내용이라 그렇습니다. 학교나 회사 네트워크가 막아 두어 안 되는 경우가 있는데, 그때 쓰는 방법을 아래에 적어 두었습니다.

4. 인터넷이 막혀 12단원이 안 될 때

12단원과 13단원 종합04는 `jsonplaceholder.typicode.com` 이라는 연습용 서버에서 데이터를 받아옵니다. 이 주소가 막혀 있으면 **인터넷 없이 돌리는 대체본**을 켤 수 있습니다.

해당 파일(12단원 개념03·04·05·연습문제·연습문제_정답, 13단원 종합04 두 개) 위쪽에 이런 줄이 있습니다.

```
<!-- <script src="오프라인_대체.js"></script> -->
```

여기서 앞의 `<!--` 와 뒤의 `-->` 만 지우면 켜집니다.

```
<script src="오프라인_대체.js"></script>
```

켜지면 콘솔(F12) 맨 위에 `[오프라인 대체본이 켜져 있습니다]` 가 찍힙니다. **진짜 서버가 주는 데이터를 그대로 담아 두었기 때문에, 자료에 적힌 결과가 똑같이 나옵니다.** 없는 번호를 요청했을 때 404가 나오는 것까지 똑같습니다.

인터넷이 되는 곳으로 옮기면 `<!--` 와 `-->` 를 다시 붙여 꺼 두세요.

실행 방법

01~09단원 (.js 파일) — node로 실행

VS Code에서 터미널을 엽니다. `Ctrl + `` (숫자 1 왼쪽 키) 입니다.

```
cd 01_변수와_자료형
node 개념01_console_log로_출력하기.js
```

`cd` 폴더이름 은 그 폴더로 들어가라는 뜻입니다. 다시 나오려면 `cd ..` 를 칩니다.

```
cd ..
cd 02_연산자
```

더 편한 방법: VS Code 왼쪽 파일 목록에서 폴더를 마우스 오른쪽 클릭 → **"통합 터미널에서 열기"** 를 하면 그 폴더에서 터미널이 바로 열립니다. `cd` 를 안 쳐도 됩니다.

파일 이름이 길어서 치기 힘들면, `node` 개념01 까지만 치고 `Tab` 키를 누르세요. 나머지가 자동으로 채워집니다.

10~12단원 (.html 파일) — 브라우저로 실행

파일을 더블클릭하면 브라우저가 열립니다. 화면에 결과가 보이고, 개발자도구(`F12`) → **Console** 탭에도 결과가 찍힙니다.

10~12단원은 **Console** 탭을 반드시 같이 보세요. 화면에 안 나오는 결과가 거기 찍힙니다.

더블클릭했는데 **VS Code** 가 열린다면 — 파일 우클릭 → 연결 프로그램 → 크롬(또는 엣지) → "항상 이 앱으로 열기" 체크. VS Code 를 설치하면 자주 이렇게 됩니다.

`F12` 를 눌러도 아무 일이 없다면 — 노트북은 `Fn + F12` 이거나, `Ctrl + Shift + J` 로도 열립니다. 화면 아무 곳이나 우클릭 → 검사 도 됩니다.

코드를 고친 뒤에는 (10~12단원에서 제일 많이 하는 실수)

.html 파일은 저장한다고 화면이 저절로 바뀌지 않습니다.

VS Code 에서 고친다 → `Ctrl + S` 로 저장 → 브라우저로 가서 `F5` (새로고침)

이 세 단계를 매번 해야 합니다. "코드는 고쳤는데 화면이 그대로예요" 의 1순위 원인이 **저장을 안 했거나 새로고침을 안 한 것**입니다. 10~12단원의  **직접 해보기** 는 전부 이 과정을 요구합니다.

파일 읽는 법

개념 파일 (개념01_..., 개념02_...)

설명이 주석으로 들어있는 코드입니다. 읽지만 말고 직접 쳐 보세요.

```
// — 섹션 1: 변수에 값 담기 —  
  
let drink = "아메리카노";  
console.log(drink);  
// 출력: 아메리카노
```


변수 이름은 영어로 씁니다. 한글로도 만들 수는 있지만, 나중에 배울 React·서버 코드는 전부 영어 이름을 쓰기 때문에 지금부터 습관을 들입니다. 대신 **주석과 화면에 보이는 글자는 한글**입니다.

파일은 항상 같은 순서로 되어 있습니다.

1. **맨 위 헤더** — 이 파일이 무엇을 다루는지, 어떻게 실행하는지
2. — **섹션 N: 제목** — 주제가 바뀌는 지점
3.  **직접 해보기** · 섹션마다 나오는 짧은 과제. **정답은 파일 맨 아래 직접 해보기 정답 블록**에 주석으로 있습니다. 먼저 스스로 해 보세요.
4. **섹션 N: 자주 하는 실수** · 학생들이 실제로 자주 하는 실수 모음. // [실수 1] 처럼 번호가 붙어 있고 **주석 처리되어 있습니다**. 주석을 풀고 실행해서 결과를 눈으로 본 다음, **반드시 다시 주석 처리하세요**.

단, **SyntaxError** 라고 적힌 것은 **눈으로만 보세요**. 다른 예러는 주석을 풀어도 그 줄에서만 나고 앞의 출력은 그대로 나옵니다. 그런데 **SyntaxError** 는 자바스크립트가 파일을 아예 못 읽는 상태라 **출력이 한 줄도 안 나옵니다**. 여러분이 망가뜨린 게 아니니 놀라지 마세요. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다. 각 파일의 실수 섹션 맨 위에도 같은 안내를 적어 두었습니다.
5. — **정리** — 그 파일의 핵심 5~6줄. 복습할 때 여기만 봐도 됩니다.

결과를 적어 둔 주석 3가지

주석	뜻	나오는 곳
// 출력:	터미널에 찍히는 결과	01~09단원, 13단원 .js
// 콘솔:	개발자도구 Console 탭에 찍히는 결과	10~12단원
// 화면:	브라우저 화면이 어떻게 바뀌는지	10~12단원

전부 **"실행하면 이렇게 나온다"** 는 뜻입니다. 직접 실행해서 같은지 확인하세요. 다르게 나오면 어딘가 잘못된 것입니다.

연습 파일 (연습문제.js / 연습문제.html)

TODO가 적힌 빈칸을 채우는 파일입니다. 각 문제는 이렇게 생겼습니다.

```
// —— 문제 1 ——
// 삼각김밥 가격 1200을 변수 riceBallPrice 에 담고 출력하세요.
//
// 기대 출력:
// 1200

// TODO: 여기에 코드를 쓰세요
```

- **기대 출력**과 똑같이 나오면 정답입니다.
- 막히면 **연습문제_정답.js**를 보세요. 다만 **먼저 10분은 혼자 고민하고** 보세요.
- 문제는 **기본 → 응용 → [도전]** 순서로 어려워집니다. **[도전]**은 못 풀어도 괜찮습니다.
- 맨 마지막 문제는 **에러를 직접 보는 문제**입니다. 주석을 풀어 에러를 확인하고 다시 주석 처리하세요.

10~12단원 연습문제는 화면을 바꾸는 문제가 많아서, 기대 출력 대신 문제 문장 안에 "이렇게 되어야 합니다" 가 적혀 있는 경우가 있습니다.

13단원 종합 실습도 같은 방식입니다. 문제 파일의 TODO를 채우고, 막히면 **_정답** 파일을 보세요.

심화 파일 (심화문제.js / 심화문제.html) — 선택

01~12단원에 **연습문제와 별개로** 심화문제가 3개씩 들어 있습니다. 총 36문제입니다.

- **안 풀어도 됩니다.** 다음 단원으로 넘어가는 데 아무 지장이 없습니다.
- 연습문제를 **다 풀 뒤에** 하세요. 실습 시간이 남을 때 쓰라고 만든 것입니다.
- 연습문제보다 확실히 어렵습니다. **답이 바로 안 보이는 게 정상입니다.**
- 정답 파일은 답보다 **"왜 그런지" 설명이 훨씬 깁니다.** 그게 심화의 목적입니다.

막혀도 좌절하지 마세요. 개념 파일을 다시 뒤져 보게 만드는 것이 이 문제들의 의도입니다.

자주 나오는 에러 4가지

에러 메시지	뜻	해결
'node'은(는) 내부 또는 외부 명령...	Node.js 가 없다	맨 위 "시작 전 준비" 를 다시 하세요. 설치 후 터미널을 새로 열어야 합니다

ReferenceError: x is not defined	없는 이름을 불렀다	변수 이름 오타 확인, 만들기 전에 쓰지 않았는지 확인
SyntaxError: Unexpected token	문법이 깨졌다	괄호 (), 중괄호 { }, 따옴표 쌍이 맞는지 확인
TypeError: x is not a function	함수가 아닌 걸 함수처럼 불렀다	이름 오타 확인 (lenght → length)

브라우저(10~12단원)에서는 에러가 화면이 아니라 **개발자도구 Console 탭**에 빨간 글씨로 나옵니다. **F12**를 눌러 확인하세요.

에러가 나면 **빨간 글씨의 맨 윗줄과 몇 번째 줄인지**를 먼저 보세요. 에러는 실패가 아니라 안내문입니다.

혼자 공부할 때

1. 개념 파일을 위에서 아래로 읽으면서 **직접 타이핑**합니다. 복사·붙여넣기 하지 마세요.
2. **// 출력:** 이 나오면 **실행하기 전에 결과를 먼저 예상**해 보고, 실행해서 맞는지 확인합니다.
3.  **직접 해보기** 는 건너뛰지 마세요. 읽는 것과 쓰는 것은 완전히 다릅니다.
4. 자주 하는 실수 는 주석을 풀어 에러를 **눈으로 본 뒤** 다시 닫습니다. **SyntaxError** 짜리만 예외입니다. 그건 풀지 말고 읽기만 하세요 (위 설명 참고).
5. 단원이 끝나면 **연습문제** 를 풀고, 그다음 **연습문제_정답** 과 비교합니다.

구조분해와 스프레드

OPEN 09_구조분해와_스프레드

```
const first1 = colors[0];  
const second1 = colors[1];  
console.log(first1, second1);  
console.log(first2, second2);
```

01 배열 구조분해

02 객체 구조분해

03 스프레드: 배열 펼치기

04 스프레드: 객체 펼치기

05 나머지 매개변수

- 연습문제

- 심화문제

배열 구조분해

```
RUN node 개념01_배열_구조분해.js
```

배열에서 값을 꺼내 변수에 담는 일을 짧게 쓰는 문법입니다.

```
const [a, b] = [1, 2];
```

왼쪽의 대괄호는 '배열을 만드는 것'이 아니라 '모양을 맞춰 꺼내는 틀'입니다.

하나씩 꺼내기 vs 구조분해

섹션 1

```
const colors = ["빨강", "파랑", "노랑"];
```

— 지금까지 하던 방법

```
const first1 = colors[0];
const second1 = colors[1];
console.log(first1, second1);
```

```
출력      빨강 파랑
```

구조분해 · 한 줄이면 됩니다

```
const [first2, second2] = colors;
console.log(first2, second2);
```

```
출력      빨강 파랑
```

왼쪽에 적은 이름 순서대로 배열의 값이 담깁니다.

```
const [ first2 , second2 ] = colors;
      ↑       ↑
      colors[0] colors[1]
```

이름은 마음대로 지어도 됩니다. 위치만 맞으면 됩니다.

```
const [a, b, c] = colors;
console.log(a, b, c);
```

출력 빨강 파랑 노랑

필요한 만큼만 꺼내도 됩니다.

```
const [onlyFirst] = colors;
console.log(onlyFirst);
```

출력 빨강

▹ 직접 해보기 1

[10, 20] 을 구조분해로 x, y 에 담아 출력해 보세요.

개수가 안 맞으면

섹션 2

```
const two = ["하나", "둘"];
```

왼쪽이 더 많으면 남은 것은 `undefined` 입니다. 에러가 아닙니다.

```
const [x1, y1, z1] = two;
console.log(x1, y1, z1);
```

출력 하나 둘 undefined

기본값을 정해 둘 수 있습니다.

```
const [x2, y2, z2 = "셋"] = two;
console.log(x2, y2, z2);
```

출력 하나 둘 셋

값이 있으면 기본값은 무시됩니다.

```
const [x3 = "기본값"] = two;
console.log(x3);
```

출력 하나

주의 · `undefined` 일 때만 기본값이 쓰입니다. `null` 은 그대로 들어갑니다.

```
const [n1 = "기본값"] = [null];
console.log(n1);
```

출력 null

▹ 직접 해보기 2

["사과"]에서 두 번째 값의 기본값을 "없음"으로 꺼내 보세요.

건너뛰기

섹션 3

```
const week = ["월", "화", "수", "목", "금"];
```

셈표만 남기면 그 자리를 건너뛸니다.

```
const [mon, , wed] = week;  
console.log(mon, wed);
```

출력 월 수

셈표 두 개면 두 칸을 건너뛸니다.

```
const [first3, , , fourth] = week;  
console.log(first3, fourth);
```

출력 월 목

건너뛰기가 많아지면 읽기 어렵습니다. 세 칸 이상이면 그냥 인덱스를 쓰세요.

```
console.log(week[0], week[4]);
```

출력 월 금

▹ 직접 해보기 3

week에서 "화"와 "금"만 꺼내 보세요.

나머지 모으기 (...)

섹션 4

마지막에 ...을 붙이면 남은 값들이 배열로 모입니다.

```
const scores = [95, 88, 76, 64, 50];  
  
const [top, ...others] = scores;  
console.log(top);
```

출력 95

```
console.log(others);
```


출력 [88, 76, 64, 50]

```
const [gold, silver, ...rest] = scores;
console.log(gold, silver);
```

출력 95 88

```
console.log(rest);
```

출력 [76, 64, 50]

남은 게 없으면 빈 배열입니다.

```
const [only, ...nothing] = [1];
console.log(only, nothing);
```

출력 1 []

규칙 · ... 은 반드시 맨 마지막에 와야 합니다.

이런 실수를 합니다

```
const [...head, last] = scores;
```

SyntaxError: Rest element must be last element

▹ 직접 해보기 4

[1, 2, 3, 4] 에서 첫 값과 나머지를 나눠 출력해 보세요.

값 교환

섹션 5

구조분해로 두 변수의 값을 한 줄에 맞바꿀 수 있습니다.

```
let p = "왼쪽";
let q = "오른쪽";

[p, q] = [q, p];

console.log(p, q);
```

출력 오른쪽 왼쪽

구조분해가 없던 시절에는 임시 변수가 필요했습니다.

```
let m = 1;
let n = 2;
const temp = m;
m = n;
n = temp;
console.log(m, n);
```

출력 2 1

주의 · 위처럼 `const` 없이 쓸 때는 앞줄이 반드시 세미콜론으로 끝나야 합니다.

세미콜론을 빼먹으면 대괄호가 앞 줄과 한 문장으로 이어져 버립니다. 문법 검사는 통과하는데 실행할 때 엉뚱한 에러가 나서 더 찾기 어렵습니다.

▸ 직접 해보기

5

변수 두 개를 만들어 값을 바꿔 보세요.

실제로 쓰이는 곳

섹션 6

[1] `Object.entries` 순회 — 07단원에서 미리 봤습니다

```
const user = { name: "김민준", age: 20 };

for (const [key, value] of Object.entries(user)) {
  console.log(`${key}: ${value}`);
}
```

출력 name: 김민준
age: 20

구조분해가 없다면 `entry[0]`, `entry[1]` 이라고 써야 합니다.

[2] 함수가 값 두 개를 돌려줄 때

```
function getMinMax(numbers) {
  const sorted = numbers.slice().sort((a, b) => a - b); // 원본 보호 (06단원 개념03)
  return [sorted[0], sorted[sorted.length - 1]];
}

const [min, max] = getMinMax([5, 1, 9, 3]);
console.log(min, max);
```

출력 1 9

05단원에서 “여러 값은 `return` 할 수 없다” 고 했는데, 배열에 담아 돌려주고 구조분해로 받으면 됩니다.

[3] split 결과 받기

```
const dateText = "2026-08-11";
const [year, month, day] = dateText.split("-");
console.log(`${year}년 ${month}월 ${day}일`);
```

출력 2026년 08월 11일

[4] 배열 메소드 콜백에서

```
const pairs = [
  ["사과", 1000],
  ["바나나", 2000],
];
pairs.forEach(([name, price]) => {
  console.log(`${name} ${price}원`);
});
```

출력 사과 1000원
 바나나 2000원

자주 하는 실수

섹션 7

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

왼쪽 대괄호를 배열 만들기로 착각

```
const [1, 2] = colors;
```

SyntaxError.

왼쪽에는 '변수 이름'만 적습니다.

이런 실수를 합니다

순서를 헷갈림

```
const [wrong1, wrong2] = ["첫째", "둘째"];
console.log(wrong1);
```

출력 첫째

이름을 second, first 로 지어도 값은 위치대로 들어갑니다. 이름에 속지 마세요. 객체 구조분해(개념02)는 이름으로 찾습니다.

이런 실수를 합니다

... 을 앞이나 중간에 쓰기 (섹션 4에서 봤습니다)

이런 실수를 합니다

값 교환에서 세미콜론 빠뜨리기

```
let s = 1  
let t = 2
```

s, t = [t, s]

이런 실수를 합니다

`let t = 2[s, t] = [t, s]` 한 문장으로 이어져 버립니다. 문법 에러가 아니라서 더 헷갈립니다. 실행할 때 이런 에러가 납니다. ReferenceError: Cannot access 't' before initialization 앞줄이 `console.log(...)` 였다면 TypeError 가 납니다. 세미콜론을 붙이면 전부 해결됩니다.

이런 실수를 합니다

배열이 아닌 것을 구조분해

```
const [k] = 5;
```

TypeError: 5 is not iterable

숫자는 하나씩 꺼낼 수 있는 형태가 아닙니다. 문자열은 됩니다. 한 글자씩 꺼낼 수 있으니까요.

```
const [char1, char2] = "안녕";  
console.log(char1, char2);
```

출력

안녕

정리

- 1 `const [a, b] = 배열; —` 위치대로 값이 담긴다.
- 2 없으면 `undefined`. = 로 기본값을 정할 수 있다.
- 3 쉼표만 남기면 건너뛴다.
- 4 `const [first, ...rest] —` 나머지를 배열로 모은다. ... 은 맨 뒤에만.
- 5 `[a, b] = [b, a]` 로 값을 맞바꾼다.
- 6 `Object.entries` 순회, 여러 값 `return`, `split` 결과에 자주 쓴다.

직접 해보기 정답

```

1  const [x, y] = [10, 20];
   console.log(x, y);           // 출력: 10 20

2  const [f, s = "없음"] = ["사과"];
   console.log(f, s);           // 출력: 사과 없음

3  const [, tue, , , fri] = week;
   console.log(tue, fri);       // 출력: 화 금

4  const [head, ...tail] = [1, 2, 3, 4];
   console.log(head);           // 출력: 1
   console.log(tail);           // 출력: [ 2, 3, 4 ]

5  let one = "A";
   let other = "B";

```

`one, other = [other, one];`

```
console.log(one, other);    // 출력: B A
```

객체 구조분해

RUN node 개념02_객체_구조분해.js

배열 구조분해는 '위치'로 꺼냈습니다. 객체 구조분해는 '이름'으로 꺼냅니다. 그래서 순서가 상관없습니다.

```
const { name, age } = user;
```

실무에서 배열 구조분해보다 훨씬 많이 씁니다. React 를 배우면 매 줄 나옵니다. (React 는 이 자료를 마친 뒤에 배울, 화면 만들기를 훨씬 편하게 해 주는 도구입니다)

이름으로 꺼내기

섹션 1

```
const user = {
  name: "김민준",
  age: 20,
  city: "부산",
};
```

— 지금까지 하던 방법

```
const name1 = user.name;
const age1 = user.age;
console.log(name1, age1);
```

출력 김민준 20

— 구조분해

```
const { name, age } = user;
console.log(name, age);
```

출력 김민준 20

왼쪽 중괄호 안의 이름은 '속성 이름'과 같아야 합니다. 순서는 상관없습니다.

```
const { city, name: name2 } = user;
console.log(city, name2);
```

출력 부산 김민준

없는 속성을 꺼내면 `undefined` 입니다.

```
const { email } = user;
console.log(email);
```

출력 undefined

▸ 직접 해보기 1

{ title: "입문서", price: 25000 } 에서 title 과 price 를 구조분해로 꺼내 보세요.

다른 이름으로 받기

섹션 2

콜론을 쓰면 다른 이름으로 받을 수 있습니다.

```
const { name: userName, age: userAge } = user;
console.log(userName, userAge);
```

출력 김민준 20

읽는 법: "name 속성을 꺼내서 userName 이라는 변수에 담아라"

```
const { name : userName } = user;
      ^^^^  ^^^^^^^
      속성 이름   내가 쓸 변수 이름
```

순서를 헷갈리기 쉽습니다. 왼쪽이 원래 이름, 오른쪽이 새 이름입니다.

이미 같은 이름의 변수가 있을 때 유용합니다.

```
const product = { name: "이어폰", price: 89000 };
const { name: productName } = product;
console.log(userName, productName);
```

출력 김민준 이어폰

name 이라는 이름을 이미 위에서 썼기 때문에 다른 이름이 필요했습니다.

▸ 직접 해보기 2

product 의 price 를 productPrice 라는 이름으로 꺼내 보세요.

```
const config = { theme: "dark" };

const { theme, fontSize = 14 } = config;
console.log(theme, fontSize);
```

출력 dark 14

fontSize 가 없어서 기본값 14가 쓰였습니다.

값이 있으면 기본값은 무시됩니다.

```
const config2 = { theme: "light", fontSize: 20 };
const { fontSize: size2 = 14 } = config2;
console.log(size2);
```

출력 20

이름 바꾸기와 기본값을 함께 쓸 수도 있습니다.

```
const { language: lang = "ko" } = config;
console.log(lang);
```

출력 ko

주의 · undefined 일 때만 기본값이 쓰입니다.

```
const { theme: t = "기본테마" } = { theme: null };
console.log(t);
```

출력 null

▸ 직접 해보기 3

빈 객체에서 count 를 기본값 0 으로 꺼내 보세요.

나머지 모으기

섹션 4

```
const fullUser = {
  id: 1,
  name: "이서연",
  age: 22,
  city: "서울",
};

const { id, ...profile } = fullUser;
console.log(id);
```

출력 1

```
console.log(profile);
```

출력 { name: '이서연', age: 22, city: '서울' }

“id 만 빼고 나머지” 를 만들 때 씁니다. 원본은 그대로입니다.

```
console.log(fullUser);
```

출력 { id: 1, name: '이서연', age: 22, city: '서울' }

비밀번호처럼 빼고 넘겨야 할 값이 있을 때 자주 씁니다.

```
const account = { userId: "abc", password: "1234", nickname: "서연" };
const { password, ...safeAccount } = account;
console.log(safeAccount);
```

출력 { userId: 'abc', nickname: '서연' }

▢ 직접 해보기 4

account 에서 userId 만 빼고 나머지를 모아 보세요.

함수 매개변수에서 쓰기

섹션 5

이게 객체 구조분해의 진짜 쓸모입니다.

구조분해 없이 · 매번 user. 을 붙여야 합니다

```
function introduce1(user) {  
  console.log(`${user.name} / ${user.age}세 / ${user.city}`);  
}  
introduce1(user);
```

출력 김민준 / 20세 / 부산

— 매개변수에서 바로 구조분해

```
function introduce2({ name, age, city }) {  
  console.log(`${name} / ${age}세 / ${city}`);  
}  
introduce2(user);
```

출력 김민준 / 20세 / 부산

함수 안이 깔끔해지고, 매개변수만 봐도 “이 함수가 무엇을 쓰는지” 보입니다.

화살표 함수에서도 똑같습니다.

```
const introduce3 = ({ name, city }) => console.log(`${name}은 ${city} 거주`);  
introduce3(user);
```

출력 김민준은 부산 거주

기본값도 쓸 수 있습니다.

```
function createUser({ name, role = "일반회원" }) {  
  console.log(`${name} (${role})`);  
}  
createUser({ name: "박지훈" });
```

출력 박지훈 (일반회원)

```
createUser({ name: "최유진", role: "관리자" });
```

출력 최유진 (관리자)

05단원에서 “매개변수가 세 개를 넘어가면 순서를 외우기 어렵다” 고 했습니다. 객체로 묶어 넘기면 순서를 몰라도 되고, 이름이 보여서 읽기도 좋습니다.

```
function order1(menu, size, ice, count) {
  console.log(menu, size, ice, count);
}
order1("라떼", "L", true, 2);
```

출력 라떼 L true 2

부르는 쪽만 봐서는 `true` 가 뭔지 알 수 없습니다.

```
function order2({ menu, size, ice, count }) {
  console.log(menu, size, ice, count);
}
order2({ menu: "라떼", size: "L", ice: true, count: 2 });
```

출력 라떼 L true 2

이쪽은 부르는 쪽만 봐도 무슨 값인지 다 보입니다.

▹ 직접 해보기 5

{ name, price } 를 받아 "이름 가격원" 을 출력하는 함수를 매개변수 구조분해로 만들어 보세요.

배열 메소드 콜백에서 쓰기

섹션 6

```
const products = [
  { name: "아메리카노", price: 4000 },
  { name: "케이크", price: 6000 },
];
```

— 구조분해 없이

```
products.forEach((p) => console.log(`${p.name} ${p.price}원`));
```

출력 아메리카노 4000원
 케이크 6000원

— 구조분해

```
products.forEach(({ name, price }) => console.log(`${name} ${price}원`));
```

출력 아메리카노 4000원
 케이크 6000원

map, filter 에서도 같습니다.

```
console.log(products.map(({ name }) => name));
```

출력 ['아메리카노', '케이크']

```
console.log(products.filter(({ price }) => price >= 5000).map(({ name }) => name));
```

출력 ['케이크']

중첩 객체 구조분해

섹션 7

```
const order = {  
  id: 100,  
  customer: {  
    name: "정하늘",  
    phone: "010-0000-0000",  
  },  
};
```

★ 여기서 콜론(:)이 두 가지 뜻으로 쓰입니다. 헷갈리기 쉬우니 먼저 읽으세요.

```
const { customer: { name: customerName } } = order;
```

①

②

① 바깥 콜론은 '이름 바꾸기'가 아닙니다. "customer 안으로 더 들어가라"는 뜻입니다.

오른쪽이 이름이 아니라 또 다른 중괄호 { }라서 그렇게 읽습니다.

② 안쪽 콜론만 섹션 2에서 배운 '이름 바꾸기'입니다. name을 customerName으로.

읽는 법: "order에서 customer 안으로 들어가, 그 안의 name을 customerName에 담아라"

참고로 이렇게 하면 customer라는 변수는 만들어지지 않습니다. 들어가는 통로로만 쓰였기 때문입니다.

```
const {  
  customer: { name: customerName },  
} = order;  
console.log(customerName);
```

출력 정하늘

중첩이 깊어지면 읽기 어렵습니다. 두 단계까지만 쓰고, 더 깊으면 나눠 쓰는 게 낫습니다.

```
const { customer } = order;
const { phone } = customer;
console.log(phone);
```

출력 010-0000-0000

주의 · 중간이 없으면 에러가 납니다.

이런 실수를 합니다

```
const { seller: { name: sellerName } } = order;
```

TypeError: Cannot read properties of **undefined** 07단원의 ?. 는 구조분해에서 쓸 수 없습니다.
기본값을 주세요.

```
const { seller: { name: sellerName } = {} } = order;
console.log(sellerName);
```

출력 undefined

자주 하는 실수

섹션 8

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

이름을 틀리게 씀

```
const { nmae } = user;
console.log(nmae);
```

출력 undefined

객체 구조분해는 '이름'으로 찾습니다. 오타가 나면 조용히 **undefined** 입니다.

이런 실수를 합니다

이름 바꾸기 순서를 반대로

```
const { userName: myName } = user;
```

user 에 userName 이라는 속성이 없어서 myName 이 **undefined** 가 됩니다. 왼쪽이 원래 속성 이름, 오른쪽이 내가 쓸 새 이름입니다.

이런 실수를 합니다

배열에 중괄호를 씌

```
const arr = [1, 2, 3];
const { 0: firstItem } = arr;
console.log(firstItem);
```

출력 1

되긴 하지만 이상한 코드입니다. 배열은 대괄호로 구조분해하세요.

이런 실수를 합니다

const 없이 객체 구조분해

```
{ name } = user;
```

SyntaxError.

줄 맨 앞의 중괄호를 '블록'으로 해석합니다. `const` 나 `let` 을 붙이거나, 소괄호로 감싸야 합니다: `({ name } = user);`

이런 실수를 합니다

null 이나 undefined 를 구조분해

```
const { a } = null;
```

TypeError: Cannot destructure property 'a' of 'null'

값이 있는지 확실하지 않으면 기본값을 주세요: `= {}`

정리

- 1 `const { name, age } = 객체;` — 이름으로 꺼낸다. 순서 무관.
- 2 `const { name: newName } = 객체;` — 왼쪽이 원래 이름, 오른쪽이 새 이름.
- 3 `const { size = 14 } = 객체;` — `undefined` 일 때만 기본값.
- 4 `const { id, ...rest } = 객체;` — "id 만 빼고 나머지".
- 5 `function f({ name, age })` — 매개변수에서 바로 구조분해. 아주 많이 쓴다.
- 6 배열 메소드 콜백에서도 `{ name }` ⇒ `...` 로 쓴다.

직접 해보기 정답

- 1

```
const book = { title: "입문서", price: 25000 };  
const { title, price } = book;  
console.log(title, price);           // 출력: 입문서 25000
```
- 2

```
const { price: productPrice } = product;  
console.log(productPrice);           // 출력: 89000
```
- 3

```
const { count = 0 } = {};  
console.log(count);                  // 출력: 0
```
- 4

```
const { userId, ...withoutId } = account;  
console.log(withoutId);  
// 출력: { password: '1234', nickname: '서연' }
```
- 5

```
const printItem = ({ name, price }) => console.log(`${name} ${price}원`);  
printItem({ name: "쿠키", price: 3000 });  
// 출력: 쿠키 3000원
```

스프레드: 배열 펼치기

RUN node 개념03_스프레드_배열.js

왜 배우나? 지금까지 “원본을 바꾸지 마세요” 라는 말을 여러 번 들었습니다. `sort.reverse` 는 원본을 바꿨고, 배열을 그냥 대입하면 복사가 아니었죠. 그때마다 `slice()` 를 썼습니다. 스프레드는 그 일을 더 짧게 하고, 합치기와 “함수에 날개로 넘기기” 까지 기호 하나로 해결합니다. 06단원에서 못 했던 `Math.max(배열)` 도 이걸로 됩니다.

점 세 개(...)를 배열 앞에 붙이면 그 안의 값들이 날개로 펼쳐집니다.

```
const numbers = [1, 2, 3];
```

`...numbers` → 1, 2, 3 이 하나씩 놓인 것과 같음

구조분해의 ... 은 ‘모으기’였고, 여기서는 ‘펼치기’입니다. 위치로 구분합니다. 자세한 건 개념05에서 정리합니다.

펼쳐 보기

섹션 1

```
const numbers = [1, 2, 3];
```

```
console.log(numbers);
```

출력 [1, 2, 3]

```
console.log(...numbers);
```

출력 1 2 3

대괄호가 사라지고 값이 날개로 놓였습니다. `console.log(1, 2, 3)` 이라고 쓴 것과 같습니다.

새 배열의 재료로 쓰면 그대로 다시 배열이 됩니다.

```
console.log([...numbers]);
```

출력 [1, 2, 3]

앞뒤에 값을 더할 수 있습니다.

```
console.log([0, ...numbers, 4]);
```


출력 [0, 1, 2, 3, 4]

▹ 직접 해보기

1

["b", "c"] 앞에 "a" 를 붙인 새 배열을 만들어 보세요.

복사하기

섹션 2

06·08단원에서 이미 썼던 방법입니다. 이제 원리를 알 수 있습니다.

```
const original = [1, 2, 3];
```

잘못된 복사 · 같은 배열을 가리키는 이름만 하나 더 생깁니다

```
const notCopy = original;
notCopy.push(99);
console.log(original);
```

출력 [1, 2, 3, 99]

원본까지 바뀌었습니다.

— 진짜 복사

```
const original2 = [1, 2, 3];
const realCopy = [...original2];
realCopy.push(99);

console.log(realCopy);
```

출력 [1, 2, 3, 99]

```
console.log(original2);
```

출력 [1, 2, 3]

원본이 지켜졌습니다.

`slice()` 로도 복사할 수 있습니다. 결과는 같습니다.

```
const original3 = [1, 2, 3];
console.log(...original3);
```

출력 [1, 2, 3]

```
console.log(original3.slice());
```

출력 [1, 2, 3]

원본을 바꾸는 메소드(sort, reverse, splice) 앞에 붙여 쓰면 안전합니다.

```
const scores = [30, 10, 20];  
console.log([...scores].sort((a, b) => a - b));
```

출력 [10, 20, 30]

```
console.log(scores);
```

출력 [30, 10, 20]

⇒ 직접 해보기 2

배열을 복사한 뒤 복사본만 뒤집어 보세요. (reverse)

합치기

섹션 3

```
const group1 = ["김민준", "이서연"];  
const group2 = ["박지훈", "최유진"];  
  
console.log([...group1, ...group2]);
```

출력 ['김민준', '이서연', '박지훈', '최유진']

06단원의 concat 과 결과가 같습니다.

```
console.log(group1.concat(group2));
```

출력 ['김민준', '이서연', '박지훈', '최유진']

스프레드는 순서를 자유롭게 섞을 수 있어서 더 편합니다.

```
console.log([...group2, "정하늘", ...group1]);
```

출력 ['박지훈', '최유진', '정하늘', '김민준', '이서연']

원본은 둘 다 그대로입니다.

```
console.log(group1);
```

출력 ['김민준', '이서연']

▹ 직접 해보기 3

배열 두 개 사이에 “구분” 이라는 값을 끼워 합쳐 보세요.

함수 인자로 펼치기

섹션 4

`Math.max` 는 배열을 통째로 못 받습니다. 값을 하나씩 받습니다.

```
console.log(Math.max(3, 7, 5));
```

출력 7

```
const temps = [23, 31, 18, 27];
```

```
console.log(Math.max(temps));
```

출력 NaN

배열을 통째로 넘겨서 계산이 안 됩니다.

```
console.log(Math.max(...temps));
```

출력 31

스프레드로 펼치면 `Math.max(23, 31, 18, 27)` 이 됩니다.

```
console.log(Math.min(...temps));
```

출력 18

06단원에서 반복문으로 구하던 최댓값이 한 줄이 되었습니다.

우리가 만든 함수에도 똑같이 쓸 수 있습니다.

```
function sum3(a, b, c) {
  return a + b + c;
}
const args = [1, 2, 3];
console.log(sum3(...args));
```

출력 6

▹ 직접 해보기 4

[45, 88, 62] 의 최댓값과 최솟값을 출력해 보세요.

문자열은 글자 하나씩 꺼낼 수 있는 형태라 스프레드가 됩니다.

```
console.log(..."안녕하세요");
```

출력 ['안', '녕', '하', '세', '요']

split("") 과 결과가 같습니다.

```
console.log("안녕하세요".split(""));
```

출력 ['안', '녕', '하', '세', '요']

글자를 뒤집을 때 이렇게 씁니다.

```
const word = "자바스크립트";
console.log([...word].reverse().join(""));
```

출력 트립크스바자

▸ 직접 해보기 5

"hello" 를 뒤집어 출력해 보세요.

원본을 안 바꾸고 값 추가하기

React 같은 곳에서는 "원본을 바꾸지 말고 새 배열을 만들어라" 가 규칙입니다. 스프레드가 그 규칙을 지키는 도구입니다.

```
const todos = ["장보기", "청소"];
```

— 원본을 바꾸는 방법

```
todos.push("운동");
```

— 새 배열을 만드는 방법

```
const newTodos = [...todos, "운동"];
console.log(newTodos);
```

출력 ['장보기', '청소', '운동']

```
console.log(todos);
```

```
출력      [ '장보기', '청소' ]
```

앞에 추가

```
console.log(["급한 일", ...todos]);
```

```
출력      [ '급한 일', '장보기', '청소' ]
```

하나 빼고 새 배열 (filter 와 조합)

```
console.log(todos.filter((t) => t !== "청소"));
```

```
출력      [ '장보기' ]
```

하나 바꾸고 새 배열 (map 과 조합)

```
console.log(todos.map((t) => (t === "청소" ? "대청소" : t)));
```

```
출력      [ '장보기', '대청소' ]
```

08단원의 map / filter 도 전부 새 배열을 돌려줍니다. “원본을 안 건드린다” 는 같은 흐름입니다.

얕은 복사 — 한 겹만 복사된다

섹션 7

07단원 개념02 섹션6, 개념05 섹션7에서 봤던 함정입니다.

```
const users = [
  { name: "김민준", age: 20 },
  { name: "이서연", age: 22 },
];

const copied = [...users];
```

배열 자체는 새로 만들어졌습니다.

```
copied.push({ name: "박지훈", age: 28 });
console.log(users.length);
```

```
출력      2
```

```
console.log(copied.length);
```

```
출력      3
```

하지만 안의 객체는 원본과 같은 것을 가리킵니다.

```
copied[0].age = 99;  
console.log(users[0].age);
```

출력 99

원본까지 바뀌었습니다.

스프레드는 '한 겹만' 복사합니다. 이것을 얇은 복사라고 합니다. 안쪽 객체까지 복사하려면 map 과 함께 씁니다.

```
const users2 = [{ name: "정하늘", age: 30 }];  
const deepish = users2.map((u) => ({ ...u }));  
  
deepish[0].age = 99;  
console.log(users2[0].age);
```

출력 30

이번엔 원본이 지켜졌습니다.

더 깊은 구조라면 structuredClone 을 쓸 수도 있습니다.

```
const nested = [{ a: { b: 1 } }];  
const cloned = structuredClone(nested);  
cloned[0].a.b = 99;  
console.log(nested[0].a.b);
```

출력 1

structuredClone 은 브라우저와 node 가 기본으로 제공하는 함수입니다. 안쪽까지 통째로 복사해 줍니다. 깊은 구조를 다룰 때만 쓰면 됩니다.

▫ 직접 해보기

6

객체가 든 배열을 만들고, 겉만 복사했을 때와 map 으로 안쪽까지 복사했을 때 원본이 어떻게 다른지 확인해 보세요. `const list = [{ n: 1 }];`

자주 하는 실수

섹션 8

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

점을 두 개나 네 개로 씬 → **SyntaxError**

```
console.log(...numbers);
```

정확히 세 개입니다.

이런 실수를 합니다

배열이 아닌 것을 펼치기

```
console.log(...123);
```

```
TypeError: 123 is not iterable
```

숫자는 펼칠 수 없습니다. 배열·문자열은 됩니다.

이런 실수를 합니다

객체를 배열 대괄호 안에 펼치기

```
console.log(...{ a: 1 });
```

```
TypeError: {(intermediate value)} is not iterable
```

객체는 중괄호 안에서만 펼칠 수 있습니다. (개념04)

이런 실수를 합니다

얕은 복사인 줄 모르고 원본을 망가뜨림 (섹션 7)

이런 실수를 합니다

합칠 때 스프레드를 빠뜨림

```
console.log([group1, group2]);
```

```
출력      [ [ '김민준', '이서연' ], [ '박지훈', '최유진' ] ]
```

배열 안에 배열이 들어갔습니다. ... 을 붙여야 펼쳐집니다.

정리

- 1 [...배열] — 값들을 날개로 펼친다.
- 2 복사: `const copy = [...원본]` (원본 보호)
- 3 합치기: `[...a, ...b]`
- 4 함수 인자로 펼치기: `Math.max(...배열)`
- 5 문자열도 펼쳐진다: `[..."안녕"]`
- 6 한 겹만 복사된다(얕은 복사). 안쪽 객체는 `map((o) => ({ ...o }))` 로.

직접 해보기 정답

```
1 console.log(["a", ...["b", "c"]]);  
  // 출력: [ 'a', 'b', 'c' ]  
  
2 const arr = [1, 2, 3];  
  console.log([...arr].reverse()); // 출력: [ 3, 2, 1 ]  
  console.log(arr);                // 출력: [ 1, 2, 3 ]  
  
3 console.log([...group1, "구분", ...group2]);  
  // 출력: [ '김민준', '이서연', '구분', '박지훈', '최유진' ]  
  
4 const list = [45, 88, 62];  
  console.log(Math.max(...list)); // 출력: 88  
  console.log(Math.min(...list)); // 출력: 45  
  
5 console.log([... "hello"].reverse().join(""));  
  // 출력: olleh  
  
6 const list = [{ n: 1 }];
```

```
const shallow = [...list]; // 겹만 복사  
shallow[0].n = 99;  
console.log(list[0].n);    // 출력: 99 ← 원본이 바뀜
```

```
const list2 = [{ n: 1 }];  
const deep = list2.map((o) => ({ ...o })); // 안쪽까지 복사  
deep[0].n = 99;  
console.log(list2[0].n);    // 출력: 1 ← 원본이 지켜짐
```


스프레드: 객체 펼치기

RUN node 개념04_스프레드_객체.js

배열과 문법이 같습니다. 중괄호 안에서 쓴다는 것만 다릅니다.

```
const copy = { ...원본 };
```

07단원에서 “복사할 때 이걸 쓰라” 고 미리 봤던 문법입니다.

객체 복사

섹션 1

```
const user = { name: "김민준", age: 20 };
```

잘못된 복사 · 이름표만 하나 더 붙는 것입니다

```
const notCopy = user;
notCopy.age = 99;
console.log(user.age);
```

출력 99

원본까지 바뀌었습니다.

— 진짜 복사

```
const user2 = { name: "이서연", age: 22 };
const realCopy = { ...user2 };
realCopy.age = 99;

console.log(realCopy);
```

출력 { name: '이서연', age: 99 }

```
console.log(user2);
```

출력 { name: '이서연', age: 22 }

⇒ 직접 해보기 1

객체를 복사한 뒤 복사본만 바꿔 보세요.

시작하기 전에 짧은 표기법 하나만 보고 갑시다. 앞으로 자주 나옵니다.

넣을 값이 담긴 변수 이름과 속성 이름이 같으면, 한 번만 써도 됩니다.

```
const id = 1;
const count = 2;

console.log({ id: id, count: count });
```

```
출력      { id: 1, count: 2 }
```

```
console.log({ id, count });
```

```
출력      { id: 1, count: 2 }
```

위 두 줄은 완전히 같습니다. 아래쪽을 '속성 축약' 이라고 합니다. 13단원 종합 실습에서 이 모양이 계속 나오니 눈에 익혀 두세요.

```
const base = { name: "박지훈", age: 28 };
```

추가

```
console.log({ ...base, city: "대구" });
```

```
출력      { name: '박지훈', age: 28, city: '대구' }
```

수정 — 같은 이름을 다시 쓰면 덮어씁니다

```
console.log({ ...base, age: 30 });
```

```
출력      { name: '박지훈', age: 30 }
```

원본은 그대로입니다.

```
console.log(base);
```

```
출력      { name: '박지훈', age: 28 }
```

“원본을 안 바꾸고 하나만 고친 새 객체” 를 만드는 방법입니다. React 에서 상태를 바꿀 때 이 형태를 계속 씁니다.

▸ 직접 해보기 2

base 에 job 속성을 더한 새 객체를 만들어 보세요.

순서가 결과를 바꾼다

섹션 3

```
const defaults = { theme: "light", fontSize: 14, lang: "ko" };
const userSetting = { theme: "dark" };
```

뒤에 오는 것이 이깁니다.

```
console.log({ ...defaults, ...userSetting });
```

```
출력      { theme: 'dark', fontSize: 14, lang: 'ko' }
```

사용자 설정이 기본값을 덮어썼습니다.

순서를 바꾸면 반대가 됩니다.

```
console.log({ ...userSetting, ...defaults });
```

```
출력      { theme: 'light', fontSize: 14, lang: 'ko' }
```

기본값이 사용자 설정을 덮어써 버렸습니다. 원하는 결과가 아닙니다.

규칙: “기본값을 먼저, 덮어쓸 것을 나중에”

날개 속성도 같습니다. 나중에 쓴 것이 이깁니다.

```
console.log({ age: 1, ...base });
```

```
출력      { age: 28, name: '박지훈' }
```

```
console.log({ ...base, age: 1 });
```

```
출력      { name: '박지훈', age: 1 }
```

▸ 직접 해보기 3

defaults 를 기본으로 두고 fontSize 만 20 으로 바꾼 새 객체를 만들어 보세요.

객체 합치기

섹션 4

```
const info = { name: "최유진", age: 25 };
const contact = { phone: "010-0000-0000", email: "test@example.com" };

console.log({ ...info, ...contact });
```

```
출력      {
    name: '최유진',
    age: 25,
    phone: '010-0000-0000',
    email: 'test@example.com'
}
```

배열은 [...a, ...b], 객체는 { ...a, ...b } 입니다. 괄호만 다릅니다.

세 개 이상도 됩니다.

```
console.log({ ...info, ...contact, level: 3 });
```

```
출력      {
    name: '최유진',
    age: 25,
    phone: '010-0000-0000',
    email: 'test@example.com',
    level: 3
}
```

구조분해와 짝지어 쓰기

섹션 5

“하나만 빼고 나머지” 는 구조분해로, “하나만 바꾸기” 는 스프레드로 합니다.

```
const account = { id: 1, password: "1234", nickname: "유진" };
```

빼기 (개념02)

```
const { password, ...safe } = account;
console.log(safe);
```

```
출력      { id: 1, nickname: '유진' }
```

바꾸기 (이 파일)

```
console.log({ ...account, nickname: "새닉네임" });
```

```
출력      { id: 1, password: '1234', nickname: '새닉네임' }
```

배열 안의 객체 하나만 바꾸기 — 실무에서 아주 많이 쓰는 조합입니다.

```
const todos = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "청소", done: false },
];

const updated = todos.map((todo) => (todo.id === 2 ? { ...todo, done: true } :
todo));

console.log(updated);
```

```
출력      [
    { id: 1, text: '장보기', done: false },
    { id: 2, text: '청소', done: true }
  ]
```

```
console.log(todos[1].done);
```

```
출력      false
```

원본은 그대로입니다.

읽는 법: "id 가 2면 그 객체를 복사해서 done 만 true 로, 아니면 그대로"

⇒ 직접 해보기 4

todos 에서 id 가 1인 항목의 text 를 "대청소" 로 바꾼 새 배열을 만들어 보세요.

얕은 복사 — 여기서도 한 겹뿐

섹션 6

09

```
const order = {
  id: 100,
  customer: { name: "정하늘", city: "서울" },
};

const orderCopy = { ...order };
```

겉의 속성은 따로입니다.

```
orderCopy.id = 200;
console.log(order.id);
```

```
출력      100
```

하지만 안의 객체는 같은 것을 가리킵니다.

```
orderCopy.customer.name = "바뀐이름";
console.log(order.customer.name);
```

출력 바뀐이름

안쪽까지 복사하려면 안쪽도 펼쳐야 합니다.

```
const order2 = {
  id: 100,
  customer: { name: "정하늘", city: "서울" },
};

const deepCopy = {
  ...order2,
  customer: { ...order2.customer },
};

deepCopy.customer.name = "바뀐이름";
console.log(order2.customer.name);
```

출력 정하늘

구조가 깊으면 structuredClone 이 편합니다.

```
const order3 = { a: { b: { c: 1 } } };
const cloned = structuredClone(order3);
cloned.a.b.c = 99;
console.log(order3.a.b.c);
```

출력 1

자주 하는 실수

섹션 7

이런 실수를 합니다

배열을 중괄호 안에 펼치기

```
console.log({ ...[1, 2, 3] });
```

출력 { '0': 1, '1': 2, '2': 3 }

에러는 안 나지만 인덱스가 속성 이름인 이상한 객체가 됩니다. 배열은 대괄호 안에 펼치세요.

이런 실수를 합니다

순서를 반대로 써서 덮어쓰기가 반대로 됨 (섹션 3)

이런 실수를 합니다

얕은 복사인 줄 모르고 원본을 망가뜨림 (섹션 6)

이런 실수를 합니다

화살표 함수에서 객체를 바로 돌려줄 때 소괄호를 빼먹음

```
const toObject1 = (n) => {
  n;
};
console.log(toObject1(1));
```

출력 undefined

```
const toObject2 = (n) => ({ value: n });
console.log(toObject2(1));
```

출력 { value: 1 }

중괄호를 함수 몸통으로 해석합니다. 소괄호로 감싸세요. (08단원 개념03)

이런 실수를 합니다

null 이나 undefined 를 펼치기

```
console.log({ ...null });
```

출력 {}

에러가 나지 않고 빈 객체가 됩니다. 배열과 달리 조용히 넘어갑니다.

정리

- 1 { ...객체 } — 객체를 복사한다.
- 2 { ...객체, 속성: 값 } — 원본을 안 바꾸고 추가·수정한 새 객체를 만든다.
- 3 같은 이름이 겹치면 '나중에 쓴 것' 이 이긴다. 기본값을 먼저 쓸 것.
- 4 { ...a, ...b } 로 합친다. 배열은 [...a, ...b].
- 5 얕은 복사다. 안쪽 객체까지 지키려면 안쪽도 펼치거나 structuredClone.
- 6 map 과 함께 쓰면 "배열 안의 객체 하나만 바꾸기" 가 된다.

직접 해보기 정답

```
1  const o = { a: 1 };
   const c = { ...o };
   c.a = 99;
   console.log(c.a);      // 출력: 99
   console.log(o.a);      // 출력: 1

2  console.log({ ...base, job: "개발자" });
   // 출력: { name: '박지훈', age: 28, job: '개발자' }

3  console.log({ ...defaults, fontSize: 20 });
   // 출력: { theme: 'light', fontSize: 20, lang: 'ko' }

4  console.log(todos.map((t) => (t.id === 1 ? { ...t, text: "대청소" } : t)));
   // 출력: [
   //   { id: 1, text: '대청소', done: false },
   //   { id: 2, text: '청소', done: false }
   // ]
```


나머지 매개변수

RUN node 개념05_나머지_매개변수.js

함수가 인자를 몇 개 받을지 모를 때 씁니다.

```
function sum(...numbers) { ... }
sum(1, 2)          → numbers 는 [1, 2]
sum(1, 2, 3, 4)    → numbers 는 [1, 2, 3, 4]
```

넘어온 인자들을 '배열로 모아 줍니다'.

인자를 배열로 모으기

섹션 1

```
function showAll(...args) {
  console.log(args);
}
```

```
showAll(1, 2, 3);
```

출력 [1, 2, 3]

코드

```
showAll("가", "나");
```

```
showAll();
```

▶ 출력

▶ ['가', '나']

▶ []

배열이므로 06-08단원의 메소드를 전부 쓸 수 있습니다.

```
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
```

```
console.log(sum(1, 2));
```

출력 3

코드

```
console.log(sum(1, 2, 3, 4, 5));
```

```
console.log(sum());
```

▶ 출력

▶ 15

▶ 0

reduce 에 시작값 0 을 줬기 때문에 빈 배열도 에러가 안 납니다.

▹ 직접 해보기 1

넘어온 값들의 개수를 돌려주는 함수를 만들어 보세요.

앞에 고정 매개변수 두기

섹션 2

앞에는 정해진 매개변수를, 뒤에는 나머지를 둘 수 있습니다.

```
function introduce(name, ...hobbies) {
```

join 의 결과가 빈 문자열이면 falsy 이므로 || 로 기본값을 줍니다 (02단원)

```
  console.log(`${name}의 취미: ${hobbies.join(", ") || "(없음)"}`);
}
```

```
  introduce("김민준", "독서", "등산", "요리");
```

출력 김민준의 취미: 독서, 등산, 요리

```
  introduce("이서연", "영화");
```

출력 이서연의 취미: 영화

```
  introduce("박지훈");
```

출력 박지훈의 취미: (없음)

취미를 안 넘기면 hobbies 는 빈 배열이라 join 의 결과가 빈 문자열입니다.

규칙 · 나머지 매개변수는 반드시 맨 마지막에 하나만 쓸 수 있습니다.

이런 실수를 합니다

```
function bad(...args, last) { }
```

SyntaxError: Rest parameter must be last formal parameter

▹ 직접 해보기 2

첫 인자를 제목으로, 나머지를 목록으로 출력하는 함수를 만들어 보세요.

스프레드와 나머지 — 같은 ... 인데 반대다

섹션 3

헛갈리기 쉬운데, 어디에 쓰였는지로 구분합니다.

받는 쪽(왼쪽)에 있으면 → 모으기 (나머지)
 주는 쪽(오른쪽)에 있으면 → 펼치기 (스프레드)

모으기 · 함수를 만들 때 = 받는 쪽

```
function collect(...values) {
  return values;
}
console.log(collect(1, 2, 3));
```

출력 [1, 2, 3]

펼치기 · 함수를 부를 때 = 주는 쪽

```
const numbers = [1, 2, 3];
console.log(collect(...numbers));
```

출력 [1, 2, 3]

구조분해에서도 같습니다.

모으기 · `const` 왼쪽

```
const [head, ...tail] = [1, 2, 3, 4];
console.log(head, tail);
```

출력 1 [2, 3, 4]

펼치기 · = 오른쪽

```
const merged = [0, ...tail];
console.log(merged);
```

출력 [0, 2, 3, 4]

한 줄로 정리하면:

= 왼쪽 / function 괄호 안 → 모은다
 = 오른쪽 / 함수 호출 괄호 안 → 펼친다

배열 [10, 20, 30] 을 collect 에 펼쳐 넘겨 보세요.

둘을 같이 쓰기

섹션 4

받아서 모으고, 다시 펼쳐 넘기는 패턴입니다.

```
function logWithTime(...args) {
  console.log("[기록]", ...args);
}

logWithTime("사용자", "김민준", "로그인");
```

출력 [기록] 사용자 김민준 로그인

args 를 그냥 넘기면 배열째로 찍힙니다.

```
function logWrong(...args) {
  console.log("[기록]", args);
}

logWrong("사용자", "김민준");
```

출력 [기록] ['사용자', '김민준']

함수를 감싸서 기능을 덧붙일 때 이 패턴을 씁니다.

```
function multiply(a, b) {
  return a * b;
}

function loggedMultiply(...args) {
  console.log(`multiply(${args.join(", ")}) 호출됨`);
  return multiply(...args);
}

console.log(loggedMultiply(3, 4));
```

출력 multiply(3, 4) 호출됨
 12

실전에서 쓰는 모습

섹션 5

— 평균 구하기

```
function average(...scores) {
  if (scores.length === 0) {
    return 0; // 조기 반환으로 0으로 나누는 것을 막습니다
  }
  const total = scores.reduce((acc, n) => acc + n, 0);
  return total / scores.length;
}

console.log(average(90, 80, 70));
```

출력 80

```
console.log(average());
```

출력 0

— 최댓값

```
function max(...numbers) {
  return Math.max(...numbers);
}

console.log(max(3, 9, 5));
```

출력 9

받을 때 모으고, 넘길 때 펼칩니다. 한 줄에 둘 다 나옵니다.

— 문자열 잇기

```
function joinWords(separator, ...words) {
  return words.join(separator);
}

console.log(joinWords(" - ", "월", "화", "수"));
```

출력 월 - 화 - 수

▸ 직접 해보기 4

넘어온 숫자 중 짝수만 골라 배열로 돌려주는 함수를 만들어 보세요.

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

나머지 매개변수를 앞이나 중간에 씌 (섹션 2에서 봤습니다)

이런 실수를 합니다

배열을 그냥 넘기고 모아질 거라 기대

```
console.log(sum([1, 2, 3]));
```

출력 01,2,3

배열 하나가 인자로 들어가 numbers 는 [[1, 2, 3]] 이 됩니다. reduce 가 0 + [1, 2, 3] 을 계산하는데, 한쪽이 배열이라 문자열 "0" 과 "1,2,3" 이 이어붙어 버렸습니다.

```
console.log(sum(...[1, 2, 3]));
```

출력 6

펼쳐서 넘겨야 합니다.

이런 실수를 합니다

나머지 매개변수를 배열이 아니라고 생각

```
function check(...args) {
  console.log(Array.isArray(args));
}
check(1, 2);
```

출력 true

항상 배열입니다. 인자를 하나만 넘겨도, 아예 안 넘겨도 배열입니다.

이런 실수를 합니다

화살표 함수에서 괄호를 빼먹기

```
const f = ...args => args;
```

```
SyntaxError.
```

나머지 매개변수를 쓸 때는 괄호가 필요합니다.

```
const f = (...args) => args;
console.log(f(1, 2));
```

출력 [1, 2]

정리

- 1 `function f(...args)` — 넘어온 인자들을 배열로 모은다.
- 2 반드시 맨 마지막에 하나만 쓸 수 있다.
- 3 받는 쪽(왼쪽)이면 모으기, 주는 쪽(오른쪽)이면 펼치기.
- 4 모아서 다시 펼쳐 넘기는 패턴을 자주 쓴다: `f(...args)`
- 5 배열이므로 `reduce`, `filter`, `join` 을 바로 쓸 수 있다.

직접 해보기 정답

```
1) function countArgs(...args) {
  return args.length;
}
console.log(countArgs(1, 2, 3)); // 출력: 3
```

```
2) function printList(title, ...items) {
  console.log(title);
  items.forEach((item, i) => console.log(`${i + 1}. ${item}`));
}
```

```
}  
printList("할 일", "장보기", "청소");  
// 출력: 할 일 / 1. 장보기 / 2. 청소
```

```
3 console.log(collect(...[10, 20, 30])); // 출력: [ 10, 20, 30 ]
```

```
4 function onlyEven(...numbers) {
```

```
return numbers.filter((n) => n % 2 === 0);
```

```
}  
console.log(onlyEven(1, 2, 3, 4)); // 출력: [ 2, 4 ]
```


09단원 · 연습문제

구조분해와 스프레드

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~17은 기본, 18은 응용, 19는 [도전], 20은 에러 확인입니다.

문제 1 (개념01 배열 구조분해)

아래 배열에서 첫 값과 두 번째 값을 구조분해로 꺼내 한 줄에 출력하세요.

```
const colors1 = ["빨강", "파랑", "노랑"];
```

기대 출력 빨강 파랑

여기에 코드를 쓰세요

문제 2 (개념01 기본값)

아래 배열에서 두 값을 꺼내되, 두 번째 값이 없으면 "없음" 이 나오게 하세요.

```
const one2 = ["사과"];
```

기대 출력 사과 없음

여기에 코드를 쓰세요

문제 3 (개념01 나머지)

아래 배열에서 첫 값과 나머지를 나눠 차례로 출력하세요.

```
const nums3 = [1, 2, 3, 4];
```

기대 출력 1

— 2, 3, 4

여기에 코드를 쓰세요

문제 4 (개념01 값 교환)

아래 두 변수의 값을 구조분해로 맞바꾼 뒤 출력하세요. 임시 변수를 쓰지 마세요.

```
let left4 = "왼쪽";  
let right4 = "오른쪽";
```

기대 출력 오른쪽 왼쪽

여기에 코드를 쓰세요

문제 5 (개념01 split 과 함께)

아래 날짜 문자열을 "-" 로 나눈 뒤 구조분해로 받아 아래처럼 출력하세요.

```
const dateText5 = "2026-08-11";
```

기대 출력 2026년 08월 11일

여기에 코드를 쓰세요

문제 6 (개념02 객체 구조분해)

아래 객체에서 title 과 price 를 구조분해로 꺼내 한 줄에 출력하세요.

```
const book6 = { title: "입문서", price: 25000, author: "김작가" };
```

기대 출력 입문서 25000

여기에 코드를 쓰세요

문제 7 (개념02 이름 바꾸기)

book6 의 author 를 writer 라는 이름으로 꺼내 출력하세요.

기대 출력 김작가

여기에 코드를 쓰세요

문제 8 (개념02 기본값)

아래 객체에서 theme 과 fontSize 를 꺼내되, fontSize 가 없으면 14 가 나오게 하세요.

```
const config8 = { theme: "dark" };
```

기대 출력 dark 14

여기에 코드를 쓰세요

문제 9 (개념02 나머지)

아래 객체에서 password 만 빼고 나머지를 모아 출력하세요.

```
const account9 = { userId: "abc", password: "1234", nickname: "서연" };
```

기대 출력 { userId: 'abc', nickname: '서연' }

여기에 코드를 쓰세요

문제 10 (개념02 매개변수 구조분해)

{ name, city } 를 받아 "이름은 도시 거주" 를 출력하는 함수를 만들고, 아래 객체로 호출하세요. 함수 안에서 user10. 을 붙이면 안 됩니다.

```
const user10 = { name: "김민준", age: 20, city: "부산" };
```

기대 출력 김민준은 부산 거주

여기에 코드를 쓰세요

문제 11 (개념02 콜백 구조분해)

아래 배열을 map 으로 돌면서 이름만 뽑아 출력하세요. 콜백의 매개변수에서 구조분해를 쓰세요.

```
const products11 = [
  { name: "아메리카노", price: 4000 },
  { name: "케이크", price: 6000 },
];
```

기대 출력

— '아메리카노', '케이크'

여기에 코드를 쓰세요

문제 12 (개념03 스프레드 복사)

아래 배열을 복사한 뒤 복사본만 오름차순 정렬해 출력하고, 원본이 그대로인지 확인하기 위해 원본도 출력하세요.

```
const scores12 = [30, 10, 20];
```

기대 출력

— 10, 20, 30

— 30, 10, 20

여기에 코드를 쓰세요

문제 13 (개념03 합치기)

아래 두 배열을 스프레드로 합쳐 출력하세요. concat 을 쓰지 마세요.

```
const a13 = [1, 2];
const b13 = [3, 4];
```

기대 출력

— 1, 2, 3, 4

여기에 코드를 쓰세요

문제 14 (개념03 함수 인자로 펼치기)

아래 배열의 최댓값을 `Math.max` 로 구해 출력하세요.

```
const temps14 = [45, 88, 62];
```

기대 출력 88

여기에 코드를 쓰세요

문제 15 (개념04 객체 수정)

아래 객체의 age 만 30 으로 바꾼 '새 객체' 를 만들어 출력하고, 원본도 출력하세요. 원본은 바뀌면 안 됩니다.

```
const base15 = { name: "박지훈", age: 28 };
```

기대 출력 { name: '박지훈', age: 30 }
 { name: '박지훈', age: 28 }

여기에 코드를 쓰세요

문제 16 (개념04 병합)

기본 설정 위에 사용자 설정을 덮어쓴 결과를 출력하세요. 사용자가 정한 값이 이겨야 합니다.

```
const defaults16 = { theme: "light", fontSize: 14 };
const userSetting16 = { theme: "dark" };
```

기대 출력 { theme: 'dark', fontSize: 14 }

여기에 코드를 쓰세요

문제 17 (개념05 나머지 매개변수)

넘어온 숫자를 모두 더해서 돌려주는 sumAll 함수를 만드세요. 인자 개수는 몇 개든 됩니다.

기대 출력 15
 3
 0

여기에 코드를 쓰세요

(sumAll(1,2,3,4,5) / sumAll(1,2) / sumAll()) 을 차례로 출력)

문제 18 [응용]

아래 문자열을 뒤집어 출력하세요. 스프레드를 쓰세요.

```
const word18 = "자바스크립트";
```

기대 출력 트립크스바자

여기에 코드를 쓰세요

문제 19 [도전]

아래 할 일 목록에서 id 가 2인 항목만 done 을 **true** 로 바꾼 '새 배열' 을 만들고, 그 결과를 "제목 완료/미완료" 형태로 출력하세요. 원본 todos19 는 바뀌면 안 됩니다. 이 단원에서 가장 어렵습니다. 배열 복사와 객체 복사를 한 번에 써야 합니다.

```
const todos19 = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "청소", done: false },
];
```

힌트: 세 가지를 이어서 씁니다.

- ① map 으로 새 배열을 만든다 (원본은 그대로 남습니다)
- ② 각 항목에서 id 가 2인 것만 { ...todo, done: true } 로 바꾼다
- ③ 나머지는 그대로 돌려준다 - 삼항 연산자를 쓰면 한 줄입니다
새 객체를 안 만들고 todo.done = true 로 고치면 원본이 같이 바뀝니다.

기대 출력 장보기 미완료
 청소 완료
 원본 확인: false

여기에 코드를 쓰세요

(마지막 줄은 `console.log("원본 확인:", todos19[1].done)` 로 확인)

문제 20 (에러 확인 - 맨 마지막)

아래 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const { value } = null;
```


무슨 에러가 났나요? 값이 있을지 없을지 모를 때는 어떻게 하면 될까요?

■
■ 답
■

구조분해와 스프레드

RUN node 심화문제.js

— 이걸 선택 과제입니다

연습문제를 다 푼 사람만 하세요. 안 풀어도 다음 단원에 지장 없습니다.

심화 1 한 줄에 네 가지를 다 넣기

아래 order 에서 다음 네 가지를 '구조분해 한 번' 으로 꺼내세요.

① id ② customer 안의 name 을 customerName 이라는 이름으로 ③ customer 안의 address 안의 city (없으면 "미입력") ④ items 배열의 첫 번째 항목의 name 을 firstItem 이라는 이름으로

힌트: 중첩 구조분해 · 이름 바꾸기 · 기본값 · 배열 구조분해를 한꺼번에 씁니다.

③은 address 가 아예 없을 수도 있으니 = {} 로 막아야 합니다. (개념02 섹션 7)

④는 객체 안의 배열이라 items: [{ name: firstItem }] 모양이 됩니다.

```
const order = {
  id: 100,
  customer: {
    name: "정하늘",
    address: { city: "부산" },
  },
  items: [
    { name: "라떼", price: 4500 },
    { name: "쿠키", price: 3000 },
  ],
};
```

기대 출력 100 정하늘 부산 라떼

여기에 코드를 쓰세요

(console.log 한 번으로 네 개를 심표로 나열해 찍으세요)

심화 2 깊은 곳만 바꾼 새 객체

아래 state 에서 cart 의 id 2 번 항목의 count 만 5 로 바꾼 '새 state' 를 만드세요. 원본은 하나도 바뀌면 안 됩니다.

힌트: 곱 → 안 순서로 한 겹씩 새로 만듭니다.

① { ...state, cart: 새로운cart }

② 새로운cart 는 state.cart.map(...) 으로 만듭니다 ③ map 안에서 id 가 2 인 것만 { ...item, count: 5 } 로 바꾸고 나머지는 그대로

마지막에 원본의 count 도 함께 찍어서 안 바뀌었는지 확인하세요.

```
const state = {
  user: { name: "김민준", point: 100 },
  cart: [
    { id: 1, count: 2 },
    { id: 2, count: 1 },
  ],
};
```

기대 출력 새 state 의 2번 count: 5
 원본의 2번 count: 1
 user 객체를 그대로 재사용했나: true

여기에 코드를 쓰세요

(마지막 줄은 newState.user === state.user 를 찍으세요)

심화 3 몇 개가 오든 받아 내는 함수

이름표와 숫자 여러 개를 받아 아래 형태로 돌려주는 summarize 함수를 만드세요.

summarize("1반", 10, 20, 30) → "1반 — 합계 60 / 평균 20 / 최대 30" summarize("2반", 5) → "2반 — 합계 5 / 평균 5 / 최대 5" summarize("3반") → "3반 — 자료 없음"

조건: - 숫자는 몇 개가 와도 되어야 합니다 (나머지 매개변수) - 평균은 소수 첫째 자리까지 (toFixed(1) 은 문자열을 돌려줍니다) - 하나도 안 오면 "자료 없음"

힌트: 최대값은 `Math.max` 에 스프레드로 펼쳐 넣습니다. (개념03 섹션 5)

합계는 `reduce` 를 쓰면 한 줄입니다. (08단원)

기대 출력

1반 - 합계 60 / 평균 20.0 / 최대 30
2반 - 합계 5 / 평균 5.0 / 최대 5
3반 - 자료 없음

여기에 코드를 쓰세요

09단원 마무리

구조분해와 스프레드

자주 넘어지는 자리

1 스프레드는 **얕은 복사** — 안쪽 객체는 같이 딸려 갑니다

자주 나오는 질문

Q "`structuredClone` 이 다 해 주면 그거 쓰면 되잖아요?"
A 느리고, 함수나 화면 요소가 들어 있으면 실패합니다. 보통은 한 겹 복사로 충분합니다

DOM 조작

OPEN 10_DOM_조작

```
console.log(document.title);  
console.log(typeof document);  
console.log(title);  
console.log(firstBox);
```

01 DOM 이란과 요소 선택

02 내용 바꾸기

03 스타일과 클래스

04 요소 만들고 붙이기

05 입력값 읽기

- 연습문제

- 심화문제

DOM 이란과 요소 선택

RUN 이 파일을 브라우저로 열고 F12 → Console 탭

지금까지는 터미널에 글자만 찍었습니다. 이제 '화면'을 다룹니다.

브라우저는 HTML 을 읽어서 태그 하나하나를 자바스크립트가 다룰 수 있는 객체로 만들어 둡니다. 이 객체들의 모음을 DOM 이라고 합니다. Document Object Model = 문서를 객체로 만든 모형

1 원하는 태그를 '고른다' ← 이 파일

2 고른 것을 '바꾼다' ← 개념02부터

document — 문서 전체를 가리키는 객체

섹션 1

```
console.log(document.title);
```

F12 콘솔 10단원 개념01 - DOM 이란과 요소 선택

head 안의 title 태그 내용입니다.

```
console.log(typeof document);
```

F12 콘솔 object

document 는 브라우저가 미리 만들어 둔 객체입니다. 07단원에서 배운 객체와 똑같습니다. 점으로 속성을 꺼내고 메소드를 부릅니다.

▹ 직접 해보기

1

`console.log(document.body)` 를 찍어 보세요.

찍힌 결과 왼쪽의 ▶ 삼각형을 눌러 펼쳐 보세요. 태그가 태그를 품고 있는 나무 모양이 보입니다. 그게 DOM 입니다.

잠깐: 내 화면에서 선택자를 어떻게 알아내나

이 자료는 "id 가 title 인 요소" 처럼 답을 알려 주지만, 나중에 직접 만들 때는 스스로 찾아야 합니다. 방법은 아주 쉽습니다.

① 바꾸고 싶은 부분에 마우스 오른쪽 클릭 ② [검사] 를 누릅니다 (개발자도구가 Elements 탭으로 열립니다) ③ 파랗게 선택된 줄에서 id="..." 나 class="..." 를 읽습니다

그 값을 그대로 "#아이디" 또는 ".클래스" 로 쓰면 됩니다. HTML 을 못 읽어도 이 세 단계만 하면 선택자를 알아낼 수 있습니다.

querySelector — 하나 고르기

섹션 2

CSS 선택자를 그대로 씁니다. 조건에 맞는 '첫 번째 하나'를 돌려줍니다.

#아이디	아이디로
.클래스	클래스로
태그이름	태그로

```
const title = document.querySelector("#title");
console.log(title);
```

F12 콘솔 <h2 id="title">원래 제목</h2>

```
const firstBox = document.querySelector(".box");
console.log(firstBox);
```

F12 콘솔 <p class="box">첫 번째 상자</p>

상자가 세 개인데 첫 번째만 나왔습니다. querySelector 는 하나만 고릅니다.

```
const button = document.querySelector("button");
console.log(button);
```

F12 콘솔 <button id="myButton">버튼</button>

고른 요소도 객체입니다. 속성을 꺼내 볼 수 있습니다.

코드

```
console.log(title.textContent);
```

```
console.log(title.id);
```

▶ F12 콘솔

▶ 원래 제목

▶ title

▹ 직접 해보기

2

button 의 textContent 를 콘솔에 찍어 보세요.

못 찾으면 null

섹션 3

```
const notExist = document.querySelector("#없는아이디");
console.log(notExist);
```

F12 콘솔 null

null에서 무언가를 꺼내려 하면 에러가 납니다. (07단원에서 봤죠)

이런 실수를 합니다

```
console.log(notExist.textContent);
```

```
TypeError: Cannot read properties of null (reading 'textContent')
```

"Cannot read properties of null"은 DOM을 다룰 때 가장 많이 보는 에러입니다. 십중팔구 선택자 오타이거나, 아직 만들어지지 않은 요소를 고른 것입니다.

안전하게 쓰려면 확인하고 씁니다.

```
if (notExist) {  
  console.log("찾았습니다");  
} else {  
  console.log("그런 요소가 없습니다");  
}
```

F12 콘솔 그런 요소가 없습니다

▸ 직접 해보기 3

“.없는클래스”를 골라 보고, if로 확인해서 있으면 “있음”, 없으면 “없음”을 콘솔에 찍어 보세요.

querySelectorAll — 전부 고르기

섹션 4

```
const boxes = document.querySelectorAll(".box");  
console.log(boxes);
```

F12 콘솔 NodeList(3) [p.box, p.box, p.box]

처음 보는 영어가 나왔습니다. 읽을 수 있어야 검색도 질문도 할 수 있습니다. ‘노드(node)’는 DOM에서 태그 하나하나를 부르는 이름입니다.

이 자료에서는 ‘요소’와 거의 같은 뜻으로 보시면 됩니다.

‘NodeList’는 그 노드들을 담은 목록. 즉 “고른 요소 3개의 묶음”입니다.

```
console.log(boxes.length);
```

F12 콘솔 3

배열처럼 인덱스로 꺼낼 수 있습니다.

코드	▶ F12 콘솔
<code>console.log(boxes[0].textContent);</code>	▶ 첫 번째 상자
<code>console.log(boxes[2].textContent);</code>	▶ 세 번째 상자

forEach 도 됩니다.

<pre>boxes.forEach((box, index) => { console.log(`\${index}번 상자: \${box.textContent}`); });</pre>	
F12 콘솔	0번 상자: 첫 번째 상자 1번 상자: 두 번째 상자 2번 상자: 세 번째 상자

주의 · NodeList 는 배열처럼 보이지만 진짜 배열이 아닙니다.

<code>console.log(Array.isArray(boxes));</code>	
F12 콘솔	false

그래서 map, filter 는 쓸 수 없습니다.

<code>boxes.map(...)</code>	→ TypeError: boxes.map is not a function
-----------------------------	---

09단원의 스프레드로 진짜 배열로 바꾸면 다 쓸 수 있습니다.

<pre>const boxArray = [...boxes]; console.log(Array.isArray(boxArray));</pre>	
F12 콘솔	true
<code>console.log(boxArray.map((b) => b.textContent));</code>	
F12 콘솔	['첫 번째 상자', '두 번째 상자', '세 번째 상자']

◁ 직접 해보기 4

모든 .box 의 글자를 forEach 로 콘솔에 찍어 보세요.

getElementById — 옛날 방식

섹션 5

querySelector 가 생기기 전에 쓰던 방법입니다. 아직도 자주 보입니다.

```
const titleOld = document.getElementById("title");
console.log(titleOld);
```

F12 콘솔 <h2 id="title">원래 제목</h2>

차이점:

```
getElementById("title") → # 을 붙이지 않습니다
querySelector("#title") → # 을 붙입니다
```

아이디로 찾을 때는 getElementById 가 아주 조금 빠릅니다. 하지만 CSS 선택자를 그대로 쓸 수 있는 querySelector 가 훨씬 편해서 요즘은 querySelector 로 통일해서 씁니다. 이 자료도 그렇게 합니다.

▸ 직접 해보기 5

같은 버튼을 getElementById 와 querySelector 로 각각 고른 뒤, === 로 비교한 결과를 콘솔에 찍어 보세요. (아이디는 myButton)

선택자 더 써 보기

섹션 6

CSS 선택자를 아는 만큼 그대로 쓸 수 있습니다.

```
console.log(document.querySelector(".demo h2").textContent);
```

F12 콘솔 원래 제목

.demo 안의 h2 라는 뜻입니다.

```
console.log(document.querySelectorAll(".demo p").length);
```

F12 콘솔 3

```
console.log(document.querySelector("p.box").textContent);
```

F12 콘솔 첫 번째 상자

p 이면서 box 클래스인 것

▸ 직접 해보기 6

.demo 안에 있는 button 을 선택자 하나로 골라 textContent 를 콘솔에 찍어 보세요.

script 를 어디에 둘 것인가

섹션 7

이 파일의 script 태그는 body 의 '맨 아래' 에 있습니다. 이유가 있습니다.

브라우저는 HTML 을 위에서 아래로 읽습니다. script 를 head 에 두면, 아직 만들어지지 않은 태그를 고르려 해서 `querySelector` 가 `null` 을 돌려주고 에러가 납니다.

1 script 를 body 맨 아래에 둔다 ← 이 자료의 방식

2 head 에 두되 script 태그에 `defer` 를 붙인다

(`defer` 는 "HTML 을 다 읽은 다음에 실행해라" 라는 뜻입니다)

"Cannot read properties of `null`" 이 났는데 선택자가 맞다면 script 위치부터 확인하세요.

참고 — 주석 안에도 script 를 닫는 태그를 그대로 쓰면 안 됩니다. 브라우저는 주석인지 아닌지 따지지 않고 그 자리에서 script 를 끝내 버립니다. 그러면 아래 코드가 전부 실행되지 않고 화면에 글자로 새어 나옵니다.

◁ 직접 해보기

7

이 파일의 script 태그를 head 안으로 옮기면 어떤 에러가 날까요? 먼저 예상을 적어 본 다음 정답과 맞춰 보세요.

자주 하는 실수

섹션 8

이런 실수를 합니다

이나 . 을 빠뜨림

```
console.log(document.querySelector("title"));
```

F12 콘솔 <title>10단원 개념01 - DOM 이란과 요소 선택</title>

"title" 은 title '태그' 를 찾습니다. id 를 찾으려면 "#title" 입니다. 에러가 안 나고 엉뚱한 게 잡히니 더 헷갈립니다.

이런 실수를 합니다

querySelector 로 여러 개를 고르려 함

```
console.log(document.querySelector(".box").textContent);
```

F12 콘솔 첫 번째 상자

항상 첫 번째 하나뿐입니다. 전부 고르려면 `querySelectorAll`.

이런 실수를 합니다

querySelectorAll 결과에 바로 값을 쓰기

```
boxes.textContent = "바꾸기";
```

여러 개 묶음이라 하나처럼 다룰 수 없습니다. forEach 로 하나씩 처리해야 합니다.

이런 실수를 합니다

NodeList 에 map 을 씌 (섹션 4에서 봤습니다)

정리

- 1 document 는 문서 전체를 가리키는 객체다.
- 2 querySelector(선택자) 조건에 맞는 첫 하나. 없으면 null.
- 3 querySelectorAll(선택자) 전부. NodeList(배열 비슷한 것).
- 4 선택자는 CSS 와 같다. #아이디 .클래스 태그이름
- 5 NodeList 는 배열이 아니다. map 을 쓰려면 [...nodeList].
- 6 script 는 body 맨 아래에 둔다.

직접 해보기 정답

```
1) console.log(document.body);
```

→ body 태그와 그 안의 모든 내용이 통째로 찍힙니다.

```
2 console.log(button.textContent); // 콘솔: 버튼

3 const none = document.querySelector(".없는클래스");
  if (none) {
```

```
console.log("있음");
```

```
} else {
```

```
console.log("없음");
```

```
}
// 콘솔: 없음
```

→ 없는 것을 고르면 `null` 입니다. `null` 은 `falsy` 라서 `else` 로 갑니다.

(03단원에서 배운 `falsy` 가 여기서 그대로 쓰입니다)

```
4) document.querySelectorAll(".box").forEach((box) => {
```

```
  console.log(box.textContent);
```

```
});
```

// 콘솔: 첫 번째 상자 / 두 번째 상자 / 세 번째 상자

```
5) const b1 = document.getElementById("myButton");
```

```
const b2 = document.querySelector("#myButton");
```

```
console.log(b1 === b2);
```

// 콘솔: true

→ 찾는 방법만 다를 뿐 고른 것은 '같은 요소 하나' 입니다.

`getElementById` 는 # 없이, `querySelector` 는 # 를 붙인다는 것만 다릅니다.

```
6) console.log(document.querySelector(".demo button").textContent);
```

// 콘솔: 버튼

→ ".demo button" 은 "demo 클래스 안에 있는 button" 이라는 뜻입니다.

7) `TypeError: Cannot read properties of null` 이 납니다. → 브라우저는 위에서 아래로 읽습니다.

`head` 는 `body` 보다 먼저 읽히므로

`script` 가 실행되는 시점에는 `#title` 태그가 아직 만들어져 있지 않습니다.

그래서 `querySelector` 가 `null` 을 돌려주고, 그 `null` 에서 무언가를

꺼내려는 순간 섹션 3에서 본 그 에러가 납니다.

직접 확인해 보고 싶다면 옮겨서 본 뒤 꼭 되돌려 놓으세요.

내용 바꾸기

RUN 이 파일을 브라우저로 열고 F12 → Console 탭

요소를 골랐으니 이제 바꿔 봅시다.

```
요소.textContent = "글자"   글자만 넣기 (안전)
요소.innerHTML  = "<b>태그</b>" 태그까지 넣기 (주의)
```

textContent — 글자 읽고 쓰기

섹션 1

```
const target1 = document.querySelector("#target1");
```

읽기

```
console.log(target1.textContent);
```

F12 콘솔 이 글자는 바뀝니다

쓰기 — = 로 값을 넣으면 화면이 바로 바뀝니다

```
target1.textContent = "자바스크립트가 바꾼 글자";
```

화면 "이 글자는 바뀝니다" 가 "자바스크립트가 바꾼 글자" 로 바뀝니다

```
console.log(target1.textContent);
```

F12 콘솔 자바스크립트가 바꾼 글자

변수를 넣을 수도 있습니다. 08단원까지 배운 모든 게 여기로 연결됩니다.

```
const userName = "김민준";
const target2 = document.querySelector("#target2");
target2.textContent = `${userName}님, 안녕하세요!`;
```

화면 김민준님, 안녕하세요!

▹ 직접 해보기 1

target1의 글자를 자기 이름으로 바꿔 보세요.

숫자를 넣으면 글자로 바뀐다

섹션 2

```
const numberTarget = document.querySelector("#numberTarget");
const count = 5;

numberTarget.textContent = count;
```

화면 "여기에는 숫자가 들어갑니다" 가 "5" 로 바뀝니다

코드	▶ F12 콘솔
<code>console.log(numberTarget.textContent);</code>	▶ 5
<code>console.log(typeof numberTarget.textContent);</code>	▶ string

넣을 때는 숫자였지만 꺼내면 문자열입니다. 화면에 보이는 것은 전부 글자이기 때문입니다.

그래서 화면에서 읽은 값으로 계산하려면 `Number()` 가 필요합니다. (01단원)

코드	▶ F12 콘솔
<code>console.log(numberTarget.textContent + 1);</code>	▶ 51
<code>console.log(Number(numberTarget.textContent) + 1);</code>	▶ 6

이 함정은 개념05(입력값 읽기)에서 다시 만납니다.

▫ 직접 해보기 2

`numberTarget` 에 숫자 10 을 넣은 다음, 그 값에 5 를 더한 결과를 콘솔에 찍어 보세요. (15 가 나와야 합니다)

innerHTML — 태그까지 넣기

섹션 3

```
const target3 = document.querySelector("#target3");
```

`textContent` 로 태그를 넣으면 글자 그대로 보입니다.

```
target3.textContent = "<strong>굵게</strong>";
```

화면 굵게 ← 태그가 글자로 보입니다

`innerHTML` 로 넣으면 태그로 해석됩니다.

```
const target4 = document.querySelector("#target4");
target4.innerHTML = "<strong>굵게</strong> 그리고 <em>기울임</em>";
```

화면 굵게 그리고 기울임 ← 실제로 굵고 기울어져 보입니다

읽을 때도 다릅니다.

```
console.log(target4.innerHTML);
```

F12 콘솔 굵게 그리고 기울임

```
console.log(target4.textContent);
```

F12 콘솔 굵게 그리고 기울임

textContent 는 태그를 빼고 글자만 줍니다.

▹ 직접 해보기

3

target3 에 innerHTML 로 기울임 을 넣어 보세요.

목록 만들기 — 08·09단원과 이어지는 부분

섹션 4

```
const menu = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];
```

map 으로 태그 문자열을 만들고 join 으로 붙입니다. (08단원 개념03)

```
const html = menu.map((item) => `<li>${item.name} - ${item.price}원</li>`).join("");

console.log(html);
```

F12 콘솔 아메리카노 - 4000원라떼 - 4500원케이크 - 6000원

```
document.querySelector("#menuList").innerHTML = html;
```

화면 메뉴 세 줄이 목록으로 나타납니다

이 패턴(배열 → map → join → innerHTML)이 화면 만들기의 기본입니다. React 를 배우면 이 과정을 라이브러리가 대신해 줍니다.

join("") 을 빠뜨리면 어떻게 될까요?

```
console.log(menu.map((item) => `<li>${item.name}</li>`));
```



```
F12 콘솔    ['<li>아메리카노</li>', '<li>라떼</li>', '<li>케이크</li>']
```

배열을 그대로 innerHTML 에 넣으면 쉼표가 화면에 보입니다.

▹ 직접 해보기

4

위 menu 에서 이름만 목록으로 만들어 넣어 보세요.

언제 textContent, 언제 innerHTML

섹션 5

기본은 textContent 입니다. 이유는 안전 때문입니다.

사용자가 입력한 글을 그대로 innerHTML 에 넣으면, 누군가 <script> 나 이상한 태그를 적어 넣었을 때 그게 실제로 실행됩니다. 이런 공격을 XSS 라고 합니다.

```
const userInput = "<img src=x onerror=\"alert('공격')\">";
console.log("사용자가 이런 글을 입력했다고 해 봅시다:", userInput);
```

```
F12 콘솔    사용자가 이런 글을 입력했다고 해 봅시다: <img src=x
onerror="alert('공격')">
```

textContent 로 넣으면 그냥 글자로 보입니다. 안전합니다.

```
target1.textContent = userInput;
```

innerHTML 로 넣으면 태그로 해석되어 실행됩니다. 위험합니다.

```
target1.innerHTML = userInput;    ← 이렇게 쓰지 마세요
```

```
console.log("사용자 입력을 innerHTML 에 바로 넣지 말 것");
```

```
F12 콘솔    사용자 입력을 innerHTML 에 바로 넣지 말 것
```

정리: 글자만 넣는다 → textContent (기본) 내가 만든 태그를 넣는다 → innerHTML 사용자가 쓴 글을 넣는다 → 반드시 textContent

▹ 직접 해보기

5

위 userInput 을 target3 에 '안전한 쪽' 으로 넣어 보세요. 화면에 태그가 글자 그대로 보이면 성공입니다.

innerText 와의 차이

섹션 6

textContent 와 비슷한 innerText 도 있습니다. 차이는 "화면에 실제로 보이는가" 를 따지는지 여부입니다.

textContent : HTML 에 적힌 글자를 그대로 (숨겨진 것도 포함) innerText : 화면에 보이는 글자만

innerText 는 화면 계산을 하느라 느립니다. 특별한 이유가 없으면 textContent 를 쓰세요.

이 파일 화면에 #mixedTarget 이라는 문단이 하나 있습니다. 화면에는 “보이는 글자” 만 보이지만, 그 안에 감춰 둔 “ 숨은 글자” 가 더 있습니다.

▣ 직접 해보기 6

#mixedTarget 을 textContent 로 읽을 때와 innerText 로 읽을 때 결과가 같을까요, 다를까요? 예상한 뒤 둘 다 찍어서 비교해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

괄호를 붙임

```
target1.textContent("안녕");
```

```
TypeError: target1.textContent is not a function textContent
```

는 속성입니다. = 로 값을 넣습니다.

이런 실수를 합니다

null 에 값을 넣으려 함

```
document.querySelector("#없는아이디").textContent = "안녕";
```

```
TypeError: Cannot set properties of null (setting 'textContent')
```

선택자 오타이거나 script 위치 문제입니다. (개념01 섹션 7)

이런 실수를 합니다

innerHTML 에 배열을 그대로 넣기 (섹션 4에서 봤습니다)

이런 실수를 합니다

화면에서 읽은 값으로 바로 계산 (섹션 2)

화면의 값은 언제나 문자열입니다. Number() 로 바꾸세요.

이런 실수를 합니다

textContent 로 태그를 넣고 왜 안 되냐고 하기 (섹션 3)

정리

- 1 요소.textContent = "글자" — 글자만. 기본은 이걸 쓴다.
- 2 요소.innerHTML = "x" — 태그로 해석된다.
- 3 화면에서 읽은 값은 항상 문자열이다. 계산하려면 Number().
- 4 배열 → map → join("") → innerHTML 이 목록 만들기의 기본 패턴.
- 5 사용자가 입력한 값은 절대 innerHTML 에 바로 넣지 않는다.

직접 해보기 정답

```
1 target1.textContent = "홍길동";

2 numberTarget.textContent = 10;
  console.log(Number(numberTarget.textContent) + 5);
  // 콘솔: 15
```

→ Number() 를 빼면 "105" 가 나옵니다. 화면에서 읽은 값은 문자열이라

+ 가 더하기가 아니라 이어붙이기가 되기 때문입니다. (섹션 2)

```
3 target3.innerHTML = "<em>기울임</em>";

4 document.querySelector("#menuList").innerHTML =
```

```
menu.map((item) => `<li>${item.name}</li>`).join("");
```

```
5) target3.textContent = userInput;
  // 화면: <img src=x onerror="alert('공격')"> ← 태그가 글자로 보입니다
```

정말 안전한지 확인해 보려면

```
console.log(target3.querySelector("img"));
// 콘솔: null
```

→ img 태그가 아예 만들어지지 않았다는 뜻입니다. 글자로만 들어갔으니까요.

innerHTML 로 넣었다면 진짜 img 태그가 생기고 onerror 가 실행됩니다.

```
6) const mixed = document.querySelector("#mixedTarget");
```

```
console.log(mixed.textContent);  
// 콘솔: 보이는 글자 숨은 글자
```

→ 숨겨 둔 span 의 글자까지 나옵니다.

textContent 는 HTML 에 적힌 글자를 그대로 주기 때문입니다.

```
console.log(mixed.innerText);  
// 콘솔: 보이는 글자
```

→ 숨은 글자가 빠졌습니다. innerText 는 화면에 실제로 보이는 것만 줍니다.

이것이 두 속성의 실질적인 차이입니다.

주의 · 요소 '자신' 을 display: none 으로 감춘 경우에는 둘이 같아집니다.

그때는 innerText 도 textContent 와 같은 값을 돌려줍니다.
차이가 드러나는 건 이 예처럼 '보이는 요소 안에 숨은 자식' 이 있을 때입니다.

스타일과 클래스

RUN 이 파일을 브라우저로 열고 F12 → Console 탭

화면의 '모양'을 바꾸는 방법은 두 가지입니다.

요소.style.속성 = "값" 직접 바꾸기
 요소.classList.add("클래스") 미리 만든 클래스 붙이기 ← 이쪽을 권장

style — 직접 바꾸기

섹션 1

```
const box1 = document.querySelector("#box1");

box1.style.color = "white";
box1.style.backgroundColor = "#2d6cdf";
```

화면 상자 1 이 파란 배경에 흰 글자가 됩니다

```
console.log(box1.style.color);
```

F12 콘솔 white

CSS 속성 이름이 조금 달라집니다.

CSS	자바스크립트
color	color
background-color	backgroundColor
font-size	fontSize
text-align	textAlign

규칙: 하이픈(-)을 빼고 뒷 단어의 첫 글자를 대문자로. 카멜 표기법입니다. 변수 이름 규칙(01단원 개념02)과 같습니다.

```
box1.style.fontSize = "20px";
box1.style.textAlign = "center";
```

화면 글자가 커지고 가운데로 정렬됩니다

단위를 빼뜨리면 아무 일도 일어나지 않습니다.

```
box1.style.padding = 30;
console.log(box1.style.padding);
```

F12 콘솔

빈 값입니다. 숫자만 주면 무시됩니다.

```
box1.style.padding = "30px";
console.log(box1.style.padding);
```

F12 콘솔 30px

반드시 단위를 붙인 '문자열' 로 줘야 합니다.

▸ 직접 해보기

1

box1의 글자색을 "yellow"로 바꿔 보세요.

classList — 클래스 붙이고 떼기

섹션 2

style로 하나하나 바꾸는 것보다, CSS에 미리 만들어 둔 클래스를 붙였다 떼는 편이 훨씬 좋습니다.

이 파일 위쪽 style 태그에 .highlight .done .hidden .danger를 미리 만들어 뒀습니다.

```
const box2 = document.querySelector("#box2");
```

add — 클래스 붙이기

```
box2.classList.add("highlight");
```

화면 상자 2가 노란 배경에 굵은 글자가 됩니다

```
console.log(box2.className);
```

F12 콘솔 item highlight

원래 있던 item 클래스는 그대로 있고 highlight가 더해졌습니다.

remove — 떼기 (실제 사용 예는 섹션 7 [실수 3]에 있습니다)

```
const box3 = document.querySelector("#box3");
box3.classList.add("done");
```

화면 상자 3에 취소선이 그어지고 회색이 됩니다

contains — 붙어 있는지 확인

코드	▶ F12 콘솔
<code>console.log(box3.classList.contains("done"));</code>	▶ true
<code>console.log(box3.classList.contains("highlight"));</code>	▶ false

toggle — 있으면 떼고 없으면 붙이기

<code>const box4 = document.querySelector("#box4");</code> <code>box4.classList.toggle("hidden");</code>	
화면	상자 4 가 사라집니다 (display: none)
<code>console.log(box4.classList.contains("hidden"));</code>	
F12 콘솔	true

한 번 더 부르면 다시 나타납니다. 11단원에서 버튼과 연결해 써 봅니다.

▫ 직접 해보기 2

box2 에서 highlight 클래스를 떼어 보세요.

왜 `classList` 가 더 좋은가

섹션 3

style 로 하면 · 바꿀 게 늘어날수록 줄이 늘어납니다

<code>el.style.color = "#999";</code> <code>el.style.textDecoration = "line-through";</code> <code>el.style.opacity = "0.6";</code>	
---	--

classList 로 하면 · 한 줄입니다

<code>el.classList.add("done");</code>	
--	--

게다가 - 모양은 CSS 파일에, 동작은 JS 파일에 — 역할이 나뉩니다 - 디자인을 바꿀 때 CSS 만 고치면 됩니다 - 이름(done, danger)이 뜻을 설명해 줍니다

규칙: 모양을 바꿀 일이 있으면 먼저 CSS 에 클래스를 만들고,

자바스크립트는 그 클래스를 붙였다 떼기만 합니다.	
-----------------------------	--

▫ 직접 해보기 3

위 [style 로 하면] 세 줄과 같은 일을 box1 에 classList 한 줄로 해 보세요. (클래스 이름은 done)

```
const box5 = document.querySelector("#box5");
const stock = 0;

if (stock === 0) {
  box5.classList.add("danger");
  box5.textContent = "상자 5 - 품절입니다";
} else {
  box5.textContent = `상자 5 - 재고 ${stock}개`;
}
```

화면 상자 5 가 빨간 글자·빨간 테두리로 "품절입니다" 라고 바뀝니다

toggle 에 두 번째 인자로 true/false 를 주면 조건에 따라 붙이고 뺍니다.

```
const isSoldOut = stock === 0;
box5.classList.toggle("highlight", isSoldOut);
console.log(box5.classList.contains("highlight"));
```

F12 콘솔 true

isSoldOut 이 false 였다면 떼어졌을 겁니다.

▹ 직접 해보기 4

점수가 60 미만이면 danger 클래스를 붙이는 코드를 써 보세요.

여러 요소에 한꺼번에

```
const items = document.querySelectorAll(".item");
console.log(items.length);
```

F12 콘솔 5

forEach 로 하나씩 처리합니다. (개념01 섹션 4)

```
items.forEach((item, index) => {
  if (index % 2 === 1) {
    item.style.borderLeftWidth = "6px";
  }
});
```

화면 인덱스가 홀수인 상자, 즉 상자 2와 상자 4의 왼쪽 테두리가 굵어집니다.

다만 상자 4는 섹션 2에서 hidden 을 붙여 숨겼기 때문에
눈에는 상자 2만 굵어진 것으로 보입니다.

전부 같은 클래스를 붙이려면

```
items.forEach((item) => item.classList.add("highlight"));
```

▹ 직접 해보기 5

바로 위 주석을 참고해 모든 .item 에 highlight 를 붙이고, highlight 가 붙은 요소가 몇 개인지 콘솔에
찍어 보세요.

현재 적용된 스타일 읽기

섹션 6

주의: element.style 은 '자바스크립트로 직접 넣은 것' 만 읽습니다. box5 에는 위에서 highlight 클래스를
붙였지만 style 로 직접 넣은 적은 없습니다.

```
console.log(box5.style.backgroundColor);
```

F12 콘솔

빈 값입니다. box5 의 노란 배경은 CSS 클래스에서 온 것이니까요.

CSS 에 적힌 것까지 포함해 실제로 적용된 값을 읽으려면 getComputedStyle 을 씁니다.

```
const realStyle = getComputedStyle(box5);
console.log(realStyle.backgroundColor);
```

F12 콘솔 rgb(255, 243, 160)

#fff3a0 이 rgb 형태로 나옵니다.

반대로 box2 는 섹션 5에서 style 로 직접 테두리 굵기를 넣었기 때문에 값이 있습니다.

```
console.log(box2.style.borderLeftWidth);
```

F12 콘솔 6px

참고로 style 로 직접 넣은 값은 CSS 클래스보다 우선합니다. 그래서 같은 속성을 양쪽에서 건드리면 style
쪽이 이깁니다.

읽을 일은 드뭅니다. "왜 style 로 읽으면 비어 있지?" 할 때만 기억하세요.

box1 과 box2 의 style.backgroundColor 를 각각 찍어 보세요. 한쪽만 값이 나옵니다. 왜 그런지 말로 설명해 보세요.

자주 하는 실수

섹션 7

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

CSS 이름을 그대로 씀

```
box1.style.background-color = "red";
```

```
SyntaxError.
```

하이픈은 빼기 기호로 해석됩니다. backgroundColor 입니다.

이런 실수를 합니다

단위를 빠뜨림 (섹션 1에서 봤습니다)

이런 실수를 합니다

classList.add 에 점을 붙임

```
box3.classList.add(".highlight");
console.log(box3.className);
```

```
F12 콘솔    item done .highlight
```

클래스 이름만 씁니다. 점은 CSS 선택자에서만 붙입니다. 에러가 안 나고 이상한 이름의 클래스가 붙습니다.

```
box3.classList.remove(".highlight");
```

이런 실수를 합니다

className 에 = 로 덮어쓰기

```
box3.className = "done";
```

원래 있던 item 클래스까지 날아갑니다. 더할 때는 classList.add 를 쓰세요.

이런 실수를 합니다

여러 개를 고르고 하나처럼 다루기

```
document.querySelectorAll(".item").classList.add("x");
```

```
TypeError. NodeList
```

에는 classList 가 없습니다. forEach 로 도세요.

정리

- 1 요소.style.속성 = "값" — CSS 이름을 카멜 표기법으로. 단위 필수.
- 2 classList.add / remove / toggle / contains
- 3 모양은 CSS 클래스로 만들고 JS 는 붙였다 떼기만 하는 게 좋다.
- 4 toggle(클래스, 조건) 으로 조건에 따라 붙이고 뗄 수 있다.
- 5 element.style 은 JS 로 넣은 것만 읽는다. CSS 값은 getComputedStyle.

직접 해보기 정답

```
1 box1.style.color = "yellow";
2 box2.classList.remove("highlight");
3 box1.classList.add("done");
  console.log(box1.className);
  // 콘솔: item done
```

→ 세 줄이 한 줄이 됩니다. 회색과 취소선은 CSS 의 .done 이 이미 갖고 있습니다.

다만 이 파일의 .done 에는 opacity 가 없어서 흐려지지는 않습니다.
위 [style 로 하면] 세 줄과 똑같이 만들고 싶다면 .done 에
opacity: 0.6 을 한 줄 더 넣으면 됩니다 - 그게 바로 "모양은 CSS 에서
고친다" 는 이 섹션의 요지입니다. 자바스크립트는 그대로 둡니다.

```
4) const score = 55;
const el = document.querySelector("#box5");
if (score < 60) {
```

```
  el.classList.add("danger");
```

```
}  
또는 한 줄로: el.classList.toggle("danger", score < 60);
```

```
5) items.forEach((item) => item.classList.add("highlight"));  
console.log(document.querySelectorAll(".highlight").length);  
// 콘솔: 5
```

→ box2 와 box5 에는 이미 highlight 가 붙어 있었지만 중복으로 붙지 않습니다.

classList.add 는 없을 때만 붙입니다.

```
6) console.log(box1.style.backgroundColor);  
// 콘솔: rgb(45, 108, 223)  
console.log(box2.style.backgroundColor);  
// 콘솔:
```

→ box1 은 섹션 1에서 자바스크립트로 직접 넣었으니 값이 있습니다.

box2 의 노란 배경은 CSS 클래스에서 온 것이라 element.style 로는 안 보입니다.
넣을 때 쓴 #d6cdf 가 rgb 형태로 바뀌어 나오는 것도 함께 확인하세요.

요소 만들고 붙이기

RUN 이 파일을 브라우저로 열고 F12 → Console 탭

개념02에서는 이미 있는 태그의 '내용'만 바꿨습니다. 이번에는 태그 자체를 새로 만들어 화면에 붙입니다.

- 1 `document.createElement("li")` 만들고
- 2 `el.textContent = "글자"` 내용을 채우고
- 3 `부모.appendChild(el)` 붙인다

세 단계

섹션 1

```
const list1 = document.querySelector("#list1");
```

1) 만들기 — 아직 화면에 없습니다. 메모리에만 있습니다.

```
const li1 = document.createElement("li");
console.log(li1);
```

F12 콘솔 `<li></li>`

비어 있습니다. 화면에도 아직 안 보입니다.

2) 채우기

```
li1.textContent = "첫 번째 항목";
```

3) 붙이기 — 이 줄이 실행되는 순간 화면에 나타납니다

```
list1.appendChild(li1);
```

화면 목록 ① 에 "첫 번째 항목" 이 생깁니다

하나 더 만들어 봅시다.

```
const li2 = document.createElement("li");
li2.textContent = "두 번째 항목";
list1.appendChild(li2);
```

화면 아래에 "두 번째 항목" 이 추가됩니다

`appendChild` 는 '맨 뒤에' 붙입니다.

▹ 직접 해보기 1

“세 번째 항목”을 만들어 list1에 붙여 보세요.

append와 prepend

섹션 2

append는 appendChild와 거의 같은데 더 편합니다.

```
const li3 = document.createElement("li");
li3.textContent = "세 번째 항목 (append)";
list1.append(li3);
```

화면 맨 뒤에 추가됩니다

append는 글자를 바로 넣을 수도 있고, 여러 개를 한 번에 넣을 수도 있습니다. appendChild는 요소 하나만 받습니다.

prepend는 ‘맨 앞에’ 붙입니다.

```
const li0 = document.createElement("li");
li0.textContent = "0번 항목 (맨 앞)";
list1.prepend(li0);
```

화면 목록 맨 위에 추가됩니다

```
console.log(list1.children.length);
```

F12 콘솔 4

children은 그 요소가 품고 있는 자식 요소들의 묶음입니다. NodeList처럼 length로 개수를 세고 [0]으로 하나씩 꺼낼 수 있습니다. (개념01 섹션 4)

▹ 직접 해보기 2

“맨 앞 새 항목”이라는 li를 만들어 prepend로 붙이고, 정말 맨 앞에 갔는지 list1.children[0]으로 확인해 보세요.

반복문으로 여러 개 만들기

섹션 3

```
const fruits = ["사과", "바나나", "포도", "귤"];
const list2 = document.querySelector("#list2");

fruits.forEach((fruit, index) => {
  const li = document.createElement("li");
  li.textContent = `${index + 1}. ${fruit}`;
  list2.appendChild(li);
});
```

화면 목록 ② 에 과일 네 개가 번호와 함께 나타납니다

```
console.log(list2.children.length);
```

F12 콘솔 4

배열 하나만 고치면 화면이 통째로 바뀝니다. 이것이 “데이터로 화면을 만든다” 는 개념의 시작입니다.

▹ 직접 해보기 3

좋아하는 색 세 개를 배열로 만들어 목록으로 붙여 보세요.

클래스와 속성도 함께

섹션 4

```
const products = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
];

const cardArea = document.querySelector("#cardArea");

products.forEach((product) => {
  const div = document.createElement("div");
```

클래스 붙이기 (개념03)

```
div.classList.add("card");
if (product.stock === 0) {
  div.classList.add("sold");
}
```

내용 채우기 — 삼항 연산자로 글자를 정합니다 (03단원)

```
const stockText = product.stock === 0 ? "품절" : `재고 ${product.stock}개`;
div.textContent = `${product.name} ${product.price}원 - ${stockText}`;
```

나중에 찾기 쉽도록 데이터도 붙여 둘 수 있습니다

```
div.dataset.name = product.name;

cardArea.appendChild(div);
});
```

화면 카드 세 개가 생기고, 라떼 카드만 빨간 글씨로 "품절" 이라고 나옵니다

```
console.log(document.querySelectorAll(".card").length);
```

F12 콘솔 3

```
console.log(document.querySelector(".sold").textContent);
```

F12 콘솔 라떼 4500원 - 품절

dataset 은 data-이름 속성으로 들어갑니다. 11단원에서 다시 씁니다.

```
console.log(document.querySelector(".card").dataset.name);
```

F12 콘솔 아메리카노

▹ 직접 해보기 4

카드 세 개의 dataset.name 을 forEach 로 전부 찍어 보세요.

지우기

섹션 5

```
const willRemove = document.querySelector("#willRemove");
willRemove.remove();
```

화면 목록 ④ 의 첫 항목이 사라집니다

```
console.log(document.querySelector("#willRemove"));
```

F12 콘솔 null

화면에서도 DOM 에서도 완전히 사라졌습니다.

안의 내용을 전부 비우려면


```
요소.innerHTML = "";
```

목록을 새로 그릴 때 이 방법을 자주 씁니다. 1) 비우고 2) 데이터로 다시 만든다

```
const list3 = document.querySelector("#list3");
console.log(list3.children.length);
```

F12 콘솔 1

▹ 직접 해보기 5

list1의 첫 항목을 지워 보세요.

createElement vs innerHTML

섹션 6

개념02에서는 innerHTML로 목록을 만들었습니다. 둘을 비교해 봅시다.

— innerHTML

```
list.innerHTML = items.map((i) => `<li>${i}</li>`).join("");
```

+ 짧다. 한눈에 모양이 보인다. - 사용자 입력을 넣으면 위험하다 (XSS) - 통째로 다시 그려서 기존 요소가 전부 새것이 된다

— createElement

+ 안전하다. textContent로 넣으니 태그가 실행되지 않는다 + 필요한 것만 골라 붙이고 뗄 수 있다 - 줄이 길다

기준: 내가 만든 고정된 마크업 → innerHTML도 괜찮다 사용자가 입력한 값이 섞인다 → createElement + textContent

▹ 직접 해보기 6

사용자가 쓴 댓글로 목록을 만든다면 둘 중 어느 쪽을 써야 할까요? 이유도 한 줄로 적어 보세요.

이런 실수를 합니다

만들기만 하고 안 붙임

```
const lonely = document.createElement("li");
lonely.textContent = "화면에 안 보입니다";
console.log(lonely.textContent);
```

F12 콘솔 화면에 안 보입니다

값은 들어 있는데 화면에는 없습니다. appendChild 를 빠뜨린 것입니다. "코드는 맞는데 화면에 안 나와요" 의 1순위 원인입니다.

이런 실수를 합니다

createElement 에 태그 기호를 넣음

```
document.createElement("<li>");
```

태그 '이름' 만 씁니다. 꺾쇠는 넣지 않습니다.

이런 실수를 합니다

반복문 밖에서 요소를 한 번만 만들기

```
const wrongList = document.querySelector("#list2");
const oneLi = document.createElement("li");
["A", "B", "C"].forEach((t) => {
  oneLi.textContent = t;
  wrongList.appendChild(oneLi);
});
console.log(wrongList.children.length);
```

F12 콘솔 5

4개였는데 1개만 늘었습니다. 같은 요소를 세 번 붙이면 복사되는 게 아니라 '자리를 옮길' 뿐입니다. 요소는 반복문 '안'에서 매번 새로 만들어야 합니다.

```
oneLi.remove();
```

이런 실수를 합니다

appendChild 에 문자열을 넣음

```
list1.appendChild("안녕");
```

```
TypeError. appendChild
```

는 요소만 받습니다. 글자를 붙이려면 append("안녕") 을 쓰거나 textContent 를 쓰세요.

정리

- 1 createElement("태그이름") → textContent 채우기 → appendChild 로 붙이기
- 2 append 는 더 편하고, prepend 는 맨 앞에 붙인다.
- 3 반복문 안에서 매번 새로 만들어야 한다.
- 4 classList.add, dataset 으로 클래스와 데이터도 함께 붙인다.
- 5 요소.remove() 로 지우고, innerHTML = "" 로 안을 비운다.
- 6 사용자 입력이 섞이면 innerHTML 대신 createElement + textContent.

직접 해보기 정답

```
1 const li = document.createElement("li");
  li.textContent = "세 번째 항목";
  list1.appendChild(li);

2 const newLi = document.createElement("li");
  newLi.textContent = "맨 앞 새 항목";
  list1.prepend(newLi);
  console.log(list1.children[0].textContent);
  // 콘솔: 맨 앞 새 항목
```

→ appendChild 로 붙였다면 맨 뒤로 갔을 것입니다.

```
3) const colors = ["빨강", "파랑", "노랑"];
  colors.forEach((c) => {
```

```
const li = document.createElement("li");
li.textContent = c;
list2.appendChild(li);
```

```
});
```

```
4) document.querySelectorAll(".card").forEach((card) => {
```

```
    console.log(card.dataset.name);
```

```
});
```

```
// 콘솔: 아메리카노 / 라떼 / 케이크
```

→ 화면에 보이는 글자가 아니라, 붙여 둔 데이터만 따로 꺼낸 것입니다.

화면 글자는 "아메리카노 4000원 - 재고 10개" 처럼 섞여 있어서
이름만 필요할 때는 dataset 쪽이 훨씬 편합니다.

```
5 list1.children[0].remove(); 또는 list1.firstElementChild.remove();
```

```
6 createElement + textContent 를 써야 합니다. → 댓글은 사용자가 쓴 글입니다. innerHTML 로  
넣으면 누군가 태그를
```

적어 넣었을 때 그게 진짜 태그로 실행됩니다. (개념02 섹션 5의 XSS)
textContent 로 넣으면 무엇을 적든 글자로만 보입니다.

입력값 읽기

RUN 이 파일을 브라우저로 열고 F12 → Console 탭

화면의 입력칸에서 값을 꺼내는 방법입니다.

```
input.value      글자 / 숫자 칸
checkbox.checked   체크 여부 (true / false)
select.value     고른 option 의 value
```

value — 입력칸의 값

섹션 1

```
const nameInput = document.querySelector("#nameInput");

console.log(nameInput.value);
```

F12 콘솔 김민준

textContent 가 아니라 value 입니다. 아주 자주 틀리는 부분입니다.

```
console.log(nameInput.textContent);
```

F12 콘솔

빈 값입니다. input 은 여는 태그 하나뿐이라 안에 글자가 없습니다.

규칙: input, textarea, select → .value p, div, h1, li 등 → .textContent

값을 넣을 수도 있습니다.

```
nameInput.value = "이서연";
console.log(nameInput.value);
```

F12 콘솔 이서연

화면 이름 칸의 글자가 잠깐 "이서연" 이 됩니다

(바로 아래 줄에서 다시 "김민준" 으로 되돌리므로 눈에는 안 보입니다.
직접 보려면 아래 줄을 잠시 주석 처리하고 새로고침하세요)

```
nameInput.value = "김민준"; // 원래대로 되돌려 둡니다
```

memoArea 의 값을 콘솔에 찍어 보세요.

value 는 언제나 문자열이다

섹션 2

```
const priceInput = document.querySelector("#priceInput");
const countInput = document.querySelector("#countInput");

console.log(priceInput.value, typeof priceInput.value);
```

F12 콘솔 4500 string

type="number" 인데도 문자열입니다! 이게 함정입니다.

```
console.log(priceInput.value + countInput.value);
```

F12 콘솔 45003

곱하기를 하려 했는데 이어붙었습니다. (01단원 개념05 에서 배운 그것입니다)

만드시 Number() 로 바꾸고 계산합니다.

```
const price = Number(priceInput.value);
const count = Number(countInput.value);

console.log(price, typeof price);
```

F12 콘솔 4500 number

```
console.log(price * count);
```

F12 콘솔 13500

주의 · - * / 는 자동으로 숫자가 되지만 + 만 다릅니다.

```
console.log(priceInput.value * countInput.value);
```

F12 콘솔 13500

우연히 맞습니다. 하지만 + 를 쓰는 순간 무너지므로 습관적으로 Number() 를 붙이는 편이 안전합니다.

가격과 개수를 곱한 값에 1000 을 더해 콘솔에 찍어 보세요.

빈 값 다루기

섹션 3

```
const emptyInput = document.querySelector("#emptyInput");

console.log(emptyInput.value);
```

F12 콘솔

```
console.log(emptyInput.value === "");
```

F12 콘솔 true

비어 있으면 빈 문자열입니다. `null` 이나 `undefined` 가 아닙니다.

`placeholder` 는 값이 아닙니다. 그냥 안내 글자입니다.

```
console.log(emptyInput.placeholder);
```

F12 콘솔 여기는 비어 있습니다

빈 문자열은 `falsy` 라서 이렇게 검사합니다. (01단원, 03단원)

```
if (!emptyInput.value) {
    console.log("값을 입력해 주세요");
}
```

F12 콘솔 값을 입력해 주세요

공백만 친 경우도 걸러내려면 `trim()` 을 씁니다.

```
const onlySpace = " ";
console.log(onlySpace === "");
```

F12 콘솔 false

```
console.log(onlySpace.trim() === "");
```

F12 콘솔 true

`trim` 은 앞뒤 공백을 없앤 새 문자열을 돌려줍니다.

빈 값을 `Number` 로 바꾸면 0 입니다. `NaN` 이 아닙니다. (01단원)

```
console.log(Number(""));
```

F12 콘솔 0

“입력을 안 했는데 0으로 계산되는” 버그의 원인입니다. 먼저 비었는지 검사하세요.

▹ 직접 해보기 3

emptyInput 이 비어 있으면 “미입력”, 아니면 그 값을 콘솔에 찍는 코드를 써 보세요.

체크박스 — checked

섹션 4

```
const iceCheck = document.querySelector("#iceCheck");
const giftCheck = document.querySelector("#giftCheck");

console.log(iceCheck.checked);
```

F12 콘솔 true

```
console.log(giftCheck.checked);
```

F12 콘솔 false

value 를 보면 엉뚱한 값이 나옵니다.

```
console.log(iceCheck.value);
```

F12 콘솔 on

체크박스는 value 가 아니라 checked 를 봐야 합니다.

결과가 불리언이라 조건문에 바로 씁니다.

```
if (iceCheck.checked) {
  console.log("얼음을 추가합니다");
}
```

F12 콘솔 얼음을 추가합니다

삼항 연산자와도 잘 어울립니다. (03단원)

```
console.log(giftCheck.checked ? "선물 포장 함" : "선물 포장 안 함");
```

F12 콘솔 선물 포장 안 함

▹ 직접 해보기 4

giftCheck 를 체크된 상태로 바꿔 보세요. (checked = true)

select — 고른 값

섹션 5

```
const sizeSelect = document.querySelector("#sizeSelect");

console.log(sizeSelect.value);
```

F12 콘솔 L

화면에 보이는 “큰 것” 이 아니라 option 의 value 속성이 나옵니다.

화면에 보이는 글자가 필요하다면 이렇게 꺼냅니다.

```
console.log(sizeSelect.options[sizeSelect.selectedIndex].text);
```

F12 콘솔 큰 것

처음 보는 것이 두 개 나왔습니다. options 는 그 select 가 가진 option 들의 묶음입니다. 배열처럼 [0] 으로 꺼냅니다. .text 는 그 option 의 ‘화면에 보이는 글자’ 입니다. value 와는 다를 수 있습니다. 즉 “몇 번째가 골라졌는지 알아내서(selectedIndex), 그 option 의 글자를 꺼낸다” 입니다.

```
console.log(sizeSelect.selectedIndex);
```

F12 콘솔 1

몇 번째 option 인지 (0부터)

값을 바꾸면 화면의 선택도 따라 바뀝니다.

```
sizeSelect.value = "S";
```

◀ 직접 해보기 5

sizeSelect 의 값을 “S” 로 바꾼 다음, value 와 화면에 보이는 글자를 각각 콘솔에 찍어 보세요.

모아서 화면에 보여 주기

섹션 6

지금까지 읽은 값들을 정리해 결과 상자에 넣습니다.

```
const memo = document.querySelector("#memoArea").value;
const total = price * count;

const summary = `이름: ${nameInput.value}
사이즈: ${sizeSelect.value}
금액: ${price}원 x ${count}개 = ${total}원
얼음: ${iceCheck.checked ? "추가" : "없음"}
포장: ${giftCheck.checked ? "함" : "안 함"}
메모: ${memo || "(없음)"}`;

document.querySelector("#result").textContent = summary;
```

화면 결과 상자에 주문 내역이 여러 줄로 정리되어 나타납니다

```
console.log(summary);
```

F12 콘솔 위와 같은 내용이 여러 줄로 찍힙니다

백틱 안에서 줄을 바꾸면 그대로 줄바꿈이 됩니다. (02단원 개념05) 다만 HTML 에서는 줄바꿈이 무시되므로, CSS 에 white-space: pre-line 을 넣어 두었습니다. 그래서 화면에서도 여러 줄로 보입니다.

▹ 직접 해보기

6

memoArea 의 값을 빈 문자열로 만든 뒤, `${memo || "(없음)"}` 부분이 무엇을 내놓는지 확인해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

input 에 textContent 를 씀 (섹션 1에서 봤습니다)

이런 실수를 합니다

value 를 숫자라고 생각하고 더하기 (섹션 2)

이런 실수를 합니다

체크박스에 value 를 씀 (섹션 4)

이런 실수를 합니다

값을 페이지 열 때 한 번만 읽고 끝냄

이 파일이 그렇습니다. 입력칸을 고쳐도 결과가 안 바뀝니다. 사용자가 고칠 때마다 다시 읽으려면 11단원의 이벤트가 필요합니다.

이런 실수를 합니다

빈 값을 검사하지 않고 계산

```
const wrong = Number(emptyInput.value) * 100;
```

Number("") 은 0 이라 결과가 0 이 됩니다. "왜 계산이 0으로 나오지?" 하면 빈 칸부터 확인하세요.

정리

- 1 input / textarea / select 는 .value, 나머지 태그는 .textContent
- 2 value 는 type="number" 여도 항상 문자열이다. 계산 전 Number().
- 3 비어 있으면 빈 문자열 "". Number("") 은 0 이다.
- 4 체크박스는 .checked (true / false). value 가 아니다.
- 5 select 는 고른 option 의 value 를 준다.
- 6 입력에 반응하려면 11단원의 이벤트가 필요하다.

직접 해보기 정답

```
1 console.log(document.querySelector("#memoArea").value);
  // 콘솔: 덜 달게 해주세요

2 console.log(Number(priceInput.value) * Number(countInput.value) + 1000);
  // 콘솔: 14500

3 console.log(emptyInput.value || "미입력");
  // 콘솔: 미입력
  또는 if (emptyInput.value === "") { ... } else { ... }

4 giftCheck.checked = true;
```

→ 화면의 체크박스가 실제로 체크됩니다.

```
5) sizeSelect.value = "S";
console.log(sizeSelect.value);
// 콘솔: S
console.log(sizeSelect.options[sizeSelect.selectedIndex].text);
// 콘솔: 작은 것
```

→ value 를 바꾸면 selectedIndex 도 따라 0 으로 바뀝니다.

value("S") 와 화면 글자("작은 것") 가 다르다는 것이 이 섹션의 핵심입니다.

```
6) const memoArea = document.querySelector("#memoArea");  
memoArea.value = "";  
console.log(memoArea.value || "(없음)");  
// 콘솔: (없음)
```

→ 빈 문자열은 falsy 라서 || 의 오른쪽이 나옵니다. (03단원)

비어 있을 때 대신 보여 줄 글자를 정하는 가장 짧은 방법입니다.

10단원 · 연습문제

DOM 조작

RUN 브라우저로 열고 F12 → Console 탭

문제 1 (개념01 선택)

id 가 mainTitle 인 요소를 골라 콘솔에 찍으세요.

기대 콘솔 <h2 id="mainTitle">원래 제목</h2>

여기에 코드를 쓰세요

문제 2 (개념02 textContent)

id 가 target1 인 요소의 글자를 "바꿨습니다!" 로 바꾸세요.

기대 화면 해당 줄이 "바꿨습니다!" 로 보입니다

여기에 코드를 쓰세요

문제 3 (개념02 innerHTML)

id 가 target2 인 요소에 굵은 글자 를 넣어 실제로 굵게 보이도록 하세요.

기대 화면 그 줄이 "굵은 글자" 로 굵게 보입니다.
굵은 글자 라는 글자가 그대로 보이면 textContent 를 쓴 것입니다.

여기에 코드를 쓰세요

문제 4 (개념01 querySelectorAll)

클래스가 item 인 요소를 모두 골라 items 라는 변수에 담고, 그 개수를 콘솔에 찍으세요. (다음 문제에서 이 items 를 그대로 다시 씁니다. 변수에 꼭 담아 두세요)

기대 콘솔 3

여기에 코드를 쓰세요

문제 5 (개념01 forEach)

클래스가 item 인 요소들의 글자를 한 줄씩 콘솔에 찍으세요. 문제 4에서 만든 items 를 그대로 쓰세요. 같은 이름으로 다시 `const` 를 선언하면 `SyntaxError` 가 납니다.

기대 콘솔
항목 A
항목 B
항목 C

여기에 코드를 쓰세요

문제 6 (개념03 style)

id 가 styleTarget 인 요소의 글자색을 흰색, 배경색을 "#2d6cdf" 로 바꾸세요. CSS 이름을 그대로 쓰면 안 됩니다.

기대 화면 그 줄이 파란 배경(#2d6cdf)에 흰 글자로 바뀝니다.
아무 변화가 없으면 background-color 처럼 CSS 이름을 그대로 쓴 것입니다.

여기에 코드를 쓰세요

문제 7 (개념03 classList)

id 가 classTarget 인 요소에 highlight 클래스를 붙이세요. (CSS 에 이미 만들어져 있습니다)

기대 화면 그 줄이 노란 배경 + 굵은 글씨가 됩니다.

여기에 코드를 쓰세요

문제 8 (개념03 toggle)

id 가 toggleTarget 인 요소에 hidden 클래스를 toggle 로 붙여 숨기세요. 그다음 hidden 이 붙었는지 콘솔에 찍으세요.

기대 콘솔 true

여기에 코드를 쓰세요

문제 9 (개념04 createElement)

아래 배열을 이용해 id 가 fruitList 인 ul 안에 li 를 만들어 붙이세요. 각 항목은 "1. 사과" 처럼 번호가 붙어야 합니다.

```
const fruits = ["사과", "바나나", "포도"];
```

기대 화면

1. 사과
2. 바나나
3. 포도

여기에 코드를 쓰세요

문제 10 (개념04 클래스와 함께)

아래 상품 배열로 id 가 cardArea 인 div 안에 카드를 만들어 붙이세요. 각 카드는 div 이고 card 클래스를 가져야 하며, 내용은 "아메리카노 4000원" 형태입니다.

```
const products = [  
  { name: "아메리카노", price: 4000 },  
  { name: "케이크", price: 6000 },  
];
```

기대 화면

테두리가 있는 카드 두 개

여기에 코드를 쓰세요

문제 11 (개념04 remove)

id 가 removeMe 인 요소를 화면에서 지우세요.

기대 화면

"문제 11에서 사라집니다" 줄이 없어지고
"남아 있는 항목" 하나만 남습니다.

여기에 코드를 쓰세요

문제 12 (개념05 value)

id 가 nameInput 인 입력칸의 값을 콘솔에 찍으세요.

기대 콘솔 이서연

여기에 코드를 쓰세요

문제 13 [응용]

이름칸·가격칸·개수칸·사이즈선택·얼음체크 다섯 곳의 값을 모두 읽어 총액을 구하고, id 가 result 인 요소에 아래 형태로 넣으세요. - 가격 × 개수 = 총액 - 입력칸의 값은 문자열이라는 점에 주의하세요 - 체크박스는 .value 가 아니라 .checked 입니다 - 이름칸은 문제 12에서 이미 골라 뒀다면 그 변수를 다시 써도 됩니다

힌트: 쓸 id 는 이것들입니다 (이 파일 위쪽 body 에 있습니다).

nameInput / priceInput / countInput / sizeSelect / iceCheck / result
 값은 .value 로 읽고, 체크박스만 .checked 입니다.
 Number 로 바꾸지 않으면 4500 × 4 가 아니라 글자가 이어 붙습니다.

기대 화면 이서연님 / L 사이즈 / 18000원 / 얼음 추가

여기에 코드를 쓰세요

문제 14 [도전]

아래 배열에서 재고가 1개 이상인 상품만 골라 id 가 challengeList 인 ul 에 li 로 붙이세요. 내용은 "라떼 (재고 5개)" 형태이고, 품절 상품은 아예 만들지 않습니다. 08단원의 filter 와 09단원의 구조분해를 함께 써 보세요.

```
const stockList = [
  { name: "아메리카노", stock: 0 },
  { name: "라떼", stock: 5 },
  { name: "케이크", stock: 2 },
  { name: "쿠키", stock: 0 },
];
```

기대 화면 라떼 (재고 5개)
 케이크 (재고 2개)

여기에 코드를 쓰세요

문제 15 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 새로고침해서 콘솔에 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const nothing = document.querySelector("#존재하지않는아이디");
nothing.textContent = "안녕";
```

무슨 에러가 났나요? 이 에러가 나는 이유 두 가지는 무엇일까요?

답

DOM 조작

RUN 브라우저로 열고 F12 → Console 탭

```
const products = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
];
```

심화1 표 만들기

위 products 로 tbody 안에 tr 을 만들어 넣으세요. tr 하나에 td 세 개(이름·가격·재고)가 들어갑니다.

조건: - 가격은 "4,000원" 처럼 쉼표를 넣으세요 (toLocaleString 을 쓰면 됩니다) - 재고가 0이면 그 tr 에 soldout 클래스를 붙이고 재고칸에 "품절" 을 넣으세요

(CSS 에 이미 만들어 두었습니다. 회색 + 취소선이 됩니다)

힌트: tr 을 만들고 → td 세 개를 만들어 tr 에 붙이고 → tr 을 tbody 에 붙입니다.

두 겹이라 순서가 중요합니다. tr 안을 다 채운 뒤에 tbody 에 붙이세요.

기대 화면	표에 세 줄이 생깁니다.
	아메리카노 4,000원 10
	라떼 4,500원 품절 ← 회색 + 취소선
	케이크 6,000원 3

여기에 코드를 쓰세요

심화2 조건에 맞는 것만 세기

위에서 만든 표에서, 실제로 화면에 그려진 tr 을 골라 아래 세 가지를 콘솔에 찍으세요. products 배열이 아니라 '화면에 있는 요소' 를 세어야 합니다.

10단원 마무리 DOM 조작

자주 넘어지는 자리

1 `querySelector` 가 못 찾으면 `null`, 그 다음 줄에서 터진다는 것

자주 나오는 질문

Q “고쳤는데 화면이 그대로예요”

A `Ctrl+S` → 브라우저에서 `F5`. 이 두 단계를 매번 해야 합니다

이벤트

OPEN 11_이벤트

```
out1.textContent = "버튼을 눌렀습니다!";  
console.log("btn1 클릭됨");  
function sayClicked() {  
  console.log("함수가 실행되었습니다");  
}
```

- 01 click 이벤트
- 02 이벤트 객체
- 03 input 과 change
- 04 form submit
- 05 이벤트 위임
- 연습문제
- 심화문제

click 이벤트

RUN 브라우저로 열고 버튼을 눌러 보세요. F12 → Console 도 함께.

10단원까지는 페이지를 열 때 코드가 한 번 짝 실행되고 끝이었습니다. 이제 “사용자가 무언가 했을 때” 실행되게 만듭니다. 그것이 이벤트입니다.

```
요소.addEventListener("이벤트이름", 실행할함수);
~~~~~      ~~~~~      ~~~~~
"듣고 있어라" "click"      콜백 (08단원)
```

이렇게 붙여 둔 콜백 하나를 ‘리스너(listener)’ 라고 부릅니다. “듣고 있는 것” 이라는 뜻입니다. 앞으로 이 말이 종종 나옵니다.

넘긴 함수는 지금 실행되지 않습니다. 그 일이 생겼을 때 브라우저가 대신 불러 줍니다. 08단원에서 배운 콜백이 바로 이것입니다.

가장 기본 형태

섹션 1

```
const btn1 = document.querySelector("#btn1");
const out1 = document.querySelector("#out1");

btn1.addEventListener("click", () => {
  out1.textContent = "버튼을 눌렀습니다!";
  console.log("btn1 클릭됨");
});
```

이 줄이 실행된 시점에는 아무 일도 안 일어납니다. “누르면 이걸 해라” 라고 예약만 해 둔 것입니다.

```
console.log("예약 완료 - 아직 버튼은 안 눌렀습니다");
```

F12 콘솔 예약 완료 - 아직 버튼은 안 눌렀습니다

⇒ 직접 해보기 1

위 콜백 안에서 out1 의 글자를 자기 이름으로 바꿔 보세요.

콜백에 괄호를 붙이면 안 된다

섹션 2

```
function sayClicked() {
  console.log("함수가 실행되었습니다");
}
```

괄호 하나 차이를 눈으로 먼저 봅시다. 아래 두 줄은 실제로 실행됩니다.

```
console.log(sayClicked);
```

```
F12 콘솔    f sayClicked()
```

함수 '그 자체' 입니다. 아직 실행되지 않았습니다.

```
console.log(sayClicked());
```

```
F12 콘솔    함수가 실행되었습니다
            undefined
```

괄호를 붙이는 순간 실행됩니다. 그리고 그 '결과'가 남는데, 이 함수는 돌려주는 값이 없으니 결과가 `undefined` 입니다.

레시피 종이를 건네는 것(`sayClicked`)과 지금 요리해서 접시를 건네는 것(`sayClicked()`)의 차이입니다. 브라우저는 '나중에 놓렸을 때' 요리해야 하므로 종이를 받아야 합니다.

맞음 · 함수를 '넘긴다'

```
btn1.addEventListener("click", sayClicked);
```

틀림 · 함수를 '실행해서 그 결과를 넘긴다'

```
btn1.addEventListener("click", sayClicked());
```

→ 페이지를 여는 순간 한 번 실행되어 버립니다. → 그리고 결과인 `undefined` 는 함수가 아니라서 이벤트가 아예 안 붙습니다. → 정작 버튼을 누르면 아무 일도 일어나지 않습니다.

08단원 개념01 에서 배운 그 실수입니다. 이벤트에서 특히 자주 납니다.

인자를 넘겨야 한다면 화살표 함수로 감쌉니다.

```
function sayHello(name) {
  console.log(`${name}님 안녕하세요`);
}
```

```
btn1.addEventListener("click", sayHello("김민준")); ← 틀림
btn1.addEventListener("click", () => sayHello("김민준")); ← 맞음
```

▫ 직접 해보기 2

btn1 을 누르면 "홍길동님 안녕하세요" 가 나오도록 sayHello 를 붙여 보세요. 괄호 위치에 주의하세요.

값을 바꾸며 세기

섹션 3

```
const btn2 = document.querySelector("#btn2");
const out2 = document.querySelector("#out2");

let count = 0; // 콜백 '밖'에 만들어야 값이 유지됩니다

btn2.addEventListener("click", () => {
  count++;
  out2.textContent = count;
  console.log("현재 개수:", count);
});
```

count 를 콜백 '안'에 만들면 누를 때마다 0으로 되돌아갑니다. 05단원의 스코프 이야기가 여기서 그대로 쓰입니다.

▫ 직접 해보기 3

누를 때마다 2씩 늘어나게 바꿔 보세요.

하나에 여러 개 붙이기

섹션 4

```
const btn3 = document.querySelector("#btn3");
const out3 = document.querySelector("#out3");

btn3.addEventListener("click", () => {
  out3.textContent = "첫 번째 콜백이 실행됨";
  console.log("첫 번째");
});

btn3.addEventListener("click", () => {
  console.log("두 번째");
});
```

같은 이벤트에 여러 개를 붙이면 붙인 순서대로 전부 실행됩니다. 예전 방식인 onclick 은 나중에 쓴 것이 앞의 것을 덮어써서 하나만 남습니다.

```
btn.onclick = 함수A;
btn.onclick = 함수B; ← A 는 사라짐
```

그래서 addEventListener 를 씁니다.

▹ 직접 해보기 4

btn3 에 세 번째 콜백을 붙여 “세 번째” 를 찍어 보세요. 세 개가 어떤 순서로 나오는지 확인하세요.

켜고 끄기 (토글)

섹션 5

```
const btn4 = document.querySelector("#btn4");
const out4 = document.querySelector("#out4");

let isOn = false;

btn4.addEventListener("click", () => {
  isOn = !isOn; // 02단원의 ! - 뒤집기

  out4.classList.toggle("on", isOn); // 10단원 개념03
  out4.textContent = isOn ? "불이 켜졌습니다" : "불이 꺼져 있습니다";
  btn4.textContent = isOn ? "불 끄기" : "불 켜기";

  console.log("현재 상태:", isOn);
});
```

이 파일에서 지금까지 배운 게 전부 한자리에 모였습니다. ! (02단원) / 삼항 연산자 (03단원) / classList (10단원) / 콜백 (08단원)

▹ 직접 해보기 5

버튼 글자도 함께 바뀌도록 되어 있습니다. 글자를 “ON” / “OFF” 로 바꿔 보세요.

이벤트 떼기

섹션 6

addEventListener 로 붙인 것은 removeEventListener 로 뗄 수 있습니다. 다만 ‘같은 함수’ 여야 합니다. 그래서 이름 있는 함수로 만들어야 합니다.

```
function onceHandler() {
  console.log("이 메시지는 한 번만 나옵니다");
  btn1.removeEventListener("click", onceHandler);
}

btn1.addEventListener("click", onceHandler);
```

화살표 함수를 그 자리에서 넘기면 뗄 수 없습니다. 붙일 때와 뗄 때가 서로 다른 함수이기 때문입니다.

```
btn.addEventListener("click", () => {});
btn.removeEventListener("click", () => {}); ← 안 떼어집니다
```

한 번만 실행하고 싶다면 이 방법이 더 간단합니다.

```
btn2.addEventListener(  
  "click",  
  () => {  
    console.log("btn2 를 처음 누른 순간에만 나옵니다");  
  },  
  { once: true }  
);
```

▹ 직접 해보기 6

btn4 에 { once: true } 로 콜백을 하나 더 붙이고 두 번 눌러 보세요. 메시지가 몇 번 나오나요?

자주 쓰는 이벤트 이름

섹션 7

click	눌렀을 때	← 이 파일
input	입력칸에 글자가 바뀔 때마다	(개념03)
change	다 입력하고 벗어났을 때 / 체크·선택이 바뀔 때	(개념03)
submit	폼을 제출할 때	(개념04)
keydown	키를 눌렀을 때	
mouseover	마우스를 올렸을 때	

이름은 문자열입니다. "onclick" 이 아니라 "click" 입니다.

▹ 직접 해보기 7

btn1 에 mouseover 를 붙여, 마우스를 올리면 콘솔에 글자가 나오게 해 보세요. (누르지 않아도 나옵니다)

자주 하는 실수

섹션 8

이런 실수를 합니다

콜백에 괄호를 붙임 (섹션 2에서 봤습니다) — 가장 흔합니다.

이런 실수를 합니다

이벤트 이름에 on 을 붙임

```
btn1.addEventListener("onclick", ...);
```

에러도 안 나고 아무 일도 안 일어납니다. "click" 이 맞습니다.

이런 실수를 합니다

세는 변수를 콜백 안에 만듦 (섹션 3)

이런 실수를 합니다

요소를 못 찾은 채로 붙이기

```
document.querySelector("#없는버튼").addEventListener("click", ...);
```

```
TypeError: Cannot read properties of null
```

선택자 오타이거나 script 가 body 아래에 없는 경우입니다. (10단원 개념01)

이런 실수를 합니다

화살표 함수를 removeEventListener 로 떼려고 함 (섹션 6)

정리

- 1 요소.addEventListener("click", 콜백) — 누르면 콜백이 실행된다.
- 2 콜백에 괄호를 붙이지 않는다. 인자가 필요하다면 () ⇒ 함수(인자) 로 감싼다.
- 3 값을 유지해야 하는 변수는 콜백 밖에 만든다.
- 4 같은 이벤트에 여러 개를 붙일 수 있고, 붙인 순서대로 실행된다.
- 5 한 번만 실행하려면 { once: true }.
- 6 이벤트 이름은 "click" 이다. "onclick" 이 아니다.

직접 해보기 정답

```
1 out1.textContent = "홍길동";

2 btn1.addEventListener("click", () => sayHello("홍길동"));
  // 콘솔: 홍길동님 안녕하세요
```

→ sayHello("홍길동") 을 그대로 넘기면 안 됩니다. 그건 '지금 실행한 결과'

인 undefined 를 넘기는 것이라 이벤트가 아예 안 붙습니다.
화살표 함수로 한 겹 감싸야 '나중에 실행할 것' 을 넘기게 됩니다.

→ btn1 에는 이미 다른 콜백이 붙어 있어서 앞에 두 줄이 먼저 나옵니다.

"btn1 클릭됨" / "이 메시지는 한 번만 나옵니다" 다음에 나오면 맞습니다.

3 count += 2; (count++ 를 바꾸면 됩니다)

4 btn3.addEventListener("click", () => {

console.log("세 번째");

});

// 콘솔: 첫 번째 / 두 번째 / 세 번째

→ 붙인 순서대로 실행됩니다. 나중에 붙인 것이 앞의 것을 덮어쓰지 않습니다.

5 btn4.textContent = isOn ? "OFF" : "ON";

6 btn4.addEventListener(

"click",

() => {

console.log("한 번만 나옵니다");

},

{ once: true }

);

→ 한 번만 나옵니다. 두 번째 클릭부터는 안 나옵니다.

브라우저가 한 번 실행한 뒤 알아서 떼어 주기 때문입니다.

섹션 6의 removeEventListener 를 직접 부르지 않아도 됩니다.

7) btn1.addEventListener("mouseover", () => {

console.log("마우스가 올라왔습니다");

});

→ 클릭과 똑같은 형태입니다. 이벤트 '이름' 만 바꾸면 됩니다.

바꿀 수 있는 이름은 위 섹션 7의 목록에 있습니다.

이벤트 객체

RUN 브라우저로 열고 버튼·입력칸·링크를 눌러 보세요.

콜백은 매개변수를 하나 받을 수 있습니다.

```
btn.addEventListener("click", (e) => { ... });
                        ^
                    이벤트 객체
```

브라우저가 “무슨 일이 어디서 어떻게 일어났는지” 를 객체에 담아 넘겨 줍니다. 08단원에서 배운 “콜백의 매개변수는 실행하는 쪽이 넘겨 준다” 가 그대로 적용됩니다.

이름은 e, event, evt 무엇이든 됩니다. 보통 e 를 씁니다.

e.target — 실제로 눌린 요소

섹션 1

```
const out1 = document.querySelector("#out1");
const menuButtons = document.querySelectorAll(".menuBtn");

menuButtons.forEach((btn) => {
  btn.addEventListener("click", (e) => {
```

e.target 은 ‘실제로 눌린 요소’ 입니다

```
const name = e.target.textContent;
const price = e.target.dataset.price;

out1.textContent = `${name} ${price}원을 골랐습니다`;

console.log("눌린 요소:", e.target);
console.log("메뉴:", name, "가격:", price);
});
});
```

버튼이 세 개인데 콜백은 하나입니다. e.target 덕분에 “어느 것이 눌렸는지” 를 안에서 알 수 있습니다.

data-price 같은 속성은 dataset 으로 읽습니다. (10단원 개념04)

```
HTML: data-price="4000"
JS   : e.target.dataset.price
```

주의 · dataset 으로 읽은 값도 문자열입니다.

```
console.log(typeof menuButtons[0].dataset.price);
```

F12 콘솔 string

계산하려면 Number() 로 바꿔야 합니다.

▹ 직접 해보기

1

고른 메뉴 두 잔의 값을 계산해 출력해 보세요.

e.target 과 e.currentTarget

섹션 2

```
const cardBox = document.querySelector("#cardBox");
const out2 = document.querySelector("#out2");

cardBox.addEventListener("click", (e) => {
  console.log("target:", e.target.tagName);
  console.log("currentTarget:", e.currentTarget.tagName);

  out2.textContent = `실제 눌린 것: ${e.target.tagName} / 이벤트를 붙인 것:
  ${e.currentTarget.tagName}`;
});
```

tagName 은 그 요소의 '태그 이름' 입니다. 항상 대문자로 나옵니다.

<button> → "BUTTON", <div> → "DIV", <p> → "P"

파란 상자의 빈 곳을 누르면 → target 도 currentTarget 도 DIV 안쪽 버튼을 누르면 → target 은 BUTTON, currentTarget 은 DIV

e.target 실제로 눌린 그 요소 e.currentTarget addEventListener 를 붙인 요소

안쪽 요소를 누르면 그 이벤트가 부모에게도 전달됩니다. 이것을 '버블링' 이라고 합니다. 물속 공기방울이 위로 올라가듯 이벤트가 부모 쪽으로 올라갑니다.

버블링은 불편한 게 아니라 아주 유용합니다. 개념05에서 이걸로 큰 문제를 해결합니다.

▹ 직접 해보기

2

안쪽 버튼을 눌렀을 때와 상자 빈 곳을 눌렀을 때의 콘솔 출력을 비교해 보세요.

이벤트 객체에 뭐가 들어 있나

섹션 3

```
document.querySelector("#innerBtn").addEventListener("click", (e) => {
  console.log("type:", e.type);
```

```
F12 콘솔    type: click
```

```
console.log("눌린 좌표:", e.clientX, e.clientY);
```

```
F12 콘솔    눌린 좌표: (숫자) (숫자) ← 화면 기준 마우스 위치
```

```
});
```

자주 쓰는 것들:

```
e.type      이벤트 이름 ("click")
e.target    눌린 요소
```

e.currentTarget 이벤트를 붙인 요소

```
e.preventDefault() 기본 동작 막기 (섹션 5)
e.key              누른 키 (키보드 이벤트, 섹션 4)
```

궁금하면 `console.log(e)` 로 통째로 찍어 보세요. 속성이 아주 많습니다.

⇒ 직접 해보기 3

innerBtn 콜백 안에서 e.target 과 e.currentTarget 이 같은지 === 로 비교해 찍어 보세요. 왜 그런 결과가 나올까요?

키보드 이벤트

섹션 4

```
const keyInput = document.querySelector("#keyInput");
const out3 = document.querySelector("#out3");

keyInput.addEventListener("keydown", (e) => {
  out3.textContent = `누른 키: ${e.key}`;
  console.log("key:", e.key);
```

Enter 를 누르면 따로 처리하기 — 아주 자주 쓰는 패턴입니다

```

if (e.key === "Enter") {
  out3.textContent = `엔터! 입력값은 "${keyInput.value}" 입니다`;
  console.log("엔터가 눌렸습니다");
}
});

```

e.key 에 들어오는 값 글자 키 → "a", "1", "가" 특수 키 → "Enter", "Escape", "Backspace", "ArrowUp", " " (스페이스)

대소문자를 구분합니다. Shift + a 는 "A" 입니다.

▹ 직접 해보기

4

Escape 키를 누르면 입력칸을 비우도록 만들어 보세요.

preventDefault — 기본 동작 막기

섹션 5

```

const blockedLink = document.querySelector("#blockedLink");
const out4 = document.querySelector("#out4");

blockedLink.addEventListener("click", (e) => {
  e.preventDefault(); // 링크의 '페이지 이동' 을 막습니다

  out4.textContent = "이동을 막았습니다. 주소창을 보세요 - 그대로입니다.";
  console.log("링크 이동을 막았습니다");
});

```

브라우저가 원래 하려던 일을 취소합니다.

링크(a)	→ 페이지 이동	
폼(form)	→ 페이지 새로고침하며 제출	← 개념04 에서 핵심
체크박스	→ 체크 상태 바뀜	

e.preventDefault() 는 반드시 콜백 '안'에서, e 를 받아야 쓸 수 있습니다. 매개변수 e 를 안 적으면 ReferenceError 가 납니다.

▹ 직접 해보기

5

위 e.preventDefault(); 줄 앞에 // 를 붙여 주석 처리하고 링크를 눌러 보세요. 무엇이 달라지나요? 확인했으면 // 를 다시 지워 되돌려 놓으세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

매개변수 e 를 안 적고 e 를 씀

```
btn.addEventListener("click", () => { console.log(e.target); });
```

```
ReferenceError: e is not defined
```

괄호 안에 (e) 를 적어야 받을 수 있습니다.

이런 실수를 합니다

e.target 과 e.currentTarget 을 혼동 (섹션 2)

버튼 안에 아이콘이나 span 이 들어 있으면 e.target 이 그 안쪽 요소가 됩니다. "버튼을 눌렀는데 왜 다른 게 잡히지?" 하면 이 경우입니다. 이럴 땐 e.currentTarget 을 쓰거나 e.target.closest("button") 을 씁니다. (개념05)

이런 실수를 합니다

dataset 값을 숫자로 착각 (섹션 1)

이런 실수를 합니다

preventDefault 를 괄호 없이 씀

```
e.preventDefault;
```

아무 일도 안 일어납니다. 함수라서 괄호가 필요합니다.

이런 실수를 합니다

e.key 를 소문자로만 비교

```
if (e.key === "enter")
```

대문자 E 입니다. "Enter" 입니다.

정리

- 1 콜백의 첫 매개변수 e 가 이벤트 객체다.
- 2 e.target 은 실제로 눌린 요소, e.currentTarget 은 이벤트를 붙인 요소.
- 3 안쪽 요소의 이벤트는 부모로 올라간다(버블링).
- 4 data-이름 속성은 e.target.dataset.이름 으로 읽는다. 값은 문자열.
- 5 e.key 로 누른 키를 안다. "Enter", "Escape" 등.
- 6 e.preventDefault() 로 브라우저의 기본 동작을 막는다.

직접 해보기 정답

- 1 **const** price = **Number**(e.target.dataset.price); console.**log**(price * 2);
- 2 안쪽 버튼 → target: BUTTON / currentTarget: DIV 상자 빈 곳 → target: DIV / currentTarget: DIV
- 3 document.**querySelector**("innerBtn").**addEventListener**("click", (e) ⇒ {

```
console.log(e.target === e.currentTarget);
```

```
});  
// 콘솔: true
```

→ 이 콜백은 innerBtn '자신'에 붙어 있고, 누른 것도 innerBtn 이라

둘이 같습니다. 섹션 2의 cardBox 는 부모에 붙였기 때문에 달랐습니다.
즉 둘이 다른 경우는 "부모에 붙이고 자식을 눌렀을 때" 뿐입니다.

→ true 앞뒤로 다른 줄도 함께 나옵니다. 이 버튼에는 섹션 3의 콜백이

이미 붙어 있고, 클릭이 부모 cardBox 로 올라가 섹션 2의 콜백도
실행되기 때문입니다. 버블링을 눈으로 확인할 수 있는 자리입니다.
// 콘솔: type: click / 눌린 좌표: ... / true / target: BUTTON / currentTarget: DIV

```
4) if (e.key === "Escape") {
```

```
keyInput.value = "";  
out3.textContent = "입력칸을 비웠습니다";
```

```
}
```

5) 링크가 실제로 눌린 대로 페이지를 이동해 버립니다. → 주소창이 바뀌고 이 파일에서 벗어납니다. 뒤로 가기로 돌아오세요.

`preventDefault` 는 "브라우저가 원래 하려던 일을 하지 마라" 입니다.
링크의 원래 일이 '이동' 이었다는 걸 눈으로 확인하는 것이 이 실습입니다.

input 과 change

RUN 브라우저로 열고 직접 타이핑해 보세요.

10단원 개념05 에서 “값을 한 번만 읽어서 고쳐도 안 바뀐다” 고 했습니다. 그 문제를 해결하는 이벤트가 이것입니다.

input 글자가 바뀔 때마다 (한 글자씩)
change 다 입력하고 벗어났을 때 / 체크·선택이 바뀔 때

input — 실시간

섹션 1

```
const nameInput = document.querySelector("#nameInput");
const out1 = document.querySelector("#out1");

nameInput.addEventListener("input", (e) => {
```

e.target 이 곧 nameInput 입니다 (개념02)

```
const value = e.target.value;

out1.textContent = value ? `${value}님, 안녕하세요` : "여기에 실시간으로 나옵니다";
console.log("input:", value);
});
```

한 글자 칠 때마다 콜백이 실행됩니다. “김” → “김민” → “김민준” 이렇게 세 번 실행됩니다.

빈 값일 때 안내 문구로 되돌리려고 삼항 연산자를 썼습니다. (03단원)

▹ 직접 해보기

1

입력한 글자를 대문자로 바꿔 보여 주세요. (toUpperCase)

change — 벗어날 때

섹션 2

```
const changeInput = document.querySelector("#changeInput");
const out2 = document.querySelector("#out2");

changeInput.addEventListener("change", (e) => {
  out2.textContent = `확정된 값: ${e.target.value}`;
  console.log("change:", e.target.value);
});
```

직접 비교해 보세요. ① 칸에 타이핑 → 글자마다 바로 바뀜 ② 칸에 타이핑 → 아무 반응 없음 → Tab 을 누르거나 다른 곳을 클릭 → 그때 바뀜

언제 무엇을 쓰나? 실시간 검색, 글자 수 세기, 즉시 계산 → input 최종 값만 필요, 무거운 처리(서버 요청) → change

글자 입력칸에서는 input 을 훨씬 많이 씁니다.

▹ 직접 해보기

2

② 칸에 “커피” 를 치고 Tab 을 눌러 벗어나 보세요. 콘솔에 무엇이 언제 찍히나요? ① 칸과 비교해 보세요.

글자 수 세기

섹션 3

```
const memoArea = document.querySelector("#memoArea");
const out3 = document.querySelector("#out3");
const maxLength = 20;

memoArea.addEventListener("input", (e) => {
  const length = e.target.value.length; // 01단원의 .length

  out3.textContent = `${length} / ${maxLength}`;
```

넘치면 빨강게 (10단원 개념03)

```
out3.classList.toggle("over", length > maxLength);
});
```

toggle 의 두 번째 인자로 조건을 주면 붙였다 떼는 걸 한 줄로 씁니다.

▹ 직접 해보기

3

20자를 넘으면 “글자 수 초과” 라고 함께보여 주세요.

```
const priceInput = document.querySelector("#priceInput");
const countInput = document.querySelector("#countInput");
const out4 = document.querySelector("#out4");
```

계산하는 일을 함수로 빼 둡니다. 두 곳에서 똑같이 부를 수 있으니까요. (05단원)

```
function updateTotal() {
  const price = Number(priceInput.value); // 반드시 Number! (10단원 개념05)
  const count = Number(countInput.value);
  const total = price * count;

  out4.textContent = `합계: ${total}원`;
  console.log("합계 갱신:", total);
}

priceInput.addEventListener("input", updateTotal);
countInput.addEventListener("input", updateTotal);
```

같은 함수를 두 입력칸에 붙였습니다. 어느 쪽을 고치든 합계가 다시 계산됩니다.

여기서도 괄호를 붙이면 안 됩니다.

```
addEventListener("input", updateTotal)    ← 맞음
addEventListener("input", updateTotal())  ← 틀림
```

처음 한 번 실행해서 화면을 맞춰 둡니다.

```
updateTotal();
```

F12 콘솔 합계 갱신: 9000

▸ 직접 해보기 4

개수를 비우면 합계가 0원이 됩니다. 왜그럴까요? (힌트: Number("") 은 얼마일까요?)

체크박스과 select

섹션 5

```
const agreeCheck = document.querySelector("#agreeCheck");
const sizeSelect = document.querySelector("#sizeSelect");
const out5 = document.querySelector("#out5");

function updateOptions() {
  const agreeText = agreeCheck.checked ? "함" : "안 함";
  const size = sizeSelect.value;

  out5.textContent = `동의: ${agreeText} / 사이즈: ${size}`;
  console.log("옵션 갱신:", agreeText, size);
}
```

체크박스과 select 는 change 를 씁니다. input 도 동작하지만 change 가 관례입니다.

```
agreeCheck.addEventListener("change", updateOptions);
sizeSelect.addEventListener("change", updateOptions);
```

여기서도 처음 한 번은 직접 불러서 화면을 맞춰 둡니다. 섹션 4와 같은 이유입니다.

```
updateOptions();
```

F12 콘솔 옵션 갱신: 안 함 M

체크박스는 .checked (true/false), select 는 .value 입니다. (10단원 개념05)

⇒ 직접 해보기

5

동의하지 않으면 결과 글자를 빨강게 만들어 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

입력값을 Number 로 안 바꿈

```
const total = priceInput.value * countInput.value; ← 우연히 됨
const total = priceInput.value + countInput.value; ← "45002" 가 됨
```

습관적으로 Number() 를 붙이세요.

이런 실수를 합니다

이벤트 이름을 "onchange" 로 씀

addEventListener("onchange", ...) ← 아무 일도 안 일어남

이런 실수를 합니다

처음 화면을 안 맞춰 둠

이벤트는 '바뀔 때' 만 실행됩니다. 페이지를 열자마자의 화면은 직접 한 번 함수를 불러서 맞춰야 합니다.
(섹션 4의 `updateTotal()` 호출)

이런 실수를 합니다

체크박스에 `value` 를 씀

`agreeCheck.value` → 항상 "on" 입니다. `.checked` 를 쓰세요.

이런 실수를 합니다

`input` 이벤트가 `select` 에서도 될 거라 생각

됩니다만, 체크·선택은 `change` 를 쓰는 것이 관례입니다.

정리

- 1 `input` — 글자가 바뀔 때마다. 실시간 처리에 쓴다.
- 2 `change` — 다 입력하고 벗어났을 때 / 체크·선택이 바뀔 때.
- 3 `e.target.value` 로 값을 읽는다. 계산 전에는 `Number()`.
- 4 처리 내용을 함수로 빼면 여러 입력칸에 같은 함수를 붙일 수 있다.
- 5 페이지를 열자마자의 화면은 함수를 한 번 직접 불러서 맞춘다.

직접 해보기 정답

- 1 `out1.textContent = value.toUpperCase();`
- 2 타이핑하는 동안에는 아무것도 안 찍히다가, Tab 을 눌러 칸을 벗어나는 순간 딱 한 번 찍힙니다.

```
// 콘솔: change: 커피  
// 화면: 확정된 값: 커피
```

→ ① 칸은 "커피" 에서 한 번, "커피" 에서 또 한 번 — 글자마다 찍혔습니다.

두 글자를 치면 input 은 2번, change 는 1번 실행됩니다.
서버에 요청을 보내는 무거운 일이라면 이 차이가 아주 커집니다.

3 out3.textContent = `\${length} / \${maxLength}` + (length > maxLength ? " 글자 수 초과" : "");

4 Number("") 은 0 입니다. 그래서 가격 $\times 0 = 0$ 이 됩니다. 막으려면 빈 값을 먼저 검사하세요.

```
const count = Number(countInput.value) || 1;
```

```
5) out5.classList.toggle("over", !agreeCheck.checked);
```

form submit

RUN 브라우저로 열고 입력 후 추가 버튼 또는 Enter

입력칸이 여러 개인 화면은 form 태그로 묶습니다. 이유가 있습니다.

- 1 Enter 키만 쳐도 제출됩니다 (버튼을 안 눌러도 됨)
- 2 제출 처리를 한 곳(submit 이벤트)에서 합니다
- 3 브라우저의 기본 검사(required 등)를 쓸 수 있습니다

submit 이벤트와 preventDefault

섹션 1

```
const todoForm = document.querySelector("#todoForm");
const todoInput = document.querySelector("#todoInput");
const todoList = document.querySelector("#todoList");
const errorMsg = document.querySelector("#errorMsg");

todoForm.addEventListener("submit", (e) => {
  e.preventDefault(); // * 이 줄이 없으면 페이지가 새로고침됩니다 console.log("submit
이벤트 발생");
  handleSubmit();
});
```

e.preventDefault() 를 빠뜨리면 어떻게 될까요? form 의 기본 동작은 “입력값을 서버로 보내고 페이지를 새로고침” 입니다. 그래서 화면이 깜빡이면서 방금 만든 목록이 전부 사라집니다. “버튼을 눌렀는데 화면이 초기화돼요” 의 원인이 거의 항상 이것입니다.

▹ 직접 해보기

1

글로 읽으면 안 와닿습니다. 3초면 되니 직접 보세요.

① 할 일을 두세 개 추가해서 목록을 만듭니다. ② 위의 e.preventDefault(); 줄 맨 앞에 // 를 붙여 주석 처리합니다. ③ 저장하고 새로고침한 뒤, 다시 추가해 봅니다. → 화면이 한 번 깜빡이면서 목록이 통째로 사라집니다. → 주소창도 보세요. ?todoInput=장보기 처럼 값이 주소에 붙습니다.

"서버로 보냈다" 는 흔적입니다.

④ 확인했으면 // 를 다시 지워서 되돌려 놓으세요.

form 을 다룰 때는 preventDefault 를 반사적으로 쓴다고 생각하세요.

입력값 검사

섹션 2

```
function handleSubmit() {
  const value = todoInput.value.trim(); // 앞뒤 공백 제거 (10단원 개념05)
```

검사 1 · 비어 있는가 — 조기 반환 (05단원 개념03)

```
if (!value) {
  errorMsg.textContent = "할 일을 입력해 주세요";
  todoInput.focus(); // 커서를 다시 입력칸으로 console.log("검사 실패: 빈 값");
  return; // 여기서 함수가 끝납니다
}
```

검사 2 · 너무 긴가

```
if (value.length > 20) {
  errorMsg.textContent = "20자 이내로 입력해 주세요";
  console.log("검사 실패: 너무 김");
  return;
}
```

검사 3 · 이미 있는가

요소.children 은 그 안에 들어 있는 자식 요소들의 묶음입니다. NodeList 처럼 배열이 아니라서, map 을 쓰려면 [...] 로 펼쳐야 합니다. (10단원 개념01 섹션 4에서 boxes 에 했던 것과 같습니다)

```
const existing = [...todoList.children].map((li) => li.textContent);
if (existing.includes(value)) {
  errorMsg.textContent = "이미 있는 항목입니다";
  console.log("검사 실패: 중복");
  return;
}
```

여기까지 왔다면 통과입니다

```
errorMsg.textContent = "";
addTodo(value);
}
```

검사를 위에서 하나씩 하고 안 맞으면 바로 return 합니다. else 가 하나도 없어서 들여쓰기가 얇고 읽기 좋습니다. 조기 반환 패턴입니다.

trim() 을 쓴 이유: 공백만 친 경우도 걸러내기 위해서입니다. " " 는 빈 문자열이 아니라서 !value 로는 못 걸러냅니다.

▹ 직접 해보기 2

입력칸에 스페이스만 세 칸 치고 추가를 눌러 보세요. 무엇이 나오나요? `trim()` 이 없었다면 어떻게 됐을까요?

목록에 추가하고 입력칸 비우기

섹션 3

```
function addTodo(text) {  
  const li = document.createElement("li"); // 10단원 개념04  
  li.textContent = text;  
  todoList.appendChild(li);  
  
  todoInput.value = ""; // 입력칸 비우기  
  todoInput.focus(); // 커서를 다시 입력칸으로 - 연속 입력이  
  편해집니다 console.log("추가됨:", text, "/ 현재", todoList.children.length, "개");  
}
```

입력칸을 비우고 `focus` 를 주는 것까지가 한 세트입니다. 이게 없으면 사용자가 매번 지우고 다시 클릭해야 합니다.

폼 전체를 비우려면 이렇게 씁니다.

```
todoForm.reset();
```

입력칸이 여러 개일 때 편합니다.

▹ 직접 해보기 3

추가할 때마다 콘솔에 전체 개수를 찍고 있습니다. 화면에도 “총 N개” 를 보여 주려면 어디를 고쳐야 할까요?

버튼의 type 을 조심하세요

섹션 4

form 안의 button 은 type 을 안 쓰면 기본이 submit 입니다.

```
<button>추가</button>           → type="submit" 과 같음  
<button type="submit">추가</button> → 제출 (이 파일)  
<button type="button">그냥버튼</button> → 제출하지 않음
```

form 안에 “취소” 나 “지우기” 같은 버튼을 넣을 때 `type="button"` 을 안 쓰면 그 버튼도 폼을 제출해 버립니다. 아주 자주 겪는 문제입니다.

▹ 직접 해보기 4

이 폼에 “전부 지우기” 버튼을 넣는다면 type 을 무엇으로 해야 할까요? 안 쓰면 무슨 일이 생길까요?

click 대신 submit 을 쓰는 이유

섹션 5

입력칸에서 Enter 를 치면 브라우저가 알아서 form 을 제출합니다. 그래서 이 파일에서는 Enter 만 쳐도 할 일이 추가됩니다. 직접 해 보세요.

그럼 버튼에 click 을 붙이면 안 될까요? form 안이라면 그것도 동작합니다. Enter 를 치면 브라우저가 제출 버튼을 대신 눌러 주기 때문입니다. (섹션 4에서 “form 안의 다른 버튼도 폼을 제출해 버린다” 고 한 것이 이 동작 때문입니다)

그래도 submit 을 쓰는 이유는 이렇습니다. 1) 제출하는 길이 여러 개여도(버튼 클릭 / Enter / 나중에 추가할 다른 버튼)

받는 곳은 한 군데뿐입니다.

2) form 이 없으면 Enter 가 아예 안 먹습니다. 그때는 keydown 을 따로 달아야

하는데(개념05 가 그렇게 합니다) form 을 쓰면 그럴 필요가 없습니다.

3) input 에 required 를 달면 브라우저가 알아서 빈 칸을 막아 줍니다.

▹ 직접 해보기 5

입력칸에 “장보기” 를 치고 버튼을 누르지 말고 Enter 만 쳐 보세요. 버튼을 누른 것과 같은 일이 일어나나요?

자주 하는 실수

섹션 6

이런 실수를 합니다

preventDefault 를 빠뜨림 (섹션 1) — 1순위 실수입니다.

이런 실수를 합니다

매개변수 **e** 를 안 받고 **preventDefault** 를 씀

```
form.addEventListener("submit", () => { e.preventDefault(); });
```

ReferenceError: e is not defined

이런 실수를 합니다

검사 후 `return` 을 빠뜨림

```
if (!value) { errorMsg.textContent = "..."; }  
addTodo(value);
```

에러 메시지를 띄우고도 빈 항목이 추가됩니다. `return` 이 필요합니다.

이런 실수를 합니다

form 안의 다른 버튼에 `type="button"` 을 안 씀 (섹션 4)

이런 실수를 합니다

`trim` 을 안 해서 공백만 입력한 것이 통과

공백 세 칸 " " 는 빈 문자열이 아니라 `truthy` 입니다.

정리

- 1 `form.addEventListener("submit", (e) => { e.preventDefault(); ... })`
- 2 `preventDefault` 를 빠뜨리면 페이지가 새로고침되어 내용이 날아간다.
- 3 검사는 위에서 하나씩 하고 안 맞으면 바로 `return` (조기 반환).
- 4 `value.trim()` 으로 공백만 입력한 경우를 걸러낸다.
- 5 처리 후 입력칸을 비우고 `focus()` 를 준다.
- 6 form 안의 제출용이 아닌 버튼에는 `type="button"` 을 쓴다.

직접 해보기 정답

1) 화면이 한 번 깜빡이면서 방금 만든 목록이 통째로 사라집니다. 주소창에 `?todoInput=장보기` 가 붙는 것도 함께 보세요. → form 의 기본 동작인 "서버로 보내고 새로고침" 이 실행된 것입니다.

`preventDefault` 는 그 기본 동작을 취소하는 역할입니다.

2) // 콘솔: 검사 실패: 빈 값
// 화면: 할 일을 입력해 주세요

→ `trim()` 이 " " 를 "" 로 만들어 주어 `!value` 에 걸립니다.

`trim()` 이 없었다면 " " 는 `truthy` 라서 검사를 통과하고, 화면에 아무것도 안 보이는 빈 항목이 목록에 추가됩니다.

3) 화면에 개수를 보여 줄 요소를 하나 만들고(예: `<p id="countText">`), `addTodo` 함수 안에서 함께 갱신하면 됩니다.

```
document.querySelector("#countText").textContent =
```

```
`총 ${todoList.children.length}개`;
```

삭제 기능이 생기면 개수 갱신도 여러 곳에서 필요해집니다. 그럴 때는 갱신만 하는 함수(`updateCount`)를 따로 만들어 두고 추가·삭제 양쪽에서 부르는 것이 좋습니다.

4) `type="button"` 을 써야 합니다.

```
<button type="button">전부 지우기</button>
```

→ `type` 을 안 쓰면 기본이 `submit` 이라, 지우려고 누를 때마다 폼이

제출됩니다. `handleSubmit` 이 실행되면서 엉뚱하게 항목이 추가되거나
"할 일을 입력해 주세요" 가 뜹니다.

5) 같은 일이 일어납니다. 항목이 그대로 추가됩니다.

```
// 콘솔: submit 이벤트 발생
// 콘솔: 추가됨: 장보기 / 현재 1 개
```

→ 입력칸에서 `Enter` 를 치면 브라우저가 알아서 폼을 제출하기 때문입니다.

버튼 클릭이든 `Enter` 든 결국 같은 `submit` 이벤트로 모입니다.
이것이 `click` 대신 `submit` 을 쓰는 가장 큰 이유입니다.

이벤트 위임

RUN 브라우저로 열고 추가·완료·삭제를 해 보세요.

문제 상황부터 이해해야 합니다.

목록의 삭제 버튼마다 `addEventListener` 를 붙였습니다. 그런데 나중에 새로 만든 항목의 버튼에는 이벤트가 안 붙어 있습니다. 그때는 이미 `addEventListener` 를 다 돌린 뒤니까요.

해결책: 자식마다 붙이지 말고 '부모 하나'에 붙이고,

`e.target` 으로 "무엇이 눌렸는지" 를 안에서 판단합니다.
이것을 이벤트 위임이라고 합니다.

```
const todoInput = document.querySelector("#todoInput");
const addBtn = document.querySelector("#addBtn");
const todoList = document.querySelector("#todoList");
const status = document.querySelector("#status");

let nextId = 2; // 처음 항목이 1번이므로 2부터
```

안 되는 방법 (참고용)

섹션 1

이렇게 하면 '지금 있는' 삭제 버튼에만 붙습니다.

```
document.querySelectorAll(".deleteBtn").forEach((btn) => {

  btn.addEventListener("click", () => btn.closest("li").remove());

});
```

처음 항목은 지워지지만, 새로 추가한 항목의 버튼은 눌러도 아무 일이 없습니다. 추가할 때마다 다시 붙이는 방법도 있지만 코드가 지저분해지고 느려집니다.

▹ 직접 해보기

1

위 코드가 '지금 있는' 버튼에만 붙는 이유를 말해 보세요. (힌트: `querySelectorAll` 은 언제 실행되나요?)

되는 방법 — 부모에 한 번만

섹션 2

```

todoList.addEventListener("click", (e) => {
  console.log("눌린 요소:", e.target.className || e.target.tagName);
});

```

— 삭제 버튼을 눌렀나?

```

if (e.target.classList.contains("deleteBtn")) {
  const li = e.target.closest("li");
  console.log("삭제:", li.dataset.id);
  li.remove();
  updateStatus();
  return;
}

```

— 할 일 글자를 눌렀나?

```

if (e.target.classList.contains("text")) {
  const li = e.target.closest("li");
  li.classList.toggle("done");
  console.log("완료 토글:", li.dataset.id);
  return;
}

```

그 외(li의 빈 공간 등)는 무시합니다

```
});
```

이벤트는 자식에서 부모로 올라옵니다(버블링, 개념02 섹션 2). 그래서 li안의 버튼을 눌러도 ul이 그 이벤트를 받을 수 있습니다.

그림으로 보면 이렇습니다.

```

<ul id="todoList">
  <li>
    <button>삭제</button>
  </li>
</ul>

```

← ③ 콜백은 여기 하나만 붙어 있습니다 (e.currentTarget)
 ← ① 손가락이 실제로 닿은 곳 (e.target)
 ② 클릭이 ①에서 ③으로 타고 올라갑니다

콜백은 ③에 하나뿐인데, ①이 무엇이었는지는 e.target으로 알 수 있습니다. 그래서 “누가 눌렀는지”를 잃지 않으면서도 콜백은 하나면 됩니다.

ul은 처음부터 화면에 있으므로 한 번만 붙이면 됩니다. 나중에 li를 몇 개를 추가하든 전부 이 콜백 하나가 처리합니다.

▫ 직접 해보기 2

첫 항목의 글자를 눌러 보고, 이어서 삭제 버튼도 눌러 보세요. "눌린 요소:" 뒤에 각각 무엇이 찍히나요?

closest — 가장 가까운 조상 찾기

섹션 3

`e.target` 은 '실제로 눌린 것' 입니다. 삭제 버튼을 누르면 `button` 입니다. 그런데 우리가 지우고 싶은 건 그 버튼이 속한 `li` 전체입니다.

```
e.target.closest("li")
```

`closest` 는 자기 자신부터 시작해서 부모 쪽으로 올라가며 조건에 맞는 첫 요소를 찾아 줍니다.

```
button.closest("li") → 그 버튼을 감싸고 있는 li
```

없으면 `null` 을 돌려줍니다.

```
console.log(addBtn.closest("li"));
```

```
F12 콘솔 null
```

`addBtn` 은 `li` 안에 있지 않으니까요.

`parentElement` 를 여러 번 쓰는 것보다 훨씬 안전합니다.

```
e.target.parentElement.parentElement ← 구조가 바뀌면 깨짐  
e.target.closest("li")                 ← 구조가 바뀌어도 동작
```

▫ 직접 해보기 3

첫 삭제 버튼을 고른 뒤 `closest("li")` 로 그 버튼을 감싼 `li` 를 찾고, 그 `li` 의 `dataset.id` 를 콘솔에 찍어 보세요.

항목 추가

섹션 4

```
function addTodo() {  
  const text = todoInput.value.trim();  
  if (!text) {  
    console.log("빈 값이라 추가하지 않습니다");  
    return;  
  }  
}
```

이벤트

RUN 브라우저로 열고 직접 눌러 보세요. F12 → Console 도 함께

문제 1 (개념01 click)

btn1 을 누르면 out1 의 글자가 "눌렀습니다!" 로 바뀌게 하세요.

기대 화면 버튼을 누르면 out1 이 "눌렀습니다!" 로 바뀝니다.

여기에 코드를 쓰세요

문제 2 (개념01 카운터)

btn2 를 누를 때마다 out2 의 숫자가 1씩 올라가게 하세요. 세는 변수를 어디에 만들어야 하는지 주의하세요.

기대 화면 버튼을 누를 때마다 out2 가 1 → 2 → 3 으로 올라갑니다.
누를 때마다 계속 1 이면 세는 변수를 콜백 안에 만든 것입니다.

여기에 코드를 쓰세요

문제 3 (개념01 토글)

btn3 을 누를 때마다 out3 에 on 클래스가 붙었다 떨어지고, 글자도 "켜짐" / "꺼짐" 으로 바뀌게 하세요.

기대 화면 처음에는 "꺼짐" 입니다.
한 번 누르면 "켜짐" + 배경색이 바뀌고, 또 누르면 "꺼짐" 으로 돌아옵니다.

여기에 코드를 쓰세요

문제 4 (개념02 e.target)

menuBtn 클래스를 가진 버튼 두 개 중 무엇을 눌렀든, out4 에 “아메리카노를 골랐습니다” 처럼 그 버튼의 글자가 나오게 하세요. 콜백은 하나만 만들어서 forEach 로 붙이세요.

기대 화면 "아메리카노" 를 누르면 out4 → 아메리카노를 골랐습니다
 "케이크" 를 누르면 out4 → 케이크를 골랐습니다

여기에 코드를 쓰세요

문제 5 (개념02 dataset)

문제 4의 콜백을 고쳐서, 누른 메뉴를 '3개' 샀을 때의 금액을 콘솔에 찍으세요. 가격은 버튼의 data-price 에 들어 있고, 그 값은 문자열이라는 점에 주의하세요.

기대 콘솔 아메리카노를 누르면 12000 / 케이크를 누르면 18000

문제 4의 콜백 안에 함께 쓰세요

문제 6 (개념02 keydown)

keyInput 에서 Enter 를 누르면 out6 에 “입력한 값: 000” 이 나오게 하세요.

기대 화면 "김민준" 을 치고 Enter → out6 → 입력한 값: 김민준
 Enter 가 아닌 키를 눌렀을 때는 "대기 중" 그대로여야 합니다.

여기에 코드를 쓰세요

문제 7 (개념02 preventDefault)

myLink 를 눌러도 페이지가 이동하지 않게 막고, out7 에 “이동을 막았습니다” 를 넣으세요.

기대 화면 링크를 눌러도 페이지가 그대로 있고 out7 → 이동을 막았습니다
화면이 맨 위로 튀거나 주소창이 바뀌면 막지 못한 것입니다.

여기에 코드를 쓰세요

문제 8 (개념03 input)

nameInput 에 글자를 칠 때마다 out8 에 그 값이 바로 나오게 하세요.

기대 화면 "김" 을 치면 out8 → 김, "김민" 까지 치면 out8 → 김민 (글자마다 바로)

여기에 코드를 쓰세요

문제 9 (개념03 글자 수)

같은 입력칸의 글자 수를 out9 에 “5글자” 형태로 보여 주세요.

기대 화면 같은 입력칸에 "김민준" 을 치면 out9 → 1글자 → 2글자 → 3글자

여기에 코드를 쓰세요

문제 10 (개념03 체크박스)

agreeCheck 를 체크하면 out10 이 "동의: 함", 풀면 "동의: 안 함" 이 되게 하세요.

기대 화면 체크하면 out10 → 동의: 함, 풀면 out10 → 동의: 안 함

여기에 코드를 쓰세요

문제 11 (개념03 select)

sizeSelect 를 바꾸면 out11 에 "사이즈: L" 처럼 고른 값이 나오게 하세요.

기대 화면 "큰 것" 을 고르면 out11 → 사이즈: L
화면에 보이는 글자("큰 것")가 아니라 value("L")가 나와야 합니다.

여기에 코드를 쓰세요

문제 12 (개념04 submit)

loginForm 을 제출하면 페이지가 새로고침되지 않게 막고, out12 에 "OOO님 환영합니다" 를 넣으세요.
(OOO 은 idInput 에 친 값)

기대 화면 "김민준" 을 치고 Enter → out12 → 김민준님 환영합니다
화면이 깜빡이며 글자가 사라지면 새로고침을 못 막은 것입니다.
버튼을 눌러도 Enter 를 쳐도 똑같이 동작해야 합니다.

여기에 코드를 쓰세요

문제 13 (개념05 이벤트 위임)

colorList 안의 색을 누르면 out13 에 "고른 색: 빨강" 처럼 나오게 하세요. li 마다 붙이지 말고 colorList 하나에만 붙이세요.

기대 화면 "초록" 을 누르면 out13 → 고른 색: 초록
li 세 개에 각각 붙였다면 위임이 아닙니다. 다시 해 보세요.

여기에 코드를 쓰세요

문제 14 [응용] (개념04 form)

memoForm 을 제출하면 - 페이지가 새로고침되지 않아야 하고 - 입력이 비어 있으면(공백만 친 경우 포함) errorMsg 에

"메모를 입력해 주세요" 가 나와야 하고

- 값이 있으면 memoList 에 li 로 추가되고 입력칸이 비워져야 합니다.

기대 화면 공백만 넣고 추가 → errorMsg → 메모를 입력해 주세요 (목록은 그대로 0개)
"우유 사기" 넣고 추가 → 목록에 "우유 사기" 가 생기고, 입력칸과 errorMsg 가 비워집니다.
어느 쪽이든 화면이 새로고침되면 안 됩니다. 새로고침되면 목록이 통째로 사라집니다.

여기에 코드를 쓰세요

문제 15 [도전] (개념05 이벤트 위임)

itemList 안의 삭제 버튼을 누르면 그 항목(li)이 사라지게 하세요. 단, 버튼마다 붙이지 말고 itemList 하나에만 붙이세요. (그래야 나중에 항목을 추가해도 동작합니다)

기대 출력 처음 항목이 2개입니다. 삭제 버튼을 한 번 누르면 1개가 됩니다.

F12 콘솔 삭제됨. 남은 항목: 1

글자만 사라지고 항목이 남으면 e.target 을 그대로 지운 것입니다.

여기에 코드를 쓰세요

문제 16 (에러 확인 - 맨 마지막)

아래 줄의 주석을 풀고 버튼을 눌러 보세요. 그리고 페이지를 열자마자 콘솔에 무엇이 찍히는지 보세요.

```
document.querySelector("#btn1").addEventListener("click", console.log("눌림"));
```

버튼을 눌러도 아무 일이 없고, 페이지를 열 때 한 번 찍혔을 겁니다. 왜 그럴까요?

답

이벤트

RUN 브라우저로 열고 F12 → Console 탭

심화 1 위임 하나로 세 가지 동작

cartList 에 click 을 ‘한 번만’ 붙여서 아래 세 가지를 모두 처리하세요.

.plus 누르면 그 줄의 수량 +1
 .minus 누르면 그 줄의 수량 -1 (단, 1 아래로는 안 내려갑니다)

.remove 누르면 그 줄(li)을 삭제

그리고 무엇이 바뀌든 cartInfo 를 다시 채우세요. “담긴 상품 2종 / 총 3개” (종류 수 / 수량 합계)

힌트: e.target.classList.contains(...) 로 어느 버튼인지 가릅니다.

수량은 그 줄의 .count 요소에서 textContent 를 읽어 Number 로 바꿉니다.
 줄을 찾을 때는 closest("li") 입니다.
 정보 갱신은 함수로 빼서 세 곳에서 부르세요.

기대 화면 아메리카노의 + 를 두 번 → 수량 3, 정보 "담긴 상품 2종 / 총 5개"
 케이크의 - 를 세 번 → 수량 1 에서 멈춤 (0 이나 음수가 되면 안 됩니다)
 케이크 삭제 → 정보 "담긴 상품 1종 / 총 3개"

여기에 코드를 쓰세요

심화 2 실시간 검증

idInput 에 글자를 칠 때마다 아래를 판정하세요.

비어 있으면 → 안내 없음, 버튼 잠금 4글자 미만이면 → “4자 이상 입력하세요”, 버튼 잠금 영문이 아닌 게
 있으면 → “영문만 쓸 수 있습니다”, 버튼 잠금 둘 다 통과하면 → 안내 비우고 버튼 잠금 해제

힌트: 영문만 있는지는 아직 배운 게 없습니다. 아래 함수를 그대로 쓰세요.

(정규표현식이라는 것인데 이 자료의 범위 밖입니다)

버튼 잠금은 disabled = true / false 입니다. (12단원에서 쓰입니다)

```
function isEnglishOnly(text) {
```

영문 대소문자만 있으면 true

```
  for (const ch of text) {
    const isUpper = ch >= "A" && ch <= "Z";
    const isLower = ch >= "a" && ch <= "z";
    if (!isUpper && !isLower) {
      return false;
    }
  }
  return true;
}
```

기대 화면 "ab" → 4자 이상 입력하세요 / 버튼 잠김
 "abcd" → 안내 없음 / 버튼 눌림
 "abc1" → 영문만 쓸 수 있습니다 / 버튼 잠김
 전부 지움 → 안내 없음 / 버튼 잠김

여기에 코드를 쓰세요

심화 3 키보드 단축키

memoInput 에서 아래 세 가지를 처리하세요.

Enter → memoOut 에 “저장: 입력값” 을 넣고 입력칸 비우기

(빈 값이면 아무 일도 안 함)

Escape → 입력칸 비우기, memoOut 에 “취소했습니다” 그 외 키 → 아무 일도 안 함

힌트: e.key 로 어떤 키인지 알 수 있습니다. 대소문자를 정확히 쓰세요.

"Enter" / "Escape" 입니다.

조건에 안 맞으면 return 으로 빨리 빠져나오는 방식을 쓰세요.

기대 화면	"회의 준비" 치고 Enter	→ 저장: 회의 준비 / 입력칸 비워짐
	글자 치다가 Escape	→ 취소했습니다 / 입력칸 비워짐
	빈 칸에서 Enter	→ 아무 변화 없음

여기에 코드를 쓰세요

11단원 마무리 이벤트

자주 넘어지는 자리

- 1 `addEventListener("click", 함수())` 처럼 괄호 붙이는 실수 — **가장 흔합니다**
- 2 이벤트는 '바뀔 때'만 실행되므로 **처음 한 번은 직접 불러야** 한다는 것

자주 나오는 질문

Q "인터넷 예제엔 `onclick="hello()"` “던데 거긴 괄호가 있는데요?”

A HTML 속성 안이라 규칙이 다릅니다. 거기선 괄호가 맞고, 우리는 그 방식을 안 씁니다

비동기 fetch

OPEN 12_비동기_fetch

```
console.log("1번");  
setTimeout(() => {  
  console.log("3번");  
  console.log("타이머 시작");
```

01 동기와 비동기, setTimeout

02 콜백 지옥과 Promise

03 fetch 로 데이터 받기

04 async 와 await

05 에러 처리

- 연습문제

- 심화문제

동기와 비동기, setTimeout

RUN 브라우저로 열고 F12 → Console 을 꼭 함께 보세요.

지금까지 코드는 전부 '위에서 아래로 차례대로' 실행됐습니다. 이것을 동기(synchronous)라고 합니다.

그런데 오래 걸리는 일(서버에서 데이터 받기, 3초 기다리기)을 그렇게 하면 그동안 화면이 완전히 멈춰 버립니다.

그래서 "일단 넘어가고, 끝나면 그때 알려 줘" 하는 방식이 있습니다. 이것이 비동기(asynchronous)입니다.

실행 순서를 눈으로 보기

섹션 1

```
console.log("1번");

setTimeout(() => {
  console.log("2번 - setTimeout 0초");
}, 0);

console.log("3번");
```

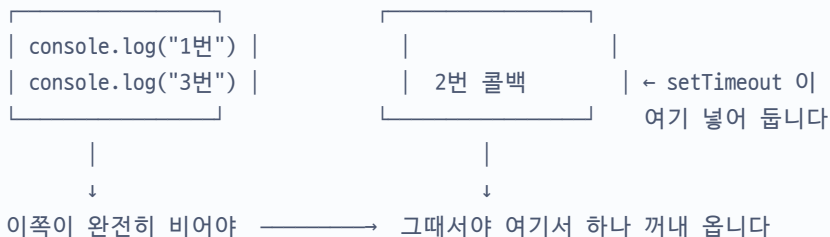
```
F12 콘솔    1번
            3번
            2번 - setTimeout 0초
```

0초인데도 2번이 마지막입니다! 왜일까요?

setTimeout 은 "이 함수를 나중에 실행해 줘" 라고 예약만 합니다. 지금 하던 일(3번 출력)을 전부 끝낸 다음에야 예약된 것을 꺼내 실행합니다.

"꺼낸다" 고 했으니 어딘가 담아 두는 곳이 있겠죠. 그림으로 보면 이렇습니다.

지금 할 일 · [예약해 둔 일]



그래서 0초여도 순서가 "1번 → 3번 → 2번" 입니다. 0초는 "기다리는 시간이 0" 이라는 뜻이지 "지금 당장" 이 아닙니다.

이것이 비동기의 핵심입니다. 시간이 0이어도 '나중' 입니다. 처음 보면 반드시 헷갈립니다. 콘솔로 순서를 직접 확인하세요.

▹ 직접 해보기 1

위 setTimeout 의 0 을 500 으로 바꾸면 출력 순서가 달라질까요? 먼저 예상하고 바꿔서 확인하세요.

setTimeout — 한 번만 나중에

섹션 2

```
setTimeout(실행할함수, 밀리초)
          ^^^^^^^^^^  ^^^^^^
          콜백!        1000 = 1초
```

08단원의 콜백이 또 나왔습니다. 여기서도 괄호를 붙이면 안 됩니다.

```
const out1 = document.querySelector("#out1");

document.querySelector("#timerBtn").addEventListener("click", () => {
  out1.textContent = "3초를 세는 중...";
  console.log("타이머 시작");

  setTimeout(() => {
    out1.textContent = "3초가 지났습니다!";
    console.log("3초 경과");
  }, 3000);

  console.log("이 줄은 3초를 기다리지 않고 바로 실행됩니다");
});
```

버튼을 누르면 콘솔에 타이머 시작 이 줄은 3초를 기다리지 않고 바로 실행됩니다 (3초 뒤) 3초 경과

중요한 것: 3초를 기다리는 동안에도 페이지는 멀쩡히 움직입니다. 다른 버튼도 누를 수 있습니다. 이것이 비동기의 장점입니다.

▹ 직접 해보기 2

1초 뒤에 콘솔에 "1초!" 를 찍어 보세요.

값을 나중에 받으면 생기는 문제

섹션 3

초보자가 반드시 한 번은 하는 실수입니다.

```
let result = "아직 없음";

setTimeout(() => {
  result = "받아온 값";
}, 1000);

console.log(result);
```

F12 콘솔 아직 없음

1초 뒤에 값이 들어오는데, 그 전에 읽어 버렸습니다.

“왜 값이 비어 있지?” 하는 문제의 정체가 이것입니다. 서버에서 데이터를 받아올 때 똑같은 일이 벌어집니다.

해결책: 값을 쓰는 코드도 ‘나중에’ 실행되게 안으로 넣습니다.

```
setTimeout(() => {
  result = "받아온 값";
  console.log("안에서 읽으면:", result);
```

F12 콘솔 안에서 읽으면: 받아들온 값

```
}, 1000);
```

이 “안으로 넣기”가 계속 겹치면 코드가 깊어집니다. 그 문제를 해결하는 것이 개념02의 Promise와 개념04의 async/await입니다.

▸ 직접 해보기

3

setTimeout을 하나 더 써서, 1.5초 뒤에 result를 ‘밖에서’ 읽어 보세요. 이번에는 값이 들어 있을까요?

setInterval — 반복해서

섹션 4

setInterval(실행할함수, 밀리초)	지정한 간격마다 계속 실행
clearInterval(아이디)	멈추기

```
const out2 = document.querySelector("#out2");
let timerId = null; // 멈추려면 아이디를 보관해야
합니다 document.querySelector("#countBtn").addEventListener("click", () => {
  if (timerId !== null) {
    console.log("이미 돌고 있습니다");
    return; // 두 번 눌러도 두 개가 돌지 않도록 (조기 반환)
  }
}
```

콜백 지옥과 Promise

RUN 브라우저로 열고 버튼을 눌러 보세요. F12 → Console 필수.

비동기 작업을 '순서대로' 여러 개 하려면 어떻게 할까요? 주문하기 → 결제하기 → 영수증 받기 각 단계가 1초씩 걸린다고 해 봅시다.

주의 · 이 파일은 버튼을 누르지 않아도 섹션 3·5의 예제가 페이지를 열 때

자동으로 실행됩니다. 그래서 아무것도 안 눌렀는데 콘솔에
"1. 주문 완료 (Promise)" 같은 줄이 이미 찍혀 있습니다. 정상입니다.

```
const out1 = document.querySelector("#out1");
const out2 = document.querySelector("#out2");
const out3 = document.querySelector("#out3");
```

콜백으로 이어 붙이기

섹션 1

“끝나면 이 함수를 불러 줘” 를 계속 넘기는 방식입니다.

```
function order(callback) {
  setTimeout(() => {
    console.log("1. 주문 완료");
    callback("주문번호-100");
  }, 1000);
}

function pay(orderNo, callback) {
  setTimeout(() => {
    console.log("2. 결제 완료");
    callback(`${orderNo} 결제됨`);
  }, 1000);
}

function receipt(payInfo, callback) {
  setTimeout(() => {
    console.log("3. 영수증 발급");
    callback(`영수증: ${payInfo}`);
  }, 1000);
}
```

```
document.querySelector("#callbackBtn").addEventListener("click", () => {
    out1.textContent = "진행 중...";
});
```

콜백 안에 콜백 안에 콜백 — 계단처럼 깊어집니다

```
order((orderNo) => {
    pay(orderNo, (payInfo) => {
        receipt(payInfo, (result) => {
            out1.textContent = result;
            console.log("최종:", result);
        });
    });
});
```

이렇게 오른쪽으로 계속 밀려나는 모양을 ‘콜백 지옥’ 이라고 부릅니다. 단계가 5개만 되어도 읽을 수 없게 됩니다. 게다가 중간에 실패하면 어디서 처리해야 할지도 막막합니다.

➤ 직접 해보기

1

① 버튼을 누르고 콘솔을 보세요. 세 단계가 몇 초 간격으로 찍히나요? 왜 그럴까요?

Promise — 나중에 줄 값에 대한 약속

섹션 2

Promise 는 “지금은 없지만 나중에 값을 주겠다” 는 약속을 담은 객체입니다.

카페 진동벨을 떠올리면 쉽습니다. 주문하면 커피 대신 ‘진동벨’ 을 먼저 줍니다. ← 이게 Promise 입니다
벨은 커피가 아닙니다. “나오면 알려 주겠다” 는 약속일 뿐입니다. 그래서 벨을 받자마자 자리로 가서 다른 일을 할 수 있습니다. ← 비동기 벨이 울리면 그때 커피를 받으러 갑니다. ← .then 재료가 떨어져 못 만들면 그것도 알려 줍니다. ← .catch

나중에 `console.log(프로미스)` 를 찍으면 값이 아니라 `Promise { <pending> }` 같은 게 나오는 이유가 이것입니다. 손에 들고 있는 게 커피가 아니라 벨이기 때문입니다.

```
new Promise((resolve, reject) => {
    성공하면 resolve(값)
    실패하면 reject(이유)
});
```

잠깐 · new 라는 키워드가 처음 나왔습니다.

“이 설계도로 새것을 하나 만들어라” 는 뜻입니다. `new Promise(...)` 는 “약속 객체를 새로 하나 만들어라” 가 됩니다. 지금은 이 정도만 알면 됩니다.

또 잠깐 · resolve 와 reject 는 어디서 왔을까요?

우리가 넘긴 콜백을 Promise 가 실행하면서 넘겨 주는 함수들입니다. 08단원에서 배운 "콜백의 매개변수는 실행하는 쪽이 넘겨 준다" 가 여기서도 그대로입니다. 이름은 관례일 뿐 마음대로 지어도 됩니다.

아래 세 함수는 섹션 1의 order / pay / receipt 를 Promise 판으로 바꾼 것입니다. 달라진 곳은 딱 두 군데입니다. ① callback 매개변수가 사라졌습니다 (받을 필요가 없어졌습니다) ② callback(값) 자리에 resolve(값) 이 들어갔습니다

```
function orderP() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("1. 주문 완료 (Promise)");
      resolve("주문번호-200");
    }, 1000);
  });
}
```

이 6줄이 무슨 순서로 도는지 한 번만 따라가 봅시다. 이 파일에서 제일 어려운 곳이니 천천히 보세요.

① orderP() 를 부른다 ② 그 자리에서 Promise(진동벨) 하나가 만들어져 밖으로 나온다

← 아직 값은 없습니다. 벨만 받은 상태입니다

③ 동시에 안쪽 setTimeout 이 "1초 뒤에 실행" 으로 예약된다 ④ orderP() 는 여기서 이미 끝납니다. 아래 줄이 먼저 실행됩니다 ⑤ 1초 뒤 예약된 것이 실행되면서 resolve("주문번호-200") 이 불린다 ⑥ 그 순간 ②에서 나갔던 벨이 울리고,

거기 붙여 둔 .then 의 콜백이 실행된다

핵심은 ②와 ⑤ 사이에 1초의 틈이 있다는 것입니다. resolve 는 "값 나왔어, 이제 .then 불러도 돼" 라고 알리는 신호입니다.

```
function payP(orderNo) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("2. 결제 완료 (Promise)");
      resolve(`${orderNo} 결제됨`);
    }, 1000);
  });
}

function receiptP(payInfo) {
  return new Promise((resolve) => {
    setTimeout(() => {
```

fetch 로 데이터 받기

RUN 브라우저로 열고 버튼 클릭. 인터넷 연결 필요.

서버에서 데이터를 받아오는 함수입니다. 브라우저가 기본으로 제공합니다.

```
fetch(주소)
  .then((res) => res.json()) ← 받은 것을 자바스크립트 값으로 변환
  .then((data) => { ... }) ← 그 값을 사용
```

fetch 는 Promise 를 돌려줍니다. 그래서 개념02 의 then 이 그대로 쓰입니다.

— 처음 나오는 용어 네 개

API 프로그램끼리 데이터를 주고받는 창구. 여기서는 “데이터를 주는 주소” 정도로

생각하면 됩니다.

JSON 데이터를 글자로 적어 주고받는 형식. 생김새가 자바스크립트 객체와 거의 같습니다.

서버는 객체를 그대로 못 보내니 글자로 바꿔서 보냅니다.

파싱 그 글자를 프로그램이 다룰 수 있는 값으로 되돌리는 일. `res.json()` 이 합니다. 상태 코드 서버가 함께 보내는 세 자리 숫자.

200 성공 / 404 그런 거 없음 / 500 서버가 고장남

연습용 무료 API 를 씁니다. 가짜 데이터를 돌려주는 사이트입니다.

```
const BASE_URL = "https://jsonplaceholder.typicode.com";
```

가장 기본

섹션 1

```
const out1 = document.querySelector("#out1");

document.querySelector("#oneBtn").addEventListener("click", () => {
  out1.textContent = "불러오는 중...";
  console.log("요청 보냄");

  fetch(`${BASE_URL}/posts/1`)
    .then((res) => {
      console.log("응답 도착:", res.status);
```

F12 콘솔 응답 도착: 200

```
return res.json(); // * 이 return 을 빠뜨리면 다음 then 이 undefined 를 받습니다
})
.then((data) => {
  console.log("데이터:", data);
  out1.textContent = data.title;
})
.catch((error) => {
  out1.textContent = "가져오지 못했습니다";
  console.log("실패:", error.message);
});

console.log("이 줄은 데이터를 기다리지 않고 먼저 실행됩니다");
});
```

- 1 fetch 가 끝나면 '응답 객체(Response)' 를 줍니다. 아직 내용은 안 읽었습니다.
- 2 res.json() 이 그 내용을 읽어 자바스크립트 값으로 바꿉니다.

이것도 시간이 걸리는 일이라 또 Promise 입니다.

그래서 then 이 두 번 필요합니다. 처음엔 다들 헷갈립니다.

▹ 직접 해보기

1

/posts/2 를 가져와 body 를 화면에 넣어 보세요.

받은 데이터의 모양

섹션 2

/posts/1 을 요청하면 이런 객체가 옵니다.

```
{
```

```

    userId: 1,
    id: 1,
    title: "sunt aut facere ...",
    body: "quia et suscipit ..."

```

```

}

```

07단원에서 배운 객체 그대로입니다. 점으로 꺼내면 됩니다.

목록을 요청하면 객체들이 담긴 '배열' 이 옵니다. (07단원 개념05) 그러면 08단원의 map, filter 를 그대로 쓸 수 있습니다.

데이터가 어떻게 생겼는지 모르겠으면 일단 `console.log(data)` 로 찍어 보세요. 이게 가장 확실한 방법입니다.

▹ 직접 해보기 2

/posts/1 을 받아서 title 과 userId 를 각각 콘솔에 찍어 보세요. (받은 것은 07단원의 그냥 객체입니다)

목록을 받아 화면에 그리기

섹션 3

```

const postList = document.querySelector("#postList");

document.querySelector("#listBtn").addEventListener("click", () => {
  postList.innerHTML = "<li>불러오는 중...</li>";

  fetch(`${BASE_URL}/posts?_limit=5`)
    .then((res) => res.json())
    .then((posts) => {
      console.log("받은 글 개수:", posts.length);
    });
});

```

F12 콘솔 받은 글 개수: 5

08단원의 map + join + innerHTML (10단원 개념02)

```

postList.innerHTML = posts
  .map((post) => `<li>${post.id}. ${post.title}</li>`)
  .join("");
});

.catch(() => {
  postList.innerHTML = "<li>불러오지 못했습니다</li>";
});
});

```

주소 뒤의 ?_limit=5 는 "5개만 주세요" 라는 뜻입니다. ? 뒤에 붙이는 이런 값을 쿼리 문자열이라고 합니다.

지금까지 배운 게 전부 여기서 만납니다. fetch(비동기) → 배열(06단원) → map(08단원) → innerHTML(10단원)

▹ 직접 해보기

3

_limit 를 3 으로 바꿔 보세요.

createElement 로 안전하게 그리기

섹션 4

```
const userList = document.querySelector("#userList");

document.querySelector("#userBtn").addEventListener("click", () => {
  userList.innerHTML = "<li>불러오는 중...</li>";

  fetch(`${BASE_URL}/users?_limit=4`)
    .then((res) => res.json())
    .then((users) => {
      userList.innerHTML = ""; // 먼저 비우기

      users.forEach((user) => {
        const li = document.createElement("li");
```

서버에서 온 값이므로 textContent 로 넣습니다 (10단원 개념02)

```
        li.textContent = `${user.name} (${user.email})`;
        userList.appendChild(li);
      });

      console.log("사용자 이름들:", users.map((u) => u.name));
    })
    .catch(() => {
      userList.innerHTML = "<li>불러오지 못했습니다</li>";
    });
  });
```

서버에서 온 데이터를 innerHTML 에 그대로 넣으면 위험합니다. 남이 입력한 글이 섞여 있을 수 있으니까요. (10단원 개념02 섹션 5) 실무에서는 이 방식(createElement + textContent)을 씁니다.

▹ 직접 해보기

4

위 users 에서 name 말고 email 만 모아 배열로 콘솔에 찍어 보세요. (08단원 map)

다시 한 번, 순서

섹션 5

버튼을 누르면 콘솔에 이 순서로 찍힙니다.

요청 보냄 이 줄은 데이터를 기다리지 않고 먼저 실행됩니다 응답 도착: 200

```
데이터: {...}
```

fetch 아래 줄이 먼저 실행됩니다. 개념01 에서 본 그대로입니다.

그래서 이렇게 쓰면 안 됩니다.

```
let posts = [];  
fetch(url).then((res) => res.json()).then((d) => { posts = d; });  
console.log(posts.length); // 0 입니다. 아직 안 왔으니까요.
```

데이터를 쓰는 코드는 반드시 then 안에 넣어야 합니다.

▹ 직접 해보기

5

바로 위의 '이렇게 쓰면 안 되는' 세 줄을 직접 실행해 posts.length 가 정말 0 으로 나오는지 확인해 보세요.

다른 요청 방법 (맛보기)

섹션 6

지금까지는 '가져오기(GET)' 만 했습니다. 보내는 것도 있습니다.

```
fetch(url, {  
  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ title: "새 글" }),  
  
})
```

JSON.stringify 는 자바스크립트 객체를 문자열로 바꿉니다. 반대는 JSON.parse 입니다. res.json() 이 그 일을 대신해 주는 것입니다.

```
console.log(JSON.stringify({ a: 1, b: "글자" }));
```

```
F12 콘솔    {"a":1,"b":"글자"}
```

```
console.log(JSON.parse('{"a":1}'));
```

```
F12 콘솔    { a: 1 }
```

주의 · 이 연습용 사이트는 POST 를 받아 주는 척만 하고 실제로 저장하지 않습니다.

그래서 새로 만든 글을 다시 GET 하면 404 가 납니다. 정상입니다.

👉 직접 해보기 6

{ name: "홍길동", age: 20 } 을 `JSON.stringify` 로 바꿔 찍고, 그 결과를 다시 `JSON.parse` 로 되돌려 age 만 꺼내 보세요.

JSON 자세히 보기 ★

섹션 7

위에서 `res.json()` 을 계속 썼습니다. 이제 그게 무엇인지 제대로 봅니다. 서버와 이야기할 때 데이터는 언제나 이 형식으로 오갑니다. 15단원 `localStorage` 에서도, 나중에 서버를 직접 만들 때도 그대로 씁니다.

① JSON 은 '글자' 다. 객체가 아니다.

```
const 사람 = { name: "홍길동", age: 20 };
const 글자 = JSON.stringify(사람);
```

```
console.log(typeof 사람);
```

```
F12 콘솔    object
```

```
console.log(typeof 글자);
```

```
F12 콘솔    string
```

```
console.log(글자);
```

```
F12 콘솔    {"name":"홍길동","age":20}
```

따옴표에 주목하세요. 전부 큰따옴표이고 키에도 붙습니다. 자바스크립트 객체는 키에 따옴표가 없어도 되고 작은따옴표도 되지만, JSON 은 안 됩니다. 비슷하게 생겼을 뿐 같은 것이 아닙니다.

② 되돌리면 '새 객체' 다. 원래 것과 다른 물건이다.

```
const 되돌린것 = JSON.parse(글자);
```

```
console.log(되돌린것.name);
```

```
F12 콘솔    홍길동
```

```
console.log(되돌린것 === 사람);
```

```
F12 콘솔    false
```

09단원에서 본 '복사' 이야기입니다. 글자로 갔다 오면 완전히 새 객체가 됩니다.

★ 여기서부터가 중요합니다. JSON 으로 갔다 오면 없어지는 것들이 있습니다. 에러가 안 나기 때문에 나중에 엉뚱한 곳에서 문제가 터집니다.

③ 함정 1 — `undefined` 인 속성은 사라진다. `null` 은 남는다.

```
console.log(JSON.stringify({ a: 1, b: undefined, c: null }));
```

```
F12 콘솔    {"a":1,"c":null}
```

b 가 통째로 없어졌습니다. "값을 안 넣었다" 를 서버에 보내고 싶다면 `undefined` 가 아니라 `null` 을 넣어야 합니다.

④ 함정 2 — 배열 안에서는 `undefined` 가 `null` 이 된다. 없어지지 않는다.

```
console.log(JSON.stringify([1, undefined, 3]));
```

```
F12 콘솔    [1,null,3]
```

객체에서는 사라지고 배열에서는 `null` 이 됩니다. 규칙이 다릅니다. 배열은 자리가 밀리면 안 되니까 자리를 지켜 주는 것입니다.

⑤ 함정 3 — 함수는 사라진다.

```
const 계산기 = {  
  값: 10,  
  더하기() {  
    return this.값 + 1;  
  },  
};  
  
console.log(JSON.stringify(계산기));
```

```
F12 콘솔    {"값":10}
```

07단원에서 만든 메소드가 통째로 없어졌습니다. JSON 은 '데이터' 를 주고받는 형식이니 '기능' 을 옮기는 것이 아닙니다.

⑥ 함정 4 — `Date` 는 글자가 된다.

```
const 기록 = { 언제: new Date("2026-01-01T00:00:00Z") };  
const 왕복 = JSON.parse(JSON.stringify(기록));  
  
console.log(typeof 왕복.언제);
```

```
F12 콘솔    string
```

```
console.log(왕복.언제);
```

```
F12 콘솔    2026-01-01T00:00:00.000Z
```

날짜처럼 보이지만 글자입니다. 왕복.언제.getFullYear() 를 부르면 터집니다. 서버에서 받은 날짜를 쓰려면 new Date(왕복.언제) 로 다시 만들어야 합니다. (15단원 개념03 에서 Date 를 배웁니다)

⑦ 함정 5 — 자기 자신을 품으면 아예 터진다.

```
const 가 = { 이름: "가" };
가.자기자신 = 가;

try {
  JSON.stringify(가);
} catch (에러) {
  console.log(에러.name);
}
```

F12 콘솔 TypeError

```
}
```

앞의 넷은 조용히 없어지는데 이것만 에러를 냅니다. 화면 요소(DOM)를 객체에 담아 두면 실제로 이렇게 됩니다.

⑧ 사람이 읽게 찍기 세 번째 인자에 칸 수를 주면 줄을 나눠서 들여쓰기까지 해 줍니다. (두 번째 인자는 지금 안 씁니다. 그래서 null 을 넣습니다)

```
JSON.stringify(사람, null, 2)
```

```
→ {
```

```
  "name": "홍길동",
  "age": 20
}
```

줄이 나뉘었는지 길이로 확인해 봅시다.

```
console.log(JSON.stringify(사람).length);
```

F12 콘솔 23

```
console.log(JSON.stringify(사람, null, 2).split("\n").length);
```

F12 콘솔 4

한 줄짜리 23글자가 4줄로 늘어났습니다. 서버가 준 데이터가 어떻게 생겼는지 들여다볼 때 이렇게 찍어 보세요. 사람이 읽으려고 쓰는 것이라, 서버로 보낼 때는 그냥 stringify 를 씁니다.

▹ 직접 해보기

7

{ 이름: "라떼", 가격: 4500, 할인: undefined, 메모: null } 을 `JSON.stringify` 로 찍어 보세요. 몇 개가 남나요?

▹ 직접 해보기

8

위 기록 객체를 왕복시킨 뒤 `왕복.언제.getFullYear()` 를 불러 보세요. 무슨 에러가 나나요? 어떻게 고치면 되나요?

자주 하는 실수

섹션 8

이런 실수를 합니다

res.json() 앞에 return 을 빠뜨림

`.then((res) => { res.json(); })` 다음 then 이 `undefined` 를 받습니다. 화살표 한 줄로 쓰면 자동 `return` 됩니다. `.then((res) => res.json())`

이런 실수를 합니다

res 를 데이터라고 생각

`.then((res) => console.log(res.title))` `undefined` 입니다. `res` 는 응답 객체이지 내용이 아닙니다.

이런 실수를 합니다

fetch 결과를 밖에서 바로 씀 (섹션 5)

이런 실수를 합니다

catch 를 안 붙임

인터넷이 끊기거나 주소가 틀리면 조용히 아무 일도 안 일어납니다.

이런 실수를 합니다

주소를 틀리게 씀

오타가 있어도 `fetch` 는 에러를 던지지 않고 404 응답을 '성공' 으로 처리합니다. 이 문제는 개념05 에서 `res.ok` 로 해결합니다.

이런 실수를 합니다

JSON 으로 갔다 온 것을 원래 그대로라고 생각 (섹션 7)

함수와 `undefined` 는 없어지고 `Date` 는 글자가 됩니다. 에러는 안 납니다. 화면에 날짜가 이상하게 찍히거나 메소드가 없다고 하면 여기를 떠올리세요.

이런 실수를 합니다

객체를 그대로 서버에 보냄

body: { title: "새 글" } 라고 쓰면 [object Object] 가 날아갑니다. 반드시 body: `JSON.stringify({ title: "새 글" })` 로 글자로 바꿔서 보냅니다.

정리

- 1 `fetch(주소).then((res) => res.json()).then((data) => { ... })`
- 2 then 이 두 번인 이유: 응답 받기 → 내용 읽기, 둘 다 시간이 걸린다.
- 3 res 는 응답 객체, data 가 진짜 내용이다.
- 4 받은 데이터는 객체나 배열이므로 07·08단원 그대로 다룬다.
- 5 데이터를 쓰는 코드는 반드시 then 안에 넣는다.
- 6 서버에서 온 값은 innerHTML 보다 textContent 가 안전하다.
- 7 JSON 은 '글자' 다. 객체와 비슷하게 생겼을 뿐 같은 것이 아니다. 보낼 때 `JSON.stringify`, 받을 때 `JSON.parse`. `res.json()` 이 후자를 대신해 준다.
- 8 JSON 으로 갔다 오면 없어지는 것이 있다 — 함수, `undefined` 인 속성. 배열 안의 `undefined` 는 없어지지 않고 `null` 이 된다. Date 는 글자가 되고, 자기 자신을 품은 객체는 `TypeError` 를 낸다.
- 9 없어지는 것들은 에러가 안 난다. 그래서 나중에 엉뚱한 곳에서 터진다.

직접 해보기 정답

1) `fetch(`${BASE_URL}/posts/2`)`

```
.then((res) => res.json())
.then((data) => {
  out1.textContent = data.body;
});
```

2) `fetch(`${BASE_URL}/posts/1`)`

```
.then((res) => res.json())
.then((data) => {
  console.log(data.title);
  console.log(data.userId);
});
```

```
// 콘솔: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
// 콘솔: 1
```

→ 서버에서 왔다는 것만 다를 뿐, 꺼내는 방법은 07단원의 객체와 똑같습니다.

```
3) fetch(`${BASE_URL}/posts?_limit=3`)
```

주소의 숫자만 바꾸면 됩니다.

```
4) fetch(`${BASE_URL}/users?_limit=4`)
```

```
.then((res) => res.json())
.then((users) => {
  console.log(users.map((u) => u.email));
});
```

```
// 콘솔: ['Sincere@april.biz', 'Shanna@melissa.tv',
//       'Nathan@yesenia.net', 'Julianne.OConner@kory.org']
```

→ 받은 것이 배열이니 08단원의 map 을 그대로 쓸 수 있습니다.

"서버에서 온 데이터" 라고 특별한 것이 아니라 그냥 배열입니다.

```
5) let posts = [];
fetch(`${BASE_URL}/posts?_limit=5`)
```

```
.then((res) => res.json())
.then((d) => {
  posts = d;
});
```

```
console.log("바로 읽으면:", posts.length);
// 콘솔: 바로 읽으면: 0
```

→ 5개를 받아 왔는데도 0 입니다. console.log 가 먼저 실행되고

데이터는 그 뒤에 도착하기 때문입니다.
개념01 섹션 3에서 setTimeout 으로 본 것과 완전히 같은 문제입니다.


```
6) const person = { name: "홍길동", age: 20 };
const text = JSON.stringify(person);
console.log(text);
// 콘솔: {"name":"홍길동","age":20}
console.log(typeof text);
// 콘솔: string
```

```
const back = JSON.parse(text);
console.log(back.age);
// 콘솔: 20
```

→ 겉모습은 객체와 비슷해 보이지만 `typeof` 가 string 입니다.

서버로 보낼 때는 글자로 바꿔서 보내고, 받을 때는 다시 객체로 되돌려야 합니다. `res.json()` 이 그 '되돌리기' 를 대신해 주는 것입니다.

```
7) const 메뉴 = { 이름: "라떼", 가격: 4500, 할인: undefined, 메모: null };
console.log(JSON.stringify(메뉴));
// 콘솔: {"이름":"라떼","가격":4500,"메모":null}
```

→ 넷을 넣었는데 셋만 남았습니다. 할인이 사라졌습니다.

`undefined` 는 없어지고 `null` 은 남습니다.

"할인 값이 아직 안 정해졌다" 를 서버에 알리고 싶었다면 실패한 것입니다.
서버는 할인이라는 항목이 있었는지조차 모릅니다.
그럴 때는 `undefined` 가 아니라 `null` 을 넣습니다. 메모처럼요.

8) 왕복.언제.getFullYear() 를 부르면 이렇게 납니다.

```
TypeError: 왕복.언제.getFullYear is not a function
```

→ 왕복.언제 가 Date 가 아니라 글자이기 때문입니다.

글자에는 `getFullYear` 가 없습니다.

고치는 법 — 글자를 다시 Date 로 만들어서 씁니다.

```
console.log(new Date(왕복.언제).getFullYear());
// 콘솔: 2026
```

★ 서버에서 받은 날짜는 언제나 글자입니다. 예외가 없습니다.

JSON 에는 '날짜' 라는 것이 아예 없기 때문입니다.
숫자·글자·불리언·null·객체·배열 여섯 가지뿐입니다.

async 와 await

RUN 브라우저로 열고 버튼 클릭. 인터넷 연결 필요.

then 을 계속 이어 붙이는 대신, 비동기 코드를 '동기처럼' 쓰는 문법입니다. 요즘 실무 코드는 거의 다 이 방식입니다.

```
async function 이름() {
  const 값 = await 프로미스;
}
```

async 이 함수 안에서 await 를 쓰겠다는 표시
await 이 Promise 가 끝날 때까지 기다렸다가 '값' 을 꺼내라

★ 여기서 반드시 짚고 갈 것 — “기다린다” 는 말에 속지 마세요.

개념01 에서 이렇게 배웠습니다.

"3초 동안 기다리면 버튼이 안 눌리고 화면이 하얗게 멈춘다"

그런데 await 도 '기다린다' 고 합니다. 그럼 화면이 멈출까요?

안 멈춥니다. 둘은 기다리는 방식이 완전히 다릅니다.

개념01 의 while 로 버티기 → 브라우저 전체가 붙잡혀 아무것도 못 함
await 로 기다리기 → 이 async 함수 하나만 잠시 비켜서 있고,
그동안 브라우저는 평소대로 움직입니다.
버튼도 눌리고 스크롤도 됩니다.

비켜서 있던 함수는 Promise 가 끝나면 멈췄던 자리에서 다시 이어집니다. 개념01 의 '예약해 두고 나중에 꺼내 쓰기' 와 똑같은 방식입니다.

즉 await 는 '동기처럼 보이게 써 주는 비동기' 입니다. 생김새만 순서대로일 뿐, 속은 개념01·02 에서 배운 그대로입니다.

```
const BASE_URL = "https://jsonplaceholder.typicode.com";
const out1 = document.querySelector("#out1");
const out2 = document.querySelector("#out2");
```

then 방식과 나란히 비교

섹션 1

then 방식 · — 개념03 에서 쓰던 것

```
function loadWithThen() {
  out1.textContent = "불러오는 중...";

  fetch(`${BASE_URL}/posts/1`)
    .then((res) => res.json())
    .then((data) => {
      out1.textContent = `[then] ${data.title}`;
      console.log("then 방식 완료");
    });
}
```

await 방식 · — 같은 일을 합니다

```
async function loadWithAwait() {
  out1.textContent = "불러오는 중...";

  const res = await fetch(`${BASE_URL}/posts/1`);
  const data = await res.json();

  out1.textContent = `[await] ${data.title}`;
  console.log("await 방식 완료");
}

document.querySelector("#thenBtn").addEventListener("click", loadWithThen);
document.querySelector("#awaitBtn").addEventListener("click", loadWithAwait);
```

차이를 보세요. then : .then((res) => res.json()).then((data) => ...)

```
await: const res = await fetch(...);
```

```
const data = await res.json();
```

await 쪽은 우리가 05단원까지 쓰던 평범한 코드처럼 생겼습니다. 변수에 값을 담고 다음 줄에서 쓰는, 익숙한 모양입니다.

▹ 직접 해보기

1

loadWithAwait 를 /posts/3 으로 바꿔 보세요.

규칙 1 · await 는 async 함수 '안'에서만 쓸 수 있다

```
const data = await fetch(url);
```

← 함수 밖이면 `SyntaxError`

규칙 2 · async 함수는 항상 Promise 를 돌려준다

```
async function getNumber() {
  return 42;
}
```

```
console.log(getNumber());
```

```
F12 콘솔    Promise {<fulfilled>: 42}
```

42 가 아니라 Promise 입니다! (표기는 브라우저마다 조금씩 다릅니다. 중요한 건 꺾데기가 Promise 라는 점입니다)

```
getNumber().then((n) => console.log("then 으로 꺼내면:", n));
```

```
F12 콘솔    then 으로 꺼내면: 42
```

그래서 async 함수의 결과를 쓰려면 그쪽에서도 await 를 쓰거나 then 을 씁니다.

▹ 직접 해보기 2

`getNumber()` 의 42 를 then 이 아니라 await 로 꺼내 보세요. (규칙 1을 기억하세요. await 를 쓸 자리를 먼저 만들어야 합니다)

순서대로 요청하기

앞의 결과가 있어야 다음 요청을 할 수 있는 경우입니다.

```
async function loadPostAndAuthor() {
  out2.textContent = "불러오는 중...";
  console.log("1단계 시작");
```

글을 가져오고

```
const postRes = await fetch(`${BASE_URL}/posts/1`);
const post = await postRes.json();
console.log("2단계: 글 받음, userId =", post.userId);
```

그 글의 작성자를 가져옵니다 (앞의 결과가 필요합니다)

```
const userRes = await fetch(`${BASE_URL}/users/${post.userId}`);
const user = await userRes.json();
console.log("3단계: 작성자 받음");

out2.textContent = `${post.title} - 작성자: ${user.name}`;
}

document.querySelector("#seqBtn").addEventListener("click", loadPostAndAuthor);
```

then 으로 쓰면 이렇게 됩니다. 비교해 보세요.

```
fetch(url1)

.then((res) => res.json())
.then((post) => fetch(`${BASE_URL}/users/${post.userId}`))
.then((res) => res.json())
.then((user) => { ... });
```

문제는 마지막 then 에서 post 를 쓸 수 없다는 것입니다. 이미 살펴보았으니까요. await 는 변수가 그대로 살아 있어서 편합니다.

▹ 직접 해보기 3

작성자의 email 도 함께 보여 주세요.

동시에 여러 개 — Promise.all

섹션 4

서로 상관없는 요청을 await 로 줄줄이 쓰면 순서대로 기다려서 느립니다.

```
const a = await fetch(url1); // 0.5초
const b = await fetch(url2); // 0.5초 → 합계 1초
```

동시에 보내면 0.5초면 됩니다.

```
async function loadAll() {
  const allList = document.querySelector("#allList");
  allList.innerHTML = "<li>불러오는 중...</li>";
  console.log("동시 요청 시작");
```

이 파일 내내 “then 대신 await” 라고 했는데 여기만 then 이 나옵니다. 이유가 있습니다. await 는 async 함수 ‘안’ 에서만 쓸 수 있는데, 아래 [] 배열 안은 함수 안이 아니라 그냥 값을 나열하는 자리입니다. 그래서 각 요청은 then 으로 json 까지 처리해 두고, 그 묶음 ‘전체’ 를 Promise.all 로 감싸 한 번만 await 합니다.

```
const [post1, post2, post3] = await Promise.all([
  fetch(`${BASE_URL}/posts/1`).then((r) => r.json()),
```

에러 처리

RUN 브라우저로 열고 버튼 클릭. 인터넷 연결 필요.

서버 요청은 실패할 수 있습니다. 인터넷이 끊겼다 / 주소가 틀렸다 / 서버가 죽었다 / 권한이 없다

실패해도 화면이 멈추지 않고 “실패했다” 고 알려 주는 것이 에러 처리입니다.

```
const BASE_URL = "https://jsonplaceholder.typicode.com";
const out1 = document.querySelector("#out1");
const out2 = document.querySelector("#out2");
const out3 = document.querySelector("#out3");
```

try / catch

섹션 1

await 방식에서는 then 의 .catch 대신 try / catch 를 씁니다.

```
try {
  위험한 코드
} catch (error) {
  실패했을 때 할 일
}
```

```
async function loadPost(url, target) {
  target.textContent = "불러오는 중...";
  target.classList.remove("error");

  try {
    const res = await fetch(url);
    const data = await res.json();

    target.textContent = data.title;
    console.log("성공:", data.id);
  } catch (error) {
    target.textContent = "가져오지 못했습니다";
    target.classList.add("error");
    console.log("catch 에서 잡음:", error.message);
  }
}

document.querySelector("#okBtn").addEventListener("click", () => {
  loadPost(`${BASE_URL}/posts/1`, out1);
});
```

```
});

document.querySelector("#failBtn").addEventListener("click", () => {
```

존재하지 않는 주소 — 네트워크 자체가 실패합니다

```
loadPost("https://이런주소는없습니다-abcxyz.example/posts/1", out1);
});
```

try 안에서 에러가 나면 그 즉시 catch 로 됩니다. try 안의 남은 줄은 실행되지 않습니다.

catch (error) 의 error 에는 에러 객체가 들어옵니다.

```
error.message 사람이 읽을 수 있는 설명
error.name     에러 종류
```

▸ 직접 해보기

1

catch 안에서 error 객체 전체를 콘솔에 찍어 보세요.

404 는 에러가 아니다

섹션 2

이것이 fetch 의 가장 큰 함정입니다.

fetch 는 '서버와 연결이 안 될 때' 만 실패로 칩니다. 서버가 "그런 거 없어요(404)" 라고 대답한 것은 fetch 입장에서는 '대답을 받았으니 성공' 입니다.

```
async function loadWithoutCheck() {
  out2.textContent = "불러오는 중...";

  try {
    const res = await fetch(`${BASE_URL}/posts/9999`); // 없는
    글 console.log("status:", res.status);
```

```
F12 콘솔    status: 404
```

```
console.log("ok:", res.ok);
```

```
F12 콘솔    ok: false
```

```
const data = await res.json();
console.log("받은 데이터:", data);
```

```
F12 콘솔    받은 데이터: {} ← 빈 객체입니다
```

```
out2.textContent = data.title;
```

화면 아무것도 안 나옵니다. data.title 이 undefined 니까요.

```
    } catch (error) {
      out2.textContent = "여기로는 오지 않습니다";
      console.log("catch:", error.message);
    }
  }

  document.querySelector("#notFoundBtn").addEventListener("click", loadWithoutCheck);
```

catch 로 가지 않고 조용히 빈 화면이 됩니다. "왜 아무것도 안 나오지?" 하는 문제의 정체입니다.

해결: res.ok 를 직접 확인하고 직접 에러를 던집니다.

```
res.ok      2xx 응답이면 true
res.status 200, 404, 500 같은 숫자
```

⇒ 직접 해보기 2

② 버튼을 누르고 콘솔을 보세요. status 와 ok 가 무엇인가요? 그리고 catch 안의 "여기로는 오지 않습니다" 가 찍히나요?

제대로 만든 버전

섹션 3

```
const goodBtn = document.querySelector("#goodBtn");
```

참고 · 아래에서 classList 대신 className 에 통째로 넣습니다.

10단원 개념03에서 "className 에 = 로 덮어쓰면 원래 클래스가 날아간다" 고 했죠. 여기서는 그게 오히려 편합니다. 로딩·성공·실패 상태를 매번 깨끗이 갈아끼워야 하니까요. 기준: 클래스를 '더하고 뺄' 때는 classList, '통째로 정할' 때는 className.

```
async function loadPostProperly(id) {
```

1) 로딩 시작 — 버튼을 잠그고 안내 문구

```
  goodBtn.disabled = true;
  out3.textContent = "불러오는 중...";
  out3.className = "output loading";

  try {
    const res = await fetch(`${BASE_URL}/posts/${id}`);
```

2) 응답 상태를 직접 확인


```
if (!res.ok) {
    throw new Error(`서버 응답 오류 (${res.status})`);
}

const data = await res.json();
```

3) 성공

```
out3.textContent = data.title;
out3.className = "output";
console.log("성공:", data.id);
} catch (error) {
```

4) 실패 — 사용자에게는 쉬운 말로, 콘솔에는 자세히

```
out3.textContent = "글을 불러오지 못했습니다. 잠시 후 다시 시도해 주세요.";
out3.className = "output error";
console.log("에러 내용:", error.message);
} finally {
```

5) 성공하든 실패하든 버튼은 다시 풀어 줘야 합니다

```
goodBtn.disabled = false;
console.log("finally 실행 - 버튼 잠금 해제");
}
}

goodBtn.addEventListener("click", () => loadPostProperly(1));
```

throw new Error("메시지") 는 "여기서 에러를 내겠다" 는 뜻입니다. 던지면 즉시 catch 로 씁니다.

finally 는 성공하든 실패하든 반드시 실행됩니다. 로딩 표시 끄기, 버튼 잠금 풀기처럼 "무조건 해야 하는 뒷정리" 에 씁니다. finally 가 없으면 실패했을 때 버튼이 영원히 잠긴 채로 남습니다.

▸ 직접 해보기

3

id 를 9999 로 바꿔 눌러 보고, 이번에는 catch 로 가는지 확인해 보세요.

좋은 에러 처리의 조건

섹션 4

1) 사용자에게 보여 주는 말과 개발자용 정보를 나눈다

화면	내용
	"불러오지 못했습니다. 다시 시도해 주세요"

- 2 로딩 중임을 알린다 아무 반응이 없으면 사용자는 고장 났다고 생각하고 계속 누릅니다.
- 3 버튼을 잠근다 (disabled) 연속으로 눌러 요청이 여러 번 나가는 것을 막습니다.
- 4 finally 로 뒷정리를 한다 실패했을 때 로딩 표시가 영원히 남는 사고를 막습니다.
- 5 실패해도 화면이 깨지지 않는다 데이터가 없으면 "내용 없음" 정도는 나와야 합니다.

▹ 직접 해보기

4

loadPostProperly 에서 finally 블록을 통째로 지우면 요청이 실패했을 때 화면이 어떻게 될까요? 말로 설명해 보세요.

then 방식의 에러 처리

섹션 5

개념02 에서 본 것과 같습니다. 같은 구조입니다.

```
fetch(url)
```

```
.then((res) => {
  if (!res.ok) throw new Error(`오류 ${res.status}`);
  return res.json();
})
.then((data) => { 성공 처리 })
.catch((error) => { 실패 처리 })
.finally(() => { 뒷정리 });
```

```
try ↔ then
catch ↔ .catch
```

finally ↔ .finally

하는 일은 같습니다. 요즘은 try/catch 쪽을 더 많이 씁니다.

▹ 직접 해보기

5

위 then 방식에서 if (!res.ok) throw ... 줄을 지우면 404 일 때 어디로 갈까요? (섹션 2를 떠올려 보세요)

try/catch 는 fetch 전용이 아니다

섹션 6

비동기가 아니어도 쓸 수 있습니다.

```
try {
  JSON.parse("이건 JSON 이 아닙니다");
} catch (error) {
  console.log("JSON 파싱 실패:", error.name);
}
```

F12 콘솔 JSON 파싱 실패: SyntaxError

```
}
console.log("에러가 났지만 프로그램은 계속됩니다");
```

F12 콘솔 에러가 났지만 프로그램은 계속됩니다

try/catch 가 없었다면 여기서 코드가 멈췄을 겁니다.

주의 · 모든 코드를 try 로 감싸지는 마세요.

오타로 인한 에러까지 조용히 삼켜 버리면 버그를 못 찾습니다. “실패할 수 있다고 예상되는 곳”에만 씁니다. 서버 요청, JSON 파싱 같은 것.

▸ 직접 해보기

6

위 `JSON.parse` 에 올바른 JSON 문자열 `{“a”:1}` 을 넣으면 catch 안의 코드는 실행될까요? 예상하고 확인해 보세요.

자주 하는 실수

섹션 7

★ 아래에서 `SyntaxError` 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (`ReferenceError`, `TypeError`)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 나옵니다. 그런데 `SyntaxError` 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다. 실수로 풀었다면 다시 // 를 붙이면 그대로 돌아옵니다.

이런 실수를 합니다

res.ok 를 확인 안 함 (섹션 2) — 이 단원의 핵심입니다.

이런 실수를 합니다

catch 를 비워 둡니다

```
catch (e) {}
```

에러가 났는데 아무 흔적도 안 남습니다. 최소한 `console.log` 는 하세요.

이런 실수를 합니다

finally 없이 로딩 표시를 끄

try 안 마지막에만 “로딩 끝” 을 두면 실패했을 때 영원히 로딩 중입니다.

이런 실수를 합니다

에러 객체를 그대로 화면에 보여 줌

```
out.textContent = error.message;
```

“Failed to fetch” 같은 영어가 사용자에게 보입니다. 사용자에게는 쉬운 말로, 자세한 건 콘솔에.

이런 실수를 합니다

await 를 try 밖에 둬

```
const res = await fetch(url);  
try { ... } catch { ... }
```

fetch 가 실패하면 try 밖이라 잡히지 않습니다.

정리

- 1 await 방식은 try / catch / finally 로 처리한다.
- 2 fetch 는 404·500 을 실패로 치지 않는다. res.ok 를 직접 확인할 것.
- 3 if (!res.ok) throw new Error(...) 로 직접 에러를 던진다.
- 4 finally 에서 로딩 해제·버튼 잠금 해제 같은 뒷정리를 한다.
- 5 사용자에게는 쉬운 말, 콘솔에는 자세한 정보.
- 6 try/catch 는 실패할 만한 곳에만 쓴다. 전부 감싸면 버그를 놓친다.

직접 해보기 정답

```
1) catch (error) {
```

```
  console.log(error);           // 에러 객체 전체  
  console.log(error.name);      // 종류  
  console.log(error.message);   // 설명
```

```
}
```

```
2) // 콘솔: status: 404
// 콘솔: ok: false
// 콘솔: 받은 데이터: {}
```

“여기로는 오지 않습니다”는 찍히지 않습니다. → 없는 글을 달라고 했는데도 catch 로 가지 않습니다.

fetch 입장에서는 서버가 “없다”고 대답해 준 것도 ‘대답을 받은 것’이라 성공이기 때문입니다. 그래서 res.ok 를 직접 봐야 합니다. 이것이 이 단원에서 가장 중요한 한 가지입니다.

```
3) goodBtn.addEventListener("click", () => loadPostProperly(9999));
```

→ res.ok 가 false 이므로 throw 가 실행되고 catch 로 갑니다. → 화면에 “글을 불러오지 못했습니다...”가 나오고

콘솔에 “에러 내용: 서버 응답 오류 (404)”가 찍힙니다.

4) 화면이 “불러오는 중...”에서 멈춘 것처럼 보이고, 버튼이 잠긴 채로 영원히 풀리지 않습니다. → 버튼 잠금 해제(goodBtn.disabled = false)가 finally 안에 있기 때문입니다.

이 줄을 try 안 마지막에 두면 성공했을 때만 실행됩니다.
실패한 사용자는 다시 시도할 방법이 없어집니다.
“무조건 해야 하는 뒷정리”를 finally 에 두는 이유가 이것입니다.

5) 성공 쪽(.then)으로 갑니다. .catch 로 가지 않습니다. → 섹션 2에서 본 것과 똑같은 함정입니다. then 방식이든 try/catch

방식이든, 404 를 실패로 만들어 주는 것은 오직 res.ok 검사뿐입니다.
그 줄을 지우면 빈 객체 {} 를 받아 화면이 조용히 비어 버립니다.

6) 실행되지 않습니다. 바꾸기 전에는 두 줄이 나왔는데

```
// 콘솔: JSON 파싱 실패: SyntaxError
// 콘솔: 에러가 났지만 프로그램은 계속됩니다
```

바꾼 뒤에는 아랫줄 하나만 남습니다.

```
// 콘솔: 에러가 났지만 프로그램은 계속됩니다
```

→ catch 는 ‘에러가 났을 때만’ 들르는 곳입니다.

try 안이 무사히 끝나면 catch 는 통째로 건너뛰집니다.
즉 “JSON 파싱 실패” 줄이 사라진 것이 정답의 증거입니다.
개념02 섹션 4의 .catch 와 같은 규칙입니다.

주의 · ③ 버튼을 눌렀다면 콘솔에 “성공: 1” 이라는 줄도 보입니다.

그건 섹션 3의 `loadPostProperly` 가 찍은 것이지 이 실습과 무관합니다.

비동기와 fetch

RUN 브라우저로 열기. 인터넷 연결 필요. F12 → Console 필수

```
const BASE_URL = "https://jsonplaceholder.typicode.com";
```

문제 1 (개념01 setTimeout)

timerBtn 을 누르면 out1 에 "기다리는 중..." 을 넣고, 2초 뒤에 "2초가 지났습니다" 로 바꾸세요.

기대 화면 누르자마자 "기다리는 중..." 이 나오고, 2초 뒤 "2초가 지났습니다" 로 바뀝니다.
누르자마자 바로 "2초가 지났습니다" 가 나오면 괄호를 붙여 즉시 실행한 것입니다.

여기에 코드를 쓰세요

문제 2 (개념01 실행 순서)

아래 세 줄을 실행하면 콘솔에 어떤 순서로 찍힐까요? 먼저 예상을 적어 보고, 주석을 풀어 확인하세요.

내 예상: _____

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

문제 3 (개념01 setInterval)

countBtn 을 누르면 out3 에 3, 2, 1 을 1초 간격으로 보여 주고 0이 되면 "끝!" 을 넣은 뒤 반드시 clearInterval 로 멈추세요.

기대 화면 3 → 2 → 1 이 1초 간격으로 바뀌고 마지막에 "끝!" 이 나옵니다.
"끝!" 뒤에도 숫자가 계속 바뀌면 clearInterval 을 안 한 것입니다.

여기에 코드를 쓰세요

문제 4 (개념02 Promise)

아래 함수는 이미 만들어져 있습니다. promiseBtn 을 누르면 delay(1000) 이 끝난 뒤 out4 에 "1초 뒤 완료" 를 넣으세요. then 을 쓰세요.

```
function delay(ms, shouldFail = false) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      if (shouldFail) {  
        reject(new Error("일부러 실패했습니다"));  
      } else {  
        resolve("작업 완료");  
      }  
    }, ms);  
  });  
}
```

기대 화면 누르면 "진행 중..." 이 나오고, 1초 뒤 "1초 뒤 완료" 로 바뀝니다.

여기에 코드를 쓰세요

문제 5 (개념02 catch)

rejectBtn 을 누르면 delay(1000, true) 를 실행하고, 실패했을 때 out4 에 "오류: 일부러 실패했습니다" 를 넣으세요. catch 를 쓰세요.

기대 화면 1초 뒤 out4 → 오류: 일부러 실패했습니다 (빨간 글씨)
then 안의 글자가 나오면 안 됩니다. 실패하면 then 은 건너뛰니다.

여기에 코드를 쓰세요

문제 6 (개념03 fetch)

oneBtn 을 누르면 `${BASE_URL}/posts/1` 을 가져와 out6 에 그 글의 title 을 넣으세요. then 방식으로 쓰세요.

기대 화면 out6 → sunt aut facere repellat provident occaecati excepturi optio reprehenderit
(연습용 서버가 주는 1번 글의 제목입니다. 라틴어처럼 보이는 게 정상입니다)

여기에 코드를 쓰세요

문제 7 (개념03 목록 그리기)

listBtn 을 누르면 `${BASE_URL}/posts?_limit=5` 를 가져와 postList 안에 li 로 "1. 제목" 형태로 그리세요.

기대 화면 postList 에 li 5개가 생깁니다. 첫 줄은
1. sunt aut facere repellat provident occaecati excepturi optio reprehenderit
항목 사이에 쉼표가 보이면 `join(", ")` 을 빠뜨린 것입니다.

여기에 코드를 쓰세요

문제 8 (개념04 async/await)

문제 6과 같은 일을 async/await 로 다시 쓰세요. awaitBtn 을 누르면 out8 에 title 이 나와야 합니다.

기대 화면 문제 6과 똑같은 제목이 out8 에 나옵니다. 결과가 같아야 맞게 옮긴 것입니다.

여기에 코드를 쓰세요

문제 9 (개념04 순서대로)

- 1 /posts/1 을 가져와 userId 를 알아내고
- 2 그 userId 로 /users/그번호 를 가져와
- 3 out9 에 "제목 — 작성자이름" 을 넣으세요.

기대 화면 out9 → sunt aut facere repellat provident occaecati excepturi optio reprehenderit – Leanne Graham

F12 콘솔 작성자: Leanne Graham

여기에 코드를 쓰세요

문제 10 (개념04 Promise.all)

allBtn 을 누르면 /posts/1, /posts/2, /posts/3 을 '동시에' 가져와 allList 에 세 개의 li 로 제목을 그리세요.

기대 화면 allList 에 li 3개.
1. sunt aut facere repellat provident occaecati excepturi optio reprehenderit
2. qui est esse
3. ea molestias quasi exercitationem repellat qui ipsa sit aut

여기에 코드를 쓰세요

문제 11 (개념05 try/catch)

tryBtn 을 누르면 존재하지 않는 주소로 fetch 를 시도하고, 실패하면 out11 에 “요청에 실패했습니다” 를 넣고 error 클래스를 붙이세요. 주소는 “https://없는주소-abcxyz.example/data” 를 쓰세요.

기대 화면 잠시 뒤 out11 → 요청에 실패했습니다 (빨간 글씨)
콘솔에도 빨간 네트워크 에러가 한 줄 뜨는데, 이건 정상입니다.
일부러 없는 주소로 보냈기 때문입니다.

여기에 코드를 쓰세요

문제 12 (개념05 res.ok)

okBtn 을 누르면 `${BASE_URL}/posts/9999` 를 요청하고 res.ok 와 res.status 를 콘솔에 찍으세요.

기대 콘솔 false 404

여기에 코드를 쓰세요

문제 13 [응용]

- 1 버튼을 잠그고(disabled = true) out13 에 "불러오는 중..." 표시
- 2 `${BASE_URL}/posts/1` 요청
- 3 `res.ok` 가 `false` 면 `throw new Error(서버 오류 ${res.status})`
- 4 성공하면 out13 에 title
- 5 실패하면 out13 에 "불러오지 못했습니다" + error 클래스,

콘솔에는 `error.message`

6) finally 에서 버튼 잠금 해제

기대 화면 누르는 순간 버튼이 잠기고 "불러오는 중..." → 잠시 뒤 1번 글 제목.
 끝나면 버튼이 다시 눌립니다.
 버튼이 잠긴 채로 남으면 finally 를 안 쓴 것입니다.

여기에 코드를 쓰세요

문제 14 [도전]

idInput 에 번호를 넣고 searchBtn 을 누르면 `${BASE_URL}/posts/번호` 를 가져와 out14 에 제목을 넣으세요. - 입력값은 문자열이라는 점에 주의하세요 - 없는 번호(예: 9999)면 "그런 글이 없습니다" 를 보여 주세요 - try/catch/finally 를 쓰세요

기대 화면 1 을 넣고 검색 → out14 에 1번 글 제목
 9999 를 넣고 검색 → out14 → 그런 글이 없습니다
 빈 칸으로 검색 → 1 이상의 번호를 넣어 달라는 안내

여기에 코드를 쓰세요

문제 15 (에러 확인 - 맨 마지막)

아래 코드의 주석을 풀고 실행해서 콘솔을 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
async function getTitle() {  
  const res = await fetch(`${BASE_URL}/posts/1`);  
  const data = res.json();  
  return data.title;  
}  
getTitle().then((t) => console.log("제목:", t));
```

제목이 `undefined` 로 나옵니다. 어느 줄이 잘못됐을까요?

답

비동기

RUN 브라우저로 열고 F12 → Console 탭

```
const BASE_URL = "https://jsonplaceholder.typicode.com";
```

아래 두 함수는 이미 만들어져 있습니다. 그대로 쓰세요.

ms 만큼 기다렸다가 value 를 돌려주는 함수

```
function delay(ms, value) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(value), ms);
  });
}
```

절반의 확률로 실패하는 불안정한 작업

```
function unstableTask() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < 0.5) {
        resolve("작업 성공");
      } else {
        reject(new Error("일시적 오류"));
      }
    }, 300);
  });
}
```

심화 1 순서대로 vs 동시에 (시간 재기)

같은 일을 두 방식으로 하고, 걸린 시간을 비교해 보세요.

seqBtn : delay(500, "A"), delay(500, "B"), delay(500, "C") 를 순서대로 await allBtn : 같은 셋을 Promise.all 로 한 번에

각각 끝나면 out1 에 "순서대로: A B C (약 1500ms)" 처럼 결과와 걸린 시간을 함께 보여 주세요.

힌트: 시간을 재려면 시작할 때와 끝날 때의 Date.now() 를 빼면 됩니다.

(밀리초 단위 숫자가 나옵니다. 15단원에서 자세히 배웁니다)

```
const start = Date.now();
... 작업 ...
const 걸린시간 = Date.now() - start;
```

기대 화면 순서대로 → 약 1500ms (500 을 세 번 기다림)
 동시예 → 약 500ms (셋을 같이 보내고 제일 늦은 것만 기다림)

숫자는 정확히 1500 이 아니라 1503, 1512 처럼 조금씩 다릅니다. 정상입니다.

여기에 코드를 쓰세요

심화 2 실패하면 다시 시도

unstableTask 는 절반 확률로 실패합니다. 실패하면 최대 3번까지 다시 시도하는 함수를 만드세요.

- 성공하면 그 값을 돌려주고 끝냅니다 - 실패하면 콘솔에 "1번째 실패, 다시 시도" 를 찍고 재시도합니다 -
 3번 다 실패하면 "3번 시도했지만 실패했습니다" 를 out2 에 넣고

error 클래스를 붙입니다

힌트: 반복문 안에서 try/catch 를 씁니다.

성공하면 **return** 으로 함수를 끝내고,
 실패하면 **catch** 에서 잡고 다음 바퀴로 넘어갑니다.
 마지막 바퀴에서도 실패했는지 알려면 i 를 확인하면 됩니다.

기대 화면 대부분 1~2번 만에 성공합니다. out2 에 "작업 성공 (2번째 시도)" 처럼
 나옵니다.
 운이 나쁘면(1/8 확률) 세 번 다 실패해 빨간 글씨가 나옵니다.
 여러 번 눌러 두 경우를 다 보세요.

여기에 코드를 쓰세요

심화 3 연타해도 한 번만

loadBtn 을 빠르게 여러 번 눌러도 요청이 한 번만 나가게 만드세요.

- 요청 중이면 버튼을 잡고 out3 에 “불러오는 중...” - 끝나면 버튼을 풀고 결과(글 제목)를 보여 줍니다 - 실패해도 버튼은 반드시 풀려야 합니다 - 콘솔에 “요청 시작” 이 몇 번 찍히는지 세어 보세요. 연타해도 한 번이어야 합니다

주소는 `${BASE_URL}/posts/1` 입니다.

힌트: 방법이 두 가지입니다. 둘 다 해 보면 좋습니다. ① 버튼 disabled 로 막기 — 눈에도 보여서 사용자에게 친절합니다 ② isLoading 같은 변수로 막기 — 버튼이 없어도 되고 더 확실합니다 실무에서는 둘을 같이 씁니다.

기대 콘솔 연타해도 "요청 시작" 이 딱 한 번만 찍힙니다

여기에 코드를 쓰세요

12단원 마무리 비동기 fetch

자주 넘어지는 자리

1 fetch 는 404를 실패로 치지 않습니다. `res.ok` 를 직접 봐야 합니다

종합 연습

OPEN 13_종합_연습

```
const students = [  
const products = [  
const cart = [  
{ id: 1, count: 2 },
```

01 성적 관리 프로그램

02 장바구니

03 할 일 목록

04 사용자 검색

성적 관리 프로그램

RUN node 종합01_성적관리.js

사용 단원: 01~09단원 전부

학생들의 점수 데이터를 가지고 여러 가지를 계산합니다. 아래 데이터는 고치지 말고, TODO 부분만 채우세요.

한 문제를 풀 때마다 실행해서 기대 출력과 맞는지 확인하세요. 앞 문제에서 만든 함수를 뒤에서 쓰는 경우가 있으니 순서대로 푸세요. 막히면 종합01_성적관리_정답.js 를 보세요. 다만 먼저 혼자 고민해 보세요.

```
const students = [
  { name: "김민준", scores: [90, 85, 70] },
  { name: "이서연", scores: [95, 88, 92] },
  { name: "박지훈", scores: [60, 45, 80] },
  { name: "최유진", scores: [75, 82, 78] },
  { name: "정하늘", scores: [50, 55, 48] },
];

const subjects = ["국어", "영어", "수학"];

console.log("===== 성적 관리 프로그램 =====");
```

출력 ===== 성적 관리 프로그램 =====

문제 1 학생 수

전체 학생이 몇 명인지 출력하세요.

기대 출력 전체 5명

여기에 코드를 쓰세요

문제 2 총점 구하는 함수

점수 배열을 받아 총점을 돌려주는 getTotal 함수를 만드세요. reduce 를 쓰세요. 그다음 김민준의 총점을 출력하세요.

기대 출력 김민준 총점 245

여기에 코드를 쓰세요

문제 3 평균 구하는 함수

점수 배열을 받아 평균을 돌려주는 getAverage 함수를 만드세요. 문제 2의 getTotal 을 안에서 쓰세요. 그다음 이서연의 평균을 소수 첫째 자리까지 출력하세요. (함수는 숫자를 그대로 돌려주고, 자리수 정리는 출력할 때 하세요. toFixed 의 결과는 문자열이라 계산에 다시 쓸 수 없기 때문입니다)

기대 출력 이서연 평균 91.7

여기에 코드를 쓰세요

문제 4 등급 정하는 함수

평균 점수를 받아 등급을 돌려주는 getGrade 함수를 만드세요. 90 이상 A / 80 이상 B / 70 이상 C / 60 이상 D / 그 외 F

기대 출력 95점 → A
72점 → C
40점 → F

여기에 코드를 쓰세요

문제 5 전체 명단 출력

모든 학생의 “이름 총점 평균 등급” 을 한 줄씩 출력하세요. `forEach` 와 위에서 만든 함수들을 쓰세요.

기대 출력 김민준 245점 평균 81.7 (B)
 이서연 275점 평균 91.7 (A)
 박지훈 185점 평균 61.7 (D)
 최유진 235점 평균 78.3 (C)
 정하늘 153점 평균 51.0 (F)

여기에 코드를 쓰세요

문제 6 합격자만

평균 60점 이상인 학생의 이름만 배열로 만들어 출력하세요. `filter` 와 `map` 을 이어 쓰세요.

기대 출력

— ‘김민준’, ‘이서연’, ‘박지훈’, ‘최유진’

여기에 코드를 쓰세요

문제 7 1등 찾기

평균이 가장 높은 학생의 이름과 평균을 출력하세요.

기대 출력 1등: 이서연 (91.7)

여기에 코드를 쓰세요

문제 8 과목별 평균

과목마다 전체 학생의 평균을 구해 출력하세요. subjects 배열과 인덱스를 함께 쓰세요.

힌트: 지금까지는 학생 한 명의 점수를 "가로로" 훑었습니다. 이번엔 반대입니다.

모든 학생의 scores[0] 만 모으면 그게 국어 점수 목록입니다.
`students.map((s) => s.scores[0])` → [90, 95, 60, 75, 50]
 subjects 를 forEach 로 돌면서, 그때의 인덱스로 위처럼 세로로 뽑으면 됩니다.
 뽑은 배열은 문제 2에서 만든 평균 함수에 그대로 넘기면 됩니다.

기대 출력 국어 평균 74.0
 영어 평균 71.0
 수학 평균 73.6

여기에 코드를 쓰세요

문제 9 등급별 인원

등급마다 몇 명인지 세어 객체로 만들어 출력하세요. reduce 를 쓰세요.

기대 출력 { B: 1, A: 1, D: 1, C: 1, F: 1 }

여기에 코드를 쓰세요

문제 10 [도전] 등수 매기기

평균이 높은 순으로 정렬해서 등수를 붙여 출력하세요. 그다음 students[0].name 을 출력해 원본이 그대로인지 확인하세요. (원본이 정렬돼 버렸다면 마지막 줄이 "이서연" 으로 나옵니다)

기대 출력	1위 이서연	91.7
	2위 김민준	81.7
	3위 최유진	78.3
	4위 박지훈	61.7
	5위 정하늘	51.0
	김민준	

여기에 코드를 쓰세요

문제 11 [도전] 재시험 대상

한 과목이라도 60점 미만인 학생의 이름과 해당 과목을 출력하세요.

힌트: 문제 8과 같은 "인덱스로 짝 맞추기" 를 씁니다.

학생 한 명의 scores 를 돌 때, 그때의 인덱스가 subjects 의 인덱스와 같습니다.
scores[1] 이 45 라면 그 과목 이름은 subjects[1] 즉 "영어" 입니다.
60점 미만인 것만 "영어(45)" 형태로 모아 두었다가,
모은 게 하나라도 있는 학생만 출력하세요.

기대 출력	박지훈: 영어(45)
	정하늘: 국어(50), 영어(55), 수학(48)

여기에 코드를 쓰세요

장바구니

RUN node 종합02_장바구니.js

사용 단원: 05~09단원 (함수·배열·객체·콜백·구조분해·스프레드)

온라인 주문 장바구니를 만듭니다. 아래 데이터는 고치지 말고 TODO 부분만 채우세요.

이 실습의 규칙 하나: 원본 데이터를 직접 고치지 않습니다. 바꿀 때는 항상 '새 배열 / 새 객체'를 만듭니다. (09단원)

앞 문제에서 만든 함수를 뒤에서 쓰니 순서대로 푸세요. 막히면 종합02_장바구니_정답.js 를 보세요. 다만 먼저 혼자 고민해 보세요.

```
const products = [
  { id: 1, name: "아메리카노", price: 4000, stock: 10 },
  { id: 2, name: "카페라떼", price: 4500, stock: 3 },
  { id: 3, name: "케이크", price: 6000, stock: 0 },
  { id: 4, name: "쿠키", price: 3000, stock: 7 },
];

const cart = [
  { id: 1, count: 2 },
  { id: 4, count: 3 },
];

console.log("===== 장바구니 =====");
```

출력 ===== 장바구니 =====

문제 1 판매 가능한 상품

재고가 1개 이상인 상품의 이름만 배열로 만들어 출력하세요.

기대 출력

— '아메리카노', '카페라떼', '쿠키'

여기에 코드를 쓰세요

문제 2 상품 찾기 함수

id 를 받아 그 상품 객체를 돌려주는 findProduct 함수를 만드세요. 없으면 undefined 가 나오면 됩니다.

기대 출력 { id: 2, name: '카페라떼', price: 4500, stock: 3 }
undefined

여기에 코드를 쓰세요 (findProduct(2) 와 findProduct(99) 를 출력)

문제 3 장바구니 한 줄씩

장바구니의 각 항목을 "이름 x개 = 금액원" 형태로 출력하세요. 이름과 가격은 findProduct 로 찾아야 합니다.

기대 출력 아메리카노 x2 = 8000원
쿠키 x3 = 9000원

여기에 코드를 쓰세요

문제 4 총액

장바구니 총액을 돌려주는 getCartTotal 함수를 만들고 결과를 출력하세요. reduce 를 쓰세요.

기대 출력 총액 17000원

여기에 코드를 쓰세요

문제 5 답기

장바구니와 상품 id, 개수를 받아 '새 장바구니 배열' 을 돌려주는 addToCart 함수를 만드세요. - 이미 담긴 상품이면 개수만 늘립니다 - 없으면 맨 뒤에 새로 추가합니다 - 원본 cart 는 바뀌면 안 됩니다

기대 출력

```
[{ id: 1, count: 2 }, { id: 4, count: 3 }, { id: 2, count: 1 }][{ id: 1, count: 5 }, { id: 4, count: 3 }][{ id: 1, count: 2 }, { id: 4, count: 3 }]
```

여기에 코드를 쓰세요

(addToCart(cart, 2, 1) / addToCart(cart, 1, 3) / 원본 cart 를 차례로 출력)

문제 6 빼기

장바구니와 상품 id 를 받아 그 항목을 뺀 '새 배열' 을 돌려주는 removeFromCart 함수를 만드세요.

기대 출력

— { id: 4, count: 3 }

여기에 코드를 쓰세요

문제 7 재고 확인

장바구니의 각 항목이 재고를 넘지 않는지 확인하는 `hasEnoughStock` 함수를 만드세요. 전부 괜찮으면 `true` 입니다.

기대 출력 `true`
 `false`

여기에 코드를 쓰세요

(원래 `cart` 와, 쿠키를 10개 담은 장바구니로 각각 확인)

문제 8 할인

총액을 받아 최종 금액을 돌려주는 `getFinalPrice` 함수를 만드세요. 3만원 이상이면 10% 할인, 아니면 그대로 소수가 나오면 `Math.round` 로 다듬을 것

기대 출력 `17000`
 `45000`

여기에 코드를 쓰세요

(17000 과 50000 을 각각 넣어 출력)

문제 9 영수증

아래 형태로 영수증을 출력하는 `printReceipt` 함수를 만드세요. 위에서 만든 함수들을 조합해서 쓰세요.

기대 출력

아메리카노 x2 = 8000원 쿠키 x3 = 9000원

합계 17000원 결제 17000원

여기에 코드를 쓰세요

문제 10 [도전] 품절 상품 담기 막기

addToCart 를 '고치지 말고', 그것을 본떠 addToCartSafe 라는 함수를 새로 만드세요. (앞 문제의 addToCart 는 그대로 두어야 앞 문제 출력이 안 깨집니다) addToCartSafe 는 재고가 0인 상품을 담으려 하면 콘솔에 "케이크은(는) 품절입니다" 를 찍고 장바구니를 그대로 돌려줍니다.

기대 출력 케이크은(는) 품절입니다

[{ id: 1, count: 2 }, { id: 4, count: 3 }]

여기에 코드를 쓰세요

할 일 목록

RUN 브라우저로 열고 F12 → Console 탭

- 1 데이터는 배열 하나(todos)에만 담는다
- 2 화면은 그 배열을 보고 '전부 다시 그린다' (render 함수)
- 3 무슨 일이 생기면 배열만 고치고 render 를 다시 부른다

화면을 직접 조금씩 고치는 것보다 이 방식이 훨씬 관리하기 쉽습니다. React 도 이 생각을 자동화한 도구입니다.

할 일 하나의 모양: { id: 1, text: "장보기", done: false }

```
let todos = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "설거지", done: true },
];

let nextId = 3;
let currentFilter = "all"; // "all" | "active" | "done"
const todoForm = document.querySelector("#todoForm");
const todoInput = document.querySelector("#todoInput");
const todoList = document.querySelector("#todoList");
const errorMsg = document.querySelector("#errorMsg");
const status = document.querySelector("#status");
const clearBtn = document.querySelector("#clearBtn");
```

문제 1 걸러내기

currentFilter 값에 따라 보여 줄 목록을 돌려주는 getVisibleTodos 함수를 만드세요. "all" → 전부
"active" → done 이 false 인 것만 "done" → done 이 true 인 것만

기대 출력 이 함수만으로는 화면이 바뀌지 않습니다. 문제 2·8 까지 해야 눈에 보입니다.
지금 확인하려면 F12 콘솔에 getVisibleTodos() 를 직접 쳐 보세요.
currentFilter 가 "all" 이므로 항목 2개짜리 배열이 나와야 합니다.

여기에 코드를 쓰세요

문제 2 화면 그리기

getVisibleTodos 로 얻은 목록을 todoList 안에 li 로 그리는 render 함수를 만드세요.

li 하나의 구조 (정답 파일과 같은 모양이어야 아래 문제들이 동작합니다):

```
<li class="done" data-id="1">
  <span class="text">장보기</span>
  <button class="deleteBtn">삭제</button>
</li>
```

- done 이 true 면 li 에 done 클래스를 붙입니다 - 사용자가 입력한 글이므로 textContent 로 넣습니다 - 그리기 전에 todoList 를 반드시 비우세요 - 보여 줄 항목이 하나도 없으면

```
<li class="empty">항목이 없습니다</li> 를 넣으세요
```

- 마지막에 setStatus() 를 부르세요

기대 화면 문제 8가지 해야 실제로 보입니다. 다 되면 목록에 두 줄이 나옵니다.
장보기 [삭제]
설거지 [삭제] ← 회색 + 취소선 (done 클래스가 붙어서)

여기에 코드를 쓰세요

문제 3 상태 표시

아래 형태로 status 에 글자를 넣는 setStatus 함수를 만드세요. "할 일 3개 (완료 1개, 남은 일 2개)" 전체 todos 기준입니다. 필터와 상관없습니다.

기대 화면 상태줄 → 할 일 2개 (완료 1개, 남은 일 1개)

여기에 코드를 쓰세요

문제 4 추가

todoForm 을 제출하면 - 새로그침을 막고 - 빈 값(공백만 포함)이면 errorMsg 에 “할 일을 입력해 주세요”
- 이미 있는 내용이면 “이미 있는 항목입니다” - 통과하면 todos 에 새 항목을 추가하고 화면을 다시
그립니다

(원본을 직접 바꾸지 말고 새 배열을 만들어 todos 에 넣으세요)

- 입력칸을 비우고 focus 를 줍니다

기대 화면 공백만 넣고 Enter → 할 일을 입력해 주세요 (목록 그대로 2개)
"장보기" 넣고 Enter → 이미 있는 항목입니다 (목록 그대로 2개)
"운동하기" 넣고 Enter → 목록 3개, 입력칸이 비워지고 오류 글자도
사라집니다
상태줄 → 할 일 3개 (완료 1개, 남은 일 2개)
Enter 를 쳤는데 화면이 깜빡이며 목록이 사라지면 preventDefault 를 빠뜨린
것입니다.

여기에 코드를 쓰세요

문제 5 완료 토글 / 삭제 (이벤트 위임)

todoList 하나에만 click 이벤트를 붙여서 - .text 를 누르면 그 항목의 done 을 뒤집고 - .deleteBtn 을
누르면 그 항목을 지우세요 어느 항목인지는 li 의 data-id 로 알아냅니다. dataset 값은 문자열이라는 점에
주의하세요.

기대 화면 위 상태(장보기 / 설거지 / 운동하기)에서
 "장보기" 글자를 누르면 → 취소선이 생기고 상태줄 → 할 일 3개 (완료 2개, 남은 일 1개)
 "장보기" 의 삭제를 누르면 → 목록 2개, 상태줄 → 할 일 2개 (완료 1개, 남은 일 1개)
 글자를 눌렀는데 아무 일도 없으면 dataset 값을 Number 로 안 바꾼 것입니다.

여기에 코드를 쓰세요

문제 6 필터 버튼

.filterBtn 을 누르면 - currentFilter 를 그 버튼의 data-filter 값으로 바꾸고 - 눌린 버튼에만 active 클래스가 붙게 하고 - 화면을 다시 그리세요

기대 화면 설거지(완료) / 운동하기(미완료) 두 개가 있을 때

미완료 → 운동하기만 보임 [완료] → 설거지만 보임 [전체] → 둘 다

누른 버튼만 파란색이 되어야 합니다. 상태줄은 필터와 상관없이 전체 기준이라 계속 "할 일 2개 (완료 1개, 남은 일 1개)" 입니다.

여기에 코드를 쓰세요

문제 7 완료 항목 삭제

clearBtn 을 누르면 done 이 true 인 항목을 전부 지우세요.

기대 화면 위 상태에서 [완료 항목 삭제] → 목록에 운동하기만 남습니다.
 상태줄 → 할 일 1개 (완료 0개, 남은 일 1개)

여기에 코드를 쓰세요

문제 8 * 필수 (이걸 빼면 화면에 아무것도 안 나옵니다)

페이지를 열자마자 위 데이터가 화면에 보이도록 render 를 한 번 부르세요. (이벤트는 '바뀔 때' 만 실행되므로 처음 한 번은 직접 불러야 합니다)

딱 한 줄입니다. 어렵지는 않지만 이게 없으면 앱이 통째로 비어 보입니다. 참고로 이 실습에서 가장 어려운 것은 문제 5(위임 + dataset 문자열 변환)입니다.

기대 화면 새로그침하자마자 목록에 장보기 / 설거지 두 줄이 보여야 합니다.
이걸 빼면 앞의 문제를 다 맞게 풀어도 화면이 텅 빈 채로 있습니다.

여기에 코드를 쓰세요

사용자 검색

RUN 브라우저로 열고 F12 → Console 탭

- 1 받아온 데이터를 allUsers 배열에 담는다
- 2 화면은 그 배열을 걸러서 다시 그린다 (render)
- 3 검색·정렬은 배열만 건드리고 render 를 다시 부른다

서버에서 오는 데이터의 모양 (필요한 것만 적었습니다) { id: 1, name: "Leanne Graham", email: "...", phone: "...",

address: { city: "Gwenborough" }, company: { name: "..." } }

```
const API_URL = "https://jsonplaceholder.typicode.com/users";

let allUsers = []; // 받아온 전체
let sortByName = false;
let hasError = false; // 불러오기에 실패했는지 (문제 1·3에서 씁니다)
const searchInput = document.querySelector("#searchInput");
const loadBtn = document.querySelector("#loadBtn");
const sortBtn = document.querySelector("#sortBtn");
const message = document.querySelector("#message");
const userList = document.querySelector("#userList");
const detail = document.querySelector("#detail");
```

문제 1 불러오기

loadUsers 라는 async 함수를 만드세요. 1) 버튼 두 개를 잠그고(disabled) message 에 “불러오는 중...” 표시

```
(message.className = "loading")
```

- 2 API_URL 로 fetch
- 3 res.ok 가 false 면 throw new Error(서버 오류 \${res.status})
- 4 성공하면 allUsers 에 담고 message 를 비운 뒤 render() 호출
- 5 실패하면 message 에 “불러오지 못했습니다. 다시 시도해 주세요.”

```
+ className = "error", 콘솔에는 error.message, 그리고 hasError = true
```

6) finally 에서 버튼 잠금 해제 (다시 시도할 때를 대비해 함수 맨 처음에 hasError = false 로 되돌리세요)

기대 화면 [불러오기] 를 누르면 버튼 두 개가 잠기고 "불러오는 중..." 이 뜬 뒤,
잠시 후 사람 10명이 목록에 나옵니다. 끝나면 버튼이 다시 눌립니다.

F12 콘솔 받은 사용자 수: 10

인터넷이 막혀 있으면 "불러오지 못했습니다. 다시 시도해 주세요." 가 나옵니다. (그때는 이 파일 위쪽
오프라인_대체.js 줄의 주석 기호를 지우세요)

여기에 코드를 쓰세요

문제 2 걸러내기

검색어에 맞는 사용자만 돌려주는 getVisibleUsers 함수를 만드세요. - 검색어가 비어 있으면 전부 - 이름
(name) 또는 도시(address.city)에 검색어가 들어 있으면 포함 - 대소문자를 구분하지 않아야 합니다
(toLowerCase) - sortByName 이 true 면 이름순으로 정렬 (원본은 바꾸지 말 것)

기대 출력 이 함수만으로는 화면이 바뀌지 않습니다. 문제 3·4 까지 해야 눈에 보입니다.

여기에 코드를 쓰세요

문제 3 화면 그리기

getVisibleUsers 결과를 userList 에 li 로 그리는 render 함수를 만드세요.

li 하나의 구조:

```
<li data-id="1">
```

```
<div class="name">Leanne Graham</div>
<div class="sub">Gwenborough · Sincere@april.biz</div>

</li>
```

- 그리기 전에 비울 것 - 결과가 없으면 message 에 "검색 결과가 없습니다" 를 넣을 것 - 결과가 있으면 message 를 비울 것 (이전 안내가 남지 않도록) - 단, hasError 가 true 면 message 를 아예 건드리지 말 것

"못 불러왔다" 와 "검색 결과가 없다" 는 다른 이야기이기 때문입니다.
이 검사가 없으면, 불러오기에 실패한 뒤 검색창에 글자만 쳐도
에러 안내가 "검색 결과가 없습니다" 로 바뀌어 원인을 알 수 없게 됩니다.

기대 화면 목록 첫 줄이 이렇게 나옵니다.
Leanne Graham
Gwenborough · Sincere@april.biz
이름이 굵게, 아래 줄이 작고 회색이면 클래스(name / sub)를 제대로 붙인
것입니다.

여기에 코드를 쓰세요

문제 4 실시간 검색

searchInput 에 글자를 칠 때마다 render() 가 다시 실행되게 하세요.

기대 화면 검색창에 글자를 칠 때마다 목록이 바로 줄어듭니다.
"gwen" → 1명 (Leanne Graham - 도시로 걸린 것입니다. 대소문자 구분 없음)
"zzz" → 0명 + 검색 결과가 없습니다
검색창을 비우면 다시 10명이 됩니다.

여기에 코드를 쓰세요

문제 5 정렬 토글

sortBtn 을 누르면 sortByName 을 뒤집고 화면을 다시 그리세요. 버튼 글자도 바꾸세요. 글자는 '지금 누르면 무엇이 되는지' 를 뜻합니다. 정렬 안 된 상태 → "이름순 정렬" (누르면 이름순이 됩니다) 정렬된 상태 → "원래 순서" (누르면 원래대로 돌아갑니다)

기대 화면 [이름순 정렬] 을 누르면 버튼 글자가 "원래 순서" 로 바뀌고, 목록 맨 위가 Chelsey Dietrich 가 됩니다 (알파벳 순). 한 번 더 누르면 버튼 글자가 "이름순 정렬" 로 돌아오고 순서도 원래대로 돌아옵니다. 원래 순서로 못 돌아오면 정렬할 때 원본 배열을 직접 건드린 것입니다.

여기에 코드를 쓰세요

문제 6 상세 보기 (이벤트 위임)

userList 하나에만 click 을 붙여서, 누른 사람의 정보를 detail 에 아래 형태로 넣으세요.

이름: Leanne Graham 이메일: Sincere@april.biz 전화: 1-770-736-8031 x56442 도시: Gwenborough 회사: Romaguera-Crona

힌트 — 이 문제는 이 실습에서 가장 어렵습니다. 두 가지를 조심하세요. 1) 문제 3에서 li 안에 div 를 넣었으므로, 누르면 e.target 은 언제나

안쪽 div 입니다. e.target.dataset.id 는 항상 undefined 입니다. e.target.closest("li") 로 항목 전체를 먼저 찾으세요. (11단원 개념05)

2) dataset 값은 숫자처럼 보여도 문자열입니다.

Number() 로 바꾸지 않으면 user.id === id 가 영원히 false 입니다.

기대 화면 목록에서 Chelsey Dietrich 를 누르면 아래 상자에
이름: Chelsey Dietrich
이메일: Lucio_Hettinger@annie.ca
전화: (254)954-1289
도시: Roscoeview
회사: Keebler LLC
아무 반응이 없으면 위 힌트의 closest / Number 두 가지를 다시 보세요.

여기에 코드를 쓰세요

문제 7 시작하자마자 불러오기 (* 딱 한 줄입니다)

페이지를 열면 버튼을 누르지 않아도 자동으로 목록이 나오게 하세요.

기대 화면 새로고침하면 [불러오기] 를 누르지 않아도 목록 10명이 저절로 나옵니다.

여기에 코드를 쓰세요

OpenAPI와 Postman

OPEN 14_OpenAPI와_Postman

```
async function 서울날씨() {  
  const 주소 = new URL(날씨주소);  
  const 응답 = await fetch(주소);  
  const 자료 = await 응답.json();  
}
```

01 Open API가 무엇인가

02 주소 만들기 URL과 쿼리스트링

03 Postman으로 먼저 확인하기

04 키가 필요한 API

05 안 되는 경우들

- 연습문제

Open API가 무엇인가

Open API는 '남이 켜 둔 서버'입니다

섹션 1

12단원에서 jsonplaceholder 를 불렀습니다.

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
```

그것도 Open API 였습니다. 다만 **연습용 가짜 데이터**였습니다. 글 제목이 전부 라틴어처럼 생긴 뜻 없는 글자였던 걸 기억하세요.

이번 단원에서는 **진짜 데이터**를 주는 서버를 부릅니다. 날씨, 지도, 환율, 공공데이터 같은 것들입니다.

★ 부르는 방법은 12단원과 **똑같습니다**. fetch 그대로입니다. 달라지는 것은 세 가지뿐입니다.

- 1 문서를 읽어야 합니다 (무슨 값을 어떻게 보내는지)
- 2 키가 필요할 수 있습니다 (개념04)
- 3 안 되는 경우가 생깁니다 (개념05)

직접 해보기 1

12단원에서 쓰던 jsonplaceholder 주소를 브라우저 주소창에 그냥 넣어 보세요.
(<https://jsonplaceholder.typicode.com/posts/1>)

문서를 읽는 법

섹션 2

API 문서에서 확인할 것은 딱 네 가지입니다.

① 주소 어디로 보내나 ② 방식 GET 인가 POST 인가 ③ 보낼 값 무엇을 꼭 넣어야 하나 ④ 응답 모양 무엇이 어떤 이름으로 오나

이번에 쓸 곳은 Open-Meteo 입니다. **키가 필요 없습니다**.

① 주소 <https://api.open-meteo.com/v1/forecast> ② 방식 GET ③ 보낼 값

latitude	위도	(꼭 필요)
longitude	경도	(꼭 필요)
current	지금 값	중 무엇을 받을지
timezone	시간대	

④ 응답 { latitude, longitude, current: { time, temperature_2m, ... } }

★ ④를 미리 안 보고 코드를 쓰면 `undefined` 만 잔뜩 보게 됩니다. **응답 모양을 먼저 눈으로 확인하세요.**
 섹션 4에서 그 방법을 봅니다.

▢ 직접 해보기 2

open-meteo.com 문서에서 위 네 가지(주소·방식·보낼 값·응답 모양)를 직접 찾아보세요. 5분이면 충분합니다.

```
const 날씨주소 = "https://api.open-meteo.com/v1/forecast";
const 서울 = { 위도: 37.5665, 경도: 126.978 };
```

진짜로 불러 봅니다

섹션 3

```
async function 서울날씨() {
  const 주소 = new URL(날씨주소);
  주소.searchParams.set("latitude", 서울.위도);
  주소.searchParams.set("longitude", 서울.경도);
  주소.searchParams.set("current", "temperature_2m,relative_humidity_2m");
  주소.searchParams.set("timezone", "Asia/Seoul");

  const 응답 = await fetch(주소);
  const 자료 = await 응답.json();
  return 자료;
}

document.getElementById("weatherBtn").addEventListener("click", async () => {
  const 칸 = document.getElementById("out1");
  칸.textContent = "받는 중...";

  const 자료 = await 서울날씨();

  칸.textContent =
    `${자료.current.time} 기준 ` +
    `${자료.current.temperature_2m}${자료.current_units.temperature_2m}, ` +
    `습도
    ${자료.current.relative_humidity_2m}${자료.current_units.relative_humidity_2m}`;

  console.log(자료.current);
});
```

```
F12 콘솔 { time: '2026-08-19T12:45', interval: 900, temperature_2m: 29.5,
  relative_humidity_2m: 70 }
```

★ 숫자는 누를 때마다 다릅니다. **진짜 지금 날씨니까요.** 이 자료의 다른 곳과 달리 값을 못 박을 수 없습니다.
 모양(어떤 이름의 값이 오는지)만 같으면 맞은 것입니다.

```
});
```

◦ 직접 해보기 3

서울 좌표를 부산(35.1796, 129.0756)으로 바꾸고 눌러 보세요.

★ 주소.`searchParams.set(...)` 이 무엇인지는 개념02 에서 자세히 봅니다. 지금은 “물음표 뒤에 값을 붙이는 안전한 방법” 정도로 알아 두세요.

응답 모양을 먼저 봅니다

섹션 4

★★ 새 API 를 쓸 때 제일 먼저 할 일입니다. 문서를 믿지 말고 **직접 받아서 눈으로 보세요**. 문서가 낡은 경우가 많습니다.

```
document.getElementById("rawBtn").addEventListener("click", async () => {  
  const 자료 = await 서울날씨();
```

`JSON.stringify` 의 세 번째 인자가 들여쓰기입니다. (12단원 개념03 섹션 7)

```
document.getElementById("out2").textContent = JSON.stringify(자료, null, 2);  
  
  console.log(자료);  
});
```

받으면 이런 모양입니다.

```
{
```

```

"latitude": 37.55,
"longitude": 127,
"generationtime_ms": 0.05257129669189453,
"utc_offset_seconds": 32400,
"timezone": "Asia/Seoul",
"timezone_abbreviation": "GMT+9",
"elevation": 34,
"current_units": {
  "time": "iso8601",
  "interval": "seconds",
  "temperature_2m": "°C",
  "relative_humidity_2m": "%"
},
"current": {
  "time": "2026-08-19T12:45",
  "interval": 900,
  "temperature_2m": 29.5,
  "relative_humidity_2m": 70
}
}

```

★ 눈여겨볼 것

- 값이 `current` 안에 한 겹 더 들어 있습니다.

``자료.temperature_2m`` 이 아니라 ``자료.current.temperature_2m`` 입니다.
한 겹을 빠뜨려서 `undefined` 가 나오는 일이 제일 흔합니다.

- 단위가 **따로** 옵니다. `current_units` 에 있습니다.

"29.5" 만 보고 섭씨라고 단정하면 안 됩니다. API 마다 다릅니다.

- `time` 이 글자입니다. Date 가 아닙니다.

JSON 에는 날짜라는 것이 없습니다. (12단원 개념03 섹션 7)

⇒ 직접 해보기 4

`current` 에 `wind_speed_10m` 을 더해 보세요. 값과 함께 `current_units` 에 단위가 하나 더 생기는지 확인하세요.

```
document.getElementById("diffBtn").addEventListener("click", async () => {
  const 자료 = await 서울날씨();

  const 줄들 = [
    `보낸 위도: ${서울.위도}   받은 위도: ${자료.latitude}`,
    `보낸 경도: ${서울.경도}   받은 경도: ${자료.longitude}`,
  ];

  document.getElementById("out3").textContent = 줄들.join(" / ");
  console.log(줄들.join("\n"));
});
```

F12 콘솔 보낸 위도: 37.5665 받은 위도: 37.55
 보낸 경도: 126.978 받은 경도: 127

★★★ 다릅니다. 오류가 아닙니다.

이 API 는 지구를 격자로 나눠 놓고 **제일 가까운 칸**의 값을 줍니다. 내가 보낸 좌표가 아니라 그 칸의 좌표를 돌려주는 것입니다.

★ 이런 건 문서에 적혀 있습니다. 안 읽으면 “값이 틀렸는데요?” 가 됩니다.

API 마다 이런 규칙이 하나씩 있습니다. 반드시 문서를 한 번 훑으세요.

★★ 그럼 격자 간격이 얼마나 — 이 자료를 만들면서 **한 번 틀렸습니다**.

서울이 37.5665 → 37.55 로 떨어지는 걸 보고 “0.05도 간격이구나” 했습니다. 그런데 다른 도시로 재 보니 안 맞았습니다.

보낸 값	받은 값	
37.5665, 126.978	→ 37.55, 127	← 0.05 로 떨어짐
35.1796, 129.0756	→ 35.2, 129.0625	← ★ 0.05 가 아닙니다
33.4996, 126.5312	→ 33.5, 126.5	
0, 0	→ 0.035149384, 0	← ★★ 딱 떨어지지도 않습니다

★★★ 서울 하나만 보고 규칙을 정했다가 틀린 것입니다.

예보 모델이 지역마다 달라서 간격도 다릅니다. 규칙은 문서에 있습니다.

★ 배울 것은 격자 간격이 아니라 **하나만 보고 규칙을 정하지 않는다**입니다.

API 를 쓸 때 “몇 개 해 보니 이렇더라” 는 자주 틀립니다.

★★ 더 중요한 습관 — 보낸 것과 받은 것을 비교해 보는 것입니다. Network 탭에서 요청 주소와 응답을 나란히 보면 대부분의 문제가 여기서 풀립니다.

▹ 직접 해보기 5

위도와 경도를 둘 다 0 으로 바꿔서 눌러 보세요. 받은 위도가 무엇으로 나오니까? 왜 딱 떨어지지 않을까요?

어떤 API 를 고르나

섹션 6

연습할 API 를 고를 때 세 가지를 봅니다.

키가 필요한가 없으면 바로 씁니다. 있으면 가입해야 합니다 (개념04) 브라우저에서 되나 CORS 가 막힌 곳이 많습니다 (개념05) 횟수 제한이 있나 하루 몇 번, 분당 몇 번 (개념05)

키 없이 브라우저에서 바로 되는 곳 (이 단원에서 씁니다)

Open-Meteo	https://open-meteo.com	날씨
JSONPlaceholder	https://jsonplaceholder.typicode.com	연습용 가짜 데이터

키가 필요한 곳 (개념04 에서 설명만 합니다)

공공데이터포털 <https://www.data.go.kr> 날씨·교통·보건 등 카카오 지도

<https://developers.kakao.com> 주소 → 좌표

★ 공공데이터포털은 이 과정의 주제(제조 현장)와 맞는 자료가 많습니다. 그런데 브라우저에서 직접 못 부릅니다. 왜 그런지는 개념05 에서 봅니다.

▹ 직접 해보기 6

개념02 를 열기 전에, 지금 주소가 어떻게 만들어지는지 Network 탭의 Request URL 로 확인해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

응답의 한 겹을 빠뜨림

```
const 자료 = await 응답.json();
console.log(자료.temperature_2m);
// 콘솔: undefined
```

→ 자료.current.temperature_2m 입니다.

★ undefined 가 나오면 **그 위 단계를 찍어 보세요.** `console.log(자료)` 부터.

▹ 직접 해보기 7

일부러 자료.temperature_2m 을 찍어 undefined 를 눈으로 보세요. 그 다음 자료를 통째로 찍어 어디에 있는지 찾으세요.

이런 실수를 합니다

.json() 을 안 붙임

```
const 자료 = await fetch(주소);  
console.log(자료.current);  
// 콘솔: undefined
```

→ fetch 가 주는 것은 응답 **봉투**입니다. 안을 꺼내야 합니다.

```
`const 자료 = await (await fetch(주소)).json();`  
(12단원 개념03 섹션 1)
```

이런 실수를 합니다

문서를 안 보고 이름을 짐작

```
주소.searchParams.set("temp", "true");
```

→ 이름이 틀리면 **오류가 안 납니다**. 그냥 무시됩니다.

응답에 그 값이 없을 뿐입니다. 그래서 더 찾기 어렵습니다.
* 값이 안 오면 제일 먼저 ****보낸 주소****를 확인하세요.

이런 실수를 합니다

단위를 짐작

→ current_units 를 보세요. km/h 인지 m/s 인지 API 마다 다릅니다.

이런 실수를 합니다

인터넷이 없는 곳에서 되게 하려고 애씀

→ 이 단원은 인터넷이 필요합니다. 막혀 있으면 **오프라인_대체.js** 를 켜세요.

* 대체본은 ****같은 모양****의 값을 돌려줍니다. 코드는 안 고쳐도 됩니다.

주소 만들기 URL과 쿼리스트링

쿼리스트링이 무엇인가

섹션 1

https://api.open-meteo.com/v1/forecast?latitude=37.57&longitude=126.98

└────────── 주소 ─────────┘ └────────── 쿼리스트링 ─────────┘

· ? 부터가 쿼리스트링입니다 · 이름=값 을 & 로 잇습니다 · 순서는 대개 상관없습니다

★ 이걸 주소의 일부입니다. 그래서 주소에 못 쓰는 글자는 못 넣습니다. 한글, 공백, &, =, %, + ... 전부 못 넣습니다. 그런데 우리는 넣어야 합니다. "3호 프레스" 를 검색해야 하니까요.

★★ 그래서 바꿔서 넣습니다. 그 규칙을 이 파일에서 봅니다.

▸ 직접 해보기 1

개념01 에서 만들어진 주소를 보고 ? 뒤를 이름=값 단위로 끊어 종이에 적어 보세요.

손으로 이어 붙이면 생기는 일

섹션 2

```
document.getElementById("b1").addEventListener("click", () => {
  const 검색어 = "3호 프레스";
```

나쁜 방법 · 글자를 그냥 이어 붙입니다

```
const 손으로 = `https://예시.com/search?q=${검색어}&page=1`;
```

좋은 방법 · URL 에게 맡깁니다

```
const 제대로 = new URL("https://예시.com/search");
제대로.searchParams.set("q", 검색어);
제대로.searchParams.set("page", "1");

console.log("손으로:", 손으로);
```

F12 콘솔 손으로: https://예시.com/search?q=3호 프레스&page=1

```
console.log("URL로 :", 제대로.toString());
```

```
F12 콘솔    URL로 : https://xn--vv4b11d.com/search?
            q=3%ED%98%B8+%ED%94%84%EB%A0%88%EC%8A%A4&page=1
```

```
document.getElementById("out1").innerHTML =
    `<b>손으로</b><br>${손으로}<br><br><b>URL로</b><br>${제대로.toString()}`;
});
```

★★★ 두 가지가 달라졌습니다.

(1) 3호 프레스 가 3%ED%98%B8+%ED%94%84%EB%A0%88%EC%8A%A4 가 됐습니다

한글과 공백이 주소에 쓸 수 있는 글자로 바뀐 것입니다.

(2) 예시.com 이 xn--vv4b11d.com 이 됐습니다

★ 한글 도메인도 실제로는 영문으로 바뀌어 나갑니다. (퓨니코드)
주소창에만 한글로 보여 주는 것입니다.

★ "손으로 한 것도 브라우저에서 되던데요?" — 맞습니다. fetch 에 넣으면 브라우저가 알아서 고쳐 줍니다.
그런데 **고쳐 주는 규칙이 자리마다 다릅니다**. 섹션 5에서 봅니다. 말기지 말고 내가 정하는 게 낫습니다.

▹ 직접 해보기

2

검색어를 "A&B" 로 바꿔서 눌러 보세요. 손으로 이어 붙인 쪽과 URL 쪽이 어떻게 달라집니까?

★★ 공백은 + 가 됩니다

섹션 3

```
document.getElementById("b2").addEventListener("click", () => {
    const 검색어 = "3호 프레스";

    const 쿼리 = new URLSearchParams();
    쿼리.set("q", 검색어);

    console.log("URLSearchParams :", 쿼리.toString());
```

```
F12 콘솔    URLSearchParams : q=3%ED%98%B8+%ED%94%84%EB%A0%88%EC%8A%A4
```

```
console.log("encodeURIComponent:", encodeURIComponent(검색어));
```

```
F12 콘솔    encodeURIComponent: 3%ED%98%B8%20%ED%94%84%EB%A0%88%EC%8A%A4
```

```
const 되읽기1 = new URLSearchParams("q=a+b").get("q");
const 되읽기2 = new URLSearchParams("q=a%20b").get("q");
console.log("`a+b` 를 되읽으면 :", JSON.stringify(되읽기1));
```


F12 콘솔 `a+b` 를 되읽으면 : "a b"

```
console.log("`a%20b` 를 되읽으면:", JSON.stringify(되읽기2));
```

F12 콘솔 `a%20b` 를 되읽으면: "a b"

```
document.getElementById("out2").innerHTML =
  `URLSearchParams → <b>${쿼리.toString()}</b><br>` +
  `encodeURIComponent → <b>${encodeURIComponent(검색어)}</b><br><br>` +
  `둘 다 되읽으면 → ${JSON.stringify(되읽기1)} / ${JSON.stringify(되읽기2)}`;
});
```

★★★ 같은 공백인데 **한쪽은 +, 한쪽은 %20** 입니다.

URLSearchParams 공백 → + encodeURIComponent 공백 → %20

★ 왜 다르냐 — 쿼리스트링에는 옛날부터 내려오는 규칙이 따로 있습니다.

폼(form)을 보낼 때 쓰던 방식이고, 거기서는 공백을 `+` 로 씁니다.
URLSearchParams 는 그 규칙을 따릅니다.

★★ 둘 다 되읽으면 "a b" 로 똑같이 나옵니다. 그래서 대개 문제가 없습니다.

쿼리스트링을 읽는 쪽은 `+` 를 공백으로 되돌리게 돼 있습니다.

★★★ 그런데 **경로(path)**에서는 다릅니다.

new URL("https://x.com/a+b").pathname → "/a+b" ← + 가 그냥 + 입니다

```
new URL("https://x.com/a%20b").pathname → "/a%20b"
```

경로에서 + 는 공백이 아니라 그냥 더하기 기호입니다. ★ 그래서 **경로에 값을 끼울 때는 encodeURIComponent 를 쓰세요.**

쿼리에는 URLSearchParams 를 쓰고요. 자리에 따라 도구가 다릅니다.

```
const 이름 = "3호 프레스";
const 주소 = `https://예시.com/설비/${encodeURIComponent(이름)}`;
```

⇒ 직접 해보기 3

콘솔에 encodeURIComponent("가+나") 를 직접 쳐 보세요. + 가 어떻게 바뀌니까?

```
document.getElementById("b3").addEventListener("click", () => {
  const 에이 = new URLSearchParams();
  에이.append("종류", "프레스");
  에이.append("종류", "절단기");

  const 비 = new URLSearchParams();
  비.set("종류", "프레스");
  비.set("종류", "절단기");

  console.log("append 두 번:", decodeURIComponent(에이.toString()));
});
```

F12 콘솔 append 두 번: 종류=프레스&종류=절단기

```
console.log("set 두 번   :", decodeURIComponent(비.toString()));
```

F12 콘솔 set 두 번 : 종류=절단기

```
document.getElementById("out3").innerHTML =
  `append 두 번 → <b>${decodeURIComponent(에이.toString())}</b><br>` +
  `set 두 번 → <b>${decodeURIComponent(비.toString())}</b>`;
});
```

★ 보기 좋으라고 `decodeURIComponent` 로 되돌려서 찍었습니다. 실제로 나가는 것은 `%EC%A2%85%EB%A5%98=...` 입니다.

★★ `set` 은 갈아 끼우고, `append` 는 덧붙입니다. 같은 이름을 여러 개 보내야 하는 API 가 있습니다. (종류=프레스&종류=절단기) 그럴 때만 `append` 를 쓰고, 평소에는 `set` 을 쓰세요. ★ 반복문 안에서 `set` 을 쓰면 **마지막 것만 남습니다**. 흔한 실수입니다.

▫ 직접 해보기 4

반복문으로 `set("종류", 값)` 을 세 번 돌려 보세요. 마지막 것만 남는 것을 확인하고, `append` 로 바꿔 보세요.

★ 값이 없을 때가 제일 위험합니다

섹션 5

```
document.getElementById("b4").addEventListener("click", () => {
  let 설비이름; // 값을 안 넣었습니다 → undefined
  const 쿼리 = new URLSearchParams();
  쿼리.set("a", 설비이름);
  쿼리.set("b", null);
  쿼리.set("c", "");
  쿼리.set("d", 0);

  console.log(쿼리.toString());
});
```

F12 콘솔 a=undefined&b=null&c=&d=0

```
document.getElementById("out4").innerHTML = `<b>${쿼리.toString()}</b>`;
});
```

★★★ undefined 가 글자 “undefined” 로 들어갔습니다.

서버는 이름이 “undefined” 인 설비를 찾게 됩니다. 당연히 0건이 나옵니다. 그런데 오류는 안 납니다. “왜 검색이 안 되지?” 하고 코드를 한참 봅니다.

★★ null 도 마찬가지로 글자 “null” 이 됩니다. 0 은 제대로 0 이 됩니다. "" 는 빈 값으로 들어갑니다.

★ 그래서 넣기 전에 거릅니다.

```
const 조건 = { 종류: "프레스", 이름: undefined, 개수: 0 };

const 쿼리 = new URLSearchParams();
for (const [이름, 값] of Object.entries(조건)) {
  if (값 === undefined || 값 === null || 값 === "") continue;
  쿼리.set(이름, 값);
}
console.log(decodeURIComponent(쿼리.toString()));
// 콘솔: 종류=프레스&개수=0
```

★★ if (!값) continue; 로 쓰면 안 됩니다. 0 이 같이 걸러집니다.

0 은 거짓으로 취급되니까요. (01단원 개념04)
값이 있는지 없는지를 물어야 할 때는 `=== undefined` 로 또렷하게 쓰세요.

▹ 직접 해보기

5

거르는 코드에서 === undefined 대신 if (!값) continue; 로 바꿔 보세요. 개수=0 이 사라지는 것을 눈으로 확인하세요.

이런 실수를 합니다

? 를 두 번 씀

```
const 주소 = "https://예시.com/search?type=a" + "?q=b";
```

→ 두 번째 ? 부터는 그냥 값의 일부가 됩니다. & 를 써야 합니다.

★ URL 을 쓰면 이런 실수가 아예 안 생깁니다.

이런 실수를 합니다

new URL() 에 경로만 넘김

```
new URL("/v1/forecast");
```

→ TypeError: Invalid URL

★ 두 번째 인자로 기준 주소를 주면 됩니다.

```
new URL("/v1/forecast", "https://api.open-meteo.com")
```

▹ 직접 해보기

6

콘솔에 new URL("/v1/forecast") 를 직접 쳐서 오류를 보세요. 그 다음 두 번째 인자를 넣어 고쳐 보세요.

이런 실수를 합니다

반복문에서 set 을 씀 (섹션 4)

이런 실수를 합니다

undefined 를 그냥 넣음 (섹션 5) ← ★ 제일 흔합니다

이런 실수를 합니다

경로에 URLSearchParams 를 씀

→ 경로에는 encodeURIComponent 를 씁니다. (섹션 3)

이런 실수를 합니다

이미 인코딩된 값을 또 인코딩

```
encodeURIComponent("a%20b") → "a%2520b"
```

→ %가 다시 %25로 바뀝니다. 한 번만 하세요.

★ 값이 어디서 왔는지 모르겠으면 **원본을 찾아 거기서 한 번만** 하세요.

Postman으로 먼저 확인하기

★★ 왜 도구를 따로 쓰나

섹션 1

코드를 써서 API 를 불렀는데 안 됩니다. 원인이 둘 중 하나입니다.

(가) 내 코드가 틀렸다 (나) 요청 자체가 틀렸다 (주소·값·키)

★ 이 둘을 못 가르면 엉뚱한 데를 고칩니다. 코드를 열 번 고쳐도 (나) 옳으면 안 됩니다.

Postman 은 (나) 만 따로 시험해 봅니다. 코드 없이 요청만 보냅니다. 되면 (가), 안 되면 (나) 입니다.

★★ 이게 전부입니다. 나머지는 편의 기능입니다. “요청이 맞는지 코드 없이 확인한다” — 이 한 줄만 기억하세요.

▫ 직접 해보기 1

지금 코드가 안 된다면 무엇부터 하겠는지 한 줄로 적어 보세요. 이 파일을 다 읽은 뒤에 다시 보면 답이 달라져 있을 것입니다.

첫 요청 보내기

섹션 2

- 1 Postman 을 켭니다
- 2 왼쪽 위 New → HTTP (또는 + 탭)
- 3 방식은 GET 그대로 두고, 주소 칸에 이렇게 넣습니다

```
https://api.open-meteo.com/v1/forecast
```

4. 주소 아래 Params 탭에서 이름·값을 채웁니다

latitude	37.5665
longitude	126.978
current	temperature_2m
timezone	Asia/Seoul

5. Send

★ 4번을 채우면 주소 칸이 저절로 길어집니다. ?latitude=37.5665&longitude=126.978&... ★★ 개념02 에서 URLSearchParams 가 해 준 일을 눈으로 보는 것입니다.

Params 표에 한 줄 쓰면 `&이름=값` 이 하나 붙습니다.

★ 한글을 값에 넣어 보세요. 주소 칸이 %ED%98%B8... 로 바뀝니다. 개념02 섹션3 에서 본 그것입니다.

▢ 직접 해보기 2

Postman(없으면 브라우저 주소창)으로 개념01 의 요청을 직접 보내 보세요. Params 표에 한 줄 넣을 때마다 주소가 어떻게 길어지는지 보세요.

응답에서 볼 것

섹션 3

Send 를 누르면 아래쪽에 응답이 나옵니다. 네 가지를 봅니다.

```
Status 200 인가. 4xx 면 내가 잘못 보낸 것, 5xx 면 저쪽 문제
Time    얼마나 걸렸나. 몇 초씩 걸리면 타임아웃을 생각해야 합니다
Size    얼마나 큰가. 목록 API 는 수 MB 가 오기도 합니다
Headers * Content-Type 을 꼭 보세요
```

★★ Headers 탭을 안 보는 사람이 많습니다. 여기에 답이 자주 있습니다.

Content-Type: application/json → .json() 을 쓰면 됩니다 Content-Type: text/html → ★ JSON 이 아닙니다. 오류 페이지일 수 있습니다 Content-Type: text/xml → 공공데이터포털이 이럴 때가 있습니다

★ 이걸 안 보고 .json() 을 쓰면 개념05 의 그 오류가 납니다.

▢ 직접 해보기 3

응답의 Headers 에서 Content-Type 을 찾아보세요. 그리고 Access-Control-Allow-Origin 도 있는지 보세요. (개념05)

저장하고 다시 쓰기

섹션 4

요청을 저장해 두면 다음에 그대로 다시 보냅니다.

Save → 컬렉션(Collection) 을 만들어 그 안에 넣습니다

★ 컬렉션은 그냥 폴더입니다. 날씨API, 우리서버 처럼 만들어 두세요.

★★ 환경변수(Environment) 가 진짜 쓸모 있습니다.

오른쪽 위 눈 모양 아이콘 → Add 로 환경을 만들고 값을 넣습니다

```
baseUrl  https://api.open-meteo.com
apiKey   (내 키)
```

그 다음 주소 칸에 이렇게 씁니다.

```
{{baseUrl}}/v1/forecast
```

★★★ 왜 이렇게 하나 —

(1) 개발용 서버와 진짜 서버를 **환경만 바꿔서** 오갑니다

localhost:3000 ↔ 진짜 주소. 요청은 안 고칩니다.

(2) ★ 키를 요청에 직접 안 씁니다.

컬렉션을 남에게 보내거나 GitHub 에 올릴 때 키가 안 팔려 갑니다.
(환경은 따로 있고, 안 보내면 안 갑니다)

★ 이걸 나중에 배울 .env 와 똑같은 생각입니다.

"값은 코드 밖에 둔다."

```
document.getElementById("b2").addEventListener("click", () => {
```

Postman 이 {{변수}} 를 채우는 일을 흉내 내 봅니다

```
const 환경 = {
  baseUrl: "https://api.open-meteo.com",
  lat: "37.5665",
  lon: "126.978",
};

const 틀 = "{{baseUrl}}/v1/forecast?latitude={{lat}}&longitude={{lon}}&current=temperature_2m";

const 채운것 = 틀.replace(/\{\{(\w+)\}\}/g, (통째로, 이름) => 환경[이름] ?? 통째로);

console.log("틀   :", 틀);
```

```
F12 콘솔   틀   : {{baseUrl}}/v1/forecast?latitude={{lat}}&longitude={{lon}}&current=temperature_2m
```

```
console.log("채운것:", 채운것);
```

```
F12 콘솔   채운것: https://api.open-meteo.com/v1/forecast?
latitude=37.5665&longitude=126.978&current=temperature_2m
```

```
document.getElementById("out2").textContent = `틀\n  ${틀}\n\n채운 것\n  ${채운것}`;
});
```

▫ 직접 해보기 4

환경 객체에서 lat 을 지우고 눌러 보세요. {{lat}} 이 그대로 남습니까?

★ ?? 통째로 를 쓴 이유 — 환경에 없는 이름이면 그대로 둡니다. Postman 도 그렇게 합니다. 그래서 `{{apiKey}}` 가 주소에 그대로 나가면 "아, 환경을 안 골랐구나" 하고 바로 압니다. ★★ 조용히 빈칸으로 만들었으면 뭐가 잘못됐는지 몰랐을 것입니다.

★★★ 코드로 내보내기

섹션 5

Postman 에서 요청이 잘 되면, 그걸 코드로 바꿔 줍니다.

오른쪽의 `</>` 아이콘 (Code) → 언어 목록에서 JavaScript - Fetch

그러면 이런 코드가 나옵니다.

```
document.getElementById("b1").addEventListener("click", () => {
```

Postman 이 하는 일을 흉내 낸 것입니다

```
const 요청 = {
  방식: "GET",
  주소: "https://api.open-meteo.com/v1/forecast",
  값: { latitude: "37.5665", longitude: "126.978", current: "temperature_2m" },
  헤더: { Accept: "application/json" },
};

const 주소 = new URL(요청.주소);
for (const [이름, 값] of Object.entries(요청.값)) 주소.searchParams.set(이름, 값);

const 페치코드 = [
  `const 응답 = await fetch("${주소}", {`,
  `  method: "${요청.방식}",`,
  `  headers: ${JSON.stringify(요청.헤더)},`,
  `});`,
  `const 자료 = await 응답.json();`,
].join("\n");

const 컬코드 =
  `curl -X ${요청.방식} "${주소}" +`
  `Object.entries(요청.헤더)`
  `.map(([이름, 값]) => `\\n -H "${이름}: ${값}"`)`
  `.join("");

console.log(페치코드);
console.log(컬코드);

document.getElementById("out1").textContent =
  `[JavaScript - Fetch]\n${페치코드}\n\n[cURL]\n${컬코드}`;
});
```

★★ 이게 Postman 의 제일 쓸모 있는 기능입니다.

Postman 에서 되는 걸 확인 → 코드로 내보냄 → 붙여넣고 다듬음

★ “주소를 잘못 옮겨 적어서 안 됐다” 가 사라집니다.

손으로 옮기지 마세요. 내보내기를 쓰세요.

★★★ 다만 그대로 쓰면 안 됩니다. 나온 코드를 반드시 다듬으세요.

· 키가 코드에 박혀 나옵니다 → 빼야 합니다 (개념04) · 오류 처리가 없습니다 → 응답.ok 를 봐야 합니다 (12단원 개념05) · 주소가 통째로 한 줄입니다 → URL 로 나누는 게 낫습니다 (개념02)

★ 내보내기는 출발점입니다. 완성품이 아닙니다.

▫ 직접 해보기 5

요청.헤더 에 한 줄을 더 넣고 눌러 보세요. fetch 코드와 curl 이 어떻게 바뀔까요?

Postman 에서는 되는데 브라우저에서 안 됩니다

섹션 6

★★★ 이 단원에서 제일 많이 겪을 일입니다.

Postman → 200, 잘 옴 브라우저 → 빨간 CORS 오류

코드가 틀린 게 아닙니다. **Postman 은 브라우저가 아니기 때문**입니다.

CORS 는 **브라우저가 스스로 거는 제한**입니다. Postman·curl·node 에는 그 제한이 없습니다. 그래서 다 됩니다.

★ 그러니 Postman 에서 됐다고 “코드로 옮기면 되겠지” 하면 안 됩니다. 자세한 것은 개념05 에서 봅니다.

▫ 직접 해보기 6

개념05 를 먼저 열어 ① 버튼을 눌러 보고 오세요. 브라우저에서 무슨 일이 생기는지 눈으로 보고 오는 게 낫습니다.

Postman 없이 하는 방법

섹션 7

도구를 못 깔아도 됩니다. 같은 일을 하는 다른 방법이 있습니다.

① 브라우저 주소창

GET 이고 키가 필요 없으면 주소를 그냥 붙여넣으면 됩니다.

★ 크롬은 JSON 을 예쁘게 보여 주지 않습니다.

F12 → Network 에서 보면 낫습니다.

② curl (터미널)

```
curl "https://api.open-meteo.com/v1/forecast?
latitude=37.5&longitude=127&current=temperature_2m"
```

★ 윈도우 PowerShell에서는 `curl`이 다른 명령입니다.
`curl.exe`라고 쓰거나 Git Bash를 쓰세요.

③ node (터미널)

```
node -e "fetch('주소').then(r => r.json()).then(console.log)"
```

★ Node 18부터 fetch가 그냥 있습니다. 설치할 게 없습니다.

④ VS Code 확장 (REST Client 등)

```
`.http` 파일에 요청을 적어 두고 누르면 보냅니다.
```

★ 요청을 **파일로 저장**하니 git에 같이 올릴 수 있습니다.

★ ②③은 브라우저가 아니어서 CORS가 안 걸립니다. Postman과 같습니다.

▫ 직접 해보기

7

터미널에서 `node -e "fetch('주소').then(r=>r.json()).then(console.log)"`로 같은 요청을 보내 보세요. 브라우저와 결과가 같습니까?

자주 하는 실수

섹션 8

이런 실수를 합니다

Postman에서 되니 다 된 줄 앎

→ 브라우저는 다릅니다. (섹션 6)

이런 실수를 합니다

Params 표에 값을 넣고 주소 칸도 손으로 고침

→ 둘이 어긋나서 이상한 주소가 나옵니다. 한쪽만 고치세요.

이런 실수를 합니다

환경을 안 고르고 Send

→ 주소에 `{{baseUrl}}`이 그대로 남습니다. ★ 오른쪽 위에서 환경이 골라져 있는지 보세요.

이런 실수를 합니다

내보낸 코드에 키가 박힌 채로 GitHub에 올림

→ ★★ 제일 흔하고 제일 아픕니다. 개념04를 꼭 읽으세요.

이런 실수를 합니다

Postman 계정에 컬렉션을 동기화해 두고 키까지 같이 올림

→ 환경 값은 **Current value** 로 두면 내 컴퓨터에만 남습니다. **Initial value** 에 적으면 같이 올라갑니다.

★ 칸이 두 개인 이유입니다.

키가 필요한 API

키가 왜 필요한가

섹션 1

개념01에서 쓴 Open-Meteo는 키가 없었습니다. 누구나 그냥 부릅니다. 그런데 대부분의 API는 키를 요구합니다. 이유가 있습니다.

· 누가 얼마나 썼는지 세야 합니다 (하루 1000번 같은 제한) · 돈을 받는 API면 청구를 해야 합니다 · 함부로 쓰는 사람을 막아야 합니다

★ 키는 **이름표**입니다. 비밀번호와 비슷하지만 조금 다릅니다. 비밀번호는 “나임을 증명” 하고, 키는 “이 요청이 누구 몫인지”를 정합니다. ★★ 그래서 남이 내 키를 쓰면 **내 몫이 줄고 내가 돈을 냅니다.**

직접 해보기 1

평소에 쓰는 서비스 중 API를 여는 곳을 하나 찾아 키가 필요한지, 무료 한도가 얼마인지 보세요.

키를 넣는 두 자리

섹션 2

```
const 가짜키 = "SAMPLE-KEY-0000-0000";

document.getElementById("b1").addEventListener("click", () => {
```

방법 1 · 쿼리스트링에 넣기

```
const 쿼리로 = new URL("https://예시api.com/v1/data");
쿼리로.searchParams.set("serviceKey", 가짜키);
쿼리로.searchParams.set("지역", "서울");
```

방법 2 · 헤더에 넣기

```
const 헤더로 = {
  주소: "https://예시api.com/v1/data?지역=서울",
  헤더: { Authorization: `Bearer ${가짜키}` },
};

console.log("쿼리로:", 쿼리로.toString());
```

F12 콘솔 쿼리로: https://xn--api-oq6n10k.com/v1/data?serviceKey=SAMPLE-KEY-0000-0000&%EC%A7%80%EC%97%AD=%EC%84%9C%EC%9A%B8

```
console.log("헤더로:", JSON.stringify(헤더로));
```

```
F12 콘솔    헤더로: {"주소":"https://예시api.com/v1/data?지역=서울","헤더":  
              {"Authorization":"Bearer SAMPLE-KEY-0000-0000"}}
```

```
document.getElementById("out1").innerHTML =  
    `<b>쿼리로</b><br>${쿼리로}<br><br><b>헤더로</b><br>${헤더로.주소}  
    <br>Authorization: Bearer ${가짜키}`;  
});
```

▫ 직접 해보기

2

가짜키를 헤더 대신 쿼리에 넣어 보세요. 주소가 어떻게 됩니까? 그 주소를 남에게 보여 줄 수 있겠습니까?

★ 어느 쪽을 쓰시지는 **API가 정합니다**. 내가 고르는 게 아닙니다. 문서를 보세요.

```
공공데이터포털    쿼리    `serviceKey=...`  
카카오            헤더    `Authorization: KakaoAK ...`
```

요즘 만든 API 헤더 `Authorization: Bearer ...`

★★ 헤더 쪽이 조금 낫습니다. 이유가 두 가지입니다.

(1) 주소창·브라우저 기록·서버 접속 기록에 키가 안 남습니다

쿼리에 넣으면 그 주소가 통째로 여기저기 기록됩니다.

(2) 주소를 남에게 보여 줄 때 키를 안 지워도 됩니다

★ 그래도 브라우저에서는 둘 다 위험합니다. 섹션 4에서 봅니다.

키를 어떻게 받아 (공공데이터포털)

섹션 3

이 과정의 주제(제조 현장)와 맞는 자료가 많은 곳입니다.

1 <https://www.data.go.kr> 가입

2 쓰고 싶은 자료를 찾아 **활용신청**

3 승인되면 **마이페이지** → **개발계정** 에서 키를 봅니다

★ 자동 승인인 것과 며칠 걸리는 것이 있습니다. 수업 전에 미리 신청하세요.

★★ 키가 두 가지 모양으로 보입니다.

```
일반 인증키(Encoding)    abc%2Bdef%2Fghi%3D  
일반 인증키(Decoding)    abc+def/ghi=
```

같은 키인데 하나는 **이미 인코딩된 것**입니다. ★★★ 여기서 사고가 납니다. 섹션 5에서 봅니다.

⇒ 직접 해보기 3

data.go.kr 에서 제조·안전과 관련된 자료를 하나 찾아보세요. 활용신청이 자동 승인인지 아닌지도 같이 보세요.

★★★ 브라우저에 넣으면 다 보입니다

섹션 4

```
document.getElementById("b2").addEventListener("click", () => {
```

우리가 쓴 코드가 이렇다고 칩시다

```
const 내코드 = `
  const 키 = "SAMPLE-KEY-0000-0000";
  const 주소 = new URL("https://예시api.com/v1/data");
  주소.searchParams.set("serviceKey", 키);
  const 응답 = await fetch(주소);
`;
```

남이 F12 → Sources 에서 이 파일을 열고 검색합니다

```
const 찾은것 = 내코드.match(/SAMPLE-[A-Z0-9-]+/);

console.log("검색 결과:", 찾은것 ? 찾은것[0] : "없음");
```

F12 콘솔 검색 결과: SAMPLE-KEY-0000-0000

```
document.getElementById("out2").innerHTML =
  `누구나 F12 → Sources 에서 이 파일을 열 수 있습니다.<br>` +
  `Ctrl+F 로 검색하면 → <b>${찾은것 ? 찾은것[0] : "없음"}</b>`;
});
```

★★★ 이걸 막을 방법이 없습니다.

브라우저에서 도는 코드는 **전부 사용자에게 전달된 파일**입니다. 숨길 수가 없습니다. 압축하고 이름을 뭉개도 글자는 그대로 남습니다.

그리고 코드를 안 보더라도 **F12 → Network** 에 요청이 그대로 보입니다. 쿼리에 넣었으면 주소에, 헤더에 넣었으면 헤더 목록에 있습니다. ★ 헤더에 넣는다고 안 보이는 게 아닙니다. 그냥 덜 남을 뿐입니다.

★★ “그럼 브라우저에서 키가 필요한 API 는 못 쓰나요?” — 못 씁니다. 정확히는 **써도 되지만 키가 공개된다는 걸 알고** 써야 합니다.

써도 되는 키가 있습니다.

- 도메인 제한을 걸 수 있는 키 (내 사이트에서만 되게)
- 공개용으로 만들어진 키 (지도 표시용 등)

절대 안 되는 키

- 돈이 나가는 것 (LLM API, 문자 발송)
- 남의 정보를 읽는 것
- 무언가를 지우거나 바꿀 수 있는 것

▹ 직접 해보기 4

F12 → Sources 에서 이 파일을 열고 Ctrl+F 로 SAMPLE-KEY 를 검색해 보세요. 정말로 보이는지 직접 확인하세요.

★★ 키를 두 번 인코딩하면

섹션 5

```
document.getElementById("b3").addEventListener("click", () => {
  const 디코딩키 = "abc+def/ghi=";          // 포털의 'Decoding' 칸 const 인코딩키 =
  "abc%2Bdef%2Fghi%3D";          // 포털의 'Encoding' 칸 const 틀림 = new URLSearchParams();
  틀림.set("serviceKey", 인코딩키); // 이미 인코딩된 것을 또 넣었습니다 const 맞춤 =
  new URLSearchParams();
  맞춤.set("serviceKey", 디코딩키);

  console.log("틀림:", 틀림.toString());
```

F12 콘솔 틀림: serviceKey=abc%252Bdef%252Fghi%253D

```
console.log("맞음:", 맞춤.toString());
```

F12 콘솔 맞춤: serviceKey=abc%2Bdef%2Fghi%3D

서버가 받아서 되 읽으면 이렇게 됩니다

```
console.log("서버가 읽는 값(틀림):",
  new URLSearchParams(틀림.toString()).get("serviceKey"));
```

F12 콘솔 서버가 읽는 값(틀림): abc%2Bdef%2Fghi%3D

```
console.log("서버가 읽는 값(맞음):",
  new URLSearchParams(맞음.toString()).get("serviceKey"));
```

F12 콘솔 서버가 읽는 값(맞음): abc+def/ghi=


```
document.getElementById("out3").innerHTML =
    `<b>틀림</b> ${틀림}<br>→ 서버가 읽는 값: ${new
    URLSearchParams(틀림.toString()).get("serviceKey")}<br><br>` +
    `<b>맞음</b> ${맞음}<br>→ 서버가 읽는 값: ${new
    URLSearchParams(맞음.toString()).get("serviceKey")}`;
});
```

★★★ %2B 의 % 가 다시 %25 로 바뀌었습니다. 그래서 서버는 abc%2Bdef%2Fghi%3D 라는 글자 그대로를 키로 읽습니다. 그런 키는 없으니 인증 실패입니다.

→ SERVICE KEY IS NOT REGISTERED ERROR

등록되지 않은 키` 같은 답이 옵니다.

★★ 헛갈리기 쉬운 게, 반대로 해도 틀립니다.

주소를 손으로 이어 붙이면서 Decoding 키를 그대로 넣으면 + 가 공백으로 읽혀서 역시 틀린 키가 됩니다. (개념02 섹션3)

★★★ 규칙 하나로 외우세요.

URLSearchParams 를 쓰면 → Decoding 키를 넣는다 주소를 손으로 이어 붙이면 → Encoding 키를 넣는다

그리고 손으로 이어 붙이지 마세요. 앞의 것만 쓰면 됩니다.

★ 이게 공공데이터포털을 처음 쓰는 사람이 100% 만나는 벽입니다. "키가 맞는데 안 돼요" 는 거의 다 이것입니다.

◀ 직접 해보기

5

틀림/맞음 을 서로 바꿔 넣어 보세요. %25 가 어디에 생깁니까?

그럼 어떻게 하나

섹션 6

★★★ 답은 서버를 하나 두는 것입니다.

지금 · 브라우저 — (키를 들고) — → 남의 API

키가 사용자에게 전달됩니다

나중 · 브라우저 ————— → 내 서버 — (키를 들고) — → 남의 API

키는 내 서버에만 있습니다

브라우저는 내 서버만 부릅니다. 키를 아예 모릅니다. 내 서버가 대신 부르고 결과만 넘겨 줍니다.

★ 이 과정 PART 3 에서 그 서버를 만듭니다. 지금은 못 합니다. 지금 알아야 할 것은 “왜 서버가 필요한가” 입니다. 그 이유가 이 섹션입니다. 서버는 데이터베이스 때문만이 아닙니다.

★★ 그때까지 이 단원에서는 —

· 키가 필요 없는 API 로 연습합니다 (Open-Meteo) · 키가 필요한 것은 **Postman 으로만** 불러 봅니다 (개념03)

Postman 은 내 컴퓨터에서 도니까 키가 안 새어 나갑니다.

▫ 직접 해보기 6

브라우저 → 내 서버 → 남의 API 그림을 종이에 그려 보세요. 키가 어디에만 있는지 화살표로 표시하세요.

키가 새는 세 가지 길

섹션 7

(1) git 에 올림

★★ 지운 뒤 커밋해도 **기록에 남습니다.** 되돌릴 수 없습니다.
키를 폐기하는 것 말고 방법이 없습니다.

(2) 스크린샷·화면 공유

질문 올리면서 코드 사진을 찍어 올립니다. 주소창에 키가 보입니다.
★ 캡처 전에 F12 를 닫고 주소를 지우세요.

(3) 배포한 파일

섹션 4 그대로입니다.

★★★ 키가 섰다 싶으면 순서가 정해져 있습니다.

1 키를 먼저 폐기합니다 (재발급이 아니라 폐기)

2 새 키를 받습니다

3 코드를 고칩니다

4 사용량을 확인합니다

★ 1번이 먼저입니다. 코드부터 고치면 그 사이에도 옛 키는 살아 있습니다.

▫ 직접 해보기 7

지금까지 만든 코드에 키나 비밀번호를 적어 둔 곳이 없는지 검색해 보세요. 있으면 지금 지우세요.
나중에는 잊어버립니다.

자주 하는 실수

섹션 8

이런 실수를 합니다

Encoding 키를 URLSearchParams 에 넣음 (섹션 5) ← ★ 제일 흔합니다

이런 실수를 합니다

헤더에 Bearer 를 빼먹음

{ Authorization: 키 } → 401 { Authorization: Bearer \${키} } → 200 ★ API 마다 앞에 붙이는 말이
다릅니다. Bearer, KakaoAK, 없는 것도 있습니다.

이런 실수를 합니다

키를 코드에 적어 두고 “나중에 빼야지” 하고 잊음

→ ★ 그 사이에 커밋합니다. 처음부터 적지 마세요.

이런 실수를 합니다

승인 직후에 바로 부름

→ 공공데이터포털은 키가 도는 데 몇십 분 걸리기도 합니다. “키가 틀렸다” 는 답이 와도 조금 기다려
보세요.

이런 실수를 합니다

키가 없어서 안 되는 건데 코드를 고침

→ ★ Postman 으로 먼저 확인하세요. (개념03 섹션1)

안 되는 경우들

★★★ CORS — 브라우저만 막습니다

섹션 1

개념03 에서 예고한 그것입니다.

Postman 에서는 200 이 옵니다. 같은 주소를 브라우저에서 부르면 빨간 오류가 납니다.

★ 코드가 틀린 게 아닙니다. **브라우저가 스스로 막는 것**입니다.

브라우저는 “지금 보고 있는 사이트가 아닌 곳” 에 요청을 보낼 때 그 서버에게 먼저 물어봅니다. “애가 나한테 요청해도 되나요?” 서버가 된다고 답해야 브라우저가 응답을 코드에 넘겨 줍니다.

그 대답이 `Access-Control-Allow-Origin` 헤더입니다.

★★ 이걸 왜 하나 — 내가 어떤 사이트를 보고 있는데, 그 사이트의 코드가 몰래 내 은행 사이트에 요청을 보내면 안 되기 때문입니다. ★ 나를 지키려고 있는 것입니다. 개발을 방해하려는 게 아닙니다.

```
async function 불러보기(주소) {
  try {
    const 응답 = await fetch(주소);
    return { 됐나: true, 상태: 응답.status };
  } catch (오류) {
    return { 됐나: false, 이름: 오류.name, 메시지: 오류.message };
  }
}

document.getElementById("b1").addEventListener("click", async () => {
  const 칸 = document.getElementById("out1");
  칸.textContent = "부르는 중...";
});
```

열려 있는 곳 (Access-Control-Allow-Origin: *)

```
const 열린곳 = await 불러보기(
  "https://api.open-meteo.com/v1/forecast?
  latitude=37&longitude=127&current=temperature_2m",
);
```

안 열어 준 곳 (그 헤더를 아예 안 보냅니다)

```
const 막힌곳 = await 불러보기("https://example.com");

console.log("열린 곳:", JSON.stringify(열린곳));
console.log("막힌 곳:", JSON.stringify(막힌곳));

칸.innerHTML =
  `<b>open-meteo</b> ${JSON.stringify(열린곳)}<br><br>` +
  `<b>example.com</b> ${JSON.stringify(막힌곳)}`;
});
```

★ 브라우저에서 누르면 이렇게 나옵니다.

```
열린 곳: {"됐나":true,"상태":200}
막힌 곳: {"됐나":false,"이름":"TypeError","메시지":"Failed to fetch"}
```

그리고 콘솔에 빨간 줄이 하나 더 뜹니다.

```
Access to fetch at 'https://example.com/' from origin 'null'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin'
header is present on the requested resource.
```

★★ 같은 코드를 node 로 돌려 보면 **둘 다 200** 이 나옵니다. 실제로 재 봤습니다.

```
열린 곳: {"됐나":true,"상태":200}
막힌 곳: {"됐나":true,"상태":200}
```

example.com 은 아무것도 안 바뀌었는데 브라우저에서만 막힙니다. ★ “CORS 는 브라우저가 거는 제한” 이라는 말이 이 뜻입니다.

★★★ 코드가 받는 것은 `Failed to fetch` 뿐입니다. 왜 막혔는지는 코드에서 알 수 없습니다. 콘솔에만 나옵니다. 일부러 그렇게 만들어 봤습니다. 알려 주면 그것도 정보가 되니까요.

★ 그래서 `catch` 만 보고 있으면 원인을 못 찾습니다. **콘솔을 보세요.**

★★ `from origin 'null'` 에 주목하세요. 이 파일을 **더블클릭해서 열었기 때문**입니다. 그때 출처가 `null` 입니다. `open-meteo` 는 * (아무나) 라서 `null` 도 통과합니다. ★ 어떤 서버는 `null` 을 막습니다. 그럴 땐 간단한 로컬 서버로 여세요.

(VS Code 의 Live Server 확장 같은 것)

▫ 직접 해보기

1

example.com 을 다른 주소로 바꿔 눌러 보세요. 어떤 곳이 되고 어떤 곳이 막힙니까?

★★ 내가 못 풁니다. 서버가 열어 줘야 합니다.

브라우저 설정으로 끄는 방법이 인터넷에 돌아다닙니다. 쓰지 마세요. 내 브라우저만 뚫리는 것이고, 다른 사이트에도 구멍이 뚫립니다.

★★★ 진짜 해법은 서버를 하나 두는 것입니다. 개념04 섹션6 과 같습니다.

브라우저 → 내 서버 → 남의 API

서버끼리는 CORS 가 없습니다. 브라우저만 있는 규칙이니깐요. 그리고 내 서버는 내가 CORS 를 열어 줄 수 있습니다.

★ PART 3 에서 그 서버를 만듭니다. 지금은 못 합니다.

공공데이터포털이 브라우저에서 안 되는 이유가 이것입니다.

★ 지금 당장 확인만 하고 싶다면 Postman·curl·node 를 쓰세요. (개념03 섹션7)

▫ 직접 해보기 2

콘솔의 빨간 줄을 끝까지 읽어 보세요. 어느 출처에서 어느 주소로 갔다가 막혔다고 적혀 있습니까?

JSON 이 아닌 것을 받았을 때

```
document.getElementById("b2").addEventListener("click", async () => {
```

서버가 오류 페이지(HTML)를 준 상황을 그대로 만들어 봅시다

```
const 가짜응답 = new Response("<!doctype html>\n<html><body>404 Not Found</body>\n</html>", {
  status: 404,
  headers: { "Content-Type": "text/html" },
});

try {
  await 가짜응답.json();
} catch (오류) {
  console.log(오류.name + ": " + 오류.message);
}
```

F12 콘솔 SyntaxError: Unexpected token '<', "<!doctype "... is not valid JSON

```
document.getElementById("out2").textContent = 오류.name + ": " + 오류.message;
}
```

빈 응답도 혼합니다 (204 No Content 등)

```
try {
  await new Response("").json();
} catch (오류) {
  console.log(오류.name + ": " + 오류.message);
}
```

F12 콘솔 SyntaxError: Unexpected end of JSON input

```
}
});
```

★★★ Unexpected token ‘<’ 는 **HTML** 을 받았다는 뜻입니다. < 는 <!doctype html> 의 첫 글자입니다.

왜 HTML 이 왔나 —

- 주소를 틀려서 404 오류 ****페이지****가 왔다
- 로그인 페이지로 넘겨졌다
- 서버가 죽어서 nginx 의 502 페이지가 왔다

★ 전부 “JSON 을 줄 줄 알았는데 아니었다” 입니다.

★★ Unexpected end of JSON input 은 **빈 응답**입니다. 204 처럼 본문이 없는 응답에 .json() 을 부르면 이렇게 됩니다.

★★★ 막는 방법 — .json() 전에 두 가지를 봅니다.

```
const 응답 = await fetch(주소);

if (!응답.ok) {
  throw new Error(`서버가 ${응답.status} 를 보냈습니다`);
}

const 종류 = 응답.headers.get("content-type") ?? "";
if (!종류.includes("application/json")) {
  throw new Error(`JSON 이 아닙니다: ${종류}`);
}

const 자료 = await 응답.json();
```

★ 응답.ok 만으로는 부족합니다. 200 인데 HTML 인 경우가 있습니다.

(개념03 섹션3 에서 Headers 탭을 보라고 한 이유입니다)

⇒ 직접 해보기 3

가짜응답의 본문을 빈 글자("")로 바꿔 보세요. 오류 메시지가 어떻게 달라집니까?

```
document.getElementById("b3").addEventListener("click", async () => {
```

429 응답을 그대로 만들어 봅니다

```
const 가짜응답 = new Response(JSON.stringify({ error: "Too Many Requests" }), {
  status: 429,
  headers: { "Content-Type": "application/json", "Retry-After": "60" },
});

console.log("상태:", 가짜응답.status);
```

F12 콘솔 상태: 429

```
console.log("ok 인가:", 가짜응답.ok);
```

F12 콘솔 ok 인가: false

```
console.log("얼마나 기다리라고?", 가짜응답.headers.get("Retry-After") + "초");
```

F12 콘솔 얼마나 기다리라고? 60초

```
document.getElementById("out3").textContent =
  `${가짜응답.status} - ${가짜응답.headers.get("Retry-After")}초 뒤에 다시 오라고
  합니다`;
});
```

★ 429 Too Many Requests 는 “너무 자주 불렀다” 는 뜻입니다. Retry-After 헤더에 몇 초 뒤에 오라고 적혀 있을 때가 있습니다.

★★ 언제 나오나 · 반복문 안에서 fetch 를 돌렸을 때 ← ★ 제일 흔합니다 · 버튼을 여러 번 눌렀을 때 · 무료 등급의 하루 한도를 다 썼을 때

★★★ 반복문에서 fetch 를 돌리면 동시에 수십 개가 나갑니다.

```
for (const 도시 of 도시들) {
  const 답 = await fetch(주소(도시));    // 하나씩 (그나마 낫습니다)
}

await Promise.all(도시들.map((도시) => fetch(주소(도시))));    // ★ 한꺼번에
```

Promise.all 은 빠르지만 한도에 잘 걸립니다. ★ 목록이 길면 몇 개씩 끊어서 보내세요.

★ 429 를 받으면 바로 다시 보내지 마세요. 더 심해집니다. 기다렸다가 한 번만 다시 보내는 게 예의입니다.

➤ 직접 해보기 4

Retry-After 헤더를 지우고 눌러 보세요. 무엇이 나오니까? 그 값이 없을 때 코드는 어떻게 해야 할까요?

시간이 오래 걸릴 때

섹션 5

```
document.getElementById("b4").addEventListener("click", async () => {
  const 칸 = document.getElementById("out4");
  칸.textContent = "부르는 중...";

  try {
```

1밀리초 안에 못 끝내면 끊습니다. 당연히 못 끝냅니다.

```
await fetch("https://api.open-meteo.com/v1/forecast?latitude=37&longitude=127", {
  signal: AbortSignal.timeout(1),
});
칸.textContent = "★ 1밀리초 만에 성공했습니다. 그럴 리가 없는데요?";
} catch (오류) {
  console.log("이름:", 오류.name);
}
```

F12 콘솔 이름: TimeoutError

```
console.log("메시지:", 오류.message);
```

```
F12 콘솔    메시지: The operation was aborted due to timeout
```

```
칸.textContent = `${오류.name} - ${오류.message}`;
}
});
```

★ `AbortSignal.timeout(밀리초)` 하나면 됩니다. 따로 설치할 게 없습니다.

```
const 응답 = await fetch(주소, { signal: AbortSignal.timeout(5000) });
```

★★ 왜 필요하냐 — fetch 는 **기본으로 안 끊습니다**. 서버가 답을 안 주면 몇 분이고 기다립니다. 그동안 화면은 “불러오는 중...” 인 채로 멈춰 있습니다.

★★★ 꿈긴 것과 실패한 것을 갈라야 합니다.

```
catch (오류) {
  if (오류.name === "TimeoutError") {
    알림("서버가 느립니다. 다시 시도해 주세요.");
  } else if (오류.name === "TypeError") {
    알림("연결할 수 없습니다. 인터넷을 확인해 주세요.");
  } else {
    알림("알 수 없는 문제입니다.");
  }
}
```

★ 오류.name 을 보세요. 오류.message 는 브라우저마다 다릅니다.

▣ 직접 해보기 5

제한을 1 에서 5000 으로 늘려 돌려 보세요. 이번엔 성공하니까? 그 다음 인터넷을 끄고 다시 돌려 오류 이름을 비교하세요.

★★ 안 될 때 보는 순서

섹션 6

이 단원에서 나온 것을 순서로 묶으면 이렇습니다.

1. F12 → Console 에 빨간 줄이 있나

`CORS policy` 라고 쓰여 있으면 → 섹션 1·2
 `Unexpected token '<`' 이면 → 섹션 3

2. F12 → Network 에서 그 요청을 찾습니다

- 요청이 아예 없다 → 코드가 안 불린 것입니다
- 주소가 이상하다 → 개념02
- 상태가 401·403 → 키 문제 (개념04)
- 상태가 429 → 섹션 4
- 상태가 200 인데 값이 없다 → 응답 모양 (개념01 섹션4)

3. 같은 요청을 Postman 으로 보내 봅니다 (개념03)

- Postman 은 되고 브라우저만 안 됨 → CORS 입니다
- Postman 도 안 됨 → 요청이 틀린 것입니다

▣ 직접 해보기 6

섹션 6의 순서를 종이에 옮겨 적어 모니터 옆에 붙이세요. PART 3·4 에서도 그대로 씁니다.

★★★ 3번이 걸림길입니다. 여기서 **어디를 고칠지**가 정해집니다. 코드를 먼저 고치기 시작하면 시간을 많이 씁니다.

★ 이 순서를 종이에 적어 옆에 두세요. 배포하고 나서도 그대로 씁니다.

연습문제

문제 1 (개념02)

조건 객체를 쿼리스트링으로 만드세요. ★ 값이 `undefined·null·빈 글자`인 것은 빼야 합니다. 0 은 넣어야 합니다.

기대 출력 종류=프레스&개수=0&정렬=이름

★ 보기 좋으라고 `decodeURIComponent` 로 되돌려 씁습니다.

```
const 조건 = { 종류: "프레스", 이름: undefined, 개수: 0, 메모: null, 정렬: "이름",
비교: "" };
```

```
function 쿼리만들기(조건) {
```

여기에 코드를 쓰세요

```
}
```

문제 2 (개념02)

설비 이름을 **경로**에 끼워 주소를 만드세요.

기대 출력 <https://예시.com/설비/3%ED%98%B8%20%ED%94%84%EB%A0%88%EC%8A%A4>

★ 힌트: 경로에는 `URLSearchParams` 를 못 씁니다. 개념02 섹션3 을 보세요.

```
function 설비주소(이름) {
```

여기에 코드를 쓰세요

}

문제 3 (개념04)

공공데이터포털이 준 두 가지 키 중 어느 것을 URLSearchParams 에 넣어야 할까요? 맞는 것을 골라 쿼리를 만드세요.

기대 출력 serviceKey=abc%2Bdef%2Fghi%3D

```
const 인코딩키 = "abc%2Bdef%2Fghi%3D";
const 디코딩키 = "abc+def/ghi=";

function 키쿼리() {
```

여기에 코드를 쓰세요

}

문제 4 (개념05)

응답을 안전하게 JSON 으로 바꾸는 함수를 만드세요. · 상태가 ok 가 아니면 → “서버가 N 를 보냈습니다” · Content-Type 이 JSON 이 아니면 → “JSON 이 아닙니다: 종류” · 그 외 → 파싱해서 돌려줍니다
오류는 throw 하지 말고 { 됐나, 값 } 모양으로 돌려주세요.

기대 출력 {"됐나":true,"값":{"이름":"3호 프레스"}}
 {"됐나":false,"값":"서버가 500 를 보냈습니다"}
 {"됐나":false,"값":"JSON 이 아닙니다: text/html"}

```
async function 안전하게읽기(응답) {
```

여기에 코드를 쓰세요

```
}
```

문제 5 (개념05)

오류를 보고 사용자에게 보여 줄 말을 정하세요. `TimeoutError` → “서버가 느립니다” `TypeError` → “연결할 수 없습니다” 그 외 → “알 수 없는 문제입니다”

기대 출력 서버가 느립니다
 연결할 수 없습니다
 알 수 없는 문제입니다

```
function 무슨말(오류) {
```

여기에 코드를 쓰세요

```
}
```

문제 6 (개념01·02) ★ 인터넷 필요

도시 이름을 받아 지금 기온을 돌려주는 함수를 만드세요. “이름 기온 단위” 모양의 글자로 돌려줍니다.

기대 출력 (숫자는 그때그때 다릅니다): 서울 29.7°C

★ `current=temperature_2m` 과 `timezone=Asia/Seoul` 을 넣으세요. ★ 단위는 응답의 `current_units` 에 있습니다. 짐작하지 마세요.

```
const 도시들 = { 서울: [37.5665, 126.978], 부산: [35.1796, 129.0756], 제주: [33.4996, 126.5312] };
```

```
async function 기온(도시이름) {
```

여기에 코드를 쓰세요

}

문제 7 (개념05) ★ 인터넷 필요

문제 6에 시간 제한을 붙이세요. 3초 안에 못 받으면 문제 5의 말을 돌려줍니다.

기대 출력 (1밀리초로 시험하면): 서버가 느립니다

```
async function 기온_시간제한(도시이름, 제한밀리초) {
```

여기에 코드를 쓰세요

}

아래는 고치지 마세요. 위에 쓴 것을 확인만 합니다.

```
document.getElementById("풀기").addEventListener("click", async () => {
  const 줄들 = [];
  const 적기 = (무엇) => {
    줄들.push(String(무엇));
    console.log(무엇);
  };

  적기(decodeURIComponent(String(쿼리만들기(조건))));
  적기(설비주소("3호 프레스"));
  적기(키쿼리());

  const 좋은응답 = new Response(JSON.stringify({ 이름: "3호 프레스" }), {
    headers: { "Content-Type": "application/json" },
  });
  const 오백 = new Response("{} ", { status: 500, headers: { "Content-Type":
```

```

"application/json" } }));
const 에이치티엠엘 = new Response("<html></html>", { headers: { "Content-Type":
"text/html" } }));

적기(JSON.stringify(await 안전하게읽기(좋은응답)));
적기(JSON.stringify(await 안전하게읽기(오백)));
적기(JSON.stringify(await 안전하게읽기(에이치티엠엘)));

적기(무슨말({ name: "TimeoutError" }));
적기(무슨말({ name: "TypeError" }));
적기(무슨말({ name: "이상한오류" }));

try {
  적기(await 기온("서울"));
  적기(await 기온_시간제한("서울", 1));
} catch (오류) {
  적기("6.7번에서 오류: " + 오류.message);
}

document.getElementById("결과").innerHTML = 줄들.join("<br>");
});

```

손으로 · 푸는 문제 — 답을 주석으로 적으세요

문제 8 (개념 03·05)

팀원이 말합니다.

"Postman에서는 200 이 잘 오는데 브라우저에서는 빨간 오류가 나요. 코드를 몇 번이나 고쳤는데 안 돼요."

(1) 코드 문제입니까? (2) 무슨 일이 벌어지고 있습니까? (3) 지금 이 과정에서 할 수 있는 일은 무엇입니까?

답:

문제 9 (개념 04)

실습 중에 학생이 이렇게 물어봅니다.

"키를 코드에 안 쓰고 파일로 빼면 안전한 거죠? key.js 에 넣고 import 하면 되잖아요."

(1) 안전해집니까? (2) 왜 그렇습니까? (3) 진짜 해결책은 무엇입니까?

답:

문제 10 (개념04)

공공데이터포털 키를 넣었는데 계속 “등록되지 않은 키” 가 옵니다. 포털에서 복사한 키는 분명히 맞습니다.

(1) 제일 먼저 의심할 것은 무엇입니까? (2) 어떻게 확인합니까? (3) 그것이 아니라면 다음으로 무엇을 봅니까?

답:

문제 11 (개념01)

새 API 를 처음 쓰게 됐습니다. 문서가 아주 깁니다.

(1) 코드를 쓰기 전에 확인할 네 가지는 무엇입니까? (2) 문서에 적힌 응답 모양을 그대로 믿어도 됩니까? (3) ★ 이 단원에서 “하나만 보고 규칙을 정했다가 틀린” 예가 있었습니다.

무엇이었고, 무엇을 배웠습니까?

답:

14단원 마무리 OpenAPI와 Postman

자주 넘어지는 자리

- 1 URLSearchParams 에 undefined 를 넣으면 글자 “**undefined**” 가 들어갑니다. 오류가 안 납니다
- 2 공백이 URLSearchParams 에서는 +, encodeURIComponent 에서는 %20 — 경로에서는 + 가 공백이 아닙니다
- 3 ★ 브라우저에 넣은 키는 **F12** 로 다 보입니다. 파일을 나눠도, 압축해도 같습니다
- 4 ★ CORS 는 브라우저만 겁니다. Postman 에서 되는 걸로 “코드 문제” 라고 판단하면 안 됩니다

React 가기 전에

OPEN 15_React_가기_전에

```
console.log(TAX_RATE);
console.log(addTax(10000));
console.log(won(25000));
import greet from "./도구모음.js";
```

01 파일 나눠 쓰기 (모듈)

02 localStorage

03 날짜와 무작위 (Date, Math.random)

04 class 맛보기

- 연습문제

- 연습문제

파일 나눠 쓰기 (모듈)

```
RUN node 개념01_모듈.js
```

지금까지는 파일 하나에 전부 담았습니다. 13단원 종합03이 300줄이 넘었죠. 실무 프로젝트는 수만 줄입니다. 한 파일에 다 넣으면 아무도 못 읽습니다.

그래서 파일을 나누고, 필요한 것만 서로 꺼내 씁니다. 내보내기 : export 가져오기 : import

이것이 React 를 배울 때 가장 먼저 만나는 문법입니다. React 파일은 예외 없이 첫 줄이 import 로 시작합니다.

시작하기 전에: package.json

이 폴더에 package.json 이라는 파일이 하나 있습니다. 열어 보면 두 줄뿐입니다.

```
{ "type": "module" }
```

“이 폴더의 .js 파일들은 모듈 문법(import/export)을 씁니다” 라는 표시입니다. 이게 없으면 node 가 경고를 냅니다. 실무 프로젝트에도 이 파일이 항상 있습니다. 지금은 이 정도만 알면 됩니다.

이름을 지정해서 가져오기

섹션 1

중괄호 안에 '가져올 이름'을 적습니다. 이름은 내보낸 쪽과 똑같아야 합니다.

```
import { TAX_RATE, addTax } from "./도구모음.js";
```

```
console.log(TAX_RATE);
```

```
출력      0.1
```

```
console.log(addTax(10000));
```

```
출력      11000
```

읽는 법

```
import { 가져올것 } from "./어느파일.js";
```

중괄호 안에 경로. ./ 로 시작하고 .js 까지 씁니다

import 는 파일 맨 위에 모아 씁니다. 자바스크립트가 먼저 읽기 때문입니다.

▹ 직접 해보기 1

위 import 줄에 formatWon 을 추가로 가져와서 `console.log(formatWon(11000))` 을 찍어 보세요.

필요한 것만 골라 온다

섹션 2

도구모음.js 에는 addTax 말고도 여러 개가 있습니다. 그중 지금 쓸 것만 가져오면 됩니다. 안 가져온 것은 없는 것과 같습니다.

```
console.log(peek);
→ ReferenceError: peek is not defined
(도구모음.js 에 있지만 여기서 import 하지 않았으니까요)
```

안 내보낸 것(SECRET)은 아예 가져올 수 없습니다.

```
import { SECRET } from "./도구모음.js";
→ SyntaxError: does not provide an export named 'SECRET'
```

export 를 붙인 것만 '공개' 됩니다. 나머지는 그 파일 안의 비밀입니다.

▹ 직접 해보기 2

도구모음.js 를 열어 보고, export 가 붙은 것과 안 붙은 것을 각각 찾아보세요.

이름 바꿔서 가져오기 (as)

섹션 3

이미 같은 이름의 변수가 있거나, 이름이 마음에 안 들 때 씁니다.

```
import { formatWon as won } from "./도구모음.js";

console.log(won(25000));
```

출력 25,000원

09단원의 구조분해 이름 바꾸기와 방향이 반대입니다. 헷갈리지 마세요.

```
구조분해 : const { name: 새이름 } = 객체;    ← 콜론, 원래이름이 왼쪽
import   : import { name as 새이름 } from ... ← as, 원래이름이 왼쪽
```

왼쪽이 원래 이름인 것은 같습니다. 기호만 다릅니다.

▹ 직접 해보기 3

addTax 를 세금붙이기 라는 이름으로 가져와 써 보세요.

export default — 대표 하나

섹션 4

default 로 내보낸 것은 중괄호 없이 가져옵니다. 그리고 이름을 내 마음대로 지어도 됩니다.

```
import greet from "./도구모음.js";

console.log(greet("김민준"));
```

출력 김민준님 안녕하세요

도구모음.js 에서는 greet 이라는 이름이었지만, 아무 이름이나 됩니다. default 는 파일에 하나뿐이라 헷갈릴 일이 없기 때문입니다.

둘을 같이 쓸 수도 있습니다. default 가 앞, 중괄호가 뒤입니다.

```
import greet, { addTax } from "./도구모음.js";
```

React 에서 매일 보게 될 모양입니다.

```
import React, { useState } from "react";
```

default 이름 지정

여기서 "react" 는 ./ 가 없습니다. 우리가 만든 파일이 아니라 남이 만들어 놓은 것을 내려받아 쓰기 때문입니다. (React 과정에서 배웁니다)

▹ 직접 해보기 4

위 import 줄의 greet 을 인사 라는 이름으로 바꿔도 잘 되는지 확인해 보세요.

실제로 어떻게 나누나

섹션 5

정답은 “같이 바뀌는 것끼리 한 파일에” 입니다.

계산기.js	세금·할인·합계 계산
화면.js	화면 그리기
데이터.js	서버에서 받아오기

13만원 종합03을 이렇게 나눠 보면 이렇게 됩니다.

할일데이터.js	todos 배열과 추가·삭제·토글 함수
화면그리기.js	render, updateStatus
main.js	둘을 가져와 이벤트를 붙이는 곳

파일이 늘어나는 게 아니라, 한 파일이 짧아지는 것입니다. 나중에 고칠 때 “어디를 고쳐야 하지?” 를 안 해매게 됩니다.

자주 하는 실수

섹션 6

★ 아래에서 SyntaxError 라고 적힌 것은 눈으로만 보세요. 주석을 풀지 마세요. 다른 에러 (ReferenceError, TypeError)는 주석을 풀어도 그 줄에서만 나고 그 앞의 출력은 그대로 다 나옵니다. 그런데 SyntaxError 는 다릅니다. 자바스크립트가 파일을 아예 못 읽어서 출력이 한 줄도 안 나옵니다. 여러분이 망가뜨린 것이 아닙니다.

이런 실수를 합니다

확장자를 빼먹음 → 에러

```
import { addTax } from "./도구모음";
```

Cannot find module ... 브라우저와 node 모두 .js 까지 적어야 합니다. (React 에서는 도구가 알아서 붙여 줘서 생략합니다. 지금은 꼭 적으세요)

이런 실수를 합니다

./ 를 빼먹음 → 에러

```
import { addTax } from "도구모음.js";
```

./ 가 없으면 “내려받은 남의 코드” 를 찾습니다. 내 파일은 ./ 로 시작합니다.

이런 실수를 합니다

default 를 중괄호로 감쌈 → SyntaxError

```
import { greet } from "./도구모음.js";
```

default 는 중괄호 없이 가져옵니다. 중괄호는 이름 지정한 것만.

이런 실수를 합니다

이름을 틀리게 씀 → **SyntaxError**

```
import { addtax } from "./도구모음.js";
```

대소문자까지 정확히 같아야 합니다. addTax 입니다.

이런 실수를 합니다

import 를 함수 안에 씀 → **SyntaxError**

```
function load() {  
  import { addTax } from "./도구모음.js";  
}
```

import 는 파일 맨 바깥, 맨 위에만 쓸 수 있습니다.

정리

- 1 export 를 붙인 것만 밖에서 가져다 쓸 수 있다.
- 2 **import** { 이름 } **from** "./파일.js"; — 중괄호, ./, .js 세 가지를 꼭 지킨다.
- 3 as 로 이름을 바꿔서 가져올 수 있다.
- 4 export default 는 파일당 하나. 중괄호 없이, 아무 이름으로나 가져온다.
- 5 import 는 파일 맨 위에 모아 쓴다.
- 6 이 폴더의 package.json 한 줄이 "모듈 문법을 쓴다" 는 표시다.

직접 해보기 정답

- 1 **import** { TAX_RATE, addTax, formatWon } **from** "./도구모음.js";
console.log(formatWon(11000)); // 출력: 11,000원
- 2 export 가 붙은 것 : TAX_RATE, addTax, formatWon, peek, greet(default) 안 붙은 것 :
SECRET → SECRET 만 도구모음.js 안에서만 쓰이는 값입니다.
- 3 **import** { addTax **as** 세금붙이기 } **from** "./도구모음.js"; console.log(세금붙이기(10000)); //
출력: 11000

(변수 이름은 영어로 쓰는 게 좋지만, 여기서는 as 를 확인하는 것이 목적입니다)

```
4) import 인사 from "./도구모음.js";  
console.log(인사("이서연"));  
// 출력: 이서연님 안녕하세요
```


→ default 는 이름을 마음대로 지어도 되기 때문에 그대로 동작합니다.

localStorage

RUN 브라우저로 열고 F12 → Console 탭

13단원에서 만든 할 일 목록에는 큰 구멍이 하나 있었습니다. 새로고침하면 전부 사라집니다.

지금까지 만든 변수는 전부 '잠깐 쓰는 것'이었습니다. 페이지를 닫으면 같이 없어집니다.

localStorage는 브라우저가 컴퓨터에 남겨 주는 작은 저장소입니다. 여기 넣어 두면 페이지를 닫았다 열어도, 컴퓨터를 껐다 켜도 남아 있습니다.

localStorage.setItem("이름", "값")	저장
localStorage.getItem("이름")	꺼내기
localStorage.removeItem("이름")	하나 지우기
localStorage.clear()	전부 지우기

07단원의 객체와 비슷합니다. 이름표를 붙여 값을 넣고 이름으로 꺼냅니다.

아래 예제의 결과를 매번 같게 만들려고, 예제에서 쓸 이름만 먼저 지웁니다. ★ localStorage.clear()로 통째로 지우면 안 됩니다. 섹션 5에서 저장한 할 일 목록까지 같이 날아가서 "새로고침해도 남는다"는 것을 확인할 수 없게 됩니다.

```
["nickname", "count", "badArray", "badObject", "todos", "temp"].forEach((key) => {
  localStorage.removeItem(key);
});
```

저장하고 꺼내기

섹션 1

```
localStorage.setItem("nickname", "김민준");

const saved = localStorage.getItem("nickname");
console.log(saved);
```

F12 콘솔 김민준

없는 이름을 꺼내면 null입니다. 에러가 아닙니다.

```
console.log(localStorage.getItem("없는이름"));
```

F12 콘솔 null

그래서 꺼낸 값은 쓰기 전에 확인해야 합니다. (03단원)

```
const nickname = localStorage.getItem("nickname") || "손님";
console.log(nickname);
```

F12 콘솔 김민준

▸ 직접 해보기 1

자기 이름을 "myName" 이라는 이름으로 저장하고 꺼내 보세요.

저장되는 건 오직 문자열

섹션 2

이게 localStorage 의 가장 큰 함정입니다.

```
localStorage.setItem("count", 5);

const count = localStorage.getItem("count");
console.log(count, typeof count);
```

F12 콘솔 5 string

숫자 5를 넣었는데 문자열 "5" 가 나왔습니다!

```
console.log(count + 1);
```

F12 콘솔 51

01단원에서 본 그 함정입니다. 이어붙이기가 되어 버립니다.

```
console.log(Number(count) + 1);
```

F12 콘솔 6

꺼낸 뒤 Number 로 바꿔야 계산이 됩니다.

객체나 배열을 그냥 넣으면 더 심합니다.

```
localStorage.setItem("badArray", [1, 2, 3]);
console.log(localStorage.getItem("badArray"));
```

F12 콘솔 1,2,3

배열이 아니라 글자가 되어 버렸습니다.

```
localStorage.setItem("badObject", { name: "김민준" });
console.log(localStorage.getItem("badObject"));
```

F12 콘솔 [object Object]

내용이 통째로 사라졌습니다. 이건 되돌릴 방법이 없습니다.

▸ 직접 해보기

2

localStorage 에 true 를 저장했다가 꺼내서 typeof 로 자료형을 확인해 보세요.

객체·배열은 JSON 으로

섹션 3

12단원에서 JSON 을 배웠습니다. “데이터를 글자로 적는 형식”이었죠. 그때는 서버가 준 글자를 객체로 되돌렸습니다(res.json()). 여기서는 반대로, 내 객체를 글자로 바꿔서 저장합니다.

JSON.stringify(값)	객체·배열 → 글자
JSON.parse(글자)	글자 → 객체·배열

```
const todos = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "설거지", done: true },
];
```

```
const asText = JSON.stringify(todos);
console.log(asText);
```

F12 콘솔 [{"id":1,"text":"장보기","done":false}, {"id":2,"text":"설거지","done":true}]

```
console.log(typeof asText);
```

F12 콘솔 string

```
localStorage.setItem("todos", asText);
```

꺼낼 때는 반대로 되돌립니다.

```
const back = JSON.parse(localStorage.getItem("todos"));
console.log(back.length);
```

F12 콘솔 2

코드

```
console.log(back[0].text);
```

```
console.log(Array.isArray(back));
```

▸ F12 콘솔

▸ 장보기

▸ true

진짜 배열로 되돌아왔습니다. map, filter 를 그대로 쓸 수 있습니다.

저장할 때 stringify, 꺼낼 때 parse. 이 한 쌍을 짝으로 외우세요.

▢ 직접 해보기 3

{ name: "이서연", age: 22 } 를 저장했다가 꺼내서 name 만 콘솔에 찍어 보세요.

지우기

섹션 4

```
localStorage.setItem("temp", "잠깐 쓸 값");
console.log(localStorage.getItem("temp"));
```

F12 콘솔 잠깐 쓸 값

```
localStorage.removeItem("temp");
console.log(localStorage.getItem("temp"));
```

F12 콘솔 null

몇 개 들어 있는지는 localStorage.length 로 볼 수 있습니다. 다만 이 파일은 섹션 5에서 할 일 목록도 저장하기 때문에 여러분 화면에서는 숫자가 그때그때 달라집니다. 그래서 여기에 적어 두지 않았습니다.

```
console.log(localStorage.length);
```

직접 찍어 보세요. 지금은 nickname, count, badArray, badObject, todos 가 들어 있으니 최소 5개입니다.

localStorage.clear() 는 전부 지웁니다. 되돌릴 수 없으니 조심하세요. 그래서 이 파일 맨 위에서도 clear() 대신 이름을 하나씩 지웠습니다.

직접 보기 · F12 → Application 탭 → 왼쪽 Local Storage 를 눌러 보세요.

방금 저장한 것들이 표로 보입니다. 여기서 직접 지울 수도 있습니다.

▢ 직접 해보기 4

localStorage.length 를 찍어 보고, badArray 와 badObject 를 removeItem 으로 지운 뒤 다시 찍어서 2 줄었는지 확인해 보세요.

진짜로 써 보기

섹션 5

아래가 13단원 할 일 목록에 저장 기능을 붙인 것입니다. 새로 배운 것은 단 두 줄뿐입니다. save() 와 load() 를 보세요.

```
const todoInput = document.querySelector("#todoInput");
const todoList = document.querySelector("#todoList");
const status = document.querySelector("#status");

const KEY = "myTodos"; // 저장할 때 쓸 이름. 상수로 빼 두면 오타가 안 납니다.
```

저장 · 배열을 글자로 바꿔서 넣는다

```
function save(list) {
  localStorage.setItem(KEY, JSON.stringify(list));
}
```

불러오기 · 글자를 배열로 되돌린다. 처음 열었으면 빈 배열

```
function load() {
  const text = localStorage.getItem(KEY);
  if (!text) {
    return []; // 저장된 게 없으면 빈 배열로 시작
  }
  return JSON.parse(text);
}

let myTodos = load();

function render() {
  todoList.innerHTML = "";

  myTodos.forEach((text, index) => {
    const li = document.createElement("li");

    const span = document.createElement("span");
    span.textContent = text;

    const btn = document.createElement("button");
    btn.textContent = "삭제";
    btn.dataset.index = index;
    btn.classList.add("deleteBtn");

    li.appendChild(span);
    li.appendChild(btn);
    todoList.appendChild(li);
  });

  status.textContent = `항목 ${myTodos.length}개`;
}

function addTodo() {
  const text = todoInput.value.trim();
  if (!text) {
    return;
  }
}
```

```

    }

    myTodos = [...myTodos, text];
    save(myTodos); // * 배열을 바꿨으면 저장도 같이 render();

    todoInput.value = "";
    todoInput.focus();
    console.log("추가 후 저장됨:", myTodos.length, "개");
  }

  document.querySelector("#addBtn").addEventListener("click", addTodo);

  todoInput.addEventListener("keydown", (e) => {
    if (e.key === "Enter") {
      addTodo();
    }
  });
});

```

삭제도 위임으로 (11단원 개념05)

```

  todolist.addEventListener("click", (e) => {
    if (!e.target.classList.contains("deleteBtn")) {
      return;
    }
    const index = Number(e.target.dataset.index);
    myTodos = myTodos.filter((_, i) => i !== index);
    save(myTodos); // * 여기도 저장 render();
  });

  document.querySelector("#clearBtn").addEventListener("click", () => {
    myTodos = [];
    save(myTodos);
    render();
    console.log("전부 지웠습니다");
  });

  render();

```

화면 처음에는 비어 있습니다. 몇 개 넣고 F5 를 눌러 보세요. 그대로 남아 있습니다.

규칙은 하나뿐입니다.

"배열을 바꿨으면 곧바로 save() 를 부른다"

이걸 한 군데라도 빠뜨리면 그 변경만 사라져서 원인을 찾기 어렵습니다.

▹ 직접 해보기 5

할 일을 두세 개 넣고 F5 로 새로고침해 보세요. 그다음 브라우저를 완전히 닫았다가 다시 열어 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

꺼낸 값을 그대로 계산에 씀

```
const n = localStorage.getItem("count") + 1; → "51"
```

전부 문자열입니다. Number 로 바꾸고 쓰세요. (섹션 2)

이런 실수를 합니다

객체를 stringify 없이 저장

```
localStorage.setItem("user", { name: "김만준" }); → "[object Object]"
```

내용이 통째로 사라집니다. 반드시 `JSON.stringify` 로 감싸세요.

이런 실수를 합니다

parse 를 빼먹음

```
const list = localStorage.getItem("todos");  
list.forEach(...) → TypeError: list.forEach is not a function
```

꺼낸 것은 아직 글자입니다. `JSON.parse` 로 되돌려야 배열이 됩니다.

이런 실수를 합니다

저장된 게 없을 때를 생각 안 함

```
const list = JSON.parse(localStorage.getItem("없는키"));
```

→ `null` 이 나옵니다. `JSON.parse(null)` 은 에러는 아니지만 `null` 입니다. 그다음 줄에서 `null.forEach` 로 터집니다. `load()` 처럼 미리 걸러 주세요.

이런 실수를 합니다

바꿔 놓고 저장을 안 함

```
myTodos.push("새 항목"); render(); ← 화면은 바뀌지만
```

새로고침하면 사라집니다. `save()` 를 안 불렀기 때문입니다.

정리

- 1 localStorage 는 새로고침·재접속에도 남는 저장소다.
- 2 `setItem(이름, 값)` / `getItem(이름)` / `removeItem(이름)` / `clear()`
- 3 없는 이름을 꺼내면 `null`. 쓰기 전에 확인한다.
- 4 저장되는 건 오직 문자열. 숫자는 `Number` 로 되돌린다.
- 5 객체·배열은 `JSON.stringify` 로 넣고 `JSON.parse` 로 꺼낸다.
- 6 데이터를 바꿨으면 곧바로 저장한다.

직접 해보기 정답

```
1 localStorage.setItem("myName", "김민준");
  console.log(localStorage.getItem("myName"));
  // 콘솔: 김민준
```

```
2 localStorage.setItem("flag", true);
  const flag = localStorage.getItem("flag");
  console.log(flag, typeof flag);
  // 콘솔: true string
```

→ `true` 로 보이지만 문자열입니다. `if (flag)` 는 항상 `true` 가 되어 버립니다.

(`"false"` 라는 글자도 빈 문자열이 아니라서 `true` 입니다. 주의하세요)

```
3 localStorage.setItem("user", JSON.stringify({ name: "이서연", age: 22 }));
  const user = JSON.parse(localStorage.getItem("user"));
  console.log(user.name);
  // 콘솔: 이서연
```

```
4 console.log(localStorage.length);
  localStorage.removeItem("badArray");
  localStorage.removeItem("badObject");
  console.log(localStorage.length);
```

→ 두 번째 숫자가 첫 번째보다 정확히 2 작으면 맞습니다.

숫자 자체는 할 일을 몇 개 저장해 뒀는지에 따라 달라집니다.

5) 넣은 항목이 그대로 보이면 성공입니다. 브라우저를 닫았다 열어도 남아 있습니다. 그게 `localStorage` 입니다.

날짜와 무작위 (Date, Math.random)

RUN node 개념03_Date와_Math.js

지금까지 만든 예제는 값이 항상 정해져 있었습니다. 실제 앱에는 '지금 몇 시인지', '무작위로 하나 골라 줘' 같은 게 늘 필요합니다. 할 일에 등록 날짜 붙이기 / 오늘의 추천 메뉴 / 주사위 / 로또 번호

두 가지만 알면 됩니다. Date 와 Math.random 입니다.

먼저 알아둘 것 · 이 파일에는 '출력:' 이 없는 console.log 가 몇 개 있습니다.

실행할 때마다 값이 달라지는 것들이라 적어 둘 수가 없습니다. 그런 줄에는 대신 "실행할 때마다 다릅니다" 라고 적어 두었습니다.

Math.random — 0 이상 1 미만

섹션 1

괄호 안에 아무것도 안 넣습니다. 부를 때마다 다른 소수가 나옵니다.

```
console.log(Math.random());
→ 0.5488135039273248  처럼  나옵니다. 실행할 때마다 다릅니다.
```

이 자료의 다른 줄들은 '출력:' 에 적힌 값이 항상 똑같이 나옵니다. 그런데 무작위 값은 매번 달라서 적어 둘 수가 없습니다. 그래서 이 파일에서는 무작위 값 자체를 찍는 대신, "범위 안에 들어왔는가" 처럼 항상 같은 답이 나오는 것을 찍습니다. 진짜 값이 궁금하면  에서 직접 찍어 보세요.

딱 세 가지만 기억하면 됩니다. ① 0 은 나올 수 있다 ② 1 은 절대 안 나온다 ③ 항상 소수다 (정수가 필요하면 우리가 바꿔야 한다)

확인해 봅시다. 이건 언제 실행해도 결과가 같습니다.

```
const r = Math.random();
console.log(r >= 0 && r < 1);
```

출력 true

```
console.log(typeof Math.random());
```

출력 number

▹ 직접 해보기 1

`Math.random()` 을 세 번 찍어 보고 값이 다 다른지 보세요.

원하는 범위의 정수 만들기

섹션 2

0~1 사이 소수만으로는 쓸 데가 없습니다. 주사위를 만들어 봅시다.

`Math.random()` 0 이상 1 미만의 소수 * 6 0 이상 6 미만의 소수 → 0.0 ~ 5.999... `Math.floor(...)` 소수점 버림 → 0, 1, 2, 3, 4, 5 + 1 1 부터 시작하게 → 1, 2, 3, 4, 5, 6

```
const dice = Math.floor(Math.random() * 6) + 1;
console.log(dice >= 1 && dice <= 6);
```

출력 true

공식으로 외워 두면 편합니다.

```
Math.floor(Math.random() * (최대 - 최소 + 1)) + 최소
```

1~45 (로또 번호 하나) 라면 최소 1, 최대 45 이므로

```
Math.floor(Math.random() * 45) + 1
```

```
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

```
const lotto = randomInt(1, 45);
console.log(lotto >= 1 && lotto <= 45);
```

출력 true

```
console.log(Number.isInteger(lotto));
```

출력 true

`Math.floor` 를 거쳤으니 소수가 아니라 정수입니다.

▹ 직접 해보기 2

`randomInt` 로 10~20 사이 숫자를 만들어 범위 안에 들어오는지 확인해 보세요.

배열에서 하나 무작위로 뽑기

섹션 3

인덱스를 무작위로 만들면 됩니다. 인덱스는 0부터 length-1 까지죠. (06단원)

```
const menus = ["김치찌개", "제육볶음", "냉면", "돈까스"];

const pickedIndex = Math.floor(Math.random() * menus.length);
const picked = menus[pickedIndex];

console.log(menus.includes(picked));
```

출력 true

뽑은 것이 원래 배열 안에 있는 값입니다.

```
console.log(picked === undefined);
```

출력 false

범위를 벗어나지 않았다는 뜻입니다.

여기서 * menus.length 인 것이 중요합니다. + 1 을 하면 안 됩니다. 길이가 4면 인덱스는 0,1,2,3 입니다. 4는 없습니다. Math.random() * 4 는 0 이상 4 미만이라 딱 맞습니다.

함수로 만들어 두면 어디서든 씁니다.

```
function pickOne(list) {
  return list[Math.floor(Math.random() * list.length)];
}

console.log(typeof pickOne(menus));
```

출력 string

▹ 직접 해보기 3

pickOne 으로 오늘의 점심을 다섯 번 뽑아 출력해 보세요.

Date — 날짜와 시간

섹션 4

괄호를 비우면 '지금' 입니다.

```
const now = new Date();
console.log(now.getFullYear() >= 2024);
```

출력 true

09단원에서 본 new 입니다. “이 설계도로 새것을 하나 만들어라” 였죠.

```
new Date() → 날짜 객체를 하나 만들어라
```

특정 날짜를 만들 수도 있습니다. 이건 값이 고정이라 결과가 항상 같습니다.

```
const someday = new Date(2026, 7, 11, 14, 30);
```

년 월 일 시 분

코드

```
console.log(someday.getFullYear());
```

```
console.log(someday.getDate());
```

▶ 출력

▶ 2026

▶ 11

```
console.log(someday.getHours(), someday.getMinutes());
```

출력 14 30

★ 여기가 가장 많이 틀리는 곳입니다.

```
console.log(someday.getMonth());
```

출력 7

8월인데 7이 나옵니다! 월은 0부터 셉니다. (0=1월, 7=8월) 배열 인덱스처럼 0부터라고 생각하세요.

```
console.log(someday.getMonth() + 1);
```

출력 8

사람에게 보여 줄 때는 +1 을 해야 합니다.

요일도 0부터입니다. 0=일요일, 1=월요일 ... 6=토요일

```
console.log(someday.getDay());
```

출력 2

2는 화요일입니다. 2026년 8월 11일은 화요일입니다.

▸ 직접 해보기 4

자기 생일로 Date 를 만들고, 무슨 요일이었는지 숫자로 출력해 보세요. (월은 실제 월보다 1 작게 넣어야 합니다)

사람이 읽을 수 있게

섹션 5

`getMonth() + 1` 을 매번 하기는 번거롭습니다. 한국식으로 바꿔 주는 게 있습니다.

코드	출력
<code>console.log(someday.toLocaleDateString("ko-KR"));</code>	▶ 2026. 8. 11.
<code>console.log(someday.toLocaleTimeString("ko-KR"));</code>	▶ 오후 2:30:00

요일 이름도 받을 수 있습니다.

<code>console.log(someday.toLocaleDateString("ko-KR", { weekday: "long" }));</code>
출력 화요일

직접 조립해도 됩니다. 배운 것만으로 충분합니다.

<code>const y = someday.getFullYear();</code> <code>const m = someday.getMonth() + 1;</code> <code>const d = someday.getDate();</code> <code>console.log(`\${y}년 \${m}월 \${d}일`);</code>
출력 2026년 8월 11일

앞에 0을 붙이고 싶으면 (08월 처럼) `padStart` 를 씁니다.

<code>console.log(String(m).padStart(2, "0"));</code>
출력 08

“글자 수가 2가 될 때까지 앞을 0으로 채워라” 라는 뜻입니다.

▸ 직접 해보기 5

`someday` 를 “2026-08-11” 형태로 만들어 출력해 보세요. (힌트: `padStart` 를 월과 일에 각각 쓰고 - 로 이으세요)

날짜 사이의 차이

섹션 6

날짜끼리 빼면 ‘밀리초’ 단위 숫자가 나옵니다.

```
const start = new Date(2026, 7, 11);
const end = new Date(2026, 7, 20);

console.log(end - start);
```

출력 777600000

밀리초라 숫자가 큼니다. 사람이 읽을 수 있게 바꿔야 합니다.

1초 = 1000, 1분 = 60초, 1시간 = 60분, 1일 = 24시간

```
const oneDay = 1000 * 60 * 60 * 24;
console.log((end - start) / oneDay);
```

출력 9

9일 차이입니다.

“며칠 남았나” 를 구할 때 이 방법을 씁니다. 시간까지 들어 있으면 소수가 나오니 `Math.floor` 나 `Math.round` 로 다듬으세요.

▸ 직접 해보기 6

2026년 1월 1일과 2026년 12월 31일 사이가 며칠인지 구해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

getMonth 를 그대로 씀

```
console.log(`${someday.getMonth()}월`);
```

출력 7월

8월인데 7월이라고 나옵니다. +1 을 빠뜨린 것입니다. 가장 흔한 실수입니다.

이런 실수를 합니다

new 를 빼먹음

```
const bad = Date();
```

에러는 안 나지만 날짜 객체가 아니라 ‘글자’ 가 나옵니다. `getFullYear` 같은 걸 부르면 `TypeError` 가 납니다. `new` 를 꼭 붙이세요.

이런 실수를 합니다

Math.random에 괄호를 안 붙임

```
console.log(typeof Math.random);
```

출력 function

괄호가 없으면 함수 그 자체입니다. 08:11단원에서 계속 본 그 실수입니다.

이런 실수를 합니다

Math.floor를 안 함

```
const wrongDice = Math.random() * 6 + 1;
console.log(Number.isInteger(wrongDice));
```

출력 false

3.7 같은 소수가 나옵니다. 주사위에 3.7은 없죠. floor 로 버려야 합니다.

이런 실수를 합니다

배열 뽑기에서 **length**에 +1

```
const bad2 = menus[Math.floor(Math.random() * (menus.length + 1))];
```

가끔 **undefined**가 나옵니다. 없는 인덱스를 골랐기 때문입니다. 에러가 안 나서 원인을 찾기 어렵습니다. length 그대로 쓰세요.

정리

- 1 **Math.random()** 은 0 이상 1 미만의 소수. 1은 안 나온다.
- 2 정수 범위: **Math.floor(Math.random() * (최대 - 최소 + 1)) + 최소**
- 3 배열에서 뽑기: **list[Math.floor(Math.random() * list.length)]** — +1 하지 않는다
- 4 **new Date()** 는 지금. **new Date(년, 월, 일)** 로 특정 날짜.
- 5 **getMonth()** 는 0부터. 보여 줄 때는 +1 을 한다. **getDay()** 는 0=일요일.
- 6 **toLocaleDateString("ko-KR")** 로 한국식 표기. **padStart(2, "0")** 로 0 채우기.
- 7 날짜끼리 빼면 밀리초. **1000*60*60*24** 로 나누면 일수.

직접 해보기 정답

```
1) console.log(Math.random());  
console.log(Math.random());  
console.log(Math.random());
```

→ 세 번 다 다른 소수가 나옵니다. 같은 값이 나오는 일은 사실상 없습니다.

```
2  const n = randomInt(10, 20);  
   console.log(n, n >= 10 && n <= 20);  
   // 출력: (매번 다른 숫자) true
```

```
3  for (let i = 0; i < 5; i++) {
```

```
console.log(pickOne(menus));
```

```
}
```

→ 같은 메뉴가 연달아 나올 수도 있습니다. 무작위라 정상입니다.

```
4) const birthday = new Date(2005, 2, 15); // 2005년 3월 15일  
   console.log(birthday.getDay());  
   // 출력: 2 (화요일)
```

→ 월에 2를 넣은 것에 주의하세요. 3월이라서 3-1=2 입니다.

```
5  const mm = String(someday.getMonth() + 1).padStart(2, "0");  
   const dd = String(someday.getDate()).padStart(2, "0");  
   console.log(`${someday.getFullYear()}-${mm}-${dd}`);  
   // 출력: 2026-08-11
```

```
6  const jan1 = new Date(2026, 0, 1);  
   const dec31 = new Date(2026, 11, 31);  
   console.log((dec31 - jan1) / (1000 * 60 * 60 * 24));  
   // 출력: 364
```

→ 365일이 아니라 364입니다. 1월 1일 '부터' 세면 365일째가 12월 31일이지만,

두 날짜의 '차이' 는 364일입니다. 하루 차이가 나는 흔한 함정입니다.

class 맛보기

RUN node 개념04_class_맛보기.js

먼저 솔직하게 말씀드립니다.

이 자료를 여기까지 배운 것만으로 앱을 만들 수 있습니다. class 를 몰라도 됩니다. 요즘 React 도 class 를 쓰지 않습니다.

그런데도 배우는 이유는 하나입니다. 남이 쓴 코드에 나오기 때문입니다. 인터넷 예제, 회사 코드, 오래된 라이브러리에 흔합니다. “쓸 줄 아는 것” 보다 “읽을 줄 아는 것” 이 목표입니다.

그래서 이 파일은 짧습니다. 부담 없이 읽으세요.

같은 모양을 여러 개 만들어야 할 때

섹션 1

07단원 방식으로 학생 세 명을 만들어 봅시다.

```
const student1 = { name: "김민준", score: 90 };
const student2 = { name: "이서연", score: 95 };
const student3 = { name: "박지훈", score: 60 };

console.log(student1.name, student2.name, student3.name);
```

출력 김민준 이서연 박지훈

세 명이면 괜찮습니다. 그런데 30명이면요? 게다가 “합격인지 알려 주는 기능” 을 붙이고 싶다면 객체 30개에 똑같은 함수를 30번 써야 합니다.

05단원에서 배운 대로, 반복되면 함수로 묶습니다. class 는 ‘객체를 찍어 내는 틀’ 을 만드는 문법입니다.

class 로 틀 만들기

섹션 2

```
class Student {
```

constructor 는 new 로 만들 때 딱 한 번 실행되는 특별한 함수입니다. “처음 값을 채워 넣는 곳” 이라고 생각하면 됩니다.

```

constructor(name, score) {
    this.name = name; // this 는 '지금 만들어지는 그 객체' this.score = score;
}
}

const s1 = new Student("김민준", 90);
const s2 = new Student("이서연", 95);

console.log(s1.name, s1.score);

```

출력 김민준 90

```

console.log(s2.name, s2.score);

```

출력 이서연 95

만들어진 것은 그냥 객체입니다. 07단원에서 쓰던 것과 똑같이 다릅니다.

```

console.log(typeof s1);

```

출력 object

class 이름은 첫 글자를 대문자로 씁니다. 약속입니다.

```

Student, Product, Timer   ← class
student, product, timer   ← 그냥 변수

```

▫ 직접 해보기 1

Student 로 자기 이름과 점수를 넣은 객체를 만들어 출력해 보세요.

메소드 — 틀에 기능 붙이기

섹션 3

```

class Product {
    constructor(name, price, stock) {
        this.name = name;
        this.price = price;
        this.stock = stock;
    }
}

```

07단원의 '줄여 쓰기' 와 모양이 같습니다. `function` 을 안 씁니다.

```
isSoldOut() {
  return this.stock === 0;
}

totalValue() {
  return this.price * this.stock;
}
```

07단원 개념03에서 배운 그 this 입니다. 점 왼쪽에 있는 것.

```
describe() {
  const state = this.isSoldOut() ? "품절" : `재고 ${this.stock}개`;
  return `${this.name} ${this.price}원 (${state})`;
}
}
```

```
const coffee = new Product("아메리카노", 4000, 10);
const cake = new Product("케이크", 6000, 0);

console.log(coffee.describe());
```

출력 아메리카노 4000원 (재고 10개)

```
console.log(cake.describe());
```

출력 케이크 6000원 (품절)

코드

```
console.log(coffee.isSoldOut(), cake.isSoldOut());
```

▶ 출력

▶ false true

```
console.log(coffee.totalValue());
```

▶ 40000

여기가 class 의 진짜 장점입니다. describe 라는 함수는 '한 번만' 썼는데, 만들어진 모든 상품이 쓸 수 있습니다. 객체를 100개 만들어도 함수는 하나뿐입니다.

만들어진 것들은 배열에 담아 지금까지 배운 대로 다루면 됩니다.

```
const products = [coffee, cake, new Product("쿠키", 3000, 5)];

const soldOut = products.filter((p) => p.isSoldOut());
console.log(soldOut.length);
```

출력 1

```
const totalAssets = products.reduce((sum, p) => sum + p.totalValue(), 0);
console.log(totalAssets);
```

출력 55000

▹ 직접 해보기 2

products 에서 품질이 아닌 상품의 이름만 뽑아 출력해 보세요.

07단원 객체와 뭐가 다른가

섹션 4

하나만 만들 거면 07단원 방식이 더 간단합니다.

```
const singleProduct = {
  name: "이어폰",
  price: 89000,
  describe() {
    return `${this.name} ${this.price}원`;
  },
};
console.log(singleProduct.describe());
```

출력 이어폰 89000원

정리하면 이렇습니다.

객체 하나만 필요하다	→ 07단원 방식 { }
같은 모양을 여러 개 만든다	→ class

class 도 결국 객체를 만드는 것입니다. 완전히 새로운 게 아닙니다. 문법만 다를 뿐, 만들어진 결과는 07단원에서 배운 그 객체입니다.

그래서 지금 써야 하나

섹션 5

아닙니다. 지금은 몰라도 됩니다. 아래만 기억하세요.

① class 이름 () 앞에 new 가 붙어 있으면 “객체를 하나 만드는 중” 이다 ② constructor 는 처음 값을 채우는 곳이다 ③ 안에 있는 함수들은 만들어진 객체가 점으로 부를 수 있는 기능이다

이 셋만 알면 남의 코드를 읽을 수 있습니다. 그게 이 파일의 목표입니다.

참고로 우리는 이미 class 를 써 왔습니다.

new Promise(...)	← 12단원
new Date(...)	← 개념03
new Error(...)	← 12단원

전부 누군가 만들어 둔 class 를 new 로 찍어 낸 것이었습니다.

```
console.log(new Date(2026, 0, 1).getFullYear());
```

출력 2026

▹ 직접 해보기 3

Product 로 상품을 하나 더 만들어 products 에 넣고, 전체 재고 자산이 얼마가 되는지 다시 구해 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

new 를 빼먹음

```
const bad = Product("아메리카노", 4000, 10);
```

```
TypeError: Class constructor Product cannot be invoked without 'new'
```

다행히 class 는 에러를 내 줍니다. new 를 붙이세요.

이런 실수를 합니다

constructor 에서 this 를 안 씀

```
constructor(name) {  
  name = name; ← 매개변수에 자기를 넣은 것뿐, 객체에는 안 담김  
}
```

만들어진 객체가 텅 비어 있습니다. this.name = name 이어야 합니다.

이런 실수를 합니다

메소드에 function 을 붙임

describe: function () { ... } ← 이건 07단원의 객체 방식 class 안에서는 describe() { ... } 처럼 씁니다.
콜론도 쉼표도 없습니다.

이런 실수를 합니다

메소드를 화살표 함수로 쓰려다 헛갈림

07단원에서 배운 대로, 객체의 메소드는 화살표 함수로 만들지 않습니다. class 안에서도 마찬가지입니다.
위 예제들처럼 줄여 쓰기로 쓰세요.

정리

- 1 class 는 '같은 모양의 객체를 여러 개 찍어 내는 틀' 이다.
- 2 new 로 만든다. new 를 빼면 에러가 난다.
- 3 constructor 는 처음 값을 채우는 곳. this.속성 = 값 으로 담는다.
- 4 안의 함수(메소드)는 만들어진 객체가 점으로 부른다. function 을 안 붙인다.
- 5 만들어진 것은 결국 07단원의 그 객체다. map, filter 를 그대로 쓴다.
- 6 지금 당장 쓸 일은 없다. 남의 코드를 읽는 게 목적이다.

직접 해보기 정답

```
1  const me = new Student("최유진", 88);  
   console.log(me.name, me.score);  
   // 출력: 최유진 88
```

```
2  const 가능 = products.filter((p) => !p.isSoldOut()).map((p) => p.name);  
   console.log(가능);  
   // 출력: [ '아메리카노', '쿠키' ]
```

(변수 이름은 영어로 쓰는 게 좋습니다. available 같은 이름이 낫습니다)

```
3) products.push(new Product("샌드위치", 5000, 4));  
const total = products.reduce((sum, p) => sum + p.totalValue(), 0);  
console.log(total);  
// 출력: 75000
```

→ 55000 에 5000 × 4 = 20000 이 더해졌습니다.

15단원 · 연습문제

React 가기 전에

RUN node 연습문제.js

TODO 자리에 코드를 쓰고, '기대 출력'과 같은지 확인하세요. 1~11은 기본, 12는 응용, 13은 [도전], 14는 에러 확인입니다.

주의 · 문제 1~3은 import 문제입니다.

`import` 는 파일 맨 위에만 쓸 수 있으니, 답도 맨 위에 쓰세요.
(아래 TODO 자리가 아니라 이 헤더 바로 밑입니다)

문제 1~3의 `import` 를 여기에 쓰세요

문제 1 (개념01 import)

연습문제_도구.js 에서 SHIPPING_FEE 와 addShipping 을 가져와 배송비와, 12000원짜리에 배송비를 더한 값을 차례로 출력하세요.

기대 출력 3000
 15000

여기에 코드를 쓰세요

문제 2 (개념01 as)

toPercent 를 percent 라는 이름으로 가져와 0.35 를 넣어 출력하세요.

기대 출력 35%

여기에 코드를 쓰세요

문제 3 (개념01 default)

연습문제_도구.js 의 default 를 가져와 "김민준" 을 넣어 출력하세요. 이름은 마음대로 지어도 됩니다.

기대 출력 우리편의점에 오신 김민준님 환영합니다

여기에 코드를 쓰세요

문제 4 (개념03 Math.random)

1 이상 100 이하의 정수를 만들어 randomScore 에 담고, 그 값이 범위 안에 들어오는지 true/false 로 출력하세요.

힌트: 무작위 값은 매번 달라서 값 자체를 기대 출력에 적을 수 없습니다.

그래서 "범위 안에 있는가" 를 확인합니다. 개념03 섹션 2의 공식을 쓰세요.

기대 출력 true

여기에 코드를 쓰세요

문제 5 (개념03 정수 확인)

문제 4에서 만든 randomScore 가 정수인지 출력하세요. (소수가 나오면 Math.floor 를 빠뜨린 것입니다)

기대 출력 true

여기에 코드를 쓰세요

문제 6 (개념03 배열에서 뽑기)

아래 배열에서 무작위로 하나를 뽑아 picked 에 담고, 뽑은 것이 원래 배열 안에 있는 값인지 출력하세요.

```
const drinks = ["아메리카노", "라떼", "녹차", "주스"];
```

기대 출력 true

여기에 코드를 쓰세요

문제 7 (개념03 Date 만들기)

2026년 3월 5일로 Date 를 만들어 openDay 에 담고, 연도와 '일' 을 차례로 출력하세요. (월을 넣을 때 주의하세요)

기대 출력 2026
5

여기에 코드를 쓰세요

문제 8 (개념03 getMonth)

openDay 의 월을 '사람이 읽는 숫자' 로 출력하세요.

기대 출력 3

여기에 코드를 쓰세요

문제 9 (개념03 날짜 포맷)

openDay 를 "2026-03-05" 형태로 출력하세요.

힌트: 월과 일에 각각 padStart(2, "0") 을 쓰고 - 로 이으세요.

padStart 는 문자열의 메소드라 String() 으로 먼저 바꿔야 합니다.

기대 출력 2026-03-05

여기에 코드를 쓰세요

문제 10 (개념04 class)

이름과 가격을 받는 Menu 라는 class 를 만들고, "아메리카노" 4000 으로 하나 만들어 이름과 가격을 함께 출력하세요.

기대 출력 아메리카노 4000

여기에 코드를 쓰세요

문제 11 (개념04 메소드)

Menu 에 label() 메소드를 추가하세요. "아메리카노 4000원" 형태를 돌려줘야 합니다.

기대 출력 아메리카노 4000원

문제 10의 class 안에 함께 쓰세요

문제 12 [응용]

아래 메뉴 배열을 Menu 객체 배열로 바꾸고(map), 4500원 이상인 것만 골라(filter) label() 결과를 출력하세요.

```
const menuData = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];
```

기대 출력

— '라떼 4500원', '케이크 6000원'

여기에 코드를 쓰세요

문제 13 [도전]

오늘부터 2026년 12월 25일까지 며칠 남았는지 구하는 daysUntilChristmas 함수를 만드세요.

이 문제만 결과가 매일 달라집니다. 그래서 아래 두 가지를 확인합니다. ① 돌려준 값이 숫자인가 ② 그 값이 정수인가

힌트: 두 날짜를 빼면 밀리초가 나옵니다. 1000*60*60*24 로 나누고

Math.ceil 이나 Math.floor 로 다듬으세요. (개념03 섹션 6)

기대 출력 number
 true

여기에 코드를 쓰세요

문제 14 (에러 확인 - 맨 마지막)

아래 줄의 주석을 풀고 실행해서 어떤 에러가 나는지 확인하세요. 확인했으면 반드시 다시 주석 처리하세요.

```
const bad = Menu("라떼", 4500);
```

무슨 에러가 났나요? 무엇을 빠뜨린 걸까요?

답

localStorage

RUN 브라우저로 열고 F12 → Console 탭

문제를 여러 번 풀어도 결과가 같도록 1~5에서 쓸 것만 먼저 비웁니다. (문제 6·7이 쓰는 값은 남겨 둡니다. 새로고침해도 남아야 하니까요)

```
["nickname", "point", "user", "colors"].forEach((k) => localStorage.removeItem(k));
```

문제 1 (개념02 저장하고 꺼내기)

“nickname” 이라는 이름으로 “김민준” 을 저장한 뒤, 꺼내서 콘솔에 찍으세요.

기대 콘솔 김민준

여기에 코드를 쓰세요

문제 2 (개념02 없는 값)

저장한 적 없는 “phone” 을 꺼내서 콘솔에 찍으세요.

기대 콘솔 null

여기에 코드를 쓰세요

문제 3 (개념02 문자열 합성)

"point" 라는 이름으로 숫자 100 을 저장하세요. 그다음 꺼내서 '값' 과 '자료형' 을 한 줄에 함께 찍고, 다음 줄에 거기에 50 을 더한 값을 찍으세요.

```
기대 콘솔    100 string
              150
```

두 번째 줄이 10050 이 나왔다면 Number 로 바꾸는 것을 빠뜨린 것입니다.

여기에 코드를 쓰세요

문제 4 (개념02 객체 저장)

아래 객체를 "user" 라는 이름으로 저장했다가 꺼내서, city 만 찍으세요.

```
const user = { name: "이서연", age: 22, city: "부산" };
```

```
기대 콘솔    부산
```

그냥 setItem 으로 넣으면 [object Object] 가 됩니다. 개념02 섹션 3을 보세요.

여기에 코드를 쓰세요

문제 5 (개념02 배열 저장)

아래 배열을 "colors" 라는 이름으로 저장했다가 꺼내서, 진짜 배열로 돌아왔는지 확인하고 개수를 찍으세요.

```
const colors = ["빨강", "초록", "파랑"];
```


기대 콘솔 true
 3

여기에 코드를 쓰세요

문제 6 [응용] 마지막 검색어 기억하기

searchInput 에 글자를 치고 Enter 를 누르면 - 그 값을 "lastSearch" 라는 이름으로 저장하고 - out6 에 "마지막 검색어: OOO" 를 넣으세요. 그리고 페이지를 열 때 저장된 값이 있으면 out6 에 바로 보여 주세요.

기대 화면 "커피" 치고 Enter → 마지막 검색어: 커피
 F5 로 새로고침해도 "마지막 검색어: 커피" 가 그대로 보입니다
 저장된 게 없으면 처음 글자("아직 검색한 적이 없습니다")를 그대로 둡니다

힌트: Enter 는 11단원 개념02의 keydown 을 씁니다.

여기에 코드를 쓰세요

문제 7 [도전] 방문 횟수 세기

페이지를 열 때마다 1씩 올라가는 방문 횟수를 만들어 out7 에 "3번째 방문입니다" 처럼 보여 주세요. resetBtn 을 누르면 0으로 되돌리고 화면도 갱신하세요.

기대 화면 처음 열면 → 1번째 방문입니다
 F5 → 2번째 방문입니다
 F5 → 3번째 방문입니다
 초기화 버튼 → 0번째 방문입니다 (그다음 F5 면 다시 1번째)

힌트: 저장된 값은 문자열입니다. Number 로 바꿔서 +1 한 뒤 다시 저장하세요.

처음 열었을 때는 저장된 게 없어 null 입니다. 그때를 처리해야 합니다.

여기에 코드를 쓰세요

15단원 마무리 React 가기 전에

자주 넘어지는 자리

- 1 `import` 는 `./` 와 `.js` 를 반드시 붙인다는 것 (React 에서는 생략되어 헛갈립니다)
- 2 `localStorage` 는 **문자열만** 저장됩니다. 객체는 `JSON.stringify` 없이 넣으면 내용이 통째로 사라집니다
- 3 `getMonth()` 는 0부터. 8월이 7로 나옵니다

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

09단원 연습문제 정답

구조분해와 스프레드

RUN node 연습문제_정답.js

문제 1

```
const colors1 = ["빨강", "파랑", "노랑"];
const [first1, second1] = colors1;
console.log(first1, second1);
```

출력 빨강 파랑

왼쪽 대괄호는 배열을 만드는 게 아니라 '모양을 맞춰 꺼내는 틀' 입니다.

문제 2

```
const one2 = ["사과"];
const [fruit2, second2 = "없음"] = one2;
console.log(fruit2, second2);
```

출력 사과 없음

값이 `undefined` 일 때만 기본값이 쓰입니다. `null` 은 그대로 들어갑니다.

문제 3

```
const nums3 = [1, 2, 3, 4];
const [head3, ...tail3] = nums3;
console.log(head3);
```

출력 1

```
console.log(tail3);
```

출력 [2, 3, 4]

`...` 은 반드시 맨 마지막에만 쓸 수 있습니다.

문제 4

```
let left4 = "왼쪽";
let right4 = "오른쪽";

[left4, right4] = [right4, left4];

console.log(left4, right4);
```

출력 오른쪽 왼쪽

앞 줄이 세미콜론으로 끝나야 합니다. 안 그러면 앞 줄과 이어져 에러가 납니다.

문제 5

```
const dateText5 = "2026-08-11";
const [year5, month5, day5] = dateText5.split("-");
console.log(`${year5}년 ${month5}월 ${day5}일`);
```

출력 2026년 08월 11일

split 의 결과가 배열이므로 바로 구조분해로 받을 수 있습니다.

문제 6

```
const book6 = { title: "입문서", price: 25000, author: "김작가" };
const { title, price } = book6;
console.log(title, price);
```

출력 입문서 25000

객체는 이름으로 찾으므로 순서가 상관없습니다.

문제 7

```
const { author: writer } = book6;
console.log(writer);
```

출력 김작가

왼쪽이 원래 속성 이름, 오른쪽이 내가 쓸 새 이름입니다. 순서를 헷갈리기 쉽습니다.

문제 8

```
const config8 = { theme: "dark" };
const { theme, fontSize = 14 } = config8;
console.log(theme, fontSize);
```

출력 dark 14

문제 9

```
const account9 = { userId: "abc", password: "1234", nickname: "서연" };
const { password, ...safe9 } = account9;
console.log(safe9);
```

출력 { userId: 'abc', nickname: '서연' }

비밀번호처럼 빼고 넘겨야 할 값이 있을 때 자주 쓰는 형태입니다.

문제 10

```
const user10 = { name: "김민준", age: 20, city: "부산" };

function introduce10({ name, city }) {
  console.log(`${name}은 ${city} 거주`);
}

introduce10(user10);
```

출력 김민준은 부산 거주

매개변수만 봐도 이 함수가 무엇을 쓰는지 보입니다. age 는 안 쓰므로 아예 안 꺼내면 됩니다.

문제 11

```
const products11 = [
  { name: "아메리카노", price: 4000 },
  { name: "케이크", price: 6000 },
];

console.log(products11.map(({ name }) => name));
```

출력 ['아메리카노', '케이크']

(p) ⇒ p.name 이라고 써도 맞습니다. 속성이 많아질수록 구조분해가 편합니다.

문제 12

```
const scores12 = [30, 10, 20];
console.log([...scores12].sort((a, b) => a - b));
```

출력 [10, 20, 30]

```
console.log(scores12);
```

출력 [30, 10, 20]

sort 는 원본을 바꿉니다. [...배열] 로 복사한 뒤 정렬해야 원본이 지켜집니다.

문제 13

```
const a13 = [1, 2];
const b13 = [3, 4];
console.log([...a13, ...b13]);
```

출력 [1, 2, 3, 4]

a13, b13 · 이라고 쓰면 배열 안에 배열이 들어갑니다. ... 을 빠뜨리지 마세요.

문제 14

```
const temps14 = [45, 88, 62];
console.log(Math.max(...temps14));
```

출력 88

`Math.max(temps14)` 라고 쓰면 `NaN` 이 나옵니다. 배열을 통째로는 못 받습니다.

문제 15

```
const base15 = { name: "박지훈", age: 28 };
console.log({ ...base15, age: 30 });
```

출력 { name: '박지훈', age: 30 }

```
console.log(base15);
```

출력 { name: '박지훈', age: 28 }

같은 이름을 나중에 다시 쓰면 덮어씁니다. 원본은 그대로입니다.

문제 16

```
const defaults16 = { theme: "light", fontSize: 14 };
const userSetting16 = { theme: "dark" };

console.log({ ...defaults16, ...userSetting16 });
```

출력 { theme: 'dark', fontSize: 14 }

순서를 바꾸면 기본값이 사용자 설정을 덮어써 버립니다. 규칙: 기본값을 먼저, 덮어쓸 것을 나중에.

문제 17

```
function sumAll(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}

console.log(sumAll(1, 2, 3, 4, 5));
```


출력	15
코드	▶ 출력
<code>console.log(sumAll(1, 2));</code>	▶ 3
<code>console.log(sumAll());</code>	▶ 0

reduce 에 시작값 0 을 줬기 때문에 인자가 하나도 없어도 에러가 나지 않습니다.

문제 18

```
const word18 = "자바스크립트";
console.log([...word18].reverse().join(""));
```

출력 트립크스바자

문자열을 스프레드로 펼치면 글자 배열이 됩니다. 배열을 뒤집고 join("") 으로 다시 붙였습니다. word18.split("") 을 써도 결과는 같습니다.

문제 19

```
const todos19 = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "청소", done: false },
];
```

id 가 2면 복사해서 done 만 바꾸고, 아니면 그대로 둡니다.

```
const updated19 = todos19.map((todo) => (todo.id === 2 ? { ...todo, done: true } :
todo));

updated19.forEach(({ text, done }) => {
  console.log(`${text} ${done ? "완료" : "미완료"}`);
});
```

출력 장보기 미완료
 청소 완료

```
console.log("원본 확인:", todos19[1].done);
```

출력 원본 확인: false

map 은 새 배열을 만들고, { ...todo } 는 새 객체를 만들기 때문에 원본은 하나도 건드려지지 않았습니다.
React 에서 계속 쓰는 형태입니다.

문제 20

```
const { value } = null;
```

이 코드는 검사에서 걸립니다

왜:

구조분해는 "이 안에서 꺼내라" 는 뜻인데 null 은 안이 없습니다.
undefined 여도 같은 에러가 납니다.

해결:

오른쪽에 기본값을 주면 됩니다.

```
const { value } = null ?? {}; // 출력: undefined
```

```
const { value } = someData || {};
```

함수 매개변수에서도 같은 방법을 씁니다.

```
function f({ name } = {}) { ... } // 인자를 안 넘겨도 에러가 안 남
```

TypeError: Cannot destructure property 'value' of 'null' as it is null.

10단원 연습문제 정답 DOM 조작

RUN 브라우저로 열고 F12 → Console 탭

문제 1

```
const mainTitle = document.querySelector("#mainTitle");
console.log(mainTitle);
```

F12 콘솔 <h2 id="mainTitle">원래 제목</h2>

getElementById("mainTitle") 로 써도 맞습니다. 이때는 # 을 붙이지 않습니다.

문제 2

```
document.querySelector("#target1").textContent = "바뀌었습니다!";
```

화면 해당 줄이 "바뀌었습니다!" 로 바뀝니다

textContent 는 속성입니다. 괄호를 붙이면 TypeError 가 납니다.

문제 3

```
document.querySelector("#target2").innerHTML = "<strong>굵은 글자</strong>";
```

화면 실제로 굵게 보입니다

textContent 로 넣었다면 이 글자 그대로 보였을 겁니다.

문제 4

```
const items = document.querySelectorAll(".item");
console.log(items.length);
```

F12 콘솔 3

querySelector 였다면 첫 하나만 잡혀서 length 를 셀 수 없습니다.

문제 5

```
items.forEach((item) => {  
  console.log(item.textContent);  
});
```

F12 콘솔 항목 A
 항목 B
 항목 C

문제 6

```
const styleTarget = document.querySelector("#styleTarget");  
styleTarget.style.color = "white";  
styleTarget.style.backgroundColor = "#2d6cdf";
```

화면 파란 배경에 흰 글자

background-color 라고 쓰면 SyntaxError 입니다. 카멜 표기법으로 씁니다.

문제 7

```
document.querySelector("#classTarget").classList.add("highlight");
```

화면 노란 배경에 굵은 글자

add(".highlight") 처럼 점을 붙이면 안 됩니다. 이름만 씁니다.

문제 8

```
const toggleTarget = document.querySelector("#toggleTarget");  
toggleTarget.classList.toggle("hidden");  
console.log(toggleTarget.classList.contains("hidden"));
```

F12 콘솔 true

화면 해당 줄이 사라집니다 (display: none)

문제 9

```
const fruits = ["사과", "바나나", "포도"];
const fruitList = document.querySelector("#fruitList");

fruits.forEach((fruit, index) => {
  const li = document.createElement("li"); // 반복문 '안'에서 매번 새로 만들기
  li.textContent = `${index + 1}. ${fruit}`;
  fruitList.appendChild(li);
});
```

화면 1. 사과 / 2. 바나나 / 3. 포도

반복문 밖에서 li를 한 번만 만들면 자리만 옮겨져 하나만 남습니다.

문제 10

```
const products = [
  { name: "아메리카노", price: 4000 },
  { name: "케이크", price: 6000 },
];
const cardArea = document.querySelector("#cardArea");

products.forEach(({ name, price }) => {
  const div = document.createElement("div");
  div.classList.add("card");
  div.textContent = `${name} ${price}원`;
  cardArea.appendChild(div);
});
```

화면 테두리가 있는 카드 두 개

콜백 매개변수에서 구조분해를 썼습니다. (product) ⇒ product.name 도 맞습니다.

문제 11

```
document.querySelector("#removeMe").remove();
```

화면 해당 항목이 사라집니다

```
console.log(document.querySelector("#removeMe"));
```

F12 콘솔 null

문제 12

```
const nameInput = document.querySelector("#nameInput");
console.log(nameInput.value);
```

F12 콘솔 이서연

input 은 textContent 가 아니라 value 입니다.

문제 13

문제 12에서 만든 nameInput 을 그대로 씁니다. 문제 12를 document.querySelector(...) 한 줄로 풀었다면 여기서 nameInput 을 다시 만들어야 합니다.

```
const price = Number(document.querySelector("#priceInput").value);
const count = Number(document.querySelector("#countInput").value);
const size = document.querySelector("#sizeSelect").value;
const ice = document.querySelector("#iceCheck").checked;

const total = price * count;

document.querySelector("#result").textContent =
  `${nameInput.value}님 / ${size} 사이즈 / ${total}원 / ${ice ? "얼음 추가" : "얼음 없음"}`;
```

화면 이서연님 / L 사이즈 / 18000원 / 얼음 추가

```
console.log(total);
```

F12 콘솔 18000

Number 를 빼먹으면 "4500" * "4" 는 우연히 18000 이 되지만, + 를 쓰는 순간 "45004" 가 되어 무너집니다. 습관적으로 Number 를 붙이세요.

문제 14

```
const stockList = [
  { name: "아메리카노", stock: 0 },
  { name: "라떼", stock: 5 },
```

```

{ name: "라떼", stock: 5 },
  { name: "케이크", stock: 2 },
  { name: "쿠키", stock: 0 },
];
const challengeList = document.querySelector("#challengeList");

stockList
  .filter(({ stock }) => stock > 0)
  .forEach(({ name, stock }) => {
    const li = document.createElement("li");
    li.textContent = `${name} (재고 ${stock}개)`;
    challengeList.appendChild(li);
  });

```

화면 라떼 (재고 5개) / 케이크 (재고 2개)

filter 로 먼저 걸러 내고 forEach 로 붙였습니다. forEach 안에서 if 로 건너뛰어도 결과는 같습니다.

```

stockList.forEach((item) => {

  if (item.stock === 0) return;
  ...

});

```

다른 방법 · innerHTML 과 map 을 쓰면 더 짧습니다.

```
challengeList.innerHTML = stockList
```

```
.filter((p) => p.stock > 0).map((p) => <li>${p.name} (재고 ${p.stock}개)</li>)
```

```
.join("");
```

문제 15

```

const nothing = document.querySelector("#존재하지않는아이디");
nothing.textContent = "안녕";

```

이 코드는 검사에서 걸립니다

왜:

querySelector 는 못 찾으면 null 을 돌려줍니다.

null 에는 textContent 라는 속성이 없으므로 값을 넣을 수 없습니다.

이 에러가 나는 이유 두 가지:

- 1) 선택자 오타 - # 이나 . 을 빠뜨렸거나 이름이 틀렸다
- 2) script 위치 - script 가 head 에 있어서 아직 그 태그가 만들어지지 않았다
(해결: script 를 body 맨 아래에 두거나 defer 를 붙인다)

DOM 을 다룰 때 가장 많이 보는 에러입니다. 이 두 가지부터 확인하세요.

TypeError: Cannot set properties of null (setting 'textContent')

11단원 연습문제 정답 이벤트

RUN 브라우저로 열고 직접 눌러 보세요. F12 → Console 도 함께

문제 1

```
const out1 = document.querySelector("#out1");

document.querySelector("#btn1").addEventListener("click", () => {
  out1.textContent = "눌렀습니다!";
});
```

콜백에 괄호를 붙이면 안 됩니다. 붙이면 페이지를 열 때 한 번 실행되고 끝입니다.

문제 2

```
const out2 = document.querySelector("#out2");

let count = 0; // * 콜백 '밖' 에 만들어야 값이
유지됩니다 document.querySelector("#btn2").addEventListener("click", () => {
  count++;
  out2.textContent = count;
});
```

콜백 안에 `let count = 0` 을 두면 누를 때마다 0으로 되돌아가 항상 1이 됩니다.

문제 3

```
const out3 = document.querySelector("#out3");
const btn3 = document.querySelector("#btn3");

let isOn = false;

btn3.addEventListener("click", () => {
  isOn = !isOn; // 뒤집기 (02단원)
  out3.classList.toggle("on", isOn); // 조건에 따라 붙였다 떼기 (10단원)
  out3.textContent = isOn ? "켜짐" : "꺼짐"; // 삼항 연산자 (03단원)
});
```

문제 4·5

```
const out4 = document.querySelector("#out4");

document.querySelectorAll(".menuBtn").forEach(btn => {
  btn.addEventListener("click", (e) => {
    const name = e.target.textContent;
    const price = Number(e.target.dataset.price); // 문자열이므로 Number 필수

    out4.textContent = `${name}를 골랐습니다`;
    console.log(price * 3);
  });
});
```

F12 콘솔 아메리카노를 누르면 12000, 케이크를 누르면 18000

버튼은 두 개인데 콜백은 하나입니다. e.target 으로 어느 것이 눌렸는지 압니다. dataset 값은 항상 문자열이라 Number 를 빼면 "4000""4000""4000" 이 됩니다.

문제 6

```
const keyInput = document.querySelector("#keyInput");
const out6 = document.querySelector("#out6");

keyInput.addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    out6.textContent = `입력한 값: ${keyInput.value}`;
  }
});
```

e.key 는 대소문자를 구분합니다. "enter" 가 아니라 "Enter" 입니다.

문제 7

```
const out7 = document.querySelector("#out7");

document.querySelector("#myLink").addEventListener("click", (e) => {
  e.preventDefault(); // 링크의 기본 동작(페이지 이동)을 막습니다
  out7.textContent = "이동을 막았습니다";
});
```

매개변수 (e) 를 안 적으면 ReferenceError 가 납니다. 괄호를 빼고 e.preventDefault 라고만 쓰면 아무 일도 일어나지 않습니다.

문제 8·9

두 문제 모두 같은 입력칸을 다루므로 콜백 하나에서 함께 처리했습니다. 문제 8과 9를 따로 addEventListener 로 붙여도 맞습니다. 둘 다 실행됩니다.

```
const nameInput = document.querySelector("#nameInput");
const out8 = document.querySelector("#out8");
const out9 = document.querySelector("#out9");

nameInput.addEventListener("input", (e) => {
  const value = e.target.value;

  out8.textContent = value || "여기에 실시간으로";
  out9.textContent = `${value.length}글자`;
});
```

하나의 콜백에서 두 곳을 함께 갱신했습니다. 따로 addEventListener 를 두 번 붙여도 됩니다. 둘 다 실행됩니다. 빈 값일 때 안내 문구로 되돌리려고 || 를 썼습니다. (02단원)

문제 10

```
const agreeCheck = document.querySelector("#agreeCheck");
const out10 = document.querySelector("#out10");

agreeCheck.addEventListener("change", () => {
  out10.textContent = `동의: ${agreeCheck.checked ? "함" : "안 함"}`;
});
```

체크박스는 .value 가 아니라 .checked 입니다. value 는 항상 "on" 입니다.

문제 11

```
const sizeSelect = document.querySelector("#sizeSelect");
const out11 = document.querySelector("#out11");

sizeSelect.addEventListener("change", (e) => {
  out11.textContent = `사이즈: ${e.target.value}`;
});
```

select 는 고른 option 의 value 를 줍니다. 화면 글자가 아닙니다.

문제 12

```
const loginForm = document.querySelector("#loginForm");
const idInput = document.querySelector("#idInput");
const out12 = document.querySelector("#out12");

loginForm.addEventListener("submit", (e) => {
  e.preventDefault(); // * 이 줄이 없으면 페이지가 새로고침됩니다
  out12.textContent = `${idInput.value}님 환영합니다`;
});
```

버튼의 click 이 아니라 form 의 submit 에 붙였습니다. 그래야 버튼을 눌러도, 입력칸에서 Enter 를 쳐도 똑같이 동작합니다.

문제 13

```
const colorList = document.querySelector("#colorList");
const out13 = document.querySelector("#out13");

colorList.addEventListener("click", (e) => {
```

ul 의 빈 곳을 눌렀을 때는 무시합니다

```
  if (e.target.tagName !== "LI") {
    return;
  }
  out13.textContent = `고른 색: ${e.target.textContent}`;
});
```

li 가 세 개인데 콜백은 하나입니다. 버블링 덕분에 ul 이 대신 받습니다. tagName 은 항상 대문자입니다. "li" 로 비교하면 영원히 false 입니다.

문제 14

```
const memoForm = document.querySelector("#memoForm");
const memoInput = document.querySelector("#memoInput");
const memoList = document.querySelector("#memoList");
const errorMsg = document.querySelector("#errorMsg");
```

```
memoForm.addEventListener("submit", (e) => {
  e.preventDefault(); // * 없으면 페이지가 새로고침되어 목록이 날아갑니다 const value
= memoInput.value.trim(); // 공백만 친 경우도 걸러냅니다 if (!value) {
  errorMsg.textContent = "메모를 입력해 주세요";
  memoInput.focus();
  return; // * return 이 없으면 빈 항목이 추가됩니다
}

errorMsg.textContent = "";

const li = document.createElement("li");
li.textContent = value; // 사용자 입력이므로 textContent (innerHTML 금지)
memoList.appendChild(li);

memoInput.value = "";
memoInput.focus();
});
```

form 으로 묶었기 때문에 버튼 클릭과 Enter 가 둘 다 동작합니다.

문제 15

```
const itemList = document.querySelector("#itemList");

itemList.addEventListener("click", (e) => {
```

삭제 버튼이 눌린 경우만 처리

```
if (!e.target.classList.contains("deleteBtn")) {
  return;
}

const li = e.target.closest("li"); // 버튼이 아니라 항목 전체를 지워야 합니다
li.remove();
console.log("삭제됨. 남은 항목:", itemList.children.length);
});
```

부모인 ul 하나에만 붙였습니다. 버블링 덕분에 자식의 클릭이 여기로 올라옵니다. 이렇게 하면 나중에 li 를 추가해도 삭제가 그대로 동작합니다.

판별 없이 e.target.remove() 를 하면 글자를 눌렀을 때 글자만 사라집니다. closest("li") 대신 parentElement 를 쓰면 구조가 바뀔 때 깨집니다.

문제 16

```
document.querySelector("#btn1").addEventListener("click", console.log("눌림"));
```

왜 이렇게 되나: `console.log("눌림")` 은 '함수' 가 아니라 '함수를 실행한 결과' 입니다. 그래서 페이지를 여는 순간 즉시 실행되어 "눌림" 이 한 번 찍히고, 그 결과인 `undefined` 가 이벤트에 등록됩니다. `undefined` 는 함수가 아니므로 버튼을 눌러도 아무 일이 없습니다.

고치는 방법:

```
addEventListener("click", () => console.log("눌림"));
```

08단원에서 배운 "콜백에 괄호를 붙이지 않는다" 가 이벤트에서 특히 자주 나옵니다. "버튼을 눌러도 반응이 없다" 면 이 실수부터 의심하세요.

12단원 연습문제 정답 비동기 fetch

RUN 브라우저로 열기. 인터넷 연결 필요. F12 → Console 필수

```
const BASE_URL = "https://jsonplaceholder.typicode.com";

const out1 = document.querySelector("#out1");
const out3 = document.querySelector("#out3");
const out4 = document.querySelector("#out4");
const out6 = document.querySelector("#out6");
const out8 = document.querySelector("#out8");
const out9 = document.querySelector("#out9");
const out11 = document.querySelector("#out11");
const out13 = document.querySelector("#out13");
const out14 = document.querySelector("#out14");
```

문제 1

```
document.querySelector("#timerBtn").addEventListener("click", () => {
  out1.textContent = "기다리는 중...";

  setTimeout(() => {
    out1.textContent = "2초가 지났습니다";
  }, 2000);
});
```

1000 = 1초입니다. 2초는 2000 입니다. `setTimeout(fn(), 2000)` 처럼 괄호를 붙이면 즉시 실행되어 버립니다.

문제 2

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

F12 콘솔

```
A
C
B
```

0초인데도 B 가 마지막입니다. setTimeout 은 “지금 하던 일을 다 끝낸 다음에” 실행하겠다는 예약입니다. 시간이 0이어도 ‘나중’ 입니다.

문제 3

```
let countTimerId = null;

document.querySelector("#countBtn").addEventListener("click", () => {
  if (countTimerId !== null) return; // 두 번 눌러도 두 개가 돌지 않게 let count = 3;
  out3.textContent = count;

  countTimerId = setInterval(() => {
    count--;

    if (count <= 0) {
      out3.textContent = "끝!";
      clearInterval(countTimerId); // * 없으면 영원히 돌니다
      countTimerId = null;
      return;
    }

    out3.textContent = count;
  }, 1000);
});
```

문제 4

```
function delay(ms, shouldFail = false) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldFail) {
        reject(new Error("일부러 실패했습니다"));
      } else {
        resolve("작업 완료");
      }
    }, ms);
  });
}

document.querySelector("#promiseBtn").addEventListener("click", () => {
  out4.className = "output";
  out4.textContent = "진행 중...";
});
```



```
out4.textContent = "1초 뒤 완료";
});
});
```

문제 5

```
document.querySelector("#rejectBtn").addEventListener("click", () => {
  out4.className = "output";
  out4.textContent = "진행 중...";

  delay(1000, true)
    .then(() => {
      out4.textContent = "여기는 실행되지 않습니다";
    })
    .catch((error) => {
      out4.textContent = `오류: ${error.message}`;
      out4.className = "output error";
    });
});
```

reject 로 넘긴 Error 객체가 catch 의 매개변수로 들어옵니다. error.message 로 메시지를 꺼냅니다.

문제 6

```
document.querySelector("#oneBtn").addEventListener("click", () => {
  out6.textContent = "불러오는 중...";

  fetch(`${BASE_URL}/posts/1`)
    .then((res) => res.json()) // * return 을 빠뜨리지 않도록 한 줄로 씁니다
    .then((data) => {
      out6.textContent = data.title;
    })
    .catch(() => {
      out6.textContent = "가져오지 못했습니다";
    });
});
```

then 이 두 번인 이유: 응답 받기 → 내용 읽기. 둘 다 시간이 걸립니다.

문제 7

```
const postList = document.querySelector("#postList");

document.querySelector("#listBtn").addEventListener("click", () => {
  postList.innerHTML = "<li>불러오는 중...</li>";

  fetch(`${BASE_URL}/posts?_limit=5`)
    .then((res) => res.json())
    .then((posts) => {
      postList.innerHTML = posts
        .map((post) => `<li>${post.id}. ${post.title}</li>`)
        .join("");
    })
    .catch(() => {
      postList.innerHTML = "<li>불러오지 못했습니다</li>";
    });
});
```

배열 → map → join("") → innerHTML 이 목록 그리기의 기본 패턴입니다. join 을 빠뜨리면 항목 사이에 쉼표가 보입니다.

문제 8

```
document.querySelector("#awaitBtn").addEventListener("click", async () => {
  out8.textContent = "불러오는 중...";

  const res = await fetch(`${BASE_URL}/posts/1`);
  const data = await res.json();

  out8.textContent = data.title;
});
```

콜백에 async 를 붙이면 그 안에서 await 를 쓸 수 있습니다. await 를 두 번 쓰는 것이 then 두 번과 짝입니다.

문제 9

```
document.querySelector("#seqBtn").addEventListener("click", async () => {
  out9.textContent = "불러오는 중...";

  const postRes = await fetch(`${BASE_URL}/posts/1`);
  const post = await postRes.json();
```

앞의 결과(post.userId)가 있어야 다음 요청을 할 수 있습니다

```
const userRes = await fetch(`${BASE_URL}/users/${post.userId}`);
const user = await userRes.json();

out9.textContent = `${post.title} - ${user.name}`;
console.log("작성자:", user.name);
});
```

앞 결과가 필요하므로 Promise.all 을 쓸 수 없습니다. 순서대로 await 합니다. then 으로 쓰면 마지막 단계에서 post 를 쓸 수 없어 불편합니다.

문제 10

```
const allList = document.querySelector("#allList");

document.querySelector("#allBtn").addEventListener("click", async () => {
  allList.innerHTML = "<li>불러오는 중...</li>";

  const posts = await Promise.all([
    fetch(`${BASE_URL}/posts/1`).then((r) => r.json()),
    fetch(`${BASE_URL}/posts/2`).then((r) => r.json()),
    fetch(`${BASE_URL}/posts/3`).then((r) => r.json()),
  ]);

  allList.innerHTML = "";
  posts.forEach((post) => {
    const li = document.createElement("li");
    li.textContent = `${post.id}. ${post.title}`;
    allList.appendChild(li);
  });
});
```

서로 상관없는 요청이므로 동시에 보냅니다. 하나씩 await 하면 3배 오래 걸립니다.

문제 11

```
document.querySelector("#tryBtn").addEventListener("click", async () => {
  out11.className = "output";
  out11.textContent = "불러오는 중...";

  try {
    const res = await fetch("https://없는주소-abcxyz.example/data");
    const data = await res.json();
    out11.textContent = JSON.stringify(data);
  } catch (error) {
    out11.textContent = "요청에 실패했습니다";
    out11.className = "output error";
    console.log("에러:", error.message);
  }
});
```

서버에 연결 자체가 안 되면 fetch 가 실패하고 catch 로 갑니다. await 를 try '안' 에 뒤에 잡힙니다. 밖에 두면 안 잡힙니다.

문제 12

```
document.querySelector("#okBtn").addEventListener("click", async () => {
  const res = await fetch(`${BASE_URL}/posts/9999`);
  console.log(res.ok, res.status);
```

```
F12 콘솔    false 404
```

```
});
```

404 는 '서버가 대답을 한 것' 이라 fetch 입장에서는 성공입니다. catch 로 가지 않으므로 res.ok 를 직접 확인해야 합니다.

문제 13

```
const goodBtn = document.querySelector("#goodBtn");

goodBtn.addEventListener("click", async () => {
  goodBtn.disabled = true; // 1) 버튼 잠금
  out13.className = "output";
  out13.textContent = "불러오는 중...";
```

13단원 ·

13단원 종합 01 정답 성적 관리 프로그램

RUN node 종합01_성적관리_정답.js

```
const students = [
  { name: "김민준", scores: [90, 85, 70] },
  { name: "이서연", scores: [95, 88, 92] },
  { name: "박지훈", scores: [60, 45, 80] },
  { name: "최유진", scores: [75, 82, 78] },
  { name: "정하늘", scores: [50, 55, 48] },
];

const subjects = ["국어", "영어", "수학"];

console.log("===== 성적 관리 프로그램 =====");
```

출력 ===== 성적 관리 프로그램 =====

문제 1

```
console.log(`전체 ${students.length}명`);
```

출력 전체 5명

문제 2

```
function getTotal(scores) {
  return scores.reduce((acc, score) => acc + score, 0);
}

console.log(`김민준 총점 ${getTotal(students[0].scores)}`);
```

출력 김민준 총점 245

reduce 의 시작값 0 을 빠뜨리면 빈 배열에서 에러가 납니다.

문제 3

```
function getAverage(scores) {  
  return getTotal(scores) / scores.length;  
}  
  
console.log(`이서연 평균 ${getAverage(students[1].scores).toFixed(1)}`);
```

출력 이서연 평균 91.7

함수는 숫자를 그대로 돌려주고, 보기 좋게 다듬는 일은 출력할 때 합니다. getAverage 가 toFixed 결과 (문자열)를 돌려주면 문제 4에서 비교를 못 합니다.

문제 4

```
function getGrade(average) {  
  if (average >= 90) return "A";  
  if (average >= 80) return "B";  
  if (average >= 70) return "C";  
  if (average >= 60) return "D";  
  return "F";  
}  
  
console.log(`95점 → ${getGrade(95)}`);
```

출력 95점 → A

코드	출력
console.log(`72점 → \${getGrade(72)}`);	72점 → C
console.log(`40점 → \${getGrade(40)}`);	40점 → F

큰 수부터 검사해야 합니다. 60을 먼저 쓰면 95점도 D가 됩니다. 조기 반환을 쓰면 else 가 필요 없습니다.

문제 5

```
students.forEach(({ name, scores }) => {
  const total = getTotal(scores);
  const average = getAverage(scores);
  const grade = getGrade(average);

  console.log(`${name} ${total}점 평균 ${average.toFixed(1)} (${grade})`);
});
```

```
출력      김민준 245점 평균 81.7 (B)
          이서연 275점 평균 91.7 (A)
          박지훈 185점 평균 61.7 (D)
          최유진 235점 평균 78.3 (C)
          정하늘 153점 평균 51.0 (F)
```

콜백 매개변수에서 구조분해를 쓰면 student.name 을 매번 안 써도 됩니다.

문제 6

```
const passed = students.filter(({ scores }) => getAverage(scores) >= 60).map(({ name }) => name);

console.log(passed);
```

```
출력      [ '김민준', '이서연', '박지훈', '최유진' ]
```

filter 로 고르고 map 으로 이름만 뽑았습니다.

문제 7

첫 번째 학생을 일단 1등으로 두고, 더 높은 사람이 나오면 갈아치웁니다.

```
let top = students[0];

students.forEach((student) => {
  if (getAverage(student.scores) > getAverage(top.scores)) {
    top = student;
  }
});

console.log(`1등: ${top.name} (${getAverage(top.scores).toFixed(1)})`);
```

출력 1등: 이서연 (91.7)

다른 방법 · reduce 로도 됩니다.

```
const top2 = students.reduce((a, b) =>
  getAverage(a.scores) >= getAverage(b.scores) ? a : b
);
```

문제 8

```
subjects.forEach((subject, index) => {
```

모든 학생의 그 과목 점수만 모읍니다

```
const scoresOfSubject = students.map((student) => student.scores[index]);
const average = getAverage(scoresOfSubject);

console.log(`${subject} 평균 ${average.toFixed(1)}`);
});
```

출력 국어 평균 74.0
 영어 평균 71.0
 수학 평균 73.6

map 으로 세로 방향(같은 과목)의 점수를 뽑아냈습니다. scores[index] 에서 index 는 과목의 순서입니다. subjects 와 scores 의 순서가 같다는 약속이 있어야 이 방법이 통합니다.

문제 9

```
const gradeCount = students.reduce((acc, { scores }) => {
  const grade = getGrade(getAverage(scores));
  acc[grade] = (acc[grade] ?? 0) + 1;
  return acc; // 이 return 을 빠뜨리면 다음 바퀴의 acc 가 undefined 가 됩니다
}, {});

console.log(gradeCount);
```

출력 { B: 1, A: 1, D: 1, C: 1, F: 1 }

시작값이 빈 객체 {} 입니다. 순서는 학생 순서대로 처음 등장한 등급 순입니다.

문제 10

원본을 지켜야 하므로 [...배열] 로 복사한 뒤 정렬합니다.

```
const ranked = [...students].sort((a, b) => getAverage(b.scores) -
getAverage(a.scores));

ranked.forEach((student, index) => {
  console.log(`${index + 1}위
  ${student.name} ${getAverage(student.scores).toFixed(1)}`);
});
```

```
출력      1위 이서연 91.7
          2위 김민준 81.7
          3위 최유진 78.3
          4위 박지훈 61.7
          5위 정하늘 51.0
```

```
console.log(students[0].name);
```

```
출력      김민준
```

원본은 그대로입니다. 복사 없이 sort 했다면 순서가 망가졌을 겁니다.

높은 순(내림차순)이므로 b - a 입니다. a - b 로 쓰면 꼴찌부터 나옵니다.

문제 11

```
students.forEach(({ name, scores }) => {
```

60점 미만인 과목만 "국어(50)" 형태로 모읍니다

```
const failed = [];

scores.forEach((score, index) => {
  if (score < 60) {
    failed.push(`${subjects[index]}(${score})`);
  }
});
```

하나라도 있으면 출력

```
if (failed.length > 0) {
  console.log(`${name}: ${failed.join(", ")}`);
}
});
```

출력 박지훈: 영어(45)
 정하늘: 국어(50), 영어(55), 수학(48)

박지훈의 국어는 60점입니다. $60 < 60$ 은 `false` 이므로 포함되지 않습니다. '미만' 과 '이하' 를 구별하세요.

다른 방법 · `filter` 와 `map` 을 쓰면 이렇게도 됩니다.

```
const failed = scores
```

```
.map((score, i) => ({ subject: subjects[i], score })) .filter(({ score }) => score < 60)
```

```
.map(({ subject, score }) => `${subject}(${score})`);
```

정리 — 이 파일에서 쓴 것들

- 01단원 변수, 자료형, `toFixed` 는 문자열이라는 점
- 02단원 산술 연산자, 템플릿 리터럴
- 03단원 조건문, 조기 반환으로 등급 나누기
- 04단원 반복문 (중첩 반복문 – 문제 11)
- 05단원 함수로 쪼개기, 함수 안에서 다른 함수 부르기
- 06단원 배열, `length`, `join`, 원본을 지키는 `slice`
- 07단원 객체, 객체 배열
- 08단원 `reduce`(합계), `filter·map`(체이닝), `forEach`, `sort`(비교 함수)
- 09단원 구조분해 (`{ name, scores }`)

1 계산은 함수로 만들어 이름을 붙인다. `getTotal`, `getAverage`, `getGrade`

2 보기 좋게 다듬는 일(`toFixed`)은 '출력할 때' 한다. 계산 중에 하지 않는다.

13단원 ·

13단원 종합 02 정답 장바구니

RUN node 종합02_장바구니_정답.js

```
const products = [
  { id: 1, name: "아메리카노", price: 4000, stock: 10 },
  { id: 2, name: "카페라떼", price: 4500, stock: 3 },
  { id: 3, name: "케이크", price: 6000, stock: 0 },
  { id: 4, name: "쿠키", price: 3000, stock: 7 },
];

const cart = [
  { id: 1, count: 2 },
  { id: 4, count: 3 },
];

console.log("===== 장바구니 =====");
```

출력 ===== 장바구니 =====

문제 1

```
console.log(products.filter(({ stock }) => stock > 0).map(({ name }) => name));
```

출력 ['아메리카노', '카페라떼', '쿠키']

filter 로 고르고 map 으로 이름만 뽑는 체이닝입니다.

문제 2

```
function findProduct(id) {
  return products.find((product) => product.id === id);
}

console.log(findProduct(2));
```

출력 { id: 2, name: '카페라떼', price: 4500, stock: 3 }

```
console.log(findProduct(99));
```

출력 undefined

find 는 못 찾으면 `undefined` 를 돌려줍니다. filter 였다면 빈 배열이 나옵니다. 결과를 바로 쓰기 전에 확인하는 습관을 들이세요. `findProduct(99)?.name`

문제 3

```
cart.forEach(({ id, count }) => {  
  const product = findProduct(id);  
  console.log(`${product.name} x${count} = ${product.price * count}원`);  
});
```

출력 아메리카노 x2 = 8000원
쿠키 x3 = 9000원

장바구니에는 id 와 개수만 있고 이름·가격은 상품 목록에 있습니다. 이렇게 데이터를 나눠 두면 가격이 바뀌어도 한 곳만 고치면 됩니다.

문제 4

```
function getCartTotal(cartItems) {  
  return cartItems.reduce((acc, { id, count }) => {  
    const product = findProduct(id);  
    return acc + product.price * count;  
  }, 0);  
}  
  
console.log(`총액 ${getCartTotal(cart)}원`);
```

출력 총액 17000원

reduce 안에서도 구조분해를 쓸 수 있습니다. 콜백에 중괄호를 썼으므로 `return` 을 반드시 적어야 합니다.

문제 5

```
function addToCart(cartItems, id, count) {  
  const exists = cartItems.find((item) => item.id === id);  
  
  if (exists) {
```

이미 있으면: 그 항목만 개수를 늘린 새 객체로 바꾼 새 배열

```
return cartItems.map((item) => (item.id === id ? { ...item, count: item.count + count } : item));
}
```

없으면: 뒤에 새 항목을 붙인 새 배열

```
return [...cartItems, { id, count }];
}
```

```
console.log(addToCart(cart, 2, 1));
```

```
출력      [ { id: 1, count: 2 }, { id: 4, count: 3 }, { id: 2, count: 1 } ]
```

```
console.log(addToCart(cart, 1, 3));
```

```
출력      [ { id: 1, count: 5 }, { id: 4, count: 3 } ]
```

```
console.log(cart);
```

```
출력      [ { id: 1, count: 2 }, { id: 4, count: 3 } ]
```

원본이 그대로입니다.

push 나 item.count++ 를 쓰면 원본이 바뀝니다. map + 스프레드로 “그 항목만 바꾼 새 배열” 을 만드는 것이 핵심 패턴입니다. { id, count } 는 { id: id, count: count } 의 줄임 표현입니다.

문제 6

```
function removeFromCart(cartItems, id) {
  return cartItems.filter((item) => item.id !== id);
}
```

```
console.log(removeFromCart(cart, 1));
```

```
출력      [ { id: 4, count: 3 } ]
```

filter 는 새 배열을 돌려주므로 원본이 안 바뀝니다. splice 를 쓰면 원본이 바뀝니다.

문제 7

```
function hasEnoughStock(cartItems) {  
  return cartItems.every(({ id, count }) => {  
    const product = findProduct(id);  
    return product.stock >= count;  
  });  
}
```

```
console.log(hasEnoughStock(cart));
```

출력 true

```
const tooMany = addToCart(cart, 4, 7); // 쿠키 3 + 7 = 10개, 재고는 7개  
console.log(hasEnoughStock(tooMany));
```

출력 false

every 는 “전부 만족해야 true” 입니다. 하나라도 넘치면 false 입니다. some 을 쓰면 “하나라도 넘치는가” 를 물을 수 있습니다.

문제 8

```
function getFinalPrice(total) {  
  if (total >= 30000) {  
    return Math.round(total * 0.9);  
  }  
  return total;  
}
```

```
console.log(getFinalPrice(17000));
```

출력 17000

```
console.log(getFinalPrice(50000));
```

출력 45000

소수 오차가 생길 수 있으므로 돈 계산은 Math.round 로 마무리합니다.

문제 9

```
function printReceipt(cartItems) {
  console.log("-----");

  cartItems.forEach(({ id, count }) => {
    const product = findProduct(id);
    console.log(`${product.name} x${count} = ${product.price * count}원`);
  });

  console.log("-----");

  const total = getCartTotal(cartItems);
  console.log(`합계 ${total}원`);
  console.log(`결제 ${getFinalPrice(total)}원`);
}

printReceipt(cart);
```

출력

```
-----
아메리카노 x2 = 8000원
쿠키 x3 = 9000원
-----
합계 17000원
결제 17000원
```

작은 함수를 조립해 큰 함수를 만들었습니다. 각 함수를 따로 확인할 수 있어 버그를 찾기 쉽습니다.

문제 10

```
function addToCartSafe(cartItems, id, count) {
  const product = findProduct(id);
```

없는 상품이면 그대로

```
if (!product) {
  console.log("그런 상품이 없습니다");
  return cartItems;
}
```

품절이면 그대로 (조기 반환)

```

if (product.stock === 0) {
  console.log(`${product.name}은(는) 품절입니다`);
  return cartItems;
}

return addToCart(cartItems, id, count);
}

console.log(addToCartSafe(cart, 3, 1)); // 케이크는 재고 0

```

```

출력      케이크은(는) 품절입니다
          [ { id: 1, count: 2 }, { id: 4, count: 3 } ]

```

`console.log` 가 함수 안에서 먼저 실행되고, 돌려준 값이 밖에서 찍힙니다. 그래서 품절 메시지가 배열보다 먼저 나옵니다.

조기 반환을 쓰면 “안 되는 경우” 를 위에서 걸러내고 아래는 정상 흐름만 남아 읽기 좋아집니다.

정리 — 이 파일에서 쓴 것들

```

03단원  조건문, 조기 반환, 삼항 연산자
05단원  함수로 쪼개고 조립하기
06단원  배열
07단원  객체, 객체 배열, 데이터를 나눠 두기(상품 목록 ↔ 장바구니)
08단원  find, filter, map, reduce, every
09단원  구조분해, 스프레드로 '새 배열 / 새 객체' 만들기

```

이 파일에서 가장 중요한 것: 원본을 직접 고치지 않고 항상 새 배열·새 객체를 만들었습니다.

```

담기 → map 으로 그 항목만 바꾼 새 배열, 또는 [...cart, 새항목]
빼기 → filter 로 걸러낸 새 배열

```

`push` 나 `item.count++` 를 쓰면 원본이 바뀝니다. React 를 배우면 이 방식이 선택이 아니라 규칙이 됩니다.

13단원 ·

13단원 종합 03 정답 할 일 목록

RUN 브라우저로 열고 F12 → Console 탭

핵심 구조 todos 배열이 '진짜 데이터' 이고, 화면은 그 그림자입니다. 무슨 일이 생기면 배열만 고치고 `render()` 를 다시 부릅니다.

```
let todos = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "설거지", done: true },
];

let nextId = 3;
let currentFilter = "all";

const todoForm = document.querySelector("#todoForm");
const todoInput = document.querySelector("#todoInput");
const todoList = document.querySelector("#todoList");
const errorMsg = document.querySelector("#errorMsg");
const status = document.querySelector("#status");
const clearBtn = document.querySelector("#clearBtn");
```

문제 1

```
function getVisibleTodos() {
  if (currentFilter === "active") {
    return todos.filter((todo) => !todo.done);
  }
  if (currentFilter === "done") {
    return todos.filter((todo) => todo.done);
  }
  return todos; // "all"
}
```

조기 반환을 쓰면 else 없이 깔끔합니다. switch 로 써도 됩니다.

문제 2

```
function render() {
  const visible = getVisibleTodos();
```

```

todoList.innerHTML = ""; // * 먼저 비우지 않으면 그럴 때마다 쌓입니다 if
(visible.length === 0) {
    const empty = document.createElement("li");
    empty.classList.add("empty");
    empty.textContent = "항목이 없습니다";
    todoList.appendChild(empty);
    updateStatus();
    return; // 조기 반환
}

visible.forEach(({ id, text, done }) => {
    const li = document.createElement("li");
    li.dataset.id = id;
    if (done) {
        li.classList.add("done");
    }

    const span = document.createElement("span");
    span.classList.add("text");
    span.textContent = text; // 사용자 입력이므로 textContent
    const btn = document.createElement("button");
    btn.classList.add("deleteBtn");
    btn.textContent = "삭제";

    li.appendChild(span);
    li.appendChild(btn);
    todoList.appendChild(li);
});

updateStatus();

```

이 줄은 문제 요구사항에는 없습니다. 화면이 몇 번 다시 그려지는지 눈으로 보려고 넣은 것이니, 없어도 정답입니다.

```

console.log("render:", visible.length, "개 표시");
}

```

innerHTML + map + join 으로 써도 됩니다. 다만 사용자가 입력한 글을 그대로 넣게 되므로 createElement + textContent 쪽이 안전합니다.

문제 3

```
function updateStatus() {
  const total = todos.length;
  const doneCount = todos.filter((todo) => todo.done).length;
  const activeCount = total - doneCount;

  status.textContent = `할 일 ${total}개 (완료 ${doneCount}개, 남은 일 ${activeCount}개)`;
}
```

필터와 상관없이 전체 todos 를 기준으로 셉니다.

문제 4

```
todoForm.addEventListener("submit", (e) => {
  e.preventDefault(); // * 없으면 새로고침되어 데이터가 날아갑니다 const text =
  todoInput.value.trim(); // 공백만 친 경우도 걸러냅니다 if (!text) {
    errorMsg.textContent = "할 일을 입력해 주세요";
    todoInput.focus();
    return;
  }

  if (todos.some((todo) => todo.text === text)) {
    errorMsg.textContent = "이미 있는 항목입니다";
    return;
  }

  errorMsg.textContent = "";
```

원본을 직접 바꾸지 않고 새 배열을 만들어 넣습니다

```
todos = [...todos, { id: nextId, text, done: false }];
nextId++;

todoInput.value = "";
todoInput.focus();
render();

console.log("추가:", text);
});
```

{ id: nextId, text, done: false } 에서 text 는 text: text 의 줄임입니다. some 은 "하나라도 만족하는가"라서 중복 검사에 딱 맞습니다.

문제 5

```
todoList.addEventListener("click", (e) => {
  const li = e.target.closest("li");
  if (!li || !li.dataset.id) {
    return; // 빈 곳이나 "항목이 없습니다" 를 눌렀을 때
  }

  const id = Number(li.dataset.id); // dataset 값은 문자열이므로 변환
  if (e.target.classList.contains("deleteBtn")) {
    todos = todos.filter((todo) => todo.id !== id);
    render();
    console.log("삭제:", id);
    return;
  }

  if (e.target.classList.contains("text")) {
```

그 항목만 done 을 뒤집은 새 객체로 바꾼 새 배열

```
    todos = todos.map((todo) => (todo.id === id ? { ...todo, done: !todo.done } :
    todo));
    render();
    console.log("완료 토글:", id);
  }
});
```

부모인 ul 하나에만 붙였습니다. render 로 li 를 몇 번을 다시 만들어도 이벤트를 새로 붙일 필요가 없습니다. 이것이 위임을 쓰는 이유입니다.

Number 로 안 바꾸면 todo.id === "1" 이 되어 항상 false 입니다.

문제 6

```
document.querySelectorAll(".filterBtn").forEach((btn) => {
  btn.addEventListener("click", (e) => {
    currentFilter = e.target.dataset.filter;
```

모든 버튼에서 active 를 떼고, 눌린 것에만 붙입니다

```
document.querySelectorAll(".filterBtn").forEach((b) => b.classList.remove("active"));
e.target.classList.add("active");

render();
console.log("필터:", currentFilter);
});
});
```

문제 7

```
clearBtn.addEventListener("click", () => {
  const before = todos.length;
  todos = todos.filter((todo) => !todo.done);

  render();
  console.log(`완료 항목 ${before - todos.length}개 삭제`);
});
```

clearBtn 에 type="button" 을 준 이유: form 밖이라 상관없지만, form 안에 있었다면 type 을 안 쓸 경우 폼이 제출되어 버립니다.

문제 8

```
render();
```

F12 콘솔 render: 2 개 표시

이벤트는 '바뀔 때' 만 실행되므로 첫 화면은 직접 한 번 그려야 합니다. 이걸 빠뜨리면 "데이터는 있는데 화면이 비어 있는" 상태가 됩니다.

정리 — 이 파일에서 쓴 것들

```
03단원  조건문, 삼항 연산자
05단원  함수, 조기 반환
06단원  배열, 인덱스
07단원  객체, 객체 배열
08단원  콜백, map, filter, forEach, some
09단원  구조분해, 스프레드
10단원  createElement,.textContent, classList, dataset
11단원  submit, preventDefault, 이벤트 위임, closest
```

03~11단원이 이 한 화면에 들어 있습니다. 12단원(비동기·fetch)은 종합04에서 씁니다.

13단원 종합 04 정답 사용자 검색

RUN 브라우저로 열고 F12 → Console 탭

```
const API_URL = "https://jsonplaceholder.typicode.com/users";

let allUsers = [];
let sortByName = false;
let hasError = false; // 불러오기에 실패했는지 (render 에서 씁니다)
const searchInput = document.querySelector("#searchInput");
const loadBtn = document.querySelector("#loadBtn");
const sortBtn = document.querySelector("#sortBtn");
const message = document.querySelector("#message");
const userList = document.querySelector("#userList");
const detail = document.querySelector("#detail");
```

문제 1

```
async function loadUsers() {
  hasError = false; // 다시 시도할 때 이전 실패 표시를 지웁니다
  loadBtn.disabled = true;
  sortBtn.disabled = true;
  message.className = "loading";
  message.textContent = "불러오는 중...";

  try {
    const res = await fetch(API_URL);
```

★ fetch 는 404·500 을 실패로 치지 않습니다. 직접 확인해야 합니다.

```
if (!res.ok) {
  throw new Error(`서버 오류 ${res.status}`);
}

allUsers = await res.json();

message.className = "";
message.textContent = "";
render();

console.log("받은 사용자 수:", allUsers.length);
```

```

} catch (error) {
  hasError = true; // render 가 이 안내를 지우지 않도록 표시해 둡니다
  message.className = "error";
  message.textContent = "불러오지 못했습니다. 다시 시도해 주세요.";
  console.log("에러 내용:", error.message);
} finally {

```

★ 성공하든 실패하든 버튼은 반드시 풀어 줘야 합니다

```

    loadBtn.disabled = false;
    sortBtn.disabled = false;
  }
}

loadBtn.addEventListener("click", loadUsers);

```

문제 2

```

function getVisibleUsers() {
  const keyword = searchInput.value.trim().toLowerCase();

  let result = allUsers;

  if (keyword) {
    result = allUsers.filter(({ name, address }) => {
      const nameMatch = name.toLowerCase().includes(keyword);
      const cityMatch = address.city.toLowerCase().includes(keyword);
      return nameMatch || cityMatch;
    });
  }

  if (sortByName) {

```

★ [...result] 로 복사한 뒤 정렬해야 원본이 안 바뀝니다

```

    result = [...result].sort((a, b) => a.name.localeCompare(b.name));
  }

  return result;
}

```

양쪽 다 toLowerCase 를 해야 대소문자 구분 없이 찾을 수 있습니다. 문자열 정렬은 a - b 가 아니라 localeCompare 를 씁니다. (08단원 개념06)

문제 3

```
function render() {  
  const visible = getVisibleUsers();  
  
  userList.innerHTML = "";
```

불러오기 자체가 실패한 상태라면 그 안내를 덮어쓰지 않습니다. “데이터를 못 받았다” 와 “검색 결과가 없다” 는 전혀 다른 이야기입니다. 이 검사가 없으면, 실패한 뒤 검색창에 글자만 쳐도 에러 안내가 “검색 결과가 없습니다” 로 바뀌어 원인을 알 수 없게 됩니다.

```
if (hasError) {  
  return;  
}  
  
if (visible.length === 0) {  
  message.className = "";  
  message.textContent = "검색 결과가 없습니다";  
  return;  
}  
  
message.className = "";  
message.textContent = "";  
  
visible.forEach(({ id, name, email, address }) => {  
  const li = document.createElement("li");  
  li.dataset.id = id;  
  
  const nameDiv = document.createElement("div");  
  nameDiv.classList.add("name");  
  nameDiv.textContent = name;  
  
  const subDiv = document.createElement("div");  
  subDiv.classList.add("sub");  
  subDiv.textContent = `${address.city} · ${email}`;  
  
  li.appendChild(nameDiv);  
  li.appendChild(subDiv);  
  userList.appendChild(li);  
});  
  
console.log("표시:", visible.length, "명");  
}
```


서버에서 온 값이므로 innerHTML 이 아니라 textContent 로 넣습니다.

문제 4

```
searchInput.addEventListener("input", render);
```

input 이벤트는 글자가 바뀔 때마다 실행됩니다. 그래서 실시간 검색이 됩니다. render 에 괄호를 붙이면 안 됩니다. 함수를 넘기는 것입니다.

문제 5

```
sortBtn.addEventListener("click", () => {
  sortByName = !sortByName;
  sortBtn.textContent = sortByName ? "원래 순서" : "이름순 정렬";
  render();
  console.log("정렬:", sortByName);
});
```

문제 6

```
userList.addEventListener("click", (e) => {
  const li = e.target.closest("li");
  if (!li) return; // 목록의 빈 곳을 눌렀을 때 const id =
  Number(li.dataset.id); // dataset 은 문자열이므로 변환 const user = allUsers.find((u)
  => u.id === id);

  if (!user) {
    detail.textContent = "정보를 찾을 수 없습니다";
    return;
  }

  detail.textContent = `이름: ${user.name}
이메일: ${user.email}
전화: ${user.phone}
도시: ${user.address.city}
회사: ${user.company.name}`;

  console.log("선택:", user.name);
});
```

e.target 은 눌린 그 요소(div)일 수 있으므로 closest("li") 로 항목을 찾습니다. 백틱 안에서 엔터를 치면 줄바꿈이 되고, CSS 의 white-space: pre-line 덕분에 화면에서도 여러 줄로 보입니다.

문제 7

```
loadUsers();
```

페이지를 열자마자 한 번 부릅니다. 이벤트는 '무슨 일이 생겼을 때' 만 실행되므로 처음 한 번은 직접 불러야 합니다.

정리 — 이 파일에서 쓴 것들

```
05단원 함수, 조기 반환  
07단원 객체, 중첩 객체(address.city)  
08단원 filter, find, forEach, sort + localeCompare  
09단원 구조분해, 스프레드 복사  
10단원 createElement,.textContent, classList, dataset  
11단원 click / input 이벤트, 이벤트 위임, closest  
12단원 async/await, res.ok, try/catch/finally, 로딩 상태
```

여기까지 왔으면 자바스크립트 기초는 끝났습니다. 다음은 이 구조를 자동으로 해 주는 React 입니다.

14단원 연습문제 정답 OpenAPI와 Postman

문제 1 (개념02)

```
const 조건 = { 종류: "프레스", 이름: undefined, 개수: 0, 메모: null, 정렬: "이름",
비고: "" };

function 쿼리만들기(조건) {
  const 쿼리 = new URLSearchParams();

  for (const [이름, 값] of Object.entries(조건)) {
```

★ !값 으로 쓰면 안 됩니다. 0 이 같이 걸러집니다.

```
if (값 === undefined || 값 === null || 값 === "") continue;
  쿼리.set(이름, 값);
}

return 쿼리;
}
```

★★ 여기서 if (!값) continue; 를 쓰면 개수=0 이 사라집니다. 0 은 거짓으로 취급되기 때문입니다.
(01단원 개념04) “값이 있느냐” 를 물을 때는 === undefined 처럼 또렷하게 쓰세요.

★ 순서는 넣은 순서 그대로 유지됩니다. 종류 → 개수 → 정렬. 이름·메모·비고 만 빠졌습니다.

문제 2 (개념02)

```
function 설비주소(이름) {
```

★ 경로에는 encodeURIComponent 를 씁니다. URLSearchParams 가 아닙니다.

```
return `https://예시.com/설비/${encodeURIComponent(이름)}`;
}
```

★★★ 왜 URLSearchParams 를 못 쓰냐 — 그건 이름=값&이름=값 모양을 만드는 도구입니다. 경로용이 아닙니다. 그리고 공백을 + 로 바꾸는데, 경로에서 + 는 공백이 아닙니다.

```
new URL("https://x.com/a+b").pathname → "/a+b"
```

3호+프레스 라는 이름의 설비를 찾게 됩니다. 그런 건 없습니다.

★ 도메인의 한글은 그대로 남습니다. `new URL()` 로 감싸지 않았으니까요. `fetch` 에 넣으면 그때 브라우저가 유니코드로 바꿔서 보냅니다.

문제 3 (개념 04)

```
const 인코딩키 = "abc%2Bdef%2Fghi%3D";
const 디코딩키 = "abc+def/ghi=";

function 키쿼리() {
  const 쿼리 = new URLSearchParams();
```

★ `URLSearchParams` 가 알아서 인코딩하니 **디코딩 키**를 넣습니다.

```
    쿼리.set("serviceKey", 디코딩키);
    return 쿼리.toString();
}
```

★★★★ 인코딩 키를 넣으면 % 가 %25 로 또 바뀝니다.

```
serviceKey=abc%252Bdef%252Fghi%253D
```

서버는 `abc%2Bdef%2Fghi%3D` 라는 글자를 키로 읽습니다. 그런 키는 없습니다. → “등록되지 않은 키”

★★ 반대로 주소를 손으로 이어 붙일 때는 **인코딩 키**를 씁니다. 디코딩 키를 그대로 이어 붙이면 + 가 공백으로 읽혀서 역시 틀립니다.

★ 헷갈리면 그냥 `URLSearchParams` 만 쓰세요. 그럼 디코딩 키 하나만 기억하면 됩니다.

문제 4 (개념 05)

```
async function 안전하게읽기(응답) {
  if (!응답.ok) {
    return { 됐나: false, 값: `서버가 ${응답.status} 를 보냈습니다` };
  }

  const 종류 = 응답.headers.get("content-type") ?? "";
  if (!종류.includes("application/json")) {
```

★ `text/html; charset=utf-8` 처럼 뒤에 뭐가 붙을 수 있으니 `===` 가 아니라 `includes` 로 봅니다.

```
return { 됐나: false, 값: `JSON 이 아닙니다: ${종류}` };
}

try {
  return { 됐나: true, 값: await 응답.json() };
} catch (오류) {
```

★ 여기까지 왔는데 파싱이 안 되는 경우도 있습니다. Content-Type 은 JSON 이라는데 내용이 깨진 경우입니다.

```
return { 됐나: false, 값: `JSON 을 읽지 못했습니다: ${오류.message}` };
}
}
```

★★ 세 겹으로 막았습니다. 하나만으로는 부족합니다.

응답.ok 만 보면 → 200 인데 HTML 인 경우를 놓칩니다 Content-Type 만 보면 → 500 인데 JSON 오류 본문인 경우를 놓칩니다 try/catch 만 하면 → 왜 실패했는지 못 알려 줍니다

★★★ throw 대신 { 됐나, 값 } 을 돌려준 이유 부르는 쪽이 try/catch 를 안 써도 됩니다. 그리고 **빠뜨릴 수가 없습니다**. **.값** 을 쓰려면 **.됐나** 를 지나야 하니까요. ★ 정답은 없습니다. 팀에서 하나로 정하고 그대로 쓰는 게 중요합니다.

문제 5 (개념05)

```
function 무슨말(오류) {
  if (오류.name === "TimeoutError") return "서버가 느립니다";
  if (오류.name === "TypeError") return "연결할 수 없습니다";
  return "알 수 없는 문제입니다";
}
```

★★ 오류.message 가 아니라 오류.name 을 봅니다. message 는 브라우저마다 다릅니다. name 은 규격에 정해져 있습니다.

★ TypeError: Failed to fetch 는 원인이 여럿입니다.

- 인터넷이 끊김
- 주소가 아예 없음
- ★ CORS 에 막힘

코드에서는 이 셋을 못 가릅니다. 그래서 말을 뭉뚱그릴 수밖에 없습니다. ★★ 개발자는 콘솔을 봐야 하고, 사용자에게는 “연결할 수 없습니다” 면 충분합니다.

문제 6 (개념01.02)

```
const 도시들 = { 서울: [37.5665, 126.978], 부산: [35.1796, 129.0756], 제주: [33.4996, 126.5312] };

async function 기온(도시이름, 옵션 = {}) {
  const 좌표 = 도시들[도시이름];
  if (!좌표) throw new Error(`모르는 도시입니다: ${도시이름}`);

  const 주소 = new URL("https://api.open-meteo.com/v1/forecast");
  주소.searchParams.set("latitude", 좌표[0]);
  주소.searchParams.set("longitude", 좌표[1]);
  주소.searchParams.set("current", "temperature_2m");
  주소.searchParams.set("timezone", "Asia/Seoul");

  const 응답 = await fetch(주소, 옵션);
  const 읽은것 = await 안전하게읽기(응답); // * 문제 4를 그대로 씁니다 if (!읽은것.
  됐다) throw new Error(읽은것.값);

  const 자료 = 읽은것.값;
```

★ 단위를 짐작하지 않고 응답에서 꺼냅니다

```
return `${도시이름} ${자료.current.temperature_2m}${자료.current_units.temperature_2m
}`;
}
```

★★ 문제 4에서 만든 것을 여기서 씁니다. 연습문제가 이어집니다. 실제 코드도 이렇게 쌓입니다. 안전 장치를 한 번 만들어 두고 계속 씁니다.

★ 옵션 을 받게 해 둔 것은 문제 7 때문입니다. 같은 함수에 시간 제한만 얹으려고요. 함수를 또 만들지 않습니다.

문제 7 (개념05)

```
async function 기온_시간제한(도시이름, 제한밀리초) {
  try {
    return await 기온(도시이름, { signal: AbortSignal.timeout(제한밀리초) });
  } catch (오류) {
    return 무슨말(오류); // * 문제 5를 그대로 씁니다
  }
}
```

★★ `AbortSignal.timeout()` 은 `fetch` 의 `signal` 로 넘깁니다. 따로 설치할 것도, 타이머를 손으로 지을 것도 없습니다.

★★★ `try` 안에 `await` 가 꼭 있어야 합니다. `return 기온(...)` 처럼 `await` 를 빼면 **`catch` 가 안 걸립니다.** 약속(Promise)만 돌려주고 함수가 끝나 버리기 때문입니다. ★ 12단원 개념05 에서 본 그 함정입니다.

실행

```
document.getElementById("풀기").addEventListener("click", async () => {
  const 줄들 = [];
  const 적기 = (무엇) => {
    줄들.push(String(무엇));
    console.log(무엇);
  };

  적기(decodeURIComponent(String(쿼리만들기(조건))));
```

F12 콘솔 종류=프레스&개수=0&정렬=이름

```
적기(설비주소("3호 프레스"));
```

F12 콘솔 <https://예시.com/설비/3%ED%98%B8%20%ED%94%84%EB%A0%88%EC%8A%A4>

```
적기(키쿼리());
```

F12 콘솔 serviceKey=abc%2Bdef%2Fghi%3D

```
const 좋은응답 = new Response(JSON.stringify({ 이름: "3호 프레스" }), {
  headers: { "Content-Type": "application/json" },
});
const 오백 = new Response("{}", { status: 500, headers: { "Content-Type":
"application/json" } });
const 에이치티엠엘 = new Response("<html></html>", { headers: { "Content-Type":
"text/html" } });

적기(JSON.stringify(await 안전하게읽기(좋은응답)));
```

F12 콘솔 {"됐나":true,"값":{"이름":"3호 프레스"}}

```
적기(JSON.stringify(await 안전하게읽기(오백)));
```

F12 콘솔 {"됐나":false,"값":"서버가 500 를 보냈습니다"}

```
적기(JSON.stringify(await 안전하게읽기(에이치티엠엘)));
```

F12 콘솔 {"됐나":false,"값":"JSON 이 아닙니다: text/html"}

```
적기(무슨말({ name: "TimeoutError" })));
```

F12 콘솔 서버가 느립니다

```
적기(무슨말({ name: "TypeError" })));
```

F12 콘솔 연결할 수 없습니다

```
적기(무슨말({ name: "이상한오류" })));
```

F12 콘솔 알 수 없는 문제입니다

```
try {  
  적기(await 기온("서울"));  
  적기(await 기온_시간제한("서울", 1));
```

F12 콘솔 서버가 느립니다

```
  } catch (오류) {  
    적기("6.7번에서 오류: " + 오류.message);  
  }  
  
  document.getElementById("결과").innerHTML = 줄들.join("<br>");  
});
```

★ 6번의 출력은 못 박지 않았습니다. 진짜 기온이라 누를 때마다 다릅니다. 서울 29.7°C 처럼 나오면 맞은 것입니다.

손으로 · 푸는 문제 — 모범 답안

문제 8

“Postman에서는 200 인데 브라우저에서는 빨간 오류”

답: (1) 코드 문제가 아닙니다. 코드를 고쳐도 안 됩니다.

(2) CORS 입니다. 그 서버가 `Access-Control-Allow-Origin` 헤더를 안 줍니다.

★ 브라우저만 이 규칙을 지킵니다. Postman·curl·node 는 안 지킵니다.
그래서 Postman에서는 되는 것입니다. **서버는 아무 잘못이 없습니다.**

(3) 지금 이 과정에서 할 수 있는 것

- 브라우저에서 되는 API 로 바꿉니다 (Open-Meteo 처럼)
- 확인만 할 거면 Postman 으로 합니다
- ★ 진짜 해결은 **내 서버를 하나 두는 것**입니다 (PART 3)

★★ 하면 안 되는 것 — 브라우저의 보안 설정을 끄는 것.

내 컴퓨터만 뚫리고, 다른 사이트에도 구멍이 납니다.
그리고 **남의 브라우저에서는 여전히 안 됩니다.** 아무것도 해결 안 됩니다.

문제 9

“키를 key.js 로 빼면 안전한가요?”

답: (1) 안전해지지 않습니다. 하나도요.

(2) `key.js` 도 브라우저가 내려받는 파일입니다.

F12 → Sources 에 그 파일이 그대로 보입니다. 주소를 알면 누구나 받습니다.

★ 파일을 나눈 것은 **사람 눈에 안 보이게** 한 것뿐입니다.

브라우저에게는 아무것도 안 숨겨졌습니다.

★★ 빌드해서 압축해도 같습니다. 글자는 그대로 남습니다.

`.gitignore` 에 넣는 것도 다릅니다. 그건 git 에만 안 올리는 것입니다.

배포하면 어차피 나갑니다.

(3) 진짜 해결책 — 키를 **서버에 두고** 브라우저는 내 서버만 부릅니다.

브라우저가 키를 아예 모르게 만드는 것 말고는 방법이 없습니다.

★ “숨긴다” 와 “안 보낸다” 는 다릅니다. 숨기는 건 다 뚫립니다.

문제 10

공공데이터포털 “등록되지 않은 키”

답: (1) ★ **이중 인코딩**을 제일 먼저 의심합니다. 이게 압도적으로 많습니다.

포털의 Encoding 키를 `URLSearchParams` 에 넣은 경우입니다.

(2) 어떻게 확인하나 — **실제로 나간 주소**를 봅니다.

F12 → Network → 그 요청 → Request URL.
`%25` 가 보이면 두 번 인코딩된 것입니다.
★ 코드를 읽지 말고 나간 것을 보세요. 5초면 끝납니다.

(3) 그게 아니라면

- 승인이 아직 안 됐거나, 승인 직후라 아직 안 도는 경우 (몇십 분)
- 신청한 것과 ****다른 API**** 를 부른 경우 (키는 API 별로 승인됩니다)
- 키 앞뒤에 공백이나 줄바꿈이 달려온 경우 (복사할 때 혼합니다)
- 개발계정 키와 운영계정 키를 헷갈린 경우

★ 그리고 Postman 으로 한 번 보내 보세요. 거기서도 안 되면 키 문제가 맞습니다.

문제 11

새 API 를 처음 쓸 때

답: (1) 네 가지 — 주소 / 방식 / 보낼 값 / 응답 모양

(2) 믿으면 안 됩니다. 문서는 낱습니다.

★ 한 번 받아서 `JSON.stringify(자료, null, 2)` 로 ****눈으로 보세요.****
이게 문서 열 페이지보다 빠릅니다.

(3) ★★★ 좌표 격자입니다.

서울이 37.5665 → 37.55 로 떨어지는 걸 보고 "0.05도 간격" 이라고 했는데,
부산은 129.0756 → 129.0625 였습니다. 안 맞았습니다.

배운 것 - ****하나만 보고 규칙을 정하지 않는다.****
"몇 개 해 보니 이렇더라" 는 자주 틀립니다.
규칙이 필요하면 문서를 찾고, 못 찾으면 ****여러 개로 확인****하세요.

★ 이건 API 만의 이야기가 아닙니다. 이 과정 내내 나옵니다.

15단원 연습문제 정답 React 가기 전에

```
RUN node 연습문제_정답.js
```

먼저 스스로 풀어 본 다음에 보세요. 다만 ‘왜 그렇게 되는지’ 설명은 꼭 읽으세요. 답보다 그게 중요합니다.

문제 1·2·3

import 는 파일 맨 위에 모아 씁니다. 세 문제의 답이 여기 다 들어 있습니다.

```
import welcome, { SHIPPING_FEE, addShipping, toPercent as percent } from "./연습문제_도구.js";
```

default(welcome) 가 앞, 중괄호가 뒤입니다. 순서가 반대면 SyntaxError 입니다. toPercent 는 as 로 이름을 바꿔 가져왔습니다.

문제 1

코드	출력
console.log(SHIPPING_FEE);	3000
console.log(addShipping(12000));	15000

문제 2

console.log(percent(0.35));	
출력	35%

원래 이름 toPercent 는 이제 쓸 수 없습니다. as 로 바꿨기 때문입니다.

문제 3

console.log(welcome("김민준"));	
출력	우리편의점에 오신 김민준님 환영합니다

default라서 이름을 마음대로 지을 수 있습니다. hello 라고 해도 똑같이 동작합니다. STORE 는 export 가 없어서 가져올 수 없지만, welcome 안에서는 쓰입니다.

문제 4

```
const randomScore = Math.floor(Math.random() * 100) + 1;  
console.log(randomScore >= 1 && randomScore <= 100);
```

출력 true

공식: $\text{Math.floor}(\text{Math.random()} * (\text{최대} - \text{최소} + 1)) + \text{최소}$

최소 1, 최대 100 이므로 $(100 - 1 + 1) = 100$ 입니다.

문제 5

```
console.log(Number.isInteger(randomScore));
```

출력 true

`Math.floor` 를 거쳤으니 정수입니다. 빠뜨렸다면 `false` 가 나옵니다.

문제 6

```
const drinks = ["아메리카노", "라떼", "녹차", "주스"];  
  
const picked = drinks[Math.floor(Math.random() * drinks.length)];  
console.log(drinks.includes(picked));
```

출력 true

`length` 에 +1 을 하면 가끔 `undefined` 가 뵈어서 `false` 가 나옵니다.

문제 7

```
const openDay = new Date(2026, 2, 5);
```

3월인데 2를 넣었습니다. 월은 0부터 세기 때문입니다.

코드	▶ 출력
<code>console.log(openDay.getFullYear());</code>	▶ 2026
<code>console.log(openDay.getDate());</code>	▶ 5

문제 8

<code>console.log(openDay.getMonth() + 1);</code>
출력 3

`getMonth()` 만 쓰면 2가 나옵니다. 보여 줄 때는 항상 +1 입니다.

문제 9

<code>const mm = String(openDay.getMonth() + 1).padStart(2, "0");</code> <code>const dd = String(openDay.getDate()).padStart(2, "0");</code> <code>console.log(`\${openDay.getFullYear()}-\${mm}-\${dd}`);</code>
출력 2026-03-05

`padStart` 는 문자열 메소드입니다. 숫자에 바로 쓰면 `TypeError` 가 납니다.

문제 10 · 11

<pre>class Menu { constructor(name, price) { this.name = name; this.price = price; } label() { return `\${this.name} \${this.price}원`; } } const americano = new Menu("아메리카노", 4000);</pre>

문제 10

```
console.log(americano.name, americano.price);
```

출력 아메리카노 4000

문제 11

```
console.log(americano.label());
```

출력 아메리카노 4000원

`class` 안에서는 `label() { }` 처럼 씁니다. `label: function ()` 이 아닙니다.

문제 12

```
const menuData = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];

const menus = menuData.map(({ name, price }) => new Menu(name, price));
const expensive = menus.filter((m) => m.price >= 4500).map((m) => m.label());

console.log(expensive);
```

출력 ['라떼 4500원', '케이크 6000원']

`map` 안에서 `new` 로 객체를 만들 수 있습니다. 만들어진 것은 그냥 객체라 `filter` 와 `map` 을 그대로 이어 쓸 수 있습니다. (08단원)

문제 13

```
function daysUntilChristmas() {
  const today = new Date();
  const christmas = new Date(2026, 11, 25); // 12월이라 11
  const oneDay = 1000 * 60 * 60 * 24;

  return Math.ceil((christmas - today) / oneDay);
}
```

```
const left = daysUntilChristmas();
console.log(typeof left);
```

출력 number

```
console.log(Number.isInteger(left));
```

출력 true

오늘이 크리스마스를 지났다면 음수가 나옵니다. 그것도 정상입니다. `Math.ceil` 을 쓴 이유: 오늘이 12월 24일 오후라면 0.5일 정도가 남는데 `floor` 를 쓰면 0일이 되어 “오늘이 크리스마스” 처럼 보입니다. `ceil` 은 1일로 올려 줍니다. 남은 날짜를 셀 때는 보통 `ceil` 이 자연스럽습니다.

문제 14

```
const bad = Menu("라떼", 4500);
```

이 코드는 검사에서 걸립니다

왜:

`class` 는 '틀' 이라서 그냥 부를 수 없습니다.

`new` 를 붙여야 그 틀로 객체를 하나 찍어 냅니다.

고치는 방법:

```
const good = new Menu("라떼", 4500);
```

다행히 `class` 는 이렇게 에러로 알려 줍니다.

07단원의 일반 함수였다면 에러 없이 `undefined` 만 나와서 더 해맸을 겁니다.

`TypeError: Class constructor Menu cannot be invoked without 'new'`

15단원 연습문제 저장하기 정답 React 가기 전에

RUN 브라우저로 열고 F12 → Console 탭

```
["nickname", "point", "user", "colors"].forEach((k) => localStorage.removeItem(k));
```

문제 1

```
localStorage.setItem("nickname", "김민준");
console.log(localStorage.getItem("nickname"));
```

F12 콘솔 김민준

문제 2

```
console.log(localStorage.getItem("phone"));
```

F12 콘솔 null

없는 이름은 에러가 아니라 `null` 입니다. 그래서 쓰기 전에 확인해야 합니다.

문제 3

```
localStorage.setItem("point", 100);

const point = localStorage.getItem("point");
console.log(point, typeof point);
```

F12 콘솔 100 string

숫자로 넣었는데 문자열로 나옵니다. `localStorage` 는 글자만 저장합니다.

```
console.log(Number(point) + 50);
```

F12 콘솔 150

`Number` 를 빼면 "100" + 50 이 되어 10050 이 나옵니다. (01단원의 그 함정)

문제 4

```
const user = { name: "이서연", age: 22, city: "부산" };

localStorage.setItem("user", JSON.stringify(user));

const savedUser = JSON.parse(localStorage.getItem("user"));
console.log(savedUser.city);
```

F12 콘솔 부산

넣을 때 stringify, 꺼낼 때 parse. 이 한 쌍은 항상 짝입니다.

문제 5

```
const colors = ["빨강", "초록", "파랑"];

localStorage.setItem("colors", JSON.stringify(colors));

const savedColors = JSON.parse(localStorage.getItem("colors"));
console.log(Array.isArray(savedColors));
```

F12 콘솔 true

```
console.log(savedColors.length);
```

F12 콘솔 3

parse 를 빼먹으면 글자 그대로라 length 가 3이 아니라 글자 수가 나옵니다.

문제 6

```
const searchInput = document.querySelector("#searchInput");
const out6 = document.querySelector("#out6");
```

페이지를 열 때 · 저장된 게 있으면 보여 준다

```
const lastSearch = localStorage.getItem("lastSearch");
if (lastSearch) {
  out6.textContent = `마지막 검색어: ${lastSearch}`;
}
```

if 로 감싼 이유: 저장된 게 없으면 null 이라 그냥 넣으면 "마지막 검색어: null" 이 보입니다.

```
searchInput.addEventListener("keydown", (e) => {
  if (e.key !== "Enter") {
    return;
  }

  const value = searchInput.value.trim();
  if (!value) {
    return;
  }

  localStorage.setItem("lastSearch", value);
  out6.textContent = `마지막 검색어: ${value}`;
  console.log("저장했습니다:", value);
});
```

문제 7

```
const out7 = document.querySelector("#out7");

function showVisit(count) {
  out7.textContent = `${count}번째 방문입니다`;
}
```

저장된 게 없으면 null 이므로 0으로 시작합니다. Number(null) 은 0 이라 이렇게 한 줄로도 됩니다.

```
const before = Number(localStorage.getItem("visitCount"));
const visitCount = before + 1;

localStorage.setItem("visitCount", visitCount);
showVisit(visitCount);
```

화면 열 때마다 1씩 올라갑니다. 새로고침도 '한 번 연 것' 으로 셉니다.

```
document.querySelector("#resetBtn").addEventListener("click", () => {
  localStorage.setItem("visitCount", 0);
  showVisit(0);
  console.log("방문 횟수를 초기화했습니다");
});
```

removeItem 을 써도 됩니다. 그러면 다음에 열 때 null → 0 → 1 이 됩니다. 여기서는 화면에 0을 보여 주려고 0을 저장했습니다.

더 생각해 볼 것

문제 7에서 `Number(localStorage.getItem(...))` 이 한 번에 되는 이유: `Number(null)` 은 0 입니다. 그래서 “저장된 게 없으면 0” 처리가 저절로 됩니다.

다만 이건 운이 좋은 경우입니다. `Number("abc")` 는 `NaN` 이라 값이 망가져 있으면 `NaN + 1 = NaN` 이 됩니다. 실무에서는 이렇게 한 겹 더 막습니다.

```
const n = Number(localStorage.getItem("visitCount"));
const safe = Number.isNaN(n) ? 0 : n;
```


부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.

09단원 심화문제 정답 구조분해와 스프레드

RUN node 심화문제_정답.js

심화 1

```
const order = {
  id: 100,
  customer: {
    name: "정하늘",
    address: { city: "부산" },
  },
  items: [
    { name: "라떼", price: 4500 },
    { name: "쿠키", price: 3000 },
  ],
};

const {
  id,
  customer: {
    name: customerName,
    address: { city = "미입력" } = {},
  },
  items: [{ name: firstItem }],
} = order;

console.log(id, customerName, city, firstItem);
```

출력 100 정하늘 부산 라떼

한 조각씩 읽어 봅시다

id 그냥 꺼내기 customer: { ... } customer 안으로 들어가라 (이름 바꾸기 아님) name: customerName
name 을 customerName 으로 (이름 바꾸기)

```
address: { city = "미입력" } = {}
```

```
바깥 = {}      address 가 없으면 빈 객체로 치고 들어가라
city = "미입력" 그 안에 city 가 없으면 이 값을 써라
```

items: [{ name: firstItem }] items 배열의 첫 번째로 들어가 name 을 꺼내라

콜론이 두 가지 뜻으로 쓰이는 것이 이 문법의 전부입니다. 오른쪽이 '이름' 이면 → 이름 바꾸기 오른쪽이 { } 나 [] 면 → 더 깊이 들어가기

이렇게까지 한 줄로 쓸 일은 실무에서 드뭅니다. 다만 React 코드에서 이런 모양을 읽어야 할 때가 있어서 한 번은 해 봐야 합니다. 직접 쓸 때는 두 단계까지만 하고 나머지는 나눠 쓰는 게 좋습니다.

심화 2

```
const state = {
  user: { name: "김민준", point: 100 },
  cart: [
    { id: 1, count: 2 },
    { id: 2, count: 1 },
  ],
};

const newState = {
  ...state,
  cart: state.cart.map((item) => (item.id === 2 ? { ...item, count: 5 } : item)),
};

console.log("새 state 의 2번 count:", newState.cart[1].count);
```

출력 새 state 의 2번 count: 5

```
console.log("원본의 2번 count:", state.cart[1].count);
```

출력 원본의 2번 count: 1

```
console.log("user 객체를 그대로 재사용했나:", newState.user === state.user);
```

출력 user 객체를 그대로 재사용했나: true

겉에서 안으로 한 겹씩 새로 만듭니다

```
{ ...state, cart: 새것 }
```

새 겹껍데기를 만들고, cart 자리만 새것으로 갈아 끼웁니다.
user 는 손대지 않았으니 원본의 그 객체를 그대로 가리킵니다.

```
state.cart.map(...)
```

map 은 새 배열을 만듭니다. 원본 cart 는 그대로입니다.

```
item.id === 2 ? { ...item, count: 5 } : item
```

바뀌야 할 것만 새 객체로 만들고, 나머지는 원래 것을 그대로 넘깁니다.

마지막 줄이 왜 `true` 인가 손대지 않은 부분은 '복사하지 않고 그대로 재사용' 합니다. 이게 낭비가 아니라 의도된 것입니다. React 는 "이 객체가 그대로면 다시 그릴 필요 없다" 를 `===` 로 판단합니다. 그래서 안 바뀐 부분을 그대로 넘겨야 화면을 덜 그립니다.

반대로 구조분해로 통째 복사하면 전부 새 객체가 되어 바뀌지 않은 곳까지 다시 그리게 됩니다.

이 패턴이 13단원 종합03의 todos 갱신과 똑같은 모양입니다.

심화 3

```
function summarize(label, ...numbers) {
  if (numbers.length === 0) {
    return `${label} - 자료 없음`; // 조기 반환
  }

  const total = numbers.reduce((sum, n) => sum + n, 0);
  const average = (total / numbers.length).toFixed(1);
  const max = Math.max(...numbers);

  return `${label} - 합계 ${total} / 평균 ${average} / 최대 ${max}`;
}

console.log(summarize("1반", 10, 20, 30));
```

출력 1반 - 합계 60 / 평균 20.0 / 최대 30

```
console.log(summarize("2반", 5));
```

출력 2반 - 합계 5 / 평균 5.0 / 최대 5

```
console.log(summarize("3반"));
```

출력 3반 - 자료 없음

나머지 매개변수는 항상 마지막에 옵니다 `summarize(label, ...numbers)` $\leftarrow$ `label` 을 먼저 받고 나머지를 전부 모읍니다 순서를 바꿔 `...numbers, label` 로 쓰면 `SyntaxError` 입니다. "나머지" 뒤에 무언가가 더 올 수는 없으니까요.

`numbers` 는 진짜 배열입니다 그래서 `length`, `reduce` 를 그대로 쓸 수 있습니다. 개념05에서 본 `arguments` 와 다른 점이 이것입니다.

`Math.max(...numbers)` 의 점 세 개 `Math.max` 는 배열을 통째로 못 받습니다. 숫자를 하나씩 받습니다. 그래서 스프레드로 펼쳐 넣습니다. 06단원에서 못 했던 그 문제의 답입니다.

같은 ... 인데 매개변수 자리에서는 '모으기', 인자 자리에서는 '펼치기' 입니다. 위치로 구분한다는 것을 여기서 한 번에 볼 수 있습니다.

toFixed(1) 은 문자열을 돌려줍니다 20.0 처럼 소수점 아래를 강제로 한 자리 보여 주려면 문자열이어야 합니다. 숫자로 두면 20 으로 나와서 자릿수가 안 맞습니다. 화면에 보여 주는 용도라 문자열로 두는 것이 맞습니다. (02단원)

10단원 심화문제 정답 DOM 조작

RUN 브라우저로 열고 F12 → Console 탭

```
const products = [
  { name: "아메리카노", price: 4000, stock: 10 },
  { name: "라떼", price: 4500, stock: 0 },
  { name: "케이크", price: 6000, stock: 3 },
];
```

심화 1

```
const tableBody = document.querySelector("#tableBody");

products.forEach((product) => {
  const tr = document.createElement("tr");

  const nameTd = document.createElement("td");
  nameTd.textContent = product.name;

  const priceTd = document.createElement("td");
  priceTd.textContent = `${product.price.toLocaleString()}원`;

  const stockTd = document.createElement("td");

  if (product.stock === 0) {
    tr.classList.add("soldout");
    stockTd.textContent = "품절";
  } else {
    stockTd.textContent = product.stock;
  }
}
```

★ tr 안을 다 채운 뒤에 tbody 에 붙입니다

```
tr.appendChild(nameTd);
tr.appendChild(priceTd);
tr.appendChild(stockTd);
tableBody.appendChild(tr);
});
```

화면 세 줄짜리 표가 생기고, 라떼 줄은 회색 + 취소선이 됩니다

`toLocaleString()` 이 하는 일 4000 → "4,000" 처럼 세 자리마다 쉼표를 넣어 줍니다. 돈을 보여 줄 때 거의 항상 씁니다. 결과는 문자열입니다.

왜 `tr` 을 먼저 만들고 나중에 붙이나 빈 `tr` 을 먼저 `tbody` 에 붙이고 `td` 를 하나씩 넣으면 브라우저가 화면을 네 번 다시 그립니다. 다 채운 뒤 한 번에 붙이면 한 번만 그립니다. (11단원 개념05와 같은 이야기)

`classList.add` 를 `tr` 에 붙인 이유 CSS 가 `tr.soldout td` 로 되어 있어서, `tr` 에 클래스를 붙이면 그 안의 `td` 세 개가 전부 회색 + 취소선이 됩니다. `td` 마다 붙일 필요가 없습니다.

심화 2

```
const allRows = document.querySelectorAll("#tableBody tr");
const soldOutRows = document.querySelectorAll("#tableBody tr.soldout");
const onSaleNames = [...document.querySelectorAll("#tableBody
tr:not(.soldout)")].map(
  (tr) => tr.querySelector("td").textContent
);

console.log(`전체 ${allRows.length} 행`);
```

F12 콘솔 전체 3 행

```
console.log(`품절 ${soldOutRows.length} 행`);
```

F12 콘솔 품절 1 행

```
console.log(`판매 중: ${onSaleNames.join(", ")}`);
```

F12 콘솔 판매 중: 아메리카노, 케이크

`onSaleNames` 자체를 찍어도 됩니다. 다만 배열을 그대로 찍으면 크롬은 (2) ['아메리카노', '케이크'] 처럼 개수를 앞에 붙이고, 다른 브라우저는 또 다르게 보여 줍니다. 비교하기 번거로우니 이 자료에서는 `join` 으로 글자를 만들어 찍습니다.

왜 `products` 배열이 아니라 화면에서 세나 "내가 그린 것이 진짜 그려졌는가" 를 확인하는 것이 목적입니다. 실무에서도 화면이 안 나올 때 이렇게 세어 보며 원인을 찾습니다. 0 이 나오면 아예 안 그려진 것이고, 개수는 맞는데 화면에 안 보이면 CSS 문제입니다.

`:not(.soldout)` 은 CSS 선택자입니다 "soldout 클래스가 없는 것" 이라는 뜻입니다. `querySelectorAll` 은 CSS 선택자를 그대로 받으므로 이런 것도 됩니다.

`tr.querySelector("td")` 는 그 행의 '첫 번째' `td` 입니다 `querySelector` 는 언제나 첫 하나만 고릅니다. 여기서는 이름칸입니다. 요소에 대고 부르면 그 요소 '안에서만' 찾습니다. `document` 전체가 아닙니다.

...로 감싼 이유

NodeList 는 배열이 아니라서 map 이 없습니다. (개념01 섹션 4)

심화 3

```
const userInput = "<b>굵게</b> 그리고 <span style='color:red'>빨강</span>";
```

```
document.querySelector("#safeBox").textContent = userInput;
```

화면 굵게 그리고 빨강 ← 글자 그대로

```
document.querySelector("#unsafeBox").innerHTML = userInput;
```

화면 굵게 그리고 빨강 ← 진짜로 굵어지고 빨개집니다

답: 사용자가 입력한 글은 textContent 로 넣어야 합니다.

왜인가 innerHTML 은 받은 글자를 'HTML 코드' 로 읽어서 실행합니다. 태그가 들어 있으면 진짜 태그가 됩니다.

지금은 굵어지는 정도라 귀엽지만, 누군가 이런 걸 입력하면 어떨까요?

```

```

화면에 그려지는 순간 그 안의 코드가 실행됩니다. 남의 사이트에서 내 정보를 빼 가는 공격이 이렇게 시작됩니다. 이것 XSS 라고 부릅니다. 이름은 몰라도 됩니다.

textContent 는 무엇이 들어오든 '글자' 로만 취급합니다. 그래서 안전합니다.

규칙 하나만 기억하세요. 내가 쓴 문자열 → innerHTML 써도 됨 (목록 만들기 등) 사용자가 쓴 글자 → 무조건 textContent

13단원 종합03-04가 사용자 입력을 textContent 로 넣은 이유가 이것입니다.

11단원 심화문제 정답 이벤트

RUN 브라우저로 열고 F12 → Console 탭

심화 1

```
const cartList = document.querySelector("#cartList");
const cartInfo = document.querySelector("#cartInfo");

function updateCartInfo() {
  const rows = cartList.querySelectorAll("li");
  let totalCount = 0;

  rows.forEach((li) => {
    totalCount += Number(li.querySelector(".count").textContent);
  });

  cartInfo.textContent = `담긴 상품 ${rows.length}종 / 총 ${totalCount}개`;
}

cartList.addEventListener("click", (e) => {
  const li = e.target.closest("li");
  if (!li) {
    return; // 빈 곳을 눌렀을 때
  }
}
```

— 삭제

```
if (e.target.classList.contains("remove")) {
  li.remove();
  updateCartInfo();
  console.log("삭제:", li.dataset.id);
  return;
}
```

여기부터는 수량을 다루므로 미리 꺼내 둡니다

```
const countEl = li.querySelector(".count");
const current = Number(countEl.textContent);
```

— 증가

```
if (e.target.classList.contains("plus")) {
  countEl.textContent = current + 1;
  updateCartInfo();
  return;
}
```

감소 · 1 아래로는 안 내려갑니다

```
if (e.target.classList.contains("minus")) {
  if (current <= 1) {
    console.log("1개 아래로는 줄일 수 없습니다");
    return;
  }
  countEl.textContent = current - 1;
  updateCartInfo();
}
});
```

화면 + 를 두 번 누르면 수량이 30이 되고 정보도 함께 바뀝니다

콜백 하나로 세 가지를 처리하는 요령 ① 먼저 li 를 찾는다 (없으면 빠져나간다) ② 어느 버튼인지 classList.contains 로 가른다 ③ 각 갈래 끝에 return 을 둔다 — 아래로 흘러내리지 않게

return 을 빼면 삭제한 다음에 수량 코드까지 실행되어 이미 사라진 요소를 만지려다 에러가 납니다.

왜 정보 갱신을 함수로 뺐나 세 곳에서 똑같이 불러야 하기 때문입니다. 한 곳이라도 빠뜨리면 “화면은 바뀌었는데 합계는 그대로” 가 됩니다. 13단원 종합03의 render 와 같은 생각입니다.

수량을 화면에서 읽는 것이 좋은가 지금은 데이터가 화면에만 있어서 이렇게 했습니다. 13단원처럼 배열을 진짜 데이터로 두고 화면은 그 그림자로 만드는 편이 훨씬 관리하기 쉽습니다. 화면에서 읽으면 문자열 변환도 매번 해야 하고요.

심화 2

```
const idInput = document.querySelector("#idInput");
const idError = document.querySelector("#idError");
const submitBtn = document.querySelector("#submitBtn");

function isEnglishOnly(text) {
  for (const ch of text) {
    const isUpper = ch >= "A" && ch <= "Z";
    const isLower = ch >= "a" && ch <= "z";
```

12단원 심화문제 정답 비동기 fetch

RUN 브라우저로 열고 F12 → Console 탭

```
const BASE_URL = "https://jsonplaceholder.typicode.com";

function delay(ms, value) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(value), ms);
  });
}

function unstableTask() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < 0.5) {
        resolve("작업 성공");
      } else {
        reject(new Error("일시적 오류"));
      }
    }, 300);
  });
}

const out1 = document.querySelector("#out1");
const out2 = document.querySelector("#out2");
const out3 = document.querySelector("#out3");
```

심화 1

```
document.querySelector("#seqBtn").addEventListener("click", async () => {
  out1.textContent = "진행 중...";
  const start = Date.now();

  const a = await delay(500, "A");
  const b = await delay(500, "B");
  const c = await delay(500, "C");

  const took = Date.now() - start;
  out1.textContent = `순서대로: ${a} ${b} ${c} (약 ${took}ms)`;
  console.log("순서대로 걸린 시간:", took);
});
```

```
});

document.querySelector("#allBtn").addEventListener("click", async () => {
  out1.textContent = "진행 중...";
  const start = Date.now();

  const [a, b, c] = await Promise.all([
    delay(500, "A"),
    delay(500, "B"),
    delay(500, "C"),
  ]);

  const took = Date.now() - start;
  out1.textContent = `동시에: ${a} ${b} ${c} (약 ${took}ms)`;
  console.log("동시에 걸린 시간:", took);
});
```

화면 순서대로는 약 1500ms, 동시에는 약 500ms

왜 세 배 차이가 나나 순서대로 : 첫 500 을 기다리고 → 그다음 500 → 그다음 500 = 1500 동시에 : 셋을 한꺼번에 시작시켜 놓고, 제일 늦게 끝나는 것만 기다림 = 500

언제 무엇을 쓰나 앞의 결과가 있어야 다음을 할 수 있으면 → 순서대로 (개념04 섹션 3) 서로 상관없으면 → Promise.all

개념04에서 글을 가져와 userId 로 작성자를 찾은 것은 순서대로여야 했습니다. 글 세 개를 각각 가져오는 것은 서로 상관없으니 Promise.all 입니다.

구조분해로 받는 것에 주의

```
const [a, b, c] = await Promise.all(...)
```

Promise.all 은 '넣은 순서 그대로' 배열을 돌려줍니다. 먼저 끝난 순서가 아닙니다. 그래서 순서를 믿고 구조분해할 수 있습니다.

심화 2

```
async function runWithRetry(maxTry) {
  for (let i = 1; i <= maxTry; i++) {
    try {
      const result = await unstableTask();
      return `${result} (${i}번째 시도)`; // 성공하면 여기서 끝
    } catch (error) {
      console.log(`${i}번째 실패, 다시 시도 -`, error.message);

      if (i === maxTry) {
```